

spintent

SPANISH PARSE INTENTS

V0.99 2026-09-15*

©2026 by Pablo González L[†]

CTAN: <https://www.ctan.org/pkg/spintent>

 <https://github.com/pablgonz/spintent>

Resumen

El paquete **spintent** proporciona una serie de utilidades para docentes de educación primaria y secundaria que necesiten crear documentos PDF *accesibles* (*tagged* PDF) en español usando Lua[†]TeX.

Índice

1	Descripción	1	3.3	El comando <code>\spmoney</code>	6
1.1	Carga del paquete	1	3.3.1	Llaves para <code>\spmoney</code>	7
1.2	El paquete <code>babel</code>	2	3.3.2	Monedas y alias para <code>\spmoney</code>	7
1.3	Fuentes tipográficas	2	3.4	El comando <code>\spshort</code>	7
1.4	El paquete <code>hyperref</code>	2	3.4.1	Llaves para <code>\spshort</code>	9
2	El comando <code>\setspintent</code>	3	3.4.2	Argumentos para <code>\spshort</code>	9
3	The big four	3	4	Utilidades generales	11
3.1	El comando <code>\spnum</code>	3	5	Utilidades para símbolos	11
3.1.1	Llaves para <code>\spnum</code>	3	6	Utilidades para números	13
3.2	El comando <code>\spunit</code>	4	7	Utilidades para geometría	17
3.2.1	Llaves para <code>\spunit</code>	4	8	Referencias	25
3.2.2	Argumentos para <code>\spunit</code>	4	9	Change history	26
3.2.3	El comando <code>\spnewunit</code>	6	10	Índice de la Documentación	27
3.2.4	El comando <code>\spunitalias</code>	6	11	Implementation	29
			12	Índice de la Implementación	145

Motivación y agradecimientos

Desde que desarrollé `scontents`[7] en 2019 y, más tarde, `enumext`[8] en 2024, me he ido interesando cada vez más por la accesibilidad en los documentos PDF. Al principio era un tema prácticamente desconocido para mí, pero a medida que fui aprendiendo sobre él comprendí la importancia que tiene, especialmente en el ámbito educativo.

El equipo del proyecto de $\text{\LaTeX}$ [19] ha realizado un trabajo extraordinario en este campo. Hoy es posible generar documentos etiquetados, validarlos con herramientas como `veraPDF`[20] o inspeccionar su estructura con `ngpdf`[21]. Sin embargo, siempre me quedó una duda: cuando un estudiante utiliza NVDA[11] junto con `MathCAT`[12], ¿realmente escucha lo que el o la docente quiso expresar?

Basta con generar un documento accesible siguiendo las recomendaciones de `The LaTeX Tagged PDF Project` y escucharlo con NVDA/`MathCAT` para darse cuenta de que, en muchos casos, la *verbalización* no coincide con lo que uno espera. Esto no resulta extraño: las reglas del español, y en particular el lenguaje matemático, dependen en gran medida del contexto en que se utilizan.

El objetivo de **spintent** nace precisamente de esta necesidad: proporcionar herramientas para que el o la docente pueda indicar explícitamente el contexto de determinadas expresiones matemáticas desde el propio código fuente de $\text{\LaTeX}$, de modo que el documento PDF resultante no solo sea *accesible*, sino que también produzca una *verbalización más natural* y adecuada para el estudiante al utilizar NVDA/`MathCAT`.

El nuevo estándar PDF 2.0[14, 18, 16], las especificaciones de WTPDF[15] y la guía más reciente de la PDF Association, «Best Practice Guide: Math in PDF»[17], coinciden en que el camino hacia una accesibilidad matemática de calidad pasa por el uso de `MathML`. En ese contexto, Lua[†]TeX ofrece una plataforma especialmente adecuada para incorporar esta tecnología.

*Este archivo describe la documentación para la versión v0.99 revisada por última vez el 2026-09-15.

[†]E-mail: «pablgonz@educarchile.cl».

Licencia y requerimientos

El paquete `spintent` carga y requiere el paquete `unicode-math`[1] y que se ejecute bajo Lua $\TeX$ por lo que es necesario contar con una distribución moderna de $\TeX$ como $\TeX$ Live 2026. Ha sido probado con las clases estándar proporcionadas por $\LaTeX$: `book`, `report`, `article` y `letter` en tamaños `10pt`, `11pt` y `12pt`.

Se otorga permiso para copiar, distribuir y/o modificar este paquete bajo los términos de $\LaTeX$ Project Public License (LPPL), versión 1.3 o posterior (<https://www.latex-project.org/lppl.txt>). El paquete tiene la condición de mantenido al estilo «*A Work in Progress*»¹.

- El requerimiento mínimo es $\TeX$ release 2026-11-01.

Este paquete solo se puede utilizar solo con Lua $\TeX$ y *tagged* PDF activo. Está presente en $\TeX$ Live y MiK $\TeX$, para instalarlo, utilice el gestor de paquetes de su distribución. Si desea una instalación manual, descargue `spintent.zip` y descomprímalo, luego ejecute `luatex spintent.ins`, mueva todos los archivos a las ubicaciones correspondientes y luego ejecute `mktexlsr`. Para generar la documentación, ejecute `arara spintent.dtx`.

```
spintent.sty  ⇒ TDS:tex/luatex/spintent/
spintent.lua  ⇒ TDS:tex/luatex/spintent/
spintent.pdf  ⇒ TDS:doc/luatex/spintent/
README.md    ⇒ TDS:doc/luatex/spintent/
spintent.dtx  ⇒ TDS:source/luatex/spintent/
spintent.ins  ⇒ TDS:source/luatex/spintent/
```

1 Descripción

El nombre `spintent` puede entenderse como «*Spanish Parse Intent*» o también como «*School Parse Intent*». Ambas interpretaciones reflejan el objetivo del paquete y el público para el que fue diseñado: docentes de educación primaria y secundaria.

Está pensado para crear «*apuntes*», «*hojas de ejercicios*», «*clases escritas*» enfocadas en el estudiante. Solo un docente sabe cual es el nivel de aprendizaje del estudiante. El paquete intenta sobrescribir muchas de las reglas dictadas por la heurística de NVDA/MathCAT teniendo esto presente.

- El paquete `spintent` usa y abusa del atributo `:intent` de MathML por medio del comando `\MathMLintent` definido en `latex-lab-mathintent`[30], un encapsulado de `\luamml_attribute:een` proveída por `luamml`[2]. ¡Gracias Marcel!

1.1 Carga del paquete

El paquete `spintent` trabaja solo con Lua $\TeX$ y *tagged* PDF activo usando `tagging=on` dentro del comando `\DocumentMetadata` al inicio del archivo. Las llaves («*keys*») aceptadas por `\DocumentMetadata` y en general para el código de *tagged* PDF están documentadas en los paquetes y `tagpdf`[29] `documentmetadata-support`[30], es deber del usuario revisar la documentación asociada a ellos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage{spintent}
```

1.2 El paquete babel

El paquete `babel-spanish`[10] está marcado como «parcialmente compatible» para *tagged* PDF, pero, es absolutamente necesario para la creación de documentos en español. En caso de encontrar problemas de compatibilidad con otros paquetes se puede desactivar usando:

```
\usepackage[spanish,shorthands=off]{babel}
```

Muchas de las «shorthands» proveídas por `babel-spanish` hoy no son necesarias, más allá de `\sptext` para las abreviaturas y ordinales el resto son solo atajos de teclas para escritura que hoy en día no se ocupan. El paquete `spintent` provee el comando `\spshort` (§3.4) que cubre estos casos, de esta manera podemos cargar `babel`[9] con todas sus funcionalidades sin problemas.

¹A *Work in Progress* es un álbum en vivo de Neil Peart (* 1952–† 2020), baterista y principal letrista de Rush. El nombre no pretende formar parte de la terminología de la LPPL; simplemente describe el estado permanente de este paquete mientras sigo aprendiendo sobre *accesibilidad* en PDF.

1.3 Fuentes tipográficas

Como `spintent` es un paquete Lua $\TeX$ debemos distinguir entre las fuentes tipográficas OpenType/TrueType versus las fuentes de 8bit estándar usadas por $\TeX$ para el *modo texto* y el *modo matemático*. Para evitar problemas con *tagged* PDF preferiremos no combinar fuentes OpenType/TrueType con fuentes 8bit y trabajaremos solo con fuentes OpenType/TrueType que tengan soporte para matemáticas.

La carga de las fuentes tipográficas para Lua $\TeX$ se maneja por medio del paquete `fontspec`[3], si no se especifica ninguna se usarán las fuentes Latin Modern para el *modo texto* y Latin Modern Math para el *modo matemático*.

El tipo de fuente tipográfica escogida es preferencia de cada usuario, en lo personal utilizo las fuente Libertinus para el *modo texto* por medio del paquete `libertinus-otf`[4] y la fuente Erewhon-Math para el modo matemático cargada por el paquete `fourier-otf`[5]. Adicionalmente cargo la fuente Source Code Pro para el texto tipo «máquina de escribir» por medio del paquete `sourcecodepro`[6].

```
% \DocumentMetadata{...}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\usepackage{spintent}
```

1.4 El paquete hyperref

El paquete `hyperref`[23] es «casi obligatorio» cuando se crean documentos *tagged* PDF, se encarga de las referencias cruzadas `\label` y `\ref`, las notas al pie `\footnote`, los hipervínculos `\href` y muchos otros elementos como marcadores, índices y metadatos.

```
\DocumentMetadata
{
  lang=es-CL,
  pdfversion=2.0,
  pdfstandard=ua-2,
  tagging=on,
  tagging-setup={math/setup={mathml-SE}},
}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\setmathfont{Erewhon-Math}[range={cal,bfcal},StylisticSet=1]
\usepackage{spintent, hyperref}
\hypersetup
{
  colorlinks      = true,
  pdfstartview    = FitH,
  pdfdisplaydoctitle = true,
  pdftitle        = {Test para el paquete spintent},
  pdfinfo         =
  {
    Title   = {Test},
    Subject = {spintent},
  },
}
```

2 El comando \setspintent

`\setspintent` `\setspintent{⟨comando⟩}{⟨key-uno = valor, key-dos = valor, key-tres = valor, ...⟩}`

El comando `\setspintent` establece las llaves (*⟨keys⟩*) de forma global para los comandos provistos por `spintent`. Puede utilizarse tanto en el preámbulo como en el cuerpo del documento tantas veces como se desee. Las llaves (*⟨keys⟩*) definidas en el *argumento opcional* [*⟨key = val⟩*] de los comandos tienen prioridad sobre las establecidas por este, anulando las configuraciones globales de `\setspintent`.

Muchas de las utilidades que provee el paquete `spintent` implementan el sistema [*⟨key = val⟩*] y están pensadas para ser utilizadas en la generación de «*hojas de ejercicios*» o «*clases escritas*». Se recomienda usar `\setspintent` y establecerlas de manera global para cada documento, para mantener la coherencia en la verbalización de NVDA/MathCAT a lo largo del mismo.

El sistema [*⟨key = val⟩*] utilizado por `spintent` se implementa usando el módulo `l3keys` de `expl3`[24]. Por diseño, las llaves que reciben un *valor* después del signo '=' NO aceptan *valores vacíos* y es obligatorio pasar uno válido; las llaves documentadas como *⟨valor prohibido⟩* no reciben *ningun valor*. En caso de omitir un *valor* válido o pasar un *valor vacío* a una llave que acepte *valor* o pasar un *valor* a una llave del tipo *⟨valor prohibido⟩* se devolverá un "error" y se detendrá la compilación del documento.

3 The big four

El paquete `spintent` provee cuatro grandes comandos («*the big four*²»): `\spnum` para el manejo de números, `\spunit` para el manejo de unidades, `\spmoney` para el manejo de dinero y `\spshort` para el manejo de abreviaturas, acrónimos y números ordinales.

- También se incluyen las utilidades `\spnewunit` y `\spunitalias` asociadas a al comando `\spunit`.

3.1 El comando \spnum

`\spnum` `\spnum{⟨número⟩}` `\spnum[⟨key = val⟩]{⟨número,número;número⟩}`
`\spnum[⟨key = val⟩]{⟨+número⟩}` `\spnum[⟨key = val⟩]{⟨número.número;número⟩}`
`\spnum[⟨key = val⟩]{⟨-número⟩}` `\spnum[⟨key = val⟩]{⟨número;número⟩}`
`\spnum[⟨key = val⟩]{⟨número,número⟩}` `\spnum[⟨key = val⟩]{⟨número e exponente⟩}`
`\spnum[⟨key = val⟩]{⟨número.número⟩}` `\spnum[⟨key = val⟩]{⟨número E exponente⟩}`

El comando `\spnum` trabaja en *modo matemático* y recibe el *argumento obligatorio* {*⟨número⟩*}, donde la «*parte entera*» puede iniciar con los signos '+' o '-', la «*parte decimal finita*» **debe** iniciar con una coma ',' o con un punto '.', y la «*parte decimal infinita (periódica)*» **debe** iniciar con un punto y coma '; '.

Si no se proporciona el *argumento opcional* [*⟨key = val⟩*] y se ejecuta `\spnum{+23}` se imprimirá '+23' en la salida PDF y NVDA/MathCAT verbalizará «*más veintitrés*», si se ejecuta `\spnum{3,14}` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce*», si se ejecuta `\spnum{3,1;4}` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma uno con cuatro periódico*».

Además, el *argumento obligatorio* {*⟨número⟩*} admite al final de este un sufijo de «*notación científica*». Este se indica mediante las letras 'e' o 'E', acompañadas de los signos opcionales '+' o '-' seguido por los dígitos del exponente. Por ejemplo `\spnum{3,14e3}` imprimirá '3,14 · 10³' en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- La coma ',' o el punto '.' son *separadores decimales* y NO de millares, por lo tanto solo pueden aparecer una «única vez» dentro del {*⟨argumento⟩*}. El {*⟨argumento⟩*} puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.1.1 Llaves para \spnum

`cifras-sep = {⟨true | false⟩}` default: *true*

Establece el *espaciado* utilizado para agrupar los dígitos en bloques de tres, tanto en la parte entera como en la parte decimal finita. Si se establece en *true* se siguen las normas de la RAE y se utiliza un *espacio fino* '\,' entre las cifras; si se establece en *false*, se respetan los espacios matemáticos introducidos en el argumento. Por ejemplo `\spnum{100000}` imprimirá '100 000' en la salida PDF y `\spnum[cifras-sep=false]{10\,00\,00}` imprimirá '10 00 00' en la salida PDF.

`read-sign = {⟨terse | clear⟩}` default: *terse*

Establece la *forma* en la que NVDA/MathCAT verbalizará los signos '+' o '-'. Si se establece en *terse*, NVDA/MathCAT verbalizará «*más*» o «*menos*» **antes** de leer el número; si se establece en *clear*, NVDA/MathCAT verbalizará «*positivo*» o «*negativo*» **después** de leer el número. Por ejemplo `\spnum{-23}` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*menos veintitrés*» y `\spnum[read-sign=clear]{-23}` imprimirá '-23' en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés negativo*».

`read-period-sep = {⟨coma | punto⟩}` default: *coma*

²Los «*cuatro grandes*» del Thrash Metal: Metallica, Megadeth, Slayer y Anthrax, que me acompañan mientras desarrollo `spintent`.

Establece la *forma* en la que NVDA/MathCAT verbalizará e imprimirá el signo de separación ‘;’ de la «*parte decimal infinita (periódica)*» cuando no se tiene la «*parte decimal finita*». Si se establece en `coma`, se imprimirá ‘,’ en la salida PDF y NVDA/MathCAT verbalizará «*coma*»; si se establece en `punto`, se imprimirá ‘.’ en la salida PDF y NVDA/MathCAT verbalizará «*punto*». Por ejemplo `\spnum{3;4}` imprimirá ‘3,4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma cuatro periódico*» y `\spnum[read-period-sep=punto]{3;4}` imprimirá ‘3.4’ en la salida PDF y NVDA/MathCAT verbalizará «*tres punto cuatro periódico*»

- Esta llave es necesaria SOLO cuando se utilizan «*decimales periódicos puros*», es decir, cuando no se ha incluido en el `{(argumento)}` una *coma* ‘,’ o un *punto* ‘.’ antes del signo de separación ‘;’ periódica.

`notation-sym = {(times | cdot)}`

default: `cdot`

Establece el *símbolo* que se utilizará para imprimir la «*notación científica*» al usar ‘e’ o ‘E’. Si se establece en `times` imprimirá ‘×’ en la salida PDF y NVDA/MathCAT verbalizará «*por*»; si se establece en `cdot` imprimirá ‘·’ en la salida PDF y NVDA/MathCAT verbalizará «*por*». Por ejemplo `\spnum{3,14e3}` imprimirá ‘3,14 · 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*» y `\spnum[notation-sym=times]{3,14e3}` imprimirá ‘3,14 × 10³’ en la salida PDF y NVDA/MathCAT verbalizará «*tres coma catorce por diez al cubo*».

- Decir «*notación científica*» aquí es un poco exagerado, en realidad es «*multiplicación por potencias de 10*», no se verifica si la entrada cumple o no con la condición de estar escrita en «*notación científica*».

3.2 El comando \spunit

<code>\spunit{(unidad)}</code>	<code>\spunit[key = val]{(número unidad*unidad)}</code>
<code>\spunit[key = val]{(unidad)}</code>	<code>\spunit[key = val]{(número unidad*unidad/unidad)}</code>
<code>\spunit[key = val]{(número unidad)}</code>	<code>\spunit[key = val]{(número unidad*unidad/unidad*unidad)}</code>

El comando `\spunit` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{(unidad)}`, el cual puede contener una «*parte numérica*» opcional la cual se procesa internamente por el comando `\spnum` al inicio del `{(argumento)}` seguido de una `{(unidad)}` o un `{(alias)}` valido descritos en §3.2.2 o una `{(unidad)}` creada por el comando `\spnewunit` (§3.2.3) o un `{(alias)}` creado por el comando `\spunitalias` (§3.2.4).

Si no se proporciona el *argumento opcional* `[key = val]` y se ejecuta `\spunit{23 m}` se imprimirá ‘23 m’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros*».

La multiplicación entre las `{(unidades)}` se realiza mediante un *asterisco* ‘*’ y las potencias de estas se indican con un *circunflejo seguido del exponente entre llaves* ‘^`{(exponente)}`’. Por ejemplo si se ejecuta `\spunit{23 m*s^2}` se imprimirá ‘23 m s²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros segundos al cuadrado*».

Se pueden usar *fracciones lineales* con las `{(unidades)}` utilizando una «*única barra* ‘/’ para la separación entre las `{(unidades)}` del numerador y el denominador, teniendo presente que el denominador NO puede contener números de ningún tipo. Por ejemplo si se ejecuta `\spunit{23 m/s^2}` se imprimirá ‘23 m/s²’ en la salida PDF y NVDA/MathCAT verbalizará «*veintitrés metros por segundos al cuadrado*».

- Solo imprimiremos *fracciones lineales*, si se requiere otra forma fraccionaria para explicaciones y otros usos se debe usar el comando por separado. El `{(argumento)}` puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo, NO puede contener más de una «*única barra* ‘/’ y NO puede contener números en el denominador. Cualquier entrada fuera de esta sintaxis devolverá un “error” deteniendo la compilación del documento.

3.2.1 Llaves para \spunit

El comando `\spunit` posee las *meta-keys* `cifras-sep`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1).

`print-cdot = {(true | false)}`

default: `false`

Establece si se *imprime* `\cdot` ‘·’ o un *espacio fino* ‘\,’ para la multiplicación de `{(unidades)}`. Si se establece en `true`, se imprimirá ‘·’ entre las `{(unidades)}` en la salida PDF; si se establece en `false` se imprimirá un *espacio fino* ‘\,’ entre las `{(unidades)}` en la salida PDF.

`read-cdot = {(true | false)}`

default: `false`

Establece la *forma* en la que NVDA/MathCAT verbalizará la multiplicación de `{(unidades)}`. Si se establece en `true`, NVDA/MathCAT verbalizará «*por*» entre las `{(unidades)}`; si se establece en `false`, NVDA/MathCAT no verbalizará nada entre las `{(unidades)}`.

`read-slash = {(por | partido-por)}`

default: `por`

Establece la *forma* en la que NVDA/MathCAT verbalizará la «*barra* ‘/’ de separación entre el numerador y el denominador. Si se establece en `por`, NVDA/MathCAT verbalizará «*por*»; si se establece en `partido-por`, NVDA/MathCAT verbalizará «*partido por*».

3.2.2 Argumentos para \spunit

Resumen de `{(unidades)}` y `{(alias)}` soportados por `\spunit`.

Entrada	Alias soportados	Nombre unidad	Salida	NVDA/MathCAT
m	metro, metros	Metro	m	«metro»
g	gramo, gramos	Gramo	g	«gramo»
s	segundo, segundos	Segundo (Tiempo)	s	«segundo (Tiempo)»
l	litro, l-litro	Litro (minúscula)	l	«litro (minúscula)»
h	hora, horas	Hora	h	«hora»
d	dia, dias	Día	d	«día»
A	amperio, ampere	Amperio	A	«amperio»
V	voltio, volt	Voltio	V	«voltio»
W	vatio, watt	Vatio	W	«vatio»
J	julio, joule	Julio	J	«julio»
N	newton	Newton	N	«newton»
Pa	pascal	Pascal	Pa	«pascal»
Hz	hercio, hertz	Hercio	Hz	«hercio»
Ω	ohmio, ohm, Omega	Ohmio	Ω	«Ohmio»
F	faradio	Faradio	F	«Faradio»
C	culombio	Culombio	C	«Culombio»
K	kelvin, Kelvin	Kelvin	K	«Kelvin»
°C	celsius, degC, °C	Grados Celsius	°C	«Grados Celsius»
°F	fahrenheit, degF, °F	Grados Fahrenheit	°F	«Grados Fahrenheit»
cal	caloria, calorías	Caloría	cal	«Caloría»
b	bit	Bit	b	«Bit»
B	byte, bytes	Byte	B	«Byte»
in	pulgada, pulgadas	Pulgada	in	«Pulgada»
ft	pie, pies	Pie	ft	«Pie»
yd	yarda, yardas	Yarda	yd	«Yarda»
mi	milla, millas	Milla	mi	«Milla»
lb	libra, libras	Libra	lb	«Libra»
oz	onza, onzas	Onza	oz	«Onza»
gal	galon, galones	Galón	gal	«Galón»
Å	angstrom	Angstrom	Å	«Angstrom»
μ	micro	Micro (Prefijo)	μ	«Micro (Prefijo)»
Ω	mho	Mho / Siemens	Ω	«Mho / Siemens»
'	minmin	Minuto (Arco)	'	«Minuto (Arco)»
"	segseg	Segundo (Arco)	"	«Segundo (Arco)»
u	u-masa	Unidad masa atómica	u	«Unidad masa atómica»
Unidades directas (sin alias):				
L	-	Litro (Mayúscula)	L	«Litro (Mayúscula)»
ℓ	-	Litro (Cursiva)	ℓ	«Litro (Cursiva)»
w	-	Vatio (Minúscula)	w	«Vatio (Minúscula)»
°K	-	Grado Kelvin	°K	«Grado Kelvin»
Bq	-	Becquerel	Bq	«Becquerel»
Da	-	Dalton	Da	«Dalton»
Gy	-	Gray	Gy	«Gray»
S	-	Siemens	S	«Siemens»
Sv	-	Sievert	Sv	«Sievert»
T	-	Tesla	T	«Tesla»
Wb	-	Weber	Wb	«Weber»
atm	-	Atmósfera	atm	«Atmósfera»
au	-	Unidad astronómica	au	«Unidad astronómica»
bar	-	Bar	bar	«Bar»
cd	-	Candela	cd	«Candela»
dB	-	Decibelio	dB	«Decibelio»
dyn	-	Dina	dyn	«Dina»
eV	-	Electronvoltio	eV	«Electronvoltio»
erg	-	Ergio	erg	«Ergio»
ha	-	Hectárea	ha	«Hectárea»
kat	-	Katal	kat	«Katal»
kg	-	Kilogramo	kg	«Kilogramo»
km	-	Kilómetro	km	«Kilómetro»
lm	-	Lumen	lm	«Lumen»

Continúa en la siguiente página...

Continuación de la tabla 1

Entrada	Alias soportados	Nombre unidad	Salida	MathCAT
lx	-	Lux	lx	« <i>Lux</i> »
cm	-	Centímetro	cm	« <i>Centímetro</i> »
min	-	Minuto (Tiempo)	min	« <i>Minuto (Tiempo)</i> »
mm	-	Milímetro	mm	« <i>Milímetro</i> »
mol	-	Mol	mol	« <i>Mol</i> »
pc	-	Pársec	pc	« <i>Pársec</i> »
rad	-	Radián	rad	« <i>Radián</i> »
rpm	-	Rev. por minuto	rpm	« <i>Rev. por minuto</i> »
sr	-	Estereorradián	sr	« <i>Estereorradián</i> »
t	-	Tonelada	t	« <i>Tonelada</i> »
a	-	Año / Área (are)	a	« <i>Año / Área (are)</i> »
y	-	Año (Year)	y	« <i>Año (Year)</i> »
°	-	Grado (Arco)	°	« <i>Grado (Arco)</i> »

3.2.3 El comando \spnewunit

```
\spnewunit \spnewunit{\nueva unidad}{\verbalización NVDA}
```

El comando `\spnewunit` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\{\langle \text{nueva unidad} \rangle\}$ que establece lo que se imprimirá en la salida PDF y $\{\langle \text{verbalización NVDA} \rangle\}$ que establece como NVDA/MathCAT verbalizará la $\{\langle \text{nueva unidad} \rangle\}$.

La `{\nueva unidad}` no debe estar registrada como `{\unidad}` o `{\alias}` para `\spunit` descritos en §3.2.2. Si la `{\nueva unidad}` ya está registrada se devolverá un “error” y se detendrá la compilación del documento.

La declaración de la $\{\langle\textit{nueva unidad}\rangle\}$ es «global» y se almacena en el módulo `spintent.lua` como una $\{\langle\textit{unidad}\rangle\}$ válida para pasar como $\{\langle\textit{argumento}\rangle\}$ a `\spunit`. Por ejemplo: `\spnewunit{px}{píxeles}` creará la $\{\langle\textit{nueva unidad}\rangle\}$ ‘px’ y cuando se utilice `\spunit{100 px}` se imprimirá ‘100 px’ en la salida PDF y NVDA/MathCAT verbalizará «cien píxeles».

3.2.4 El comando \spunitalias

`\spuntialias` `\spuntialias{nuevo alias}{expresión de unidades}`

El comando `\spunitalias` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: $\{\langle \text{*nuevo alias* \rangle}\}$ que establece es el nuevo $\{\langle \text{*alias* \rangle}\}$ de $\{\langle \text{*argumento* \rangle}\}$ para `\spunit` y $\{\langle \text{*expresión de unidades* \rangle}\}$ que puede contener multiplicación o división de $\{\langle \text{*unidades* \rangle}\}$ ya registradas que se imprimirán en la salida PDF.

El `{\nuevo alias}` no debe estar registrado como `{\unidad}` o `{\alias}` para `\spunit` descritos en §3.2.2 o estar definido como `{\unidad}` por medio del comando `\spnewunit`. Si el `{\nuevo alias}` ya está registrado se devolverá un “error” y se detendrá la compilación del documento.

La declaración de un `\{nuevo alias\}` es «global» y se almacena en el módulo `spintont.lua` como una `\{alias\}` valido para pasar como `\{argumento\}` a `\spunit`. Por ejemplo: `\spunitalias{caudal}{m^3/s}` creará el `\{alias\}` ‘caudal’ y cuando se utilice `\$spunit{50 caudal}` se imprimirá ‘50 m³/s’ en la salida PDF y NVDA/MathCAT verbalizará «cincuenta metros al cubo por segundos».

3.3 El comando \spmoney

<code>\spmoney</code>	<code>\spmoney{<i>cifra</i>}</code>
<u><code>\spmoney</code></u>	<code>\spmoney{<i>cifra moneda</i>}</code>
	<code>\spmoney[<i>key = val</i>]{<i>cifra moneda</i>}</code>

El comando `\spmoney` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{ \langle \text{cifra} \rangle \}$ que contiene la «*cifra numérica*» positiva o negativa la se cual procesa internamente por el comando `\spnum` al inicio del $\{ \langle \text{argumento} \rangle \}$ acompañada de un «*símbolo de moneda*» o «*alias de moneda*» opcional descritos en §3.3.2.

Si no se proporciona el *argumento opcional* [`(key = val)`] y se ejecuta `$\spmoney{500}$` se imprimirá ‘\$500’ en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos».

- El comportamiento de `\spmoney` depende de como se pase el *argumento obligatorio* que tiene prioridad por sobre los valores por defecto. Por ejemplo: `\$ \spmoney{500} dolares` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «quinientos dolares» aunque la moneda por defecto sean pesos.

Si se pasa un «*alias de moneda*» del tipo ISO (código de divisas) como CLP o USD (mayúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país y se ajusta la salida PDF al estándar ISO. Por ejemplo: `$\spmoney{500 CLP}` imprimirá ‘CLP500’ en la salida PDF y NVDA/MathCAT verbalizará «*quinientos pesos chilenos*».

Si se pasa un «*alias de moneda*» del tipo ISO (código de divisas) como `clp` o `usd` (minúsculas) se modifica la verbalización en **NVDA/MathCAT** agregando el país, pero, NO se modifica la salida PDF imprimiendo el

«*símbolo de moneda*» correspondiente. Por ejemplo: `\$ \spmoney{500 c\p}` imprimirá '\$500' en la salida PDF y NVDA/MathCAT verbalizará «quinientos pesos chilenos»

Pasar de manera directa en el *argumento obligatorio* un «*símbolo de moneda*» como '\$', '€', '£' o un «*alias de moneda*» como peso, euro o libra produce la salida PDF y verbalización estándar esperada para NVDA/MathCAT.

- La posición del «*símbolo de moneda*» a la izquierda o a la derecha de la «*cifra numérica*» y la verbalización para NVDA/MathCAT se manejan desde el módulo Lua `spintent.lua` y no se proveen llaves para modificar este comportamiento.
- El «*símbolo de moneda*» es tomado de la fuente tipográfica definida para el *modo matemático* al momento de ejecutar el comando, se debe tener presente en caso de querer usar un «*símbolo de moneda*» poco habitual que esté soportada por `spintent`. El `{(argumento)}` puede contener espacios matemáticos válidos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. La coma ',' o el punto '.' como *separador decimal* solo pueden aparecer una «única vez». Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.3.1 Llaves para \spmoney

El comando `\spmoney` posee la *meta-key* `cifras-sep` pasada al comando `\spnum` (§3.1.1).

`currency = {(moneda | alias)}` default: peso

Establece el *símbolo de moneda* por defecto que se imprimirá en la salida PDF y cómo lo verbalizará NVDA/MathCAT cuando SOLO se pasa una «*cifra numérica*» como `{(argumento)}` al comando `\spmoney`. El «*símbolo de moneda*» o el «*alias de moneda*» pasado a esta llave debe ser alguno de los descritos en §3.3.2.

`sub-units = {(true | false)}` default: false

Establece si se usarán *subunidades* de la moneda en curso para la verbalización en NVDA/MathCAT. La moneda en curso debe aceptar *subunidades*. Por ejemplo: `\$ \spmoney[sub-units=true]{100,5 dolares}` imprimirá '\$100,5' en la salida PDF y NVDA/MathCAT verbalizará «*cient dólares con cincuenta centavos*».

`dolar-math = {(true | false)}` default: true

Establece si el *símbolo* '\$' se imprimirá en *modo texto* o en *modo matemático*. Si se establece en `true`, se usará la fuente tipográfica definida para el *modo matemático* en curso para imprimir '\$' en la salida PDF; si se establece en `false`, se usará la fuente tipográfica definida para el *modo texto* para imprimir '\$' en la salida PDF.

3.3.2 Monedas y alias para \spmoney

Resumen de los `{(símbolos de moneda)}` y `{(alias de moneda)}` soportados como `{(argumento)}` para `\spmoney`.

3.4 El comando \spshort

`\spshort {(abreviaturas | acrónimos | números ordinales)}`
`\spshort[key = val]{(abreviaturas | acrónimos | números ordinales)}`
`\spshort[read-arg={verbalización del argumento para NVDA}]{(comandos LATEX)}`

El comando `\spshort` trabaja *solo en modo texto* y recibe un *argumento obligatorio* del tipo `{(abreviaturas)}`, `{(acrónimos)}` o `{(números ordinales)}` descritos en §3.4.2.

Si no se proporciona el *argumento opcional* `[key = val]` y se ejecuta `\spshort{dr. a}` se imprimirá 'Dr.^a' en la salida PDF y NVDA/MathCAT verbalizará «*doctora*»; si se ejecuta `\spshort{onu}` se imprimirá 'ONU' en la salida PDF y NVDA/MathCAT verbalizará «*organización de las naciones unidas*» y si se ejecuta `\spshort{mineduc}` se imprimirá 'Mineduc' en la salida PDF y NVDA/MathCAT verbalizará «*ministerio de educación*».

Para la escritura de `{(números ordinales)}` se puede ejecutar `\spshort{1. a}` que imprimirá '1.^a' en la salida PDF y NVDA/MathCAT verbalizará «*primera*»; si se ejecuta `\spshort{1. o}` imprimirá '1.^o' en la salida PDF y NVDA/MathCAT verbalizará «*primero*».

Para crear una nueva `{(abreviatura)}`, `{(acrónimo)}` o `{(número ordinal)}` que no se encuentre registrada por `spintent` dentro del módulo Lua `spintent.lua` descritas en §3.4.2 se debe utilizar la llave `read-arg` y definir esta dentro del *argumento obligatorio* que puede contener `{(comandos LATEX)}` en *modo texto* los cuales se imprimirán en salida PDF.

- Para la escritura de `{(números ordinales)}` se debe tener cuidado y no confundir la «*letra o voladita*» 'o' con el símbolo de grado '°'. Si la fuente tipográfica del *modo texto* está utilizando `Numbers=OldStyle` los números pueden quedar muy separados de la «*letra voladita*», la solución para esto es agregar `\addfontfeatures{Numbers=Lining}` dentro de un grupo junto al comando: `{\addfontfeatures{Numbers=Lining}\spshort{1. o}}`.
- El comando `\spshort` está pensado como un reemplazo para *shorthands* y el comando `\sptext` provistos por el paquete `babel` en español compatible con *tagged* PDF. Es de las pocas utilidades que provee el paquete `spintent` que trabaja *solo en modo texto*. Internamente utiliza la llave `E` (*expansion text*) proveída por el paquete `tagpdf`[29].

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
\$	peso, pesos	peso	\$	«peso / pesos»
€	euro, euros	euro	€	«euro / euros»
£	libra, libras	libra	£	«libra / libras»
¥	yen, yenes	yen	¥	«yen / yenes»
¥	yuan, yuanes	yuan	¥	«yuan / yuanes»
₩	won, wons	won	₩	«won / wons»
₪	shekel, shekels, sequel, sequeles	séquel	₪	«séquel / séqueles»
₹	rupia, rupias	rupia	₹	«rupia / rupias»
₽	rublo, rublos	rublo	₽	«rublo / rublos»
₺	lira, liras	lira	₺	«lira / liras»
₴	grivna, grivnas	grivna	₴	«grivna / grivnas»
¢	centavo, centavos	centavo	¢	«centavo / centavos»

Divisas en formato ISO (Mayúsculas)

CLP	—	peso chileno	CLP	«peso chileno / pesos chilenos»
MXN	—	peso mexicano	MXN	«peso mexicano / pesos mexicanos»
USD	—	dólar	USD	«dólar estadounidense / dólares estadounidenses»
EUR	—	euro	EUR	«euro / euros»
GBP	—	libra	GBP	«libra / libras»
JPY	—	yen	JPY	«yen / yenes»
CNY	—	yuan	CNY	«yuan / yuanes»
KRW	—	won	KRW	«won / wons»
ILS	—	séquel	ILS	«séquel / séqueles»
INR	—	rupia	INR	«rupia / rupias»
RUB	—	rublo	RUB	«rublo / rublos»
TRY	—	lira	TRY	«lira / liras»
UAH	—	grivna	UAH	«grivna / grivnas»

Códigos ISO en minúsculas

clp	—	peso chileno	\$	«peso chileno / pesos chilenos»
mxn	—	peso mexicano	\$	«peso mexicano / pesos mexicanos»
usd	dolar, dolares	dólar	\$	«dólar estadounidense / dólares estadounidenses»
eur	—	euro	€	«euro / euros»
gbp	—	libra	£	«libra / libras»
jpy	—	yen	¥	«yen / yenes»
cny	—	yuan	¥	«yuan / yuanes»
krw	—	won	₩	«won / wons»
ils	—	séquel	₪	«séquel / séqueles»
inr	—	rupia	₹	«rupia / rupias»
rub	—	rublo	₽	«rublo / rublos»
try	—	lira	₺	«lira / liras»
uah	—	grivna	₴	«grivna / grivnas»

3.4.1 Llaves para \spshort

`read-arg = {⟨verbalización NVDA⟩}` default: no usado

Establece la verbalización del argumento {⟨comandos $\LaTeX$ ⟩} de la nueva {⟨abreviatura⟩}, {⟨acrónimo⟩} o {⟨número ordinal⟩} para NVDA. El valor de esta llave *siempre* debe pasarse entre ‘{ }’. Por ejemplo: `\spshort[read-arg={octavo}]{8.\textsuperscript{\emph{avo}}}` creará una nueva {⟨abreviatura⟩} que imprimirá ‘8.^{avo}’ en la salida PDF y NVDA verbalizará «octavo».

`small-caps = {⟨true | false⟩}` default: false

Establece si se usarán versalitas para imprimir {⟨acrónimos⟩} y para la «letra volada» en {⟨abreviaturas⟩} y {⟨números ordinales⟩} respetando las reglas de la RAE para el uso de estas. Si se establece en `true`, las abreviaturas con mayúscula como EE. UU. y los acrónimos de cinco o más letras como Unicef no se verán afectados; si se establece en `false`, no se aplicarán versalitas. Por ejemplo: `\spshort[small-caps=true]{dr.a}` imprimirá ‘Dr.^a’ en la salida PDF y NVDA verbalizará «doctora».

3.4.2 Argumentos para \spshort

El cuadro 2 resume las abreviaturas, expresiones latinas y títulos registrados en el módulo Lua `spintent.lua`.

Cuadro 2: Abreviaturas soportadas

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Abreviaturas con letras voladas			
dr.a	dr. ^a	Dr. ^a	«doctora»
dr.as	—	Dr. ^{as}	«doctoras»
prof.a	—	Prof. ^a	«profesora»
d.a	d. ^a	D. ^a	«doña»
m.a	—	M. ^a	«maría»
n.o	n. ^o	N. ^o	«número»
n.os	—	N. ^{os}	«números»
c.ia	—	C. ^{ia}	«compañía»
Abreviaturas comunes y expresiones latinas			
pág.	pag.	pág.	«página»
vol.	—	vol.	«volumen»
etc.	—	etc.	«etcétera»
et al.	et. al.	et al.	«y otros»
ibíd.	ibid.	ibíd.	«ibídem»
op. cit.	op.cit.	op. cit.	«obra citada»
loc. cit.	loc.cit.	loc. cit.	«lugar citado»
v. gr.	v.gr.	v. gr.	«verbigracia»
i. e.	i.e.	i. e.	«esto es»
e. g.	e.g.	e. g.	«por ejemplo»
p. ej.	p.ej.	p. ej.	«por ejemplo»
Títulos, profesiones y cargos			
sr.	—	Sr.	«señor»
sra.	—	Sra.	«señora»
srta.	—	Srta.	«señorita»
dr.	—	Dr.	«doctor»
dra.	—	Dra.	«doctora»
dres.	—	Dres.	«doctores»
dras.	—	Dras.	«doctoras»
prof.	—	Prof.	«profesor»
profa.	—	Profa.	«profesora»
profs.	—	Profs.	«profesores»
ing.	—	Ing.	«ingeniero»
ings.	—	Ings.	«ingenieros»
lic.	—	Lic.	«licenciado»
v. b.	v.b.	V. B.	«visto bueno»
Abreviaturas compuestas e institucionales			
s. a.	s.a.	S. A.	«sociedad anónima»
ee. uu.	ee.uu.	EE. UU.	«estados unidos»
dd. hh.	dd.hh.	DD. HH.	«derechos humanos»
jj. oo.	jj.oo.	JJ. OO.	«juegos olímpicos»

Continúa en la siguiente página...

Cuadro 2: (Continuación - Abreviaturas soportadas)

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
ff. aa.	ff.aa.	FF. AA.	«fuerzas armadas»
rr. ee.	rr.ee.	RR. EE.	«relaciones exteriores»
cc. aa.	cc.aa.	CC. AA.	«comunidades autónomas»
p. d.	p.d.	P. D.	«posdata»
a. c.	a.c.	a. C.	«antes de Cristo»
d. c.	d.c.	d. C.	«después de Cristo»
d. n. i.	d.n.i.	D. N. I.	«documento nacional de identidad»
r. u. t.	r.u.t.	R. U. T.	«rol único tributario»
r. u. n.	r.u.n.	R. U. N.	«rol único nacional»

El cuadro 3 detalla las siglas y acrónimos registrados en el módulo Lua `spintent.lua`.

Cuadro 3: Siglas y acrónimos soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Siglas (Deletreo o lectura expandida)			
dni	—	DNI	«documento nacional de identidad»
rut	—	RUT	«rol único tributario»
run	—	RUN	«rol único nacional»
onu	—	ONU	«organización de las naciones unidas»
rae	—	RAE	«real academia española»
ong	ongs	ONG	«organización no gubernamental»
urss	—	URSS	«unión de repúblicas socialistas soviéticas»
oea	—	OEA	«organización de los estados americanos»
oms	—	OMS	«organización mundial de la salud»
fmi	—	FMI	«fondo monetario internacional»
bid	—	BID	«banco interamericano de desarrollo»
otan	—	OTAN	«organización del tratado del atlántico norte»
ue	—	UE	«unión europea»
ru	—	RU	«reino unido»
Acrónimos silábicos			
unicef	—	Unicef	«fondo de las naciones unidas para la infancia»
unesco	—	Unesco	«organización de las naciones unidas para la educación, la ciencia y la cultura»
mercosur	—	Mercosur	«mercado común del sur»
cepal	—	Cepal	«comisión económica para américa latina»
mineduc	—	Mineduc	«ministerio de educación»
conicet	—	Conicet	«consejo nacional de investigaciones»

El cuadro 4 muestra ejemplos de números ordinales registrados en el módulo Lua `spintent.lua`.

Cuadro 4: Números ordinales soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Sufijos para ordinales (Ejemplos)			
1.a	1. ^a	1. ^a	«primera»
2.o	2. ^o	2. ^o	«segundo»
3.er	—	3. ^{er}	«tercer»
4.os	—	4. ^{os}	«cuartos»
5.as	—	5. ^{as}	«quintas»

4 Utilidades generales

Además de los «cuatro grandes» y sus asociados, el paquete **spintent** provee tres utilidades de uso general: `\spsiglo` para la escritura y lectura de siglos, `\sptime` para la escritura y lectura de tiempo y `\spdate` para la escritura y lectura de fechas.

El comando `\spsiglo`

```
\spsiglo \spsiglo{<siglo>}
\spsiglo[<key = val>]{<siglo>}
```

El comando `\spsiglo` trabaja en *modo texto* y en *modo matemático*, recibe el *argumento obligatorio* `{<siglo>}`, el cual puede ser un *número arábigo* ‘21’ o un *número romano* ‘XXI’ mayor que ‘0’ y menor que ‘4000’.

Si no se proporciona el *argumento opcional* `[<key = val>]` y se ejecuta `\spsiglo{21}` o `\spsiglo{XXI}` se imprimirá ‘xxi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

- El *argumento obligatorio* `{<siglo>}` «siempre» es convertido a *número romano en versalitas* y es verbalizado por NVDA/MathCAT como *número arábigo*. Esta es la única utilidad que provee el paquete **spintent** que trabaja en *modo texto* y en *modo matemático*. Internamente utiliza la llave `alt` proveída por el paquete **tagpdf** en *modo texto* y usa `\MathMLintent` para el *modo matemático*.

Llaves para `\spsiglo`

```
print-era = {<ac | dc | none>} default: none
```

Establece si se *imprime* o no la abreviatura después de imprimir el *argumento* `{<siglo>}`. Si se establece en *ac*, `\spsiglo[print-era=ac]{XXI}` imprimirá ‘xi a. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno antes de cristo»; si se establece en *dc*, `\spsiglo[print-era=dc]{XXI}` imprimirá ‘xi d. C.’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno después de cristo»; si se establece en *none*, `\spsiglo[print-era=none]{XXI}` imprimirá ‘xi’ en la salida PDF y NVDA/MathCAT verbalizará «veintiuno».

El comando `\sptime`

```
\sptime \sptime{<horas : minutos>}
\sptime{<horas : minutos : segundos>}
```

El comando `\sptime` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<horas : minutos>}` o `{<horas : minutos : segundos>}`. Por ejemplo: `\sptime{23:45}` imprimirá ‘23:45’ en *modo texto* la salida PDF y NVDA/MathCAT verbalizará «las veintitrés y cuarenta y cinco minutos».

El comando `\spdate`

```
\spdate \spdate{<DD/ MM/YYYY>} \spdate{<DD- MM-YYYY>}
\spdate{<YYYY/ MM/DD>} \spdate{<YYYY- MM-DD>}
```

El comando `\spdate` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<DD/MM/YYYY>}`, `{<DD-MM-YYYY>}` o `{<YYYY-MM-DD>}`. Por ejemplo: `\spdate{1981-01-02}` y `\spdate{02/01/1981}` imprimirán en *modo texto* la salida PDF ‘1981-01-02’ y ‘02/01/1981’ respectivamente y en ambos casos, NVDA/MathCAT verbalizará «dos de enero de mil novecientos ochenta y uno».

- Si se pasa el *argumento* `{<YYYY/MM/DD>}`, internamente se convertirá en `{<DD/MM/YYYY>}` y se imprimirá de esa forma por compatibilidad con NVDA/MathCAT.

5 Utilidades para símbolos

El paquete **spintent** provee tres comandos `\spread`, `\spdsym` y `\spmsym` para trabajar con «símbolos matemáticos» descritos en esta sección. Están pensando desde el punto de vista docente, para situaciones donde la lectura de un «símbolo» (acorde al contexto) es relevante.

- A diferencia de las otras utilidades implementados por **spintent** (que analizan y dan formato a la salida), estos no procesan los «símbolos matemáticos», solo afectan la *forma* (semántica) en que serán verbalizados por NVDA/MathCAT.

El comando `\spread`

```
\spread \spread{<verbalización NVDA>}{<símbolo matemático>}
```

El comando `\spread` trabaja en *modo matemático* y recibe dos *argumentos obligatorios*: El primer *argumento* `{<verbalización NVDA>}` establece la «forma» en la cual NVDA/MathCAT verbalizará al segundo *argumento* `{<símbolo matemático>}`. Por ejemplo: `\spread{es-paralela}{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralela»; `\spread{es-paralelo}{\parallel}` imprimirá ‘||’ en la salida PDF y NVDA/MathCAT verbalizará «es paralelo».

El comando `\spusep`

```
\spusep <spusep{<separador>}
\spusep[<key = val>]{<separador>}
```

El comando `\spusep` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<separador>}`, el cual debe ser uno de los «*delimitadores*» soportados: ‘(’, ‘)’, ‘[’, ‘]’, ‘{’, ‘}’ o ‘\’. Imprime el *delimitador* indicado con el tamaño establecido por la llave `size` y controla si NVDA/MathCAT lo verbaliza o no mediante la llave `silent`. Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spusep{< >}$` imprimirá ‘(’ en la salida PDF y NVDA/MathCAT verbalizará «*paréntesis izquierdo*».

- El motivo de existencia de `\spusep` es que los comandos `\left`, `\right`, `\Uleft` y `\Uright` provistos por LuaTeX crean un *grupo* implícito que dificulta el código para *tagged* PDF cuando se necesita interrumpir algo entre ellos, por ejemplo al construir expresiones complejas con `\spnZ`, `\spmQ` o `\spdiv`. El comando `\spusep` imprime el *delimitador* sin crear ningún grupo.

Llaves para `\spusep`

```
size = {<none | big | Big | bigg | Bigg>} default: none
```

Establece el *tamaño* del `{<delimitador>}` que se imprimirá en la salida PDF. Los valores corresponden a los cuatro tamaños fijos provistos por L^AT_EX: si se establece en `none`, el `{<delimitador>}` se imprime en el tamaño por defecto del modo matemático en curso; si se establece en `big`, se usa `\bigl/\bigr`; si se establece en `Big`, se usa `\Bigl/\Bigr`; si se establece en `bigg`, se usa `\biggl/\biggr`; y si se establece en `Bigg`, se usa `\Biggl/\Biggr`. El tamaño se aplica automáticamente en el lado correcto (izquierdo o derecho) según el `{<delimitador>}` pasado.

```
silent = {<true | false>} default: false
```

Establece si NVDA/MathCAT verbalizará el `{<delimitador>}` impreso por `\spusep`. Si se establece en `true`, el `{<delimitador>}` se marcará con el *intent* `:silent` y NVDA/MathCAT no lo verbalizará; si se establece en `false`, NVDA/MathCAT verbalizará el `{<delimitador>}` de forma normal.

El comando `\spdsym`

```
\spdsym
\spdsym[<key = val>]
```

El comando `\spdsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[<key = val>]` mediante el cual se establece la «*forma*» en la que se imprimirá el *símbolo de división* y como se verbalizará en NVDA/MathCAT. Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spdsym$` imprimirá ‘÷’ en la salida PDF y NVDA/MathCAT verbalizará «*dividido*».

Llaves para `\spdsym`

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «*antes*» de verbalizar el `{<valor>}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

```
suffix = {<sufijo>} default: no usado
```

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «*después*» de verbalizar el `{<valor>}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

```
set-sym = {<old-style | two-dots | slash>} default: two-dots
```

Establece el *símbolo de división* que se imprimirá en la salida PDF. Si se establece en `old-style`, imprimirá ‘÷’; si se establece en `two-dots`, imprimirá ‘:’ y si se establece `slash`, imprimirá ‘/’.

```
read-sym = {<dividido | dividido-por | dividido-entre>} default: dividido
```

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de división*. Si se establece en `dividido`, NVDA/MathCAT verbalizará «*dividido*»; si se establece en `dividido-por`, NVDA/MathCAT verbalizará «*dividido por*» y si se establece en `dividido-entre`, NVDA/MathCAT verbalizará «*dividido entre*».

El comando `\spmsym`

```
\spmsym
\spmsym[<key = val>]
```

El comando `\spmsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[<key = val>]` mediante el cual se establece la «*forma*» en la que se imprimirá el *símbolo de multiplicación* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `$\spmsym$` imprimirá ‘·’ en la salida PDF y NVDA/MathCAT verbalizará «*por*».

Llaves para `\spmsym`

`prefix = {\langle prefijo \rangle}` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el `\{ \langle valor \rangle \}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves `\{ \}`.

`suffix = {\langle sufijo \rangle}` default: no usado

Establece el *sufijo* que será verbalizado por NVDA/MathCAT «después» de verbalizar el `\{ \langle valor \rangle \}` establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves `\{ \}`.

`set-sym = {\langle cross | dot | asterisk \rangle}` default: dot

Establece el *símbolo de multiplicación* que se imprimirá en la salida PDF. Si se establece en `cross`, imprimirá `×`; si se establece en `dot`, imprimirá `·` y si se establece en `asterisk`, imprimirá `*`.

`read-sym = {\langle por | multiplicado-por | veces \rangle}` default: por

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de multiplicación*. Si se establece en `por`, NVDA/MathCAT verbalizará «por»; si se establece en `multiplicado-por`, NVDA/MathCAT verbalizará «multiplicado por» y si se establece en `veces`, NVDA/MathCAT verbalizará «veces».

`not-print = {\langle true | false \rangle}` default: false

Deshabilita la *impresión* del *símbolo de multiplicación*. Cuando se establece esta llave, NVDA/MathCAT verbalizará el *símbolo de multiplicación* acorde al valor establecido por la llave `read-sym`, pero, no se imprimirá este en la salida PDF.

6 Utilidades para números

El paquete `spintent` provee utilidades para trabajar con números descritas en esta sección.

El comando `\spdiv`

`\spdiv` `\spdiv{\langle dividendo \rangle}{\langle divisor \rangle}`
`\spdiv[\langle key = val \rangle]{\langle dividendo \rangle}{\langle divisor \rangle}`

El comando `\spdiv` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* `\{ \langle dividendo \rangle \}` y `\{ \langle divisor \rangle \}`.

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]`, `\$ \spdiv{20}{26} \$` imprimirá `20 : 26` en la salida PDF y NVDA/MathCAT verbalizará «veinte dividido veinte y seis»

Llaves para `\spdiv`

El comando `\spdiv` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 12).

`wrap-parens = {\langle true | false \rangle}` default: false

Establece si se *imprimen* paréntesis redondos `(...)` alrededor de los *argumentos* `\{ \langle dividendo \rangle \}` y `\{ \langle divisor \rangle \}` pasados a `\spdiv`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {\langle true | false \rangle}` default: true

Establece si NVDA/MathCAT verbalizará los paréntesis redondos `(...)` alrededor de los *argumentos* `\{ \langle dividendo \rangle \}` y `\{ \langle divisor \rangle \}` pasados a `\spdiv`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spmul`

`\spmul` `\spmul{\langle factor \rangle}{\langle factor \rangle}`
`\spmul[\langle key = val \rangle]{\langle factor \rangle}{\langle factor \rangle}`

El comando `\spmul` trabaja en *modo matemático* y recibe dos *argumentos obligatorios* `\{ \langle factor \rangle \}` y `\{ \langle factor \rangle \}`.

Si no se proporciona el *argumento opcional* `[\langle key = val \rangle]`, `\$ \spmul{20}{26} \$` imprimirá `20 · 26` en la salida PDF y NVDA/MathCAT verbalizará «veinte por veinte y seis».

Llaves para `\spmul`

El comando `\spmul` posee las *meta-keys* `cifras-sep`, `read-sign`, `read-period-sep` y `notation-sym` pasadas al comando `\spnum` (§3.1.1) junto a las *meta-keys* `set-sym` y `read-sym` pasadas al comando `\spmsym` (pág. 13).

`wrap-parens = {\langle true | false \rangle}` default: false

Establece si se *imprimen* paréntesis redondos `(...)` alrededor de los *argumentos* `\{ \langle factor \rangle \}` y `\{ \langle factor \rangle \}` pasados a `\spmul`. Si se establece en `true`, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {\langle true | false \rangle}` default: true

Establece si NVDA/MathCAT verbalizará los paréntesis redondos ‘(…)’ alrededor de los *argumentos* $\{ \langle \textit{factor} \rangle \}$ y $\{ \langle \textit{factor} \rangle \}$ pasados a `\spmul`. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

El comando `\spnD`

```
\spnD \spnD
\spnD(\<número natural>)
\spnD[key = val](\<número natural>)
```

El comando `\spnD` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* $(\langle \textit{número natural} \rangle)$, por ejemplo: ‘(6)’. Si se usa sin $(\langle \textit{argumento} \rangle)$, `\spnD` imprimirá ‘D(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de *n*».

Si se usa con $(\langle \textit{argumento} \rangle)$, por ejemplo, `\spnD(6)`, imprimirá ‘D(6)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de seis».

Llaves para `\spnD`

`prefix = { \<prefijo> }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar `\spnD(\<argumento>)`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { \<true | false> }` default: false

Establece si se *imprime* el resultado de calcular los *divisores* del $(\langle \textit{número natural} \rangle)$ pasado a `\spnD`. Si se establece en `true`, `\spnD[result=true](6)` imprimirá ‘D(6) = {1, 2, 3, 6}’ en la salida PDF y NVDA/MathCAT verbalizará «los divisores de seis son uno, dos, tres y seis»; si se establece en `false`, no se imprimirá en la salida PDF el resultado de calcular los *divisores*.

`result-num = { \<número natural> }` default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo `5` valores y luego se añadirá ‘...’ en la salida PDF.

`sample-arg = { \<true | false> }` default: false

Desactiva la *validación* interna del $(\langle \textit{argumento} \rangle)$ pasado a `\spnD`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`, `\spnD[sample-arg=true](m)` imprimirá ‘D(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «divisores de *m*»; si se establece en `false`, el $(\langle \textit{argumento} \rangle)$ debe ser un número natural, en caso contrario se devolverá un “error”.

`silent-cmd = { \<true | false> }` default: false

Establece si se *silencia* la verbalización para NVDA/MathCAT de `\spnD(\<argumento>)` cuando se establece `sample-arg=true` o cuando `\spnD` se usa sin $(\langle \textit{argumento} \rangle)$. Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spnM`

```
\spnM \spnM
\spnM(\<número natural>)
\spnM[key = val](\<número natural>)
```

El comando `\spnM` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* $(\langle \textit{número natural} \rangle)$, por ejemplo: ‘(6)’. Si se usa sin $(\langle \textit{argumento} \rangle)$, `\spnM` imprimirá ‘M(*n*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de *n*».

Si se usa con $(\langle \textit{argumento} \rangle)$, por ejemplo, `\spnM(6)`, imprimirá ‘M(6)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de seis».

Llaves para `\spnM`

`prefix = { \<prefijo> }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar `\spnM(\<argumento>)`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = { \<true | false> }` default: false

Establece si se *imprime* el resultado de calcular los *múltiplos* del $(\langle \textit{número natural} \rangle)$ pasado a `\spnM`. Si se establece en `true`, `\spnM[result=true](6)` imprimirá ‘M(6) = {6, 12, 18, 24, 30, ...}’ en la salida PDF y NVDA/MathCAT verbalizará «los múltiplos de seis son seis, doce, dieciocho, veinticuatro, treinta puntos suspensivos»; si se establece en `false`, no se imprimirá en la salida PDF el resultado de calcular los *múltiplos*.

`result-num = { \<número natural> }` default: 5

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF seguidos de ‘...’; si no se establece, se imprimirán por defecto los `5` primeros valores y luego se añadirá ‘...’ en la salida PDF.

`sample-arg = { \<true | false> }` default: false

Desactiva la *validación* interna del (*argumento*) pasado a `\spnM`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`, `\spnM[sample-arg=true](m)` imprimirá ‘M(*m*)’ en la salida PDF y NVDA/MathCAT verbalizará «múltiplos de *m*»; si se establece en `false`, el (*argumento*) debe ser un número natural, en caso contrario se devolverá un “error”.

`silent-cmd = {\true | false}`

default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spnM`(*argumento*) cuando se establece `sample-arg=true` o cuando `\spnM` se usa sin (*argumento*). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false` NVDA/MathCAT lo verbalizará.

El comando `\spmc`

`\spmc` `\spmc`
`\spmc`(*números separados por coma*)
`\spmc`[*key = val*](*números separados por coma*)

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (*números separados por coma*), el cual debe ser una lista de *números naturales*, por ejemplo: ‘(2, 3, 6)’. Si se usa sin (*argumento*) `\spmc` imprimirá ‘mcd’ en la salida PDF, y NVDA/MathCAT verbalizará «máximo común divisor» o «*m c d*», acorde al {*valor*} establecido por la llave `read-cmd`.

Si se usa con (*argumento*) `\spmc(2, 3, 6)` imprimirá ‘mcd(2, 3, 6)’ en la salida PDF, y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis» o «*m c d* entre dos, tres y seis» acorde al {*valor*} establecido por la llave `read-cmd`.

Llaves para `\spmc`

`upper-case = {\true | false}`

default: `false`

Establece si se usarán *mayúsculas* o *minúsculas* en la salida PDF para el comando `\spmc`. Si se establece en `true`, se imprimirá ‘MCD’ en la salida PDF; si se establece en `false`, se imprimirá ‘mcd’ en la salida PDF.

`read-cmd = {\terse | clear}`

default: `clear`

Establece la *forma* en la que NVDA/MathCAT verbalizará el comando `\spmc`. Si se establece en `terse`, NVDA/MathCAT verbalizará «*m c d*»; si se establece en `clear`, NVDA/MathCAT verbalizará «máximo común divisor».

`prefix = {\prefijo}`

default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT «antes» de verbalizar el valor establecido por la llave `read-cmd`. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

`result = {\true | false}`

default: `false`

Establece si se *imprime* el resultado de calcular el «máximo común divisor» del (*argumento*) pasado a `\spmc`. Si se establece en `true`: `\spmc[result=true](2, 3, 6)` imprimirá ‘mcd(2, 3, 6) = 1’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre dos, tres y seis es igual a uno»; si se establece en `false` no se imprimirá en la salida PDF el resultado de calcular el «máximo común divisor».

`sample-arg = {\true | false}`

default: `false`

Desactiva la *validación* interna del (*argumento*) pasado a `\spmc`, permitiendo usar ejemplos «no numéricos» como entrada. Si se establece en `true`: `\spmc[sample-arg=true](m, n)`, imprimirá ‘mcd(*m*, *n*)’ en la salida PDF y NVDA/MathCAT verbalizará «máximo común divisor entre *m* y *n*»; si se establece en `false` el (*argumento*) debe ser una lista de *números naturales*, en caso contrario se devolverá un “error”.

`silent-cmd = {\true | false}`

default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spmc`(*argumento*) cuando se establece `sample-arg=true` o cuando `\spmc` se usa sin (*argumento*). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spmc`

`\spmc` `\spmc`
`\spmc`(*números separados por coma*)
`\spmc`[*key = val*](*números separados por coma*)

El comando `\spmc` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (*números separados por coma*), el cual debe ser una lista de *números naturales*, por ejemplo: ‘(2, 3, 6)’. Si se usa sin (*argumento*) `\spmc` imprimirá ‘mcm’ en la salida PDF, y NVDA/MathCAT verbalizará «mínimo común múltiplo» o «*m c m*», acorde al {*valor*} establecido por la llave `read-cmd`.

Si se usa con (*argumento*) `\spmc(2, 3, 6)` imprimirá ‘mcm(2, 3, 6)’ en la salida PDF, y NVDA/MathCAT verbalizará «mínimo común múltiplo entre dos, tres y seis» o «*m c m* entre dos, tres y seis» acorde al {*valor*} establecido por la llave `read-cmd`.

Llaves para `\spmc`

<code>upper-case = {⟨true false⟩}</code>	default: <i>false</i>
Establece si se usarán <i>mayúsculas</i> o <i>minúsculas</i> en la salida PDF para el comando <code>\spmc</code> . Si se establece en <i>true</i> se imprimirá ‘MCM’ en la salida PDF; si se establece en <i>false</i> se imprimirá ‘mcm’ en la salida PDF.	
<code>read-cmd = {⟨terse clear⟩}</code>	default: <i>clear</i>
Establece la <i>forma</i> en la que NVDA/MathCAT verbalizará el comando <code>\spmc</code> . Si se establece en <i>terse</i> , NVDA/MathCAT verbalizará « <i>m c m</i> »; si se establece en <i>clear</i> , NVDA/MathCAT verbalizará « <i>mínimo común múltiplo</i> ».	
<code>prefix = {⟨prefijo⟩}</code>	default: <i>no usado</i>
Establece el <i>prefijo</i> que será verbalizado por NVDA/MathCAT « <i>antes</i> » de verbalizar el valor establecido por la llave <code>read-cmd</code> . El valor de esta llave <i>siempre</i> debe pasarse entre llaves ‘{ }’.	
<code>result = {⟨true false⟩}</code>	default: <i>false</i>
Establece si se <i>imprime</i> el resultado de calcular el « <i>mínimo común múltiplo</i> » del (⟨ <i>argumento</i> ⟩) pasado a <code>\spmc</code> . Si se establece en <i>true</i> : <code>\spmc[result=true](2, 3, 6)</code> imprimirá ‘mcm(2, 3, 6) = 6’ en la salida PDF y NVDA/MathCAT verbalizará « <i>mínimo común múltiplo entre dos, tres y seis es igual a seis</i> »; si se establece en <i>false</i> no se imprimirá en la salida PDF el resultado de calcular el « <i>mínimo común múltiplo</i> ».	
<code>sample-arg = {⟨true false⟩}</code>	default: <i>false</i>
Desactiva la <i>validación</i> interna del (⟨ <i>argumento</i> ⟩) pasado a <code>\spmc</code> , permitiendo usar ejemplos « <i>no numéricos</i> » como entrada. Si se establece en <i>true</i> : <code>\spmc[sample-arg=true](m, n)</code> , imprimirá ‘mcm(<i>m</i> , <i>n</i>)’ en la salida PDF y NVDA/MathCAT verbalizará « <i>mínimo común múltiplo entre m y n</i> »; si se establece en <i>false</i> el (⟨ <i>argumento</i> ⟩) debe ser una lista de <i>números naturales</i> , en caso contrario se devolverá un “error”.	
<code>silent-cmd = {⟨true false⟩}</code>	default: <i>false</i>
Establece si se <i>silencia</i> la verbalización en NVDA/MathCAT de <code>\spmc(⟨argumento⟩)</code> cuando se establece <code>sample-arg=true</code> o cuando <code>\spmc</code> se usa sin (⟨ <i>argumento</i> ⟩). Si se establece en <i>true</i> NVDA/MathCAT NO lo verbalizará; si se establece en <i>false</i> NVDA/MathCAT SÍ lo verbalizará.	

El comando `\spnZ`

`\spnZ` `\spnZ{⟨número entero⟩}`
`\spnZ[⟨key = val⟩]{⟨número entero⟩}`

El comando `\spnZ` trabaja en *modo matemático* y recibe el *argumento* obligatorio {⟨*número entero*⟩} el cual se procesa internamente bajo el comando `\spnum`. Este comando está dedicado exclusivamente al manejo de «*números enteros*», y añade algunas utilidades prácticas para el demarcado e interacción con NVDA/MathCAT.

Llaves para `\spnZ`

El comando `\spnZ` posee las ⟨*meta-keys*⟩ *cifras-sep* y *read-sign* pasadas al comando `\spnum` (§3.1.1).

<code>wrap-parens = {⟨true false⟩}</code>	default: <i>false</i>
Establece si se <i>imprimen</i> paréntesis redondos ‘()’ alrededor del argumento {⟨ <i>número entero</i> ⟩} pasado a <code>\spnZ</code> . Si se establece en <i>true</i> , se imprimirán paréntesis redondos alrededor del {⟨ <i>número entero</i> ⟩}; si se establece en <i>false</i> , no se imprimirán los paréntesis.	
<code>read-parens = {⟨true false⟩}</code>	default: <i>false</i>
Establece si NVDA/MathCAT <i>verbalizará</i> los paréntesis redondos ‘()’ alrededor del <i>argumento</i> {⟨ <i>número entero</i> ⟩} pasado a <code>\spnZ</code> . Si se establece en <i>true</i> , NVDA/MathCAT verbalizará la lectura del paréntesis ‘()’; si se establece en <i>false</i> , NVDA/MathCAT no los verbalizará.	

El comando `\spmQ`

`\spmQ` `\spmQ{⟨número entero⟩}{⟨numerador⟩}{⟨denominador⟩}`
`\spmQ[⟨key = val⟩]{⟨número entero⟩}{⟨numerador⟩}{⟨denominador⟩}`

El comando `\spmQ` trabaja en *modo matemático* y recibe *tres argumentos obligatorios*: {⟨*número entero*⟩}, {⟨*numerador*⟩} y {⟨*denominador*⟩}, los procesa internamente y devuelve un «*número mixto*».

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩], `\spmQ{2}{5}{7}` imprimirá ‘2⁵/₇’ en la salida PDF y NVDA/MathCAT verbalizará «*dos y cinco séptimos*».

- En el caso de las fracciones, y en especial con los «*números mixtos*», no se pueden manejar muchas cosas desde el lado de *spintent*. Por ejemplo, la entrada `\frac{5}{7}` con *tagged* PDF se verbaliza por NVDA/MathCAT como «*tres más cinco séptimos*», lo cual no está mal, pero pedagógicamente hablando no es del todo correcto.

Llaves para `\spmQ`

`join-word = {⟨entero | enteros | y⟩}`

default: *y*

Establece la *forma* en la que NVDA/MathCAT verbaliza la conexión entre el entero y la fracción que forman el *número mixto*. Si se establece en `entero`, al usar `\spmQ[join-word=entero]{1}{5}{7}` imprimirá $1\frac{5}{7}$ en la salida PDF, y NVDA/MathCAT verbalizará «un entero cinco séptimos»; si se establece en `enteros`, al usar `\spmQ[join-word=enteros]{2}{5}{7}` imprimirá $2\frac{5}{7}$ en la salida PDF, y NVDA/MathCAT verbalizará «dos enteros cinco séptimos»; si se establece en `y`, al usar `\spmQ[join-word=y]{2}{5}{7}` imprimirá $2\frac{5}{7}$ en la salida PDF, y NVDA/MathCAT verbalizará «dos y cinco séptimos».

`use-spnZ = {⟨true | false⟩}`

default: *true*

Establece si el `{⟨numerador⟩}` y el `{⟨denominador⟩}` que forman la fracción del *número mixto* se procesan internamente bajo el comando `\spnZ`, añadiendo así *semántica de entero* en la estructura MathML de la salida PDF. Si se establece en `true`, `{⟨numerador⟩}` y `{⟨denominador⟩}` se pasarán a `\spnZ` y NVDA/MathCAT los verbalizará correctamente como *números ordinales*; si se establece en `false`, los argumentos se tipografían directamente sin procesamiento semántico adicional.

`print-dfrac = {⟨true | false⟩}`

default: *false*

Establece si la fracción del *número mixto* se imprime usando `\dfrac` en lugar de `\frac`. Si se establece en `true`, al usar `\spmQ[print-dfrac=true]{2}{5}{7}` imprimirá la fracción en tamaño *display* $2\frac{5}{7}$ en la salida PDF; si se establece en `false`, la fracción se imprime con `\frac` siguiendo el tamaño matemático del contexto en curso.

`wrap-parens = {⟨true | false⟩}`

default: *false*

Establece si se *imprimen* paréntesis redondos ‘(...)’ alrededor del *número mixto* completo devuelto por `\spmQ`. Si se establece en `true`, se imprimirán paréntesis redondos cuyo tamaño se ajusta automáticamente al estilo matemático en curso; si se establece en `false`, no se imprimirán los paréntesis.

`read-parens = {⟨true | false⟩}`

default: *true*

Establece si NVDA/MathCAT verbalizará los paréntesis redondos ‘(...)’ alrededor del *número mixto* cuando la llave `wrap-parens` está activa. Si se establece en `true`, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en `false`, NVDA/MathCAT no los verbalizará.

7 Utilidades para geometría

El paquete `spintent` provee comandos para trabajar con los objetos y medidas de la *geometría plana* descritos en esta sección. Todos los comandos trabajan en *modo matemático* y comparten el mismo módulo `spintent.lua` para construir la verbalización correcta.

- Todos los comandos de esta sección aceptan como puntos solo *letras mayúsculas* como base, con modificadores opcionales de subíndice ‘`_{\dots}`’ o prima ‘`’`’. La única excepción es `\sprecta` cuando el *argumento* es de la forma `{⟨letra⟩}`, que acepta *letras minúsculas*. Los subíndices siempre deben escribirse de la forma correcta usando `_{\dots}` como `A_{\dots}` y nunca como `A_n` sin llaves que funciona solo por coincidencia.

El comando `\spPoint`

`\spPoint {⟨punto⟩}`

`\spPoint [⟨key = val⟩] {⟨punto⟩}`

El comando `\spPoint` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{⟨punto⟩}` el cual es una *única letra mayúscula* con un modificador opcional de subíndice ‘`_{\dots}`’ o prima ‘`’`’.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `\spPoint{A}` imprimirá ‘A’ en la salida PDF y NVDA/MathCAT verbalizará «punto A»; `\spPoint{A_{1}}` imprimirá ‘A₁’ y verbalizará «punto A sub uno».

- El comando `\spPoint` solo acepta *un punto*. Para pares o listas de puntos (rectas, rayos, lados, ángulos, arcos, polígonos), véanse los comandos correspondientes de esta sección.

Llaves para `\spPoint`

`read-cmd = {⟨school | mathcat⟩}`

default: *school*

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento `{⟨punto⟩}`. Si se establece en `school`, NVDA/MathCAT verbalizará el argumento `{⟨punto⟩}` sin el prefijo «mayúscula», `\spPoint{A}` se verbaliza «punto A»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {⟨prefijo⟩}`

default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «punto». El valor de esta llave *siempre* debe pasarse entre llaves ‘`{ }`’.

`concept = {⟨true | false⟩}`

default: *true*

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «punto» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto; si se establece en `false`, NVDA/MathCAT solo verbalizará el `{⟨argumento⟩}` acorde al valor establecido por la llave `read-cmd`.

`silent-cmd = { <true | false> }` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «punto» y del {<argumento>} pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando \sprecta

`\sprecta`

`\sprecta{<punto, punto>}`
`\sprecta[<key = val>]{<punto, punto>}`

`\sprecta{<letra>}`
`\sprecta[<key = val>]{<letra>}`

El comando `\sprecta` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma {<letra>} o {<punto, punto>}. Si el *argumento* es de la forma {<punto, punto>} se aplica `\overleftrightharrow` en la impresión; si el *argumento* no contiene coma y es de la forma {<letra>} (mayúscula o minúscula), se imprimirá solo la {<letra>} sin aplicar `\overleftrightharrow`.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\sprecta{AB}` imprimirá $\overleftrightarrow{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «recta A B»; `\sprecta{m}` imprimirá m en la salida PDF y NVDA/MathCAT verbalizará «recta m».

Llaves para \sprecta

`read-cmd = { <school | mathcat> }` default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento {<punto, punto>} y el argumento {<letra>} cuando está en mayúscula. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sprecta{AB}` se verbaliza «recta A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = { <prefijo> }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el concepto «recta». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept = { <true | false> }` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «recta» al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto; si se establece en `false`, NVDA/MathCAT solo verbalizará el {<argumento>} acorde al valor establecido por la llave `read-cmd`.

`silent-cmd = { <true | false> }` default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del concepto «recta» y del {<argumento>} pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando \sprayo

`\sprayo`

`\sprayo{<punto, punto>}`
`\sprayo[<key = val>]{<punto, punto>}`

El comando `\sprayo` trabaja en *modo matemático* y recibe el *argumento obligatorio* {<punto, punto>}. Representa un *rayo* o *semirrecta* con origen en el primer punto y dirección hacia el segundo, aplicando `\overrightarrow` en la impresión.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\sprayo{AB}` imprimirá $\overrightarrow{AB}$ en la salida PDF y NVDA/MathCAT verbalizará «rayo A B».

Llaves para \sprayo

`read-cmd = { <school | mathcat> }` default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará el argumento {<punto, punto>}. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sprayo{AB}` se verbaliza «rayo A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty = { <rayo | semirrecta> }` default: rayo

Establece el *concepto* que verbalizará NVDA/MathCAT el ejecutar el comando sin modificar la impresión de este. Si se utiliza `\sprayo[read-sty=rayo]{AB}`, NVDA/MathCAT verbalizará «rayo A B»; si se utiliza `\sprayo[read-sty=semirrecta]{AB}`, NVDA/MathCAT verbalizará «semirrecta A B».

`prefix = { <prefijo> }` default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept = { <true | false> }` default: true

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el

concepto; si se establece en `false`, NVDA/MathCAT solo verbalizará el $\{\langle \text{argumento} \rangle\}$ acorde al valor establecido por la llave `read-cmd`.

`silent-cmd` = $\{\langle \text{true} \mid \text{false} \rangle\}$ default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` y del $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

- La impresión ' $\overline{AB}$ ' en la salida PDF es idéntica en ambos casos. La elección depende de la terminología del libro de texto o del currículo nacional: en Chile y España se prefiere «semirrecta»; en México y otros países de América Latina se usa «rayo».

El comando `\spside`

`\spside` $\{\langle \text{punto}, \text{punto} \rangle\}$
 $\{\langle \text{key} = \text{val} \rangle\} \{\langle \text{punto}, \text{punto} \rangle\}$

El comando `\spside` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle \text{punto}, \text{punto} \rangle\}$. Representa un *lado*, *segmento* o *trazo* determinado por dos puntos, aplicando `\overline{\hspace{1cm}}` en la impresión.

Si no se proporciona el *argumento opcional* $[\langle \text{key} = \text{val} \rangle]$, $\$ \backslash \text{spside} \{AB\} \$$ imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «lado A B».

Llaves para `\spside`

`read-cmd` = $\{\langle \text{school} \mid \text{mathcat} \rangle\}$ default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento $\{\langle \text{punto}, \text{punto} \rangle\}$. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: $\$ \backslash \text{spside} \{AB\} \$$ se verbaliza «lado A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty` = $\{\langle \text{lado} \mid \text{segmento} \mid \text{trazo} \rangle\}$ default: `lado`

Establece el *concepto* que verbalizará NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. Si se utiliza $\$ \backslash \text{spside} [\text{read-sty}=\text{lado}] \{AB\} \$$, NVDA/MathCAT verbalizará «lado A B»; si se utiliza $\$ \backslash \text{spside} [\text{read-sty}=\text{segmento}] \{AB\} \$$, NVDA/MathCAT verbalizará «segmento A B» y si se utiliza $\$ \backslash \text{spside} [\text{read-sty}=\text{trazo}] \{AB\} \$$, NVDA/MathCAT verbalizará «trazo A B».

`prefix` = $\{\langle \text{prefijo} \rangle\}$ default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el «concepto» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves ' $\{ \}$ '.

`concept` = $\{\langle \text{true} \mid \text{false} \rangle\}$ default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el concepto; si se establece en `false`, NVDA/MathCAT solo verbalizará el $\{\langle \text{argumento} \rangle\}$ acorde al valor establecido por la llave `read-cmd`.

`read-arg` = $\{\langle \text{verbalización NVDA} \rangle\}$ default: `no usado`

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ' $\{ \}$ '. Por ejemplo: $\$ \backslash \text{spside} [\text{read-arg}=\{\text{hipotenusa}\}] \{AB\} \$$ imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «hipotenusa A B».

`silent-cmd` = $\{\langle \text{true} \mid \text{false} \rangle\}$ default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del «concepto» establecido por la llave `read-sty` y del $\{\langle \text{argumento} \rangle\}$ pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spmside`

`\spmside` $\{\langle \text{punto}, \text{punto} \rangle\}$
 $\{\langle \text{key} = \text{val} \rangle\} \{\langle \text{punto}, \text{punto} \rangle\}$

El comando `\spmside` trabaja en *modo matemático* y recibe el *argumento obligatorio* $\{\langle \text{punto}, \text{punto} \rangle\}$. Representa la *medida* del *lado*, *segmento* o *trazo* determinado por dos puntos. La distinción entre `\spside` y `\spmside` es semántica: `\spside` representa el *objeto geométrico* ' $\overline{AB}$ '; `\spmside` representa su *medida* (un número real positivo).

Si no se proporciona el *argumento opcional* $[\langle \text{key} = \text{val} \rangle]$, $\$ \backslash \text{spmside} \{AB\} \$$ imprimirá ' AB ' en la salida PDF y NVDA/MathCAT verbalizará «medida del lado A B».

Llaves para `\spmside`

`read-cmd` = { `<punto | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmside{AB}` se verbaliza «*medida del lado A B*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`read-sty` = { `<lado | segmento | trazo>` } default: `lado`

Establece el *concepto* que verbalizará NVDA/MathCAT después de verbalizar el término «*medida del*» al ejecutar el comando sin modificar la impresión de este. Si se utiliza `\spmside[read-sty=lado]{AB}`, NVDA/MathCAT verbalizará «*medida del lado A B*»; si se utiliza `\spmside[read-sty=segmento]{AB}`, NVDA/MathCAT verbalizará «*medida del segmento A B*» y si se utiliza `\spmside[read-sty=trazo]{AB}`, NVDA/MathCAT verbalizará «*medida del trazo A B*».

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «*medida del*» seguido por el «*concepto*» establecido por la llave `read-sty`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`concept` = { `<true | false>` } default: `true`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*medida del*» y del «*concepto*» establecido por la llave `read-sty` al ejecutar el comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT verbalizará el término «*medida del*» antes del concepto; si se establece en `false`, NVDA/MathCAT solo verbalizará el { `<argumento>` } acorde al valor establecido por la llave `read-cmd`.

`read-arg` = { `<verbalización NVDA>` } default: `no usado`

Sobrescribe el *concepto* establecido por la llave `read-sty` que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }' y se debe usar en conjunto con la llave `concept`.

Por ejemplo: `\spmside[concept=false,read-arg={medida de la cuerda}]{AB}` imprimirá ' $\overline{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*medida de la cuerda A B*».

`print-m` = { `<true | false>` } default: `false`

Establece si se *imprime* la notación visual 'm' antes aplicar `\overline` al { `<argumento>` }. Si se establece en `true`, `\spmside[print-m=true]{AB}` imprimirá ' $m\overline{AB}$ ' en la salida PDF, si se establece en `false` imprimirá ' $\overline{AB}$ ' en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «*medida del lado A B*».

`silent-cmd` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*medida del*», del «*concepto*» establecido por la llave `read-sty` y del { `<argumento>` } pasado al comando sin modificar la impresión de este. Si se establece en `true`, NVDA/MathCAT NO verbalizará la salida del comando; si se establece en `false`, NVDA/MathCAT sí verbalizará la salida del comando.

El comando `\spangle`

<u><code>\spangle</code></u>	<code>\spangle{<punto letra griega número>}</code>	<code>\spangle{<punto, punto, punto>}</code>
	<code>\spangle[<key = val>]{<punto letra griega número>}</code>	<code>\spangle[<key = val>]{<punto, punto, punto>}</code>

El comando `\spangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma { `<punto | letra griega | número>` } o { `<punto, punto, punto>` } aplicando `\angle` o `\rightangle` en la impresión.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spangle{A}` imprimirá ' $\angle A$ ' en la salida PDF y NVDA/MathCAT verbalizará «*ángulo en A*»; `\spangle{\alpha}` imprimirá ' $\angle \alpha$ ' en la salida PDF y NVDA/MathCAT verbalizará «*ángulo alfa*» y `\spangle{A, 0, B}` imprimirá ' $\angle AOB$ ' en la salida PDF y NVDA/MathCAT verbalizará «*ángulo A O B*».

Llaves para `\spangle`

`read-cmd` = { `<school | mathcat>` } default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento { `<punto>` } o { `<punto, punto, punto>` }. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spangle{A, 0, B}` se verbaliza «*ángulo A O B*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = { `<prefijo>` } default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «*ángulo*». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`silent-cmd` = { `<true | false>` } default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «*ángulo*». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

recto = {true | false}

default: false

Establece si se usará `\rightangle` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en `true`, `\spangle[recto=true]{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto en A»; `\spangle[recto=true]{A, O, B}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «ángulo recto A O B»; si se establece en `false`, imprimirá usando `\angle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

El comando `\spmangle`

`\spmangle`
`\spmangle{<punto | letra griega | número>}`
`\spmangle[<key = val>]{<punto | letra griega | número>}`

`\spmangle{<punto, punto, punto>}`
`\spmangle[<key = val>]{<punto, punto, punto>}`

El comando `\spmangle` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `{<punto | letra griega | número>}` o `{<punto, punto, punto>}` aplicando `\measuredangle` o `\measuredrightangle` en la impresión. La distinción entre `\spangle` y `\spmangle` es semántica: `\spangle` representa el *objeto geométrico* ‘ $\angle AOB$ ’; `\spmangle` representa su *medida* (un valor en grados).

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\spmangle{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo en A»; `\spmangle{\alpha}` imprimirá ‘ $\angle \alpha$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo alfa» y `\spmangle{A, O, B}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo A O B».

Llaves para `\spmangle`

read-cmd = {<school | mathcat>}

default: school

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento `{<punto>}` o `{<punto, punto, punto>}`. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmangle{A, O, B}` se verbaliza «medida del ángulo A O B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

prefix = {<prefijo>}

default: no usado

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del ángulo». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

silent-cmd = {<true | false>}

default: false

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del ángulo». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

recto = {<true | false>}

default: false

Establece si se usará `\measuredrightangle` en la impresión añadiendo el término «recto» a la verbalización en NVDA/MathCAT. Si se establece en `true`, `\spmangle[recto=true]{A}` imprimirá ‘ $\angle A$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto en A»; `\spmangle[recto=true]{A, O, B}` imprimirá ‘ $\angle AOB$ ’ en la salida PDF y NVDA/MathCAT verbalizará «medida del ángulo recto A O B»; si se establece en `false`, imprimirá usando `\measuredangle` en la salida PDF y NVDA/MathCAT omitirá el término «recto».

print-m = {<true | false>}

default: false

Establece *si se imprime* la notación visual ‘m’ antes del argumento. Si se establece en `true`, `\spmangle[print-m=true]{A}`, imprimirá ‘ $m\angle AOB$ ’ en la salida PDF, si se establece en `false` imprimirá ‘ $\angle AOB$ ’ en la salida PDF. En ambos casos NVDA/MathCAT verbalizará «medida del ángulo A O B».

El comando `\spang`

`\spang`
`\spang{<grados>}`
`\spang{<grados : minutos>}`
`\spang{<grados : minutos : segundos>}`

El comando `\spang` trabaja en *modo matemático* y recibe el *argumento obligatorio* `{<grados>}`, `{<grados : minutos>}` o `{<grados : minutos : segundos>}`. Los procesa internamente bajo los comandos `\spnum` y `\spunit` para imprimir y verbalizar el «ángulo sexagesimal». Por ejemplo, `\spang{45:30:15}` imprimirá ‘45°30’15” en la salida PDF y NVDA/MathCAT verbalizará «cuarenta y cinco grados treinta minutos quince segundos».

El comando `\sparc`

`\sparc`
`\sparc{<punto, punto>}`
`\sparc[<key = val>]{<punto, punto>}`

`\sparc{<punto, punto, punto>}`
`\sparc[<key = val>]{<punto, punto, punto>}`

El comando `\sparc` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma `{<punto, punto>}` o `{<punto, punto, punto>}`. Representa un arco determinado por dos o tres puntos aplicando `\widearc` en la impresión.

Si no se proporciona el *argumento opcional* `[<key = val>]`, `\sparc{A, B}` imprimirá ‘ $\widehat{AB}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «arco A B».

Llaves para `\sparc`

`read-cmd` = {`<school | mathcat>`} default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento {`<punto, punto>`} o {`<punto, punto, punto>`}. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\sparc{A, B}` se verbaliza «arco A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = {`<prefijo>`} default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «arco». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`silent-cmd` = {`<true | false>`} default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «arco». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\sparc`

<code>\sparc</code>	<code>\sparc{<punto, punto>}</code>	<code>\sparc{<punto, punto, punto>}</code>
	<code>\sparc[<key = val>]{<punto, punto>}</code>	<code>\sparc[<key = val>]{<punto, punto, punto>}</code>

El comando `\sparc` trabaja en *modo matemático* y recibe un *argumento obligatorio* el cual puede ser de la forma {`<punto, punto>`} o {`<punto, punto, punto>`}. Representa la *medida* del arco determinado por dos o tres puntos. La distinción entre `\sparc` y `\spmarc` es semántica: `\sparc` representa el *objeto geométrico* $\widehat{AB}$; `\spmarc` representa su *medida* (un valor en grados).

Si no se proporciona el *argumento opcional* [`<key = val>`], `\sparc{A, B}` imprimirá ' $\widehat{AB}$ ' en la salida PDF y NVDA/MathCAT verbalizará «medida del arco A B».

Llaves para `\spmarc`

`read-cmd` = {`<school | mathcat>`} default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento {`<punto, punto>`} o {`<punto, punto, punto>`}. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spmarc{A, B}` se verbaliza «medida del arco A B»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = {`<prefijo>`} default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «medida del arco». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`silent-cmd` = {`<true | false>`} default: `false`

Establece si se *silencia* la verbalización en NVDA/MathCAT del término «medida del arco». Si se establece en `true`, NVDA/MathCAT NO verbalizará el término; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spPoly`

<code>\spPoly</code>	<code>\spPoly{<punto, punto, punto, ...>}</code>
	<code>\spPoly[<key = val>]{<punto, punto, punto, ...>}</code>

El comando `\spPoly` trabaja en *modo matemático* y recibe el *argumento obligatorio* {`<punto, punto, punto, ...>`}, una lista de al menos tres {`<puntos>`} separados por comas. El tipo de *símbolo* en la impresión y la verbalización se determinan acorde al número de puntos en el argumento: Si son tres puntos se aplica `\triangle` y añade el término «triángulo» a la verbalización; si son cuatro puntos se aplica `\mdlgwhtsquare` y añade el término «cuadrado» a la verbalización; si son cinco o más puntos no se aplica ningún símbolo ni se añade verbalización.

Si no se proporciona el *argumento opcional* [`<key = val>`], `\spPoly{A, B, C}` imprimirá ' $\triangle ABC$ ' en la salida PDF y NVDA/MathCAT verbalizará «triángulo A B C»; `\spPoly{A, B, C, D}` imprimirá ' $\square ABCD$ ' y verbalizará «cuadrado A B C D»; `\spPoly{A, B, C, D, E}` imprimirá ' $ABCDE$ ' y verbalizará «A B C D E».

Llaves para `\spPoly`

`read-cmd` = {`<school | mathcat>`} default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento {`<punto, punto, punto, ...>`}. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPoly{A, B, C}` se verbaliza «triángulo A B C»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix` = {`<prefijo>`} default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el argumento o los términos «triángulo» y «cuadrado» establecido por la llave `print-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`print-sym` = {`<true | false>`} default: `true`

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión añadiendo el término correspondiente a la verbalización en NVDA/MathCAT. Si se establece en `true`, `\spPoly[print-sym=true]{A, B, C}` imprimirá ' $\triangle ABC$ ' en la salida PDF y NVDA/MathCAT verbalizará «*triángulo A B C*»; si se establece en `false`, imprimirá ' ABC ' en la salida PDF y NVDA/MathCAT omitirá el término «*triángulo*», verbalizando solo «*A B C*».

`silent-cmd = {\true | false}`

default: `false`

Alias de `print-sym=false`, incluido por consistencia con el resto de los comandos de geometría plana.

El comando `\spAfig`

`\spAfig` `\spAfig`
`\spAfig((punto, punto, punto, ...))`
`\spAfig(número | letra | letra_{número})`
`\spAfig[key = val]((punto, punto, punto, ...))`

El comando `\spAfig` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis*, el cual es *completamente opcional*, representando el área de una figura poligonal en la impresión aplicando `\mathcal{A}`.

El argumento admite tres formas: una *lista de vértices* (`(punto, punto, punto, ...)`) (al menos tres); un *número o letra sueltos* (`(1)`), (`(n)`); o una *letra de una tabla fija* (F, P, B, L) con subíndice numérico opcional, (`(F)`), (`(F_{1})`).

Si no se proporciona el *argumento opcional* [`key = val`] ni el argumento delimitado por paréntesis, `\spAfig` imprimirá ' $\mathcal{A}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área*». Con vértices, `\spAfig(A, B, C)` imprimirá ' $\mathcal{A}_{ABC}$ ' y verbalizará «*área del triángulo A B C*». Con un número o letra sueltos, `\spAfig(1)` imprimirá ' $\mathcal{A}_1$ ' y verbalizará «*área uno*»; `\spAfig(n)` imprimirá ' $\mathcal{A}_n$ ' y verbalizará «*área n*». Con una letra de la tabla, `\spAfig(F)` imprimirá ' $\mathcal{A}_F$ ' y verbalizará «*área de la figura*»; `\spAfig(F_{1})` imprimirá ' $\mathcal{A}_{F_1}$ ' y verbalizará «*área de la figura uno*».

Llaves para `\spAfig`

`read-cmd = {school | mathcat}`

default: `school`

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spAfig(A, B, C)` se verbaliza «*área del triángulo A B C*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

`prefix = {prefijo}`

default: `no usado`

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «*área*». El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`print-sym = {auto | triangle | square | none}`

default: `auto`

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión. Con vértices, `auto` infiere el símbolo según el número de puntos (tres o cuatro); `none` lo omite y NVDA/MathCAT verbaliza solo los vértices sin nombrar la figura: `\spAfig[print-sym=none](A, B, C)` imprimirá ' $\mathcal{A}_{ABC}$ ' y verbalizará «*área A B C*». Forzar `triangle/square` exige que coincida con el número real de vértices; en caso contrario se produce un error. Con un número o letra sueltos, `auto` y `none` se comportan igual (sin símbolo); `triangle/square` anida el argumento bajo el símbolo: `\spAfig[print-sym=triangle](1)` imprimirá ' $\mathcal{A}_{\triangle_1}$ ' y verbalizará «*área del triángulo uno*». Esta llave no puede combinarse con una letra de la tabla (F, P, B, L).

`read-sub = {true | false}`

default: `false`

Establece si NVDA/MathCAT antepone la palabra «sub» al verbalizar el número o letra del argumento único. Si se establece en `true`, `\spAfig[read-sub=true](1)` verbaliza «*área sub uno*»; si se establece en `false`, verbaliza «*área uno*». No tiene efecto en la notación de vértices ni cuando el argumento es una letra de la tabla sin subíndice.

`silent-cmd = {true | false}`

default: `false`

Establece si se *silencia por completo* la verbalización en NVDA/MathCAT, sin afectar la impresión. Si se establece en `true`, NVDA/MathCAT NO verbalizará nada; si se establece en `false`, NVDA/MathCAT verbalizará normalmente.

`read-arg = {nombre}`

default: `no usado`

Sobrescribe el *nombre de la figura* determinado automáticamente por el número de puntos en el argumento, que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'. Por ejemplo: `\spAfig[read-arg={trapezio}](A, B, C, D)` imprimirá ' $\square_{ABCD}$ ' en la salida PDF y NVDA/MathCAT verbalizará «*área del trapecio A B C D*».

El comando `\spPfig`

```
\spPfig \spPfig
\spPfig(<punto, punto, punto, ...>)
\spPfig(<número | letra | letra_{número}>)
\spPfig[<key = val>](<punto, punto, punto, ...>)
```

El comando `\spPfig` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis*, el cual es *completamente opcional*, representando el *perímetro* de una figura poligonal en la impresión aplicando `\mathcal{P}`.

El argumento admite las mismas tres formas que `\spAfig` (§7): una *lista de vértices*, un *número o letra sueltos*, o una *letra de la tabla fija* (F, P, B, L) con subíndice numérico opcional.

Si no se proporciona el *argumento opcional* [`<key = val>`] ni el argumento delimitado por paréntesis, `\spPfig` imprimirá ‘ $\mathcal{P}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro*». Con vértices, `\spPfig(A, B, C)` imprimirá ‘ $\mathcal{P}_{\triangle ABC}$ ’ y verbalizará «*perímetro del triángulo A B C*». Con un número o letra sueltos, `\spPfig(2)` imprimirá ‘ $\mathcal{P}_2$ ’ y verbalizará «*perímetro dos*». Con una letra de la tabla, `\spPfig(P_{2})` imprimirá ‘ $\mathcal{P}_{P_2}$ ’ y verbalizará «*perímetro del polígono dos*».

Llaves para `\spPfig`

```
read-cmd = {<school | mathcat>} default: school
```

Establece la *forma* en la que NVDA/MathCAT verbalizará al argumento. Si se establece en `school`, NVDA/MathCAT verbaliza las letras del argumento sin el prefijo «mayúscula»: `\spPfig(A, B, C)` se verbaliza «*perímetro del triángulo A B C*»; si se establece en `mathcat`, NVDA/MathCAT aplica sus propias reglas, que pueden incluir el prefijo «mayúscula» dependiendo de la configuración del usuario.

```
prefix = {<prefijo>} default: no usado
```

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *antes* de verbalizar el término «*perímetro*». El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’.

```
print-sym = {<auto | triangle | square | none>} default: auto
```

Establece si se usará `\triangle` o `\mdlgwhtsquare` en la impresión, con la misma semántica que en `\spAfig` (§7).

```
read-sub = {<true | false>} default: false
```

Establece si NVDA/MathCAT antepone la palabra «sub» al verbalizar el número o letra del argumento único, con la misma semántica que en `\spAfig` (§7).

```
silent-cmd = {<true | false>} default: false
```

Establece si se *silencia por completo* la verbalización en NVDA/MathCAT, sin afectar la impresión. Si se establece en `true`, NVDA/MathCAT NO verbalizará nada; si se establece en `false`, NVDA/MathCAT verbalizará normalmente.

```
read-arg = {<nombre>} default: no usado
```

Sobrescribe el *nombre de la figura* determinado automáticamente por el número de puntos en el argumento, que será verbalizado por NVDA/MathCAT al ejecutar el comando sin modificar la impresión de este. El valor de esta llave *siempre* debe pasarse entre llaves ‘{ }’. Por ejemplo: `\spPfig[read-arg={trapecio}](A, B, C, D)` imprimirá ‘ $\mathcal{P}_{\square ABCD}$ ’ en la salida PDF y NVDA/MathCAT verbalizará «*perímetro del trapecio A B C D*».

8 Referencias

- [1] ROBERTSON, WILL . “unicode-math - Unicode mathematics support for X_YTeX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/unicode-math>, 2023.
- [2] KRUEGER, MARCEL. “LuaMML - Automatically generate MathML from LuaTeX math mode material”. Available from CTAN, <https://ctan.org/pkg/luamml>, 2026.
- [3] The L^AT_EX Project. “fontspec - Advanced font selection for X_YL^AT_EX and LuaTeX”. Available from CTAN, <https://www.ctan.org/pkg/fontspec>, 2025.
- [4] VOß, HERBERT. “libertinus-otf - Support for Libertinus OpenType”. Available from CTAN, <https://www.ctan.org/pkg/libertinus-otf>, 2025.
- [5] FLIPO, DANIEL. “erewhon-math - Utopia based OpenType Math font”. Available from CTAN, <https://ctan.org/pkg/erewhon-math>, 2026.
- [6] HOFSTRA, SILKE. “sourcecodepro - Use SourceCodePro with T_EX(-alike) systems”. Available from CTAN, <https://ctan.org/pkg/sourcecodepro>, 2025.
- [7] GONZÁLEZ, PABLO. “scontents - Stores L^AT_EX contents in memory or files”. Available from CTAN, <https://www.ctan.org/pkg/scontents>, 2026.
- [8] GONZÁLEZ, PABLO. “enumext - Enumerate exercise sheets”. Available from CTAN, <https://www.ctan.org/pkg/enumext>, 2026.
- [9] BRAAMS, JOHANNES AND BEZOS, JAVIER. “babel - Multilingual support for L^AT_EX, LuaL^AT_EX, X_YL^AT_EX, and Plain T_EX”. Available from CTAN, <https://www.ctan.org/pkg/babel>, 2026.
- [10] BEZOS, JAVIER. “babel-spanish - Babel support for Spanish”. Available from CTAN, <https://www.ctan.org/pkg/babel-spanish>, 2026.
- [11] NV ACCESS. “NVDA - NonVisual Desktop Access screen reader”. Available from NV ACCESS, <https://www.nvaccess.org/>, 2026.
- [12] SOIFFER, NEIL. “MathCAT - Math to speech and braille translation”. Available from GITHUB, <https://nsoiffer.github.io/MathCAT/>, 2026.
- [13] ADOBE INC. “Adobe Acrobat Reader - Reliable PDF viewer”. Available from ADOBE, <https://get.adobe.com/reader/>, 2026.
- [14] ISO. “ISO 32000-2:2020 - Portable Document Format (PDF 2.0)”. Available from PDF ASSOCIATION, <https://pdfa.org/resource/iso-32000-2/>, 2020.
- [15] PDF ASSOCIATION. “WTPDF - Well-Tagged PDF specification”. Available from PDFA, <https://pdfa.org/wtpdf/>, 2024.
- [16] PDF ASSOCIATION. “PDF 2.0 Errata Collection 3”. Available from PDFA, <https://pdfa.org/PDF-2-0-errata-collection-3-now-available/>, 2026.
- [17] L^AT_EX PROJECT LWG. “Best Practice Guide: Math in PDF”. Available from PDFA, <https://pdfa.org/resource/best-practice-guide-math-in-PDF/>, 2026.
- [18] JOHNSON, DUFF. “PDF 2.0: New Features, Real-World Impact”. Available from PDFA, <https://pdfa.org/PDF-2-0-new-features-real-world-impact/>, 2026.
- [19] The L^AT_EX Project. “LaTeX Tagged PDF Project - Documentation and resources”. Available from L^AT_EX PROJECT, <https://latex3.github.io/tagging-project/>, 2026.
- [20] VERAPDF CONSORTIUM. “veraPDF - Industry supported PDF/A and PDF/UA validator”. Available from VERAPDF, <https://verapdf.org/>, 2026.
- [21] DUAL LAB. “ngPDF - Next Generation PDF demonstrator and validator”. Available from NGPDF, <https://ngpdf.com/>, 2026.
- [22] BERRY, KARL. “L^AT_EX 2_ε: An Unofficial Reference Manual”. Available from CTAN, <https://ctan.org/pkg/latex2e-help-texinfo>, 2026.
- [23] The L^AT_EX Project. “Extensive support for hypertext in L^AT_EX”. Available from CTAN, <https://www.ctan.org/pkg/hyperref>, 2026.
- [24] The L^AT_EX Project. “The expl3 package”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.

- [25] The $\text{\TeX}$ Project. “The $\text{\TeX}$ ₃ Interfaces”. Available from CTAN, <https://www.ctan.org/pkg/l3kernel>, 2026.
- [26] The $\text{\TeX}$ Project. “The $\text{\TeX}$ _{2 ϵ} sources”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2026.
- [27] The $\text{\TeX}$ Project. “ $\text{\TeX}$ News, Issue 41, June 2025”. Available from CTAN, <https://ctan.org/tex-archive/macros/latex/base>, 2025.
- [28] The $\text{\TeX}$ Project. “ $\text{\TeX}$ for authors current version”. Available from CTAN, <https://ctan.org/pkg/latex-base>, 2026.
- [29] FISCHER, ULRIKE. “tagpdf – $\text{\TeX}$ kernel code for PDF tagging”. Available from CTAN, <https://www.ctan.org/pkg/tagpdf>, 2026.
- [30] The $\text{\TeX}$ Project. “latex-lab – $\text{\TeX}$ laboratory”. Available from CTAN, <https://www.ctan.org/pkg/latex-lab>, 2026.

9 Change history

v0.99 (ctan), 2026-09-15 – First public release.

10 Índice de la Documentación

Los números en cursiva indican las páginas donde se describe la entrada correspondiente.

C

Document class:

article

1

book

1

letter

1

report

1

Commands provide by **spintent**:

\setspintent

3

\spAfig

23

\spPfig

24

\spPoint

17

\spPoly

22

\spangle

20

\spang

21

\sparc

21

\spdate

11

\spdiv

13

\spdsym

12

\spmQ

16

\spmangle

21

\spmarc

22

\spmcd

15

\spmcm

15

\spmoney

6

\spmside

19

\spmsym

12

\spmul

13

\spnD

14

\spnM

14

\spnZ

16

\spnewunit

6

\spnum

3

\sprayo

18

\spread

11

\sprecta

18

\spshort

7

\spside

19

\spsiglo

11

\sptime

11

\spunitalias

6

\spunit

4

\spusep

12

K

Keys for \spAfig provide by **spintent**:

prefix

23

print-sym

23

read-arg

23

read-cmd

23

read-sub

23

silent-cmd

23

Keys for \spangle provide by **spintent**:

prefix

20

read-cmd

20

recto

21

silent-cmd

20

Keys for \sparc provide by **spintent**:

prefix

22

read-cmd

22

silent-cmd

22

Keys for \spdiv provide by **spintent**:

read-parens

13

wrap-parens

13

Keys for \spdsym provide by **spintent**:

prefix

12

read-sym

12

set-sym

12

suffix

12

Keys for \spmangle provide by **spintent**:

prefix

21

print-m

21

read-cmd

21

recto

21

silent-cmd

21

Keys for \spmarc provide by **spintent**:

prefix

22

read-cmd

22

silent-cmd

22

Keys for \spmcd provide by **spintent**:

prefix

15

read-cmd

15

result

15

sample-arg

15

silent-cmd

15

upper-case

15

Keys for \spmcm provide by **spintent**:

prefix

16

read-cmd

16

result

16

sample-arg

16

silent-cmd

16

upper-case

16

Keys for \spmoney provide by **spintent**:

currency

7

dolar-math

7

sub-units

7

Keys for \spmQ provide by **spintent**:

join-word

17

print-dfrac

17

read-parens

17

use-spnZ

17

wrap-parens

17

Keys for \spmside provide by **spintent**:

concept

20

prefix

20

print-m

20

read-arg

20

read-cmd

20

read-sty

20

silent-cmd

20

Keys for \spmsym provide by **spintent**:

not-print

13

prefix

13

read-sym

13

set-sym

13

suffix

13

Keys for \spmul provide by **spintent**:

read-parens

13

wrap-parens

13

Keys for \spnD provide by **spintent**:

prefix

14

©2026 by Pablo González L

28 / 152

result-num	14	size	12
result	14	Keys for \spdiv provide by spintent :	
sample-arg	14	cifras-sep	13
silent-cmd	14	notation-sym	13
Keys for \spnM provide by spintent :		read-period-sep	13
prefix	14	read-sign	13
result-num	14	read-sym	13
result	14	set-sym	13
sample-arg	14	Keys for \spdsym provide by spintent :	
silent-cmd	15	read-sym	12
Keys for \spnum provide by spintent :		Keys for \spmcd provide by spintent :	
cifras-sep	3	read-cmd	15
notation-sym	4	Keys for \spmcm provide by spintent :	
read-period-sep	3	read-cmd	16
read-sign	3	Keys for \spmoney provide by spintent :	
Keys for \spnZ provide by spintent :		cifras-sep	7
read-parens	16	Keys for \spmQ provide by spintent :	
wrap-parens	16	wrap-parens	17
Keys for \spPfig provide by spintent :		Keys for \spmsym provide by spintent :	
prefix	24	read-sym	13
print-sym	24	Keys for \spm\ provide by spintent :	
read-arg	24	cifras-sep	13
read-cmd	24	notation-sym	13
read-sub	24	read-period-sep	13
silent-cmd	24	read-sign	13
Keys for \spPoint provide by spintent :		read-sym	13
concept	17	set-sym	13
prefix	17	Keys for \spnZ provide by spintent :	
read-cmd	17	cifras-sep	16
silent-cmd	18	read-sign	16
Keys for \spPoly provide by spintent :		Keys for \spPoly provide by spintent :	
prefix	22	print-sym	23
print-sym	22	Keys for \spside provide by spintent :	
read-cmd	22	read-sty	19, 20
silent-cmd	23	Keys for \spunit provide by spintent :	
Keys for \sprayo provide by spintent :		cifras-sep	4
concept	18	notation-sym	4
prefix	18	read-period-sep	4
read-cmd	18	Keys for \spusep provide by spintent :	
read-sty	18	silent	12
silent-cmd	19	size	12
Keys for \sprecta provide by spintent :			
concept	18		
prefix	18		
read-cmd	18		
silent-cmd	18		
Keys for \spshort provide by spintent :			
read-arg	9		
small-caps	9		
Keys for \spside provide by spintent :			
concept	19		
prefix	19		
read-arg	19		
read-cmd	19		
read-sty	19		
silent-cmd	19		
Keys for \spsiglo provide by spintent :			
print-era	11		
Keys for \spunit provide by spintent :			
print-cdot	4		
read-cdot	4		
read-slash	4		
Keys for \spusep provide by spintent :			
silent	12		

M

Lua module provide by spintent :	
spintent.lua	6, 7, 9, 10, 17

P

Packages :	
babel-spanish	2
babel	2, 7
documentmetadata-support	1
enumext	1
expl3	3
fontspec	2
fourier-otf	2
hyperref	2
l3keys	3
latex-lab-mathintent	1
libertinus-otf	2
luamml	1
scontents	1
sourcecodepro	2
tagpdf	1, 7, 11
unicode-math	1

11 Implementation

The most recent publicly released version of `spintent` is available at CTAN: <https://www.ctan.org/pkg/spintent>. While general feedback via email is welcomed, specific bugs or feature requests should be reported through the issue tracker: <https://github.com/pablgonz/spintent/issues>.

- The documentation presented here is far from professional, it contains a lot of obvious information that to the eye of a TeXpert are superfluous, but, after so many years developing this project is the only way to remember what does what.

11.1 General conventions

Functions of the form `\__spintent_luafun_some:n` or `\__spintent_luafun_some:nn` are defined (created) in the Lua module `spintent.lua` and only their variants are declared in `expl3`.

Similarly, variables of the form `\l__spintent_luaset_some_tl` or `\l__spintent_luaset_some_str` are defined in the Lua module `spintent.lua` and only declared in `expl3`.

- All variables and functions defined in this package are private and are NOT intended to work or be used by another package or module.

11.2 Initial set up

Start the DocStrip guards.

```
1 (*package)
```

Identify the internal prefix (L^AT_EX3 DocStrip convention) for `l3doc` class.

```
2 (@@=spintent)
```

11.3 Declaration of the package

First we will make sure we have a minimum (super updated) release version of L^AT_EX to work correctly, otherwise, we will return an “error”.

```
3 \NeedsTeXFormat{LaTeX2e}[2026-11-01]
4 \IfFormatAtLeastF { 2026-11-01 }
5 {
6   \PackageError { spintent }
7     { LaTeX~kernel~too~old~need~2026-11-01~or~later }
8     {
9       spintent~requires~a~LaTeX2e~release~2026-11-01~or~
10      later.~Update~your~TeX~distribution.
11     }
12 }
```

Now declare the `spintent` package and we will add the only key `mathcal` that can be passed as a package option.

```
13 \ProvidesExplPackage {spintent} {2026-09-15} {0.99} {Spanish parse intents}
14 \keys_define:nn { spintent }
15 {
16   mathcal .usage:n = load,
17   mathcal .bool_set:N = \l__spintent_mathcal_bool,
18   mathcal .initial:n = true,
19   mathcal .value_required:n = true,
20   AFghost .usage:n = load,
21   AFghost .bool_set:N = \l__spintent_debug_ghost_bool,
22   AFghost .initial:n = false,
23   AFghost .value_required:n = true,
24 }
25 \ProcessKeyOptions [ spintent ]
```

We check if the requirements for running the `spintent` package are met; that is: `\DocumentMetadata` must be active, Lua_T_EX must be running, and the `unicode-math` or `lua-unicode-math` package must be loaded. If neither is present, we will load `unicode-math`. If the conditions cannot be met, we will return a “fatal error”.

```
26 \IfDocumentMetadataTF
27 {
28   \sys_if_engine luatex:TF
29   {
30     \msg_new:nnn { spintent } { package-not-load }
31     {
32       The~'#1'~package~will~be~loaded~as~a~dependency.
33     }
34   }
35   \IfPackageLoadedF { unicode-math }
36   {
37     \IfPackageLoadedF { lua-unicode-math }
38     {
39       \msg_info:nnn { spintent } { package-not-load } { unicode-math }
40       \RequirePackage{unicode-math}
41     }
42   }
43 }
```

```

40     }
41   }
42   \msg_new:nnn { spintent } { env-success }
43   { Everything~looks~correct,~spintent~will~make~the~intent! }
44   \msg_info:nn { spintent } { env-success }
45 }
46 {
47   \msg_new:nnnn { spintent } { fatal-engine-requires-luatex }
48   { The~spintent~package~requires~LuaLaTeX. }
49   { This~package~relies~on~Lua~module.~Compile~with~lualatex. }
50   \msg_fatal:nn { spintent } { fatal-engine-requires-luatex }
51 }
52 }
53 {
54   \msg_new:nnnn { spintent } { fatal-metadata-missing }
55   { \token_to_str:N \DocumentMetadata \c_space_tl is-missing. }
56   {
57     The~spintent~package~requires~tagged~PDF~active.\\
58     You~must~declare~\token_to_str:N \DocumentMetadata { tagging=on } ~
59   }
60   \msg_fatal:nn { spintent } { fatal-metadata-missing }
61 }

```

11.4 Some kernel variants

Non-standard kernel variants.

```

\str_if_eq:nVTF
\str_if_eq:nVF
\str_uppercase:V
\str_lowercase:V
\seq_set_from_clist:Ne

```

```

62 \cs_generate_variant:Nn \str_if_eq:nnTF { V }
63 \cs_generate_variant:Nn \str_if_eq:nnF { V }
64 \cs_generate_variant:Nn \str_uppercase:n { V }
65 \cs_generate_variant:Nn \str_lowercase:n { V }
66 \cs_generate_variant:Nn \seq_set_from_clist:Nn { Ne }

```

(End of definition for `\str_if_eq:nVTF` and others.)

11.5 Variables for the key `\mathcal` and `\rightanglesqr`

```

\__spintent_luaset_cal_fam_ok_str
\__spintent_luaset_sym_fam_ok_str

```

First, we declare the variables `\__spintent_luaset_cal_fam_ok_str` and `\__spintent_luaset_sym_fam_ok_str` before loading the module `spintent.lua`. This is necessary because of the implementation of the functions `\__spintent_luafun_classic_mathcal:nn` and `\__spintent_luafun_right_angle_sqr:` within the Lua module; otherwise, `expl3` will complain about duplicate variable declarations.

```

67 \str_new:N \__spintent_luaset_cal_fam_ok_str
68 \str_new:N \__spintent_luaset_sym_fam_ok_str

```

(End of definition for `\__spintent_luaset_cal_fam_ok_str` and `\__spintent_luaset_sym_fam_ok_str`.)

11.6 Loading the Lua module and functions

Now we load the Lua module `spintent.lua` and generate the variants for the functions `\__spintent_luafun_spunit_lookup_alias:V` used by `\spunit` (§11.15), `\__spintent_luafun_spmoney_lookup_data:V` used by `\spmoney` (§11.17), `\__spintent_luafun_spmQ_scale_int:Vn` used by `\spmQ` (§11.32) and `\__spintent_luafun_clean_split_and_set:V` used by `\spnum` (§11.14) defined in it.

```

69 \lua_load_module:n { spintent.lua }
70 \bool_if:NTF \__spintent_debug_ghost_bool
71 { \__spintent_luafun_set_debug_ghost:n { true } }
72 { \__spintent_luafun_set_debug_ghost:n { false } }
73 \int_new:N \__spintent_scope_attr_int
74 \int_set:Nn \__spintent_scope_attr_int { \__spintent_scope_luaset_attr_str }
75 \cs_generate_variant:Nn \__spintent_luafun_spunit_lookup_alias:n { V }
76 \cs_generate_variant:Nn \__spintent_luafun_spmoney_lookup_data:n { V }
77 \cs_generate_variant:Nn \__spintent_luafun_spmQ_scale_int:nn { Vn }
78 \cs_generate_variant:Nn \__spintent_luafun_clean_split_and_set:n { V }

```

(End of definition for `\__spintent_luafun_spunit_lookup_alias:V` and others.)

11.7 Internal implementation for `\mathcal` and `\rightanglesqr`

To simplify user configuration and obtain the expected output style pdfTeX for `\mathcal`, we will provide a fallback to the font NewCMMath-Regular.otf, which contains the glyphs for these characters. Using this same font, we will obtain the symbol `\rightanglesqr` for the key `recto`.

```

mathcal-fallback
right-angle-sqr-fallback

```

First, we'll define two error messages in case our fallback fails.

```

79 \hook_gput_code:nnn { begindocument } { spintent }
80 {
81   \__spintent_luafun_init_math_fallbacks:

```

```

82 }
83 \msg_new:nnn { spintent } { mathcal-fallback }
84 {
85   Using~classic~\token_to_str:N \mathcal \c_space_tl as~fallback~(font~not~found~or~
86   no~free~math~family~available).~\msg_line_context:
87 }
88 \msg_new:nnn { spintent } { right-angle-sqr-fallback }
89 {
90   Using~\token_to_str:N \rightangle \c_space_tl as~fallback~for~the~right-angle~
91   glyph.~\msg_line_context:
92 }

```

(End of definition for `mathcal-fallback` and `right-angle-sqr-fallback`.)

`\__spintent_mathcal:n` Now we define the internal function `\__spintent_mathcal:n` for `\mathcal` which checks the state of the variable `\__spintent_mathcal_bool` set by the package key `mathcal`; if its state is true, we check the string variable `\__spintent_luaset_cal_fam_ok_str` and if the latter has the value true, we call the function `\__spintent_luafun_classic_mathcal:n` and otherwise (we cannot find the source in the system) we return a warning message and directly use `\mathcal`. Otherwise, that is, `mathcal=false` we simply use `\mathcal`.

```

93 \cs_new_protected:Nn \__spintent_mathcal:n
94 {
95   \bool_if:NTF \__spintent_mathcal_bool
96   {
97     \str_if_eq:VnTF \__spintent_luaset_cal_fam_ok_str { true }
98     { \__spintent_luafun_classic_mathcal:n {#1} }
99     {
100       \msg_warning:nn { spintent } { mathcal-fallback }
101       \mathcal {#1}
102     }
103   }
104   { \mathcal {#1} }
105 }

```

(End of definition for `\__spintent_mathcal:n`.)

`\__spintent_right_angle_sqr:` The internal function `\__spintent_right_angle_sqr:` establishes the internal copy of the command `\rightanglesqr`. First, we check the string variable `\__spintent_luaset_sym_fam_ok_str`. If it is true, we call the function `\__spintent__luafun_right_angle_sqr:`; otherwise (we do not find the font in the system), we return a warning message and use `\rightangle` instead.

```

106 \cs_new_protected:Nn \__spintent_right_angle_sqr:
107 {
108   \str_if_eq:VnTF \__spintent_luaset_sym_fam_ok_str { true }
109   { \__spintent_luafun_right_angle_sqr: }
110   {
111     \msg_warning:nn { spintent } { right-angle-sqr-fallback }
112     \rightangle
113   }
114 }

```

(End of definition for `\__spintent_right_angle_sqr:`.)

11.8 Shared annotation mechanisms: the queue and the ghosts

Almost all commands in the `spintent` package share three annotation functions, the mechanism of which is defined in the Lua module `spintent.lua`.

`\__spintent_luafun_wrap_mrow_intent_arg:nn` The internal function `\__spintent_luafun_wrap_mrow_intent_arg:nn` — called directly, with no `expl3` wrapper — pushes one entry onto a queue: a concept `intent` (`#1`, possibly empty) and a Lua table literal `arg=/intent=(#2)`. A `pre_mlist_to_hlist_filter` callback (installed once, before `luamml[2]`'s own) walks the formula's top-level noads in order and, for each one whose nucleus or subscript was typeset inside a `spintent` command's own group (marked via a LuaTeX `\attribute` activated by every command that uses this queue), claims it for the next pending entry: `arg`/a literal `intent` land on that leaf, and, when the concept `intent` is non-empty, the leaf is wrapped in a native `<mrow>` carrying it — no token, no ghost involved. Entries are consumed strictly in order; any left unclaimed when the formula ends are discarded rather than carried into the next one. A native `\MathMLarg/\MathMLintent` claim is invisible to this queue: mixing the two for the same content risks the queue claiming something already tagged natively, or never being claimed at all.

(End of definition for `\__spintent_luafun_wrap_mrow_intent_arg:nn`.)

The other two are “ghost” characters (an invisible U+2063, self-tagged `:silent` through their own `mathml_filter` — so the queue above never mistakes either for one of its own free targets) dropped in as an extra sibling so the resulting `<mrow>` is not the formula's sole top-level content, which `luamml` would otherwise promote straight to `<math intent=...>` instead of leaving it nested (issue #26).

`\__spintent_invisible_sep:` Placed once, before opening the formula’s own outer `\MathMLintent` — protects the *whole formula* from collapsing when that outer group is its only top-level content. Fires at most once per document: past the first call, in any command, it does nothing, since one sibling anywhere in the formula is already enough.

```

115 \cs_new_protected:Nn \__spintent_invisible_sep:
116 {
117   \__spintent_luafun_invisible_sep:
118 }

```

Executing this function writes the following line directly to the output file `-luamml-mathml.html`:

```
<mi intent=":silent" mathvariant="normal"></mi>
```

and if the package key `AFghost=true` is used, the output is:

```
<mi intent=":silent" mathvariant="normal" sptmp="outer"></mi>
```

If you wish to use math-AF mode, you can remove the line generated by `AFghost=true` using the Lua script `spiafcls`.

(End of definition for `\__spintent_invisible_sep:`.)

`\__spintent_inner_sep:` Placed just inside a native `\MathMLintent` group — protects *that specific group* when it too would otherwise hold only one child. Never deduplicates: called once per such group, however many appear, nested or sibling, in the same formula.

```

119 \cs_new_protected:Nn \__spintent_inner_sep:
120 {
121   \__spintent_luafun_inner_sep:
122 }

```

Executing this function writes the following line directly to the output file `-luamml-mathml.html`:

```
<mi intent=":silent" mathvariant="normal"></mi>
```

and if the package key `AFghost=true` is used, the output is:

```
<mi intent=":silent" mathvariant="normal" sptmp="inner"></mi>
```

If you wish to use math-AF mode, you can remove the line generated by `AFghost=true` using the Lua script `spiafcls`.

(End of definition for `\__spintent_inner_sep:`.)

`\c__spintent_invisible_sep_tl` A raw, untagged copy of the same U+2063 character, via the LuaTeX primitive `\Umathchardef` — for use as ordinary content *inside* an explicit `\MathMLintent{intent}` call (typically `:silent`), not as a sibling on its own.

```

123 \hook_gput_code:nnn { begindocument } { spintent }
124 {
125   \Umathchardef \c__spintent_invisible_sep_tl = "0 "0 "2063
126 }

```

(End of definition for `\c__spintent_invisible_sep_tl`.)

11.9 Normalize argument and affixes

We normalize the `{⟨argument⟩}` to support affixes (prefixes/suffixes) by trimming leading and trailing spaces, converting internal spaces to hyphens, collapsing consecutive hyphens, and removing leading or trailing hyphens.

`\__spintent_collapse_hyphens:N` A recursive helper to collapse multiple hyphens into a single one.

```

127 \cs_new_protected:Nn \__spintent_collapse_hyphens:N
128 {
129   \tl_if_in:NnT #1 { -- }
130   {
131     \tl_replace_all:Nnn #1 { -- } { - }
132     \__spintent_collapse_hyphens:N #1
133   }
134 }

```

(End of definition for `\__spintent_collapse_hyphens:N`.)

`\__spintent_sanitize_affix:Nn` The function `\__spintent_sanitize_affix:Nn` checks if the `{⟨argument⟩}` passed in ‘#2’ is *blank*. If so, it clears the target token list variable passed in ‘#1’. Otherwise, trimming bounds, converting spaces ‘`␣`’ to hyphens ‘`-`’, collapsing multiple hyphens, and safely stripping leading/trailing hyphens. For example, if you pass a target variable and raw text:

```
\__spintent_sanitize_affix:Nn \l__spintent_spsomesym_prefix_tl{ foo bar baz }
```

The function will “store” the normalized string foo-bar-baz inside `\__spintent_spsomesym_prefix_tl`, ready to be conditionally appended.

```

135 \cs_new_protected:Nn \__spintent_sanitize_affix:Nn
136 {
137   \tl_if_blank:nTF {#2}
138   { \tl_clear:N #1 }
139   {
140     \tl_set:Ne #1 {#2}
141     \tl_trim_spaces:N #1
142     \tl_replace_all:Nnn #1 { ~ } { - }
143     \__spintent_collapse_hyphens:N #1
144     \tl_put_left:Nn #1 { \q_mark }
145     \tl_put_right:Nn #1 { \q_stop }
146     \tl_replace_all:Nnn #1 { \q_mark - } { \q_mark }
147     \tl_replace_all:Nnn #1 { - \q_stop } { \q_stop }
148     \tl_remove_all:Nn #1 { \q_mark }
149     \tl_remove_all:Nn #1 { \q_stop }
150   }
151 }
152 \cs_generate_variant:Nn \__spintent_sanitize_affix:Nn { cn }

```

(End of definition for `\__spintent_sanitize_affix:Nn`.)

11.10 The command `\setspintent`

The `\setspintent` command is used to configure *keys* “globally” for the utilities provided by `spintent`.

11.10.1 Internal variables and error messages for `\setspintent`

```

\__spintent_setspintent_input_arg_cmd_tl
unknown-command-setup

```

The variable `\__spintent_setspintent_input_arg_cmd_tl` will store the command or command name passed to the “first” `{(argument)}` ‘#1’ of the command `\setspintent`.

```

153 \tl_new:N \__spintent_setspintent_input_arg_cmd_tl

```

Error message if the `{(argument)}` ‘#1’ passed to `\setspintent` is a “non-existent” command.

```

154 \msg_new:nnnn { spintent } { unknown-command-setup }
155 {
156   Cannot~configure~unknown~command~'\exp_not:c{#1}'.
157 }
158 {
159   You~attempted~to~use~\token_to_str:N \setspintent{#1}{...},~but~the~
160   command~\exp_not:c{#1}~is~not~defined.~Please~check~for~typos.
161 }

```

(End of definition for `\__spintent_setspintent_input_arg_cmd_tl` and `unknown-command-setup`.)

```

\__spintent_setspintent_set_keys:nn
\__spintent_setspintent_set_keys:en

```

The function `\__spintent_setspintent_set_keys:nn` will check if the command passed as `{(argument)}` exists in ‘#1’ and set the *keys* passed as `{(argument)}` in ‘#2’ to the correct path; otherwise, it will return an error.

```

162 \cs_new_protected:Nn \__spintent_setspintent_set_keys:nn
163 {
164   \cs_if_exist:cTF { #1 }
165   {
166     \keys_set:nn { spintent / #1 } { #2 }
167   }
168   {
169     \msg_error:nnn { spintent } { unknown-command-setup } { #1 }
170   }
171 }
172 \cs_generate_variant:Nn \__spintent_setspintent_set_keys:nn { en }

```

(End of definition for `\__spintent_setspintent_set_keys:nn`.)

```

\__spintent_setspintent_parse_args:nn
\__spintent_setspintent_parse_args:Vn

```

The function `\__spintent_setspintent_parse_args:nn` will check if the argument passed in #1 begins with ‘\’ or is just a *command name*. If it begins with ‘\’, it will remove it using `\cs_to_str:N` and then execute `\__spintent_setspintent_set_keys:en`. Otherwise, i.e., if ‘\’ was not passed, it will directly execute `\__spintent_setspintent_set_keys:nn`.

```

173 \cs_new_protected:Nn \__spintent_setspintent_parse_args:nn
174 {
175   \tl_if_single:nTF { #1 }
176   {
177     \token_if_cs:NTF #1
178     {
179       \__spintent_setspintent_set_keys:en { \cs_to_str:N #1 } { #2 }

```

```

180         }
181         {
182             \__spintent_setspintent_set_keys:nn { #1 } { #2 }
183         }
184     }
185     {
186         \__spintent_setspintent_set_keys:nn { #1 } { #2 }
187     }
188 }
189 \cs_generate_variant:Nn \__spintent_setspintent_parse_args:nn { Vn }

```

(End of definition for __spintent_setspintent_parse_args:nn.)

\setspintent Now we define the user command **\setspintent**.

```

190 \NewDocumentCommand \setspintent { m m }
191 {
192     \tl_if_blank:nTF { #1 }
193     {
194         \msg_error:nnn { spintent } { unknown-command-setup } { <empty> }
195     }
196     {
197         \tl_set:Nn \__spintent_setspintent_input_arg_cmd_tl { #1 }
198         \tl_trim_spaces:N \__spintent_setspintent_input_arg_cmd_tl
199         \__spintent_setspintent_parse_args:Vn \__spintent_setspintent_input_arg_cmd_tl { #2 }
200     }
201 }

```

(End of definition for \setspintent. This function is documented on page 3.)

11.11 Handling unknown keys

We define the variable `\__spintent_command_name_str` which will *store* the “name” of each command within the implementation along with the messages `unknown-choice` used by *<keys>* that use `.choice:n` in their implementation and the messages `cmd-key-unknown`, `cmd-key-value-unknown` used by the *<key>* `unknown` in commands that use `[<key = val>]`.

```

202 \str_new:N \__spintent_command_name_str
203 \msg_new:nnn { spintent } { unknown-choice }
204 {
205     The~value~'#3'~for~'#1'~key~is~invalid~use~('#2').
206 }
207 \msg_new:nnnn { spintent } { cmd-key-unknown }
208 {
209     The~key~'#1'~is~unknown~by~command~
210     \c_backslash_str \__spintent_command_name_str \c_space_tl
211     and~is~being~ignored~\msg_line_context:.
212 }
213 {
214     The~command~\c_backslash_str \__spintent_command_name_str \c_space_tl
215     does~not~have~a~key~called~'#1'.\\
216     Check~that~you~have~spelled~the~key~name~correctly.
217 }
218 \msg_new:nnnn { spintent } { cmd-key-value-unknown }
219 {
220     The~key~'#1=#2'~is~unknown~by~command~
221     \c_backslash_str \__spintent_command_name_str \c_space_tl
222     and~is~being~ignored~\msg_line_context:.
223 }
224 {
225     The~command~\c_backslash_str \__spintent_command_name_str \c_space_tl
226     does~not~have~a~key~called~'#1'.\\
227     Check~that~you~have~spelled~the~key~name~correctly.
228 }

```

(End of definition for __spintent_command_name_str and others.)

The internal functions `\__spintent_unknown_keys:nn` and `\__spintent_unknown_keys:n` used by the *<key>* `unknown`.

```

229 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:nn
230 {
231     \tl_if_blank:nTF {#2}
232     { \msg_error:nnn { spintent } { cmd-key-unknown } {#1} }
233     { \msg_error:nnnn { spintent } { cmd-key-value-unknown } {#1} {#2} }

```

```

234     }
235     \cs_generate_variant:Nn \__spintent_unknown_keys_cmd:n { Vn }
236     \cs_new_protected:Nn \__spintent_unknown_keys_cmd:n
237     {
238         \__spintent_unknown_keys_cmd:Vn \l_keys_key_str {#1}
239     }

```

(End of definition for `\__spintent_unknown_keys:n` and `\__spintent_unknown_keys:n`.)

11.12 Definition of core base variables

```

\__spintent_luafun_clean_split_and_set:n
\__spintent_luafun_clean_split_and_set:V

```

The function `\__spintent_luafun_clean_split_and_set:n` defined in the Lua module `spintent.lua`, is common to several commands in this implementation. It analyzes, processes, cleans and splits the $\{\langle argument \rangle\}$ passed to commands using internal functions defined in the Lua module `spintent.lua`. It isolates signs, integer components, decimal separators, decimal parte, decimal periodic part, exponents and final physical “units”, assigning them directly to `expl3` variables.

This function assigns the states of the variables ‘_str’ or ‘_tl’ described below to `true`, `false`, `error` or *stores* information according to the role it fulfills within the implementation.

(End of definition for `\__spintent_luafun_clean_split_and_set:n`.)

```

\l__spintent_luaset_sign_tl
\l__spintent_luaset_part_int_tl
\l__spintent_luaset_dec_sep_tl
\l__spintent_luaset_part_dec_tl
\l__spintent_luaset_part_period_tl
\l__spintent_luaset_has_integer_str
\l__spintent_luaset_has_natural_str

```

The variable `\l__spintent_luaset_sign_tl` stores the explicit ‘+’ or ‘-’ sign if present. The variable `\l__spintent_luaset_part_int_tl` stores the raw digit sequence of the “integer part”. The variable `\l__spintent_luaset_dec_sep_tl` stores the matched ‘,’ or ‘.’ used as the decimal separator. The variable `\l__spintent_luaset_part_dec_tl` stores the raw digits of the “finite decimal part” that follow the separator, and the variable `\l__spintent_luaset_part_period_tl` stores the digits of the “infinite periodic decimal part” that follow a semicolon ‘;’.

The variable `\l__spintent_luaset_has_integer_str` store the state `true` when it is *only* an “integer”. The variable `\l__spintent_luaset_has_natural_str` store the state `true` when it is *only* an “natural number”.

```

240 \tl_new:N \l__spintent_luaset_sign_tl
241 \tl_new:N \l__spintent_luaset_part_int_tl
242 \tl_new:N \l__spintent_luaset_dec_sep_tl
243 \tl_new:N \l__spintent_luaset_part_dec_tl
244 \tl_new:N \l__spintent_luaset_part_period_tl
245 \str_new:N \l__spintent_luaset_has_integer_str
246 \str_new:N \l__spintent_luaset_has_natural_str

```

(End of definition for `\l__spintent_luaset_sign_tl` and others.)

```

\l__spintent_luaset_only_part_int_str
\l__spintent_luaset_only_part_dec_str
\l__spintent_luaset_only_part_period_str
\l__spintent_luaset_format_part_int_str
\l__spintent_luaset_format_part_dec_str

```

The literal entries of the numbers that we will pass to the *intent function* :number of `\MathMLintent` are stored in `\l__spintent_luaset_only_part_int_str` for the “integer part”, in `\l__spintent_luaset_only_part_dec_str` for the “finite decimal part”, and in `\l__spintent_luaset_only_part_period_str` for the “infinite periodic decimal part”.

Formatted versions (`cifras-sep=true`) are stored in `\l__spintent_luaset_format_part_int_str` for the “integer part” and `\l__spintent_luaset_format_part_dec_str` for the “finite decimal part”.

```

247 \str_new:N \l__spintent_luaset_only_part_int_str
248 \str_new:N \l__spintent_luaset_only_part_dec_str
249 \str_new:N \l__spintent_luaset_only_part_period_str
250 \str_new:N \l__spintent_luaset_format_part_int_str
251 \str_new:N \l__spintent_luaset_format_part_dec_str

```

(End of definition for `\l__spintent_luaset_only_part_int_str` and others.)

```

\l__spintent_luaset_millions_str
\l__spintent_luaset_decimal_period_str
\l__spintent_luaset_not_decimal_period_str
\l__spintent_luaset_not_decimal_not_period_str
\l__spintent_luaset_dec_is_all_zeros_str
\l__spintent_luaset_has_exponent_str
\l__spintent_luaset_exponent_tl

```

The flag variable `\l__spintent_luaset_millions_str` indicates whether the integer part translates exactly to a multiple of one million when passed to `\MathMLintent`. To properly structure the reading and writing of numbers in MathML, we use the flag variables `\l__spintent_luaset_decimal_period_str`, `\l__spintent_luaset_not_decimal_period_str`, and `\l__spintent_luaset_not_decimal_not_period_str`. The flag variable `\l__spintent_luaset_dec_is_all_zeros_str` monitors whether the “finite decimal part” is completely zero (useful for “pure periodic decimals”).

The flag variable `\l__spintent_luaset_has_exponent_str` indicates whether active “scientific notation” was detected at the end of the entered number, passed with ‘e’ or ‘E’. If true, it will store the digits of the exponent in `\l__spintent_luaset_exponent_tl`.

```

252 \str_new:N \l__spintent_luaset_millions_str
253 \str_new:N \l__spintent_luaset_decimal_period_str
254 \str_new:N \l__spintent_luaset_not_decimal_period_str
255 \str_new:N \l__spintent_luaset_not_decimal_not_period_str
256 \str_new:N \l__spintent_luaset_dec_is_all_zeros_str
257 \str_new:N \l__spintent_luaset_has_exponent_str
258 \tl_new:N \l__spintent_luaset_exponent_tl

```

(End of definition for `\__spintent_luaset_millions_str` and others.)

```
\__spintent_luaset_has_units_str
\__spintent_luaset_status_str
\__spintent_luaset_arg_above_tl
\__spintent_luaset_arg_below_tl
\__spintent_luaset_denom_has_number_str
```

Since the function `\__spintent_luafun_clean_split_and_set:n` is common to `\spnum` (§11.14) and `\spunit` (§11.15) (as well as others), it determines the following unit-related variables after processing the `{⟨arguments⟩}` via the Lua module `spintent.lua`.

First, it checks if there is any residual text *after* the processed number that is a candidate for physical “units”. It sets the flag variable `\__spintent_luaset_has_units_str` accordingly. Following this, it verifies if this text contains more than one “/”. It then sets the variable `\__spintent_luaset_status_str` to `valid` or `multislash`. If the structure is `valid`, it splits the unit into `\__spintent_luaset_arg_above_tl` and `\__spintent_luaset_arg_below_tl`. Finally, it analyzes `\__spintent_luaset_arg_below_tl`; if the denominator starts with a number, it sets `\__spintent_luaset_denom_has_number_str` to `true`, otherwise to `false`.

```
259 \str_new:N \__spintent_luaset_has_units_str
260 \str_new:N \__spintent_luaset_status_str
261 \tl_new:N \__spintent_luaset_arg_above_tl
262 \tl_new:N \__spintent_luaset_arg_below_tl
263 \str_new:N \__spintent_luaset_denom_has_number_str
```

(End of definition for `\__spintent_luaset_has_units_str` and others.)

11.13 Common messages for text mode and math mode

Messages shared by the different commands for text mode or math mode.

```
264 \msg_new:nnnn { spintent } { need-math-mode }
265 {
266   Math~mode~required~for~'#1'~\msg_line_context:.
267 }
268 {
269   Use~inside~$...$~or~in~math~environment.
270 }
271 \msg_new:nnnn { spintent } { need-text-mode }
272 {
273   Text~mode~required~for~'#1'~\msg_line_context:.
274 }
275 {
276   Use~outside~of~$...$~or~math~environment.
277 }
```

11.14 The command `\spnum`

The user command `\spnum` is *first* of “the big four” provided by the `spintent` package and is fundamental to the implementation of other commands. It handles the processing of numerical input, scientific notation, and units of measurement, using the Lua module `spintent.lua`, which is responsible for separating and assigning variables defined in `expl3`.

From the `expl3` side, we handle the printing logic and the `intent` functions passed to `\MathMLintent` to achieve a valid MathML structure that is accepted by MathCAT.

11.14.1 Definition of keys and variables for `\spnum`

```
\__spintent_spnum_read_terse_bool
\__spintent_spnum_read_period_sep_str
\__spintent_spnum_notation_sym_tl
\__spintent_spnum_intent_read_sign_str
\__spintent_spnum_intent_read_dec_sep_str
```

The variable `\__spintent_spnum_read_terse_bool` handled by the key `read-sign`, controls the `intent` passed to `\MathMLintent` to spoken the ‘+’ or ‘−’. If set to `clear`, it will say “positivo” or “negativo” *after* reading the number; if set to `terse`, it will say “más” or “menos” *before* reading the number.

The variable `\__spintent_spnum_read_period_sep_str` handled by the key `read-period-sep`, controls how the decimal separator ‘;’ (periodic part) is printed and spoken when the `{⟨argument⟩}` is a *pure decimal periodic* and is passed the `intent` to `\MathMLintent`. If set to `coma`, it will say “coma” and print ‘,’; if set to `punto` it will say “punto” and print ‘.’.

The variable `\__spintent_spnum__notation_sym_tl`, handled by the key `notation-sym`, determines whether ‘×’ will be printed when set to `times` or ‘·’ when set to `cdot`.

The variable `\__spintent_spnum_intent_read_sign_str` will *store* the spoken translation used in the `intent` function passed to `\MathMLintent` for the ‘+’ or ‘−’ according to the value passed to the key `read-sign`.

The variable `\__spintent_spnum_intent_read_dec_sep_str` *stores* the spoken translation that will be used in the `intent` function passed to `\MathMLintent` for reading the decimal separation as a “coma” or a “punto” according to the value found in the variable `\__spintent_luaset_dec_sep_tl`.

```
278 \bool_new:N \__spintent_spnum_read_terse_bool
279 \str_new:N \__spintent_spnum_read_period_sep_str
280 \tl_new:N \__spintent_spnum_notation_sym_tl
281 \str_new:N \__spintent_spnum_intent_read_sign_str
282 \str_new:N \__spintent_spnum_intent_read_dec_sep_str
```

(End of definition for `\l__spintent_spnum_read_terse_bool` and others.)

Now we add the keys `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym`.

```

283 \keys_define:nn { spintent/spnum }
284 {
285   cifras-sep      .bool_set:N = \l__spintent_spnum_cifras_sep_bool,
286   cifras-sep      .initial:n = true,
287   cifras-sep      .value_required:n = true,
288   read-sign       .choices:nn =
289     { terse , clear }
290   {
291     \int_case:nn { \l_keys_choice_int }
292     {
293       {1} { \bool_set_true:N \l__spintent_spnum_read_terse_bool }
294       {2} { \bool_set_false:N \l__spintent_spnum_read_terse_bool }
295     }
296   },
297   read-sign       .initial:n = terse,
298   read-sign       .value_required:n = true,
299   read-sign / unknown .code:n =
300     \msg_error:nneee { spintent } { unknown-choice }
301     { read-sign } { terse , ~clear } { \exp_not:n {#1} },
302   read-period-sep .choices:nn =
303     { coma , punto }
304   {
305     \int_case:nn { \l_keys_choice_int }
306     {
307       {1} { \str_set:Nn \l__spintent_spnum_read_period_sep_str { coma } }
308       {2} { \str_set:Nn \l__spintent_spnum_read_period_sep_str { punto } }
309     }
310   },
311   read-period-sep .initial:n = coma,
312   read-period-sep .value_required:n = true,
313   read-period-sep / unknown .code:n =
314     \msg_error:nneee { spintent } { unknown-choice }
315     { read-period-sep } { coma , ~punto } { \exp_not:n {#1} },
316   notation-sym    .choices:nn =
317     { times , cdot }
318   {
319     \int_case:nn { \l_keys_choice_int }
320     {
321       {1} { \tl_set:Nn \l__spintent_spnum_notation_sym_tl { \times } }
322       {2} { \tl_set:Nn \l__spintent_spnum_notation_sym_tl { \cdot } }
323     }
324   },
325   notation-sym    .initial:n = cdot,
326   notation-sym    .value_required:n = true,
327   notation-sym / unknown .code:n =
328     \msg_error:nneee { spintent } { unknown-choice }
329     { notation-sym } { times , ~ cdot } { \exp_not:n {#1} },
330   unknown         .code:n =
331     {
332       \str_set:Nn \l__spintent_command_name_str { spnum }
333       \__spintent_unknown_keys_cmd:n {#1}
334     },
335 }

```

(End of definition for `cifras-sep` and others.)

11.14.2 Internal functions for `\spnum`

The function `\__spintent_spnum_render_part:n` processes the $\langle argument \rangle$ passed in ‘#1’ used by the functions `\__spintent_spnum_render_int:` and `\__spintent_spnum_render_dec:` to define the *intent function* :number for the integer and decimal part passed to `\MathMLintent`. It evaluates whether the key `cifras-sep` is active or not.

```

336 \cs_new_protected:Nn \__spintent_spnum_render_part:n
337 {
338   \MathMLintent { \str_use:c { \l__spintent_luaset_only_part_#1_str } :number }
339   {
340     \bool_if:NTF \l__spintent_spnum_cifras_sep_bool
341     { \str_use:c { \l__spintent_luaset_format_part_#1_str } }
342     { \tl_use:c { \l__spintent_luaset_part_#1_tl } }

```

```

343     }
344   }
345   \cs_new_protected:Nn \__spintent_snum_render_int:
346   {
347     \__spintent_snum_render_part:n { int }
348   }
349   \cs_new_protected:Nn \__spintent_snum_render_dec:
350   {
351     \__spintent_snum_render_part:n { dec }
352   }

```

(End of definition for `\__spintent_snum_render_part:n`, `\__spintent_snum_render_int:`, and `\__spintent_snum_render_dec:`.)

`\__spintent_snum_render_only_period:` The function `\__spintent_snum_render_only_period:` isolates the “infinite periodic decimal part” from the “finite decimal part”, assigning the *intent function* `:number` passed to `\MathMLintent` which will be used by `\MathMLarg` in the function `\__spintent_snum_print_period_bar:`.

```

353   \cs_new_protected:Nn \__spintent_snum_render_only_period:
354   {
355     \MathMLintent { \__spintent_luaset_only_part_period_str :number }
356     {
357       \__spintent_luaset_part_period_tl
358     }
359   }

```

The function `\__spintent_snum_print_period_bar:` inserts the function `\__spintent_snum_render_only_period:` as an argument within the *intent function* `_periódico:postfix($dp)` passed to `\MathMLintent`, generating the correct spoken for the “infinite periodic decimal part”.

```

360   \cs_new_protected:Nn \__spintent_snum_print_period_bar:
361   {
362     \MathMLintent { _periódico:postfix($dp) }
363     {
364       {
365         \overline
366         {
367           \MathMLarg { dp } { \__spintent_snum_render_only_period: }
368         }
369       }
370     }
371   }

```

The function `\__spintent_snum_render_period_sep:` sets how the decimal separator handled by the key `read-period-sep` will be printed and spoken when a “pure periodic decimal” is passed.

```

372   \cs_new_protected:Nn \__spintent_snum_render_period_sep:
373   {
374     \str_if_eq:VnTF \__spintent_snum_read_period_sep_str { coma }
375     {
376       \MathMLintent { _coma:prefix($pp) }
377       {
378         \mathord{,}
379         \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
380       }
381     }
382     {
383       \MathMLintent { _punto:prefix($pp) }
384       {
385         \mathord{.}
386         \MathMLarg { pp } { \__spintent_snum_print_period_bar: }
387       }
388     }
389   }

```

(End of definition for `\__spintent_snum_render_only_period:`, `\__spintent_snum_print_period_bar:`, and `\__spintent_snum_render_period_sep:`.)

`\__spintent_snum_render_decimal_sep:` The function `\__spintent_snum_render_decimal_sep:` parses the value of the variable `\__spintent_luaset_read_dec_sep_str` and sets the variable `\__spintent_snum_intent_read_dec_sep_str` with the function `_coma:prefix($dp)` for reading a “coma” or `_punto:prefix($dp)` for reading a “punto” to `\MathMLintent`. We print the decimal separator stored in `\__spintent_luaset_dec_sep_tl` using `\mathord` to remove spaces around it and then use the command `\MathMLarg` where we insert `\__spintent_invisible_sep:` for compatibility with MathML and finally call the function `\__spintent_snum_render_dec:` to print the finite decimal part.

```

390   \cs_new_protected:Nn \__spintent_snum_render_decimal_sep:

```

```

391 {
392   \str_if_eq:VnTF \l__spintent_luaset_dec_sep_tl { , }
393   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _coma } }
394   { \str_set:Nn \l__spintent_snum_intent_read_dec_sep_str { _punto } }
395   \MathMLintent { \l__spintent_snum_intent_read_dec_sep_str :prefix($dp) }
396   {
397     \mathord { \l__spintent_luaset_dec_sep_tl }
398     \MathMLarg { dp }
399     {
400       \__spintent_luafun_inner_sep:
401       \__spintent_snum_render_dec:
402     }
403   }
404   % period part
405   \str_if_eq:VnT \l__spintent_luaset_decimal_period_str { true }
406   {
407     \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
408     {
409       \__spintent_luafun_inner_sep:
410       \__spintent_snum_print_period_bar:
411     }
412     {
413       \MathMLintent { _con:prefix($pnum) }
414       {
415         \__spintent_luafun_inner_sep:
416         \MathMLarg { pnum }
417         {
418           \__spintent_snum_print_period_bar:
419         }
420       }
421     }
422   }
423 }

```

(End of definition for `\__spintent_snum_render_decimal_sep:`)

`\__spintent_snum_print_part_dec:`
`\__spintent_snum_print_part_period:`

The function `\__spintent_snum_print_part_dec:` checks if a decimal part exists. If it does, it determines whether the decimal separator is a comma or a period to set the correct speech intent (“coma” or “punto”). It then prints the separator, renders the decimal digits, and if a periodic sequence follows, it decides how to attach the periodic bar.

The function `\__spintent_snum_print_part_period:` handles cases where a periodic part exists. It evaluates if it should render the periodic separator directly, particularly when the number has a periodic part but lacks a standard decimal part.

```

424 \cs_new_protected:Nn \__spintent_snum_print_part_dec:
425 {
426   \tl_if_empty:NF \l__spintent_luaset_part_dec_tl
427   {
428     \__spintent_snum_render_decimal_sep:
429   }
430 }
431 \cs_new_protected:Nn \__spintent_snum_print_part_period:
432 {
433   \tl_if_empty:NF \l__spintent_luaset_part_period_tl
434   {
435     \str_if_eq:VnTF \l__spintent_luaset_not_decimal_period_str { true }
436     { \__spintent_snum_render_period_sep: }
437     {
438       \tl_if_empty:NT \l__spintent_luaset_part_dec_tl
439       {
440         \__spintent_snum_render_period_sep:
441       }
442     }
443   }
444 }

```

(End of definition for `\__spintent_snum_print_part_dec:` and `\__spintent_snum_print_part_period:`)

`\__spintent_snum_print_integer:`
`\__spintent_snum_print_num_start:`
`\__spintent_snum_print_number_core:`

The function `\__spintent_snum_print_integer:` checks if the number consists only of an integer. If so, it simply renders it. If a decimal or periodic part exists, it sets up a `MathML` intent to link the integer part (respecting digit separators if enabled) to the subsequent decimal or periodic functions.

The function `\__spintent_snum_print_num_start:` checks if an explicit sign is present. If it finds one, it evaluates the terse reading flag to decide whether the sign should be read as a prefix (“*más*” or “*menos*”) or as a postfix (“*positivo*” or “*negativo*”), and wraps the intent around the rest of the number.

The function `\__spintent_snum_print_number_core:` acts as the main wrapper. It triggers the sign and base number evaluation, and then checks if a scientific exponent is present. If it is, it appends the correct base-10 notation symbol and exponent to complete the sequence.

📌 Note for `\__spintent_invisible_sep:`.

```

445 \cs_new_protected:Nn \__spintent_snum_print_integer:
446 {
447   \str_if_eq:VnTF \__spintent_luaset_not_decimal_not_period_str { true }
448   { \__spintent_snum_render_int: }
449   {
450     \MathMLintent { \__spintent_luaset_only_part_int_str :number:prefix($next) }
451     {
452       \bool_if:NTF \__spintent_snum_cifras_sep_bool
453       { \__spintent_luaset_format_part_int_str }
454       { \__spintent_luaset_part_int_tl }
455       \MathMLarg { next }
456       {
457         \tl_if_empty:NTF \__spintent_luaset_part_dec_tl
458         { \__spintent_snum_print_part_period: }
459         { \__spintent_snum_print_part_dec: }
460       }
461     }
462   }
463 }
464 \cs_new_protected:Nn \__spintent_snum_print_num_start:
465 {
466   \tl_if_empty:NTF \__spintent_luaset_sign_tl
467   {
468     \__spintent_invisible_sep: % need
469     \__spintent_snum_print_integer:
470   }
471   {
472     \bool_if:NTF \__spintent_snum_read_terse_bool
473     {
474       \str_if_eq:VnTF \__spintent_luaset_sign_tl { + }
475       { \str_set:Nn \__spintent_snum_intent_read_sign_str { _más:prefix($n) } }
476       { \str_set:Nn \__spintent_snum_intent_read_sign_str { _menos:prefix($n) } }
477     }
478     {
479       \str_if_eq:VnTF \__spintent_luaset_sign_tl { + }
480       { \str_set:Nn \__spintent_snum_intent_read_sign_str { _positivo:postfix($n) } }
481       { \str_set:Nn \__spintent_snum_intent_read_sign_str { _negativo:postfix($n) } }
482     }
483     \__spintent_invisible_sep: % need
484     \MathMLintent { \__spintent_snum_intent_read_sign_str }
485     {
486       \__spintent_luaset_sign_tl
487       \MathMLarg { n } { \__spintent_snum_print_integer: }
488     }
489   }
490 }
491 \cs_new_protected:Nn \__spintent_snum_print_number_core:
492 {
493   \__spintent_snum_print_num_start:
494   \str_if_eq:VnTF \__spintent_luaset_has_exponent_str { true }
495   {
496     \MathMLintent { _por } { \__spintent_snum_notation_sym_tl }
497     10^{\__spintent_luaset_exponent_tl }
498   }
499 }

```

(End of definition for `\__spintent_snum_print_integer:`, `\__spintent_snum_print_num_start:`, and `\__spintent_snum_print_number_core:`.)

11.14.3 Definition of error messages

spnum-has-units
spnum-no-digits

We define two error messages for the `\spnum` command. The first “error” triggers if the user includes “*units*” in the `{\argument}` passed in ‘#1’, advising them to use the `\spunit` command instead. The second “error”

is raised if the provided $\{\langle argument \rangle\}$ contains no numerical digits.

```

500 \msg_new:nnnn { spintent } { spnum-has-units }
501 {
502   The~\token_to_str:N \spnum \c_space_tl command~does~not~accept~units~('#1').
503 }
504 {
505   Use~\token_to_str:N \spunit \c_space_tl if~you~need~to~print~numbers~with~units.
506 }
507 \msg_new:nnnn { spintent } { spnum-no-digits }
508 {
509   The~\token_to_str:N \spnum \c_space_tl command~requires~a~valid~numerical~value.
510 }
511 {
512   You~provided~'#1',~which~contains~no~digits.
513 }

```

(End of definition for *spnum-has-units* and *spnum-no-digits*.)

11.14.4 Implementation of the `\spum` command

`\spum` The public user command `\spum` accepts an optional $[(key = val)]$ argument in ‘#1’ and a mandatory $\{\langle rational\ number \rangle\}$ in ‘#2’. It first checks if it is executed inside math mode, raising an error if it is not. If correct, it opens a local group to apply the options and passes the input to Lua for parsing via `__spintent_luafun_clean_split_and_set:n`.

Before printing, it validates the extracted data. It throws an error if it finds trailing units in the input or if the integer part is completely empty (no valid digits). If all checks pass, it calls `__spintent_spnum_print_number_core:` to render the final number, and finally closes the group.

```

514 \NewDocumentCommand \spum { O{} m }
515 {
516   \mode_if_math:TF
517   {
518     \group_begin:
519     \keys_set:nn { spintent/spnum } { #1 }
520     __spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
521     \str_if_eq:VnTF \__spintent_luaset_has_units_str { true }
522     { \msg_error:nnn { spintent } { spnum-has-units } { #2 } }
523     {
524       \tl_if_empty:NTF \__spintent_luaset_part_int_tl
525       { \msg_error:nnn { spintent } { spnum-no-digits } { #2 } }
526       { __spintent_spnum_print_number_core: }
527     }
528     \group_end:
529   }
530   { \msg_error:nnn { spintent } { need-math-mode } { \spum } }
531 }

```

(End of definition for `\spum`. This function is documented on page 3.)

11.15 The command `\spunit`

The user command `\spunit` is the *second* of “the big four” provided by the `spintent` package and is used to parse and print physical units and angular systems. It sends the $\{\langle argument \rangle\}$ to Lua module `spintent.lua` via `__spintent_luafun_clean_split_and_set:n` to separate and clean the $\{\langle argument \rangle\}$.

This ensures that all “units” (such as SI fractions, products, or sexagesimal angles) are processed consistently by the `spintent.lua`.

11.15.1 Definition of variables

These string variables store the data returned by the Lua module `spintent.lua`. They hold the canonical “unit symbol”, the dictionary lookup status, the correct speech text for NVDA/MathCAT, and specific flags that indicate if the “unit” is compact or represents a sexagesimal angle.

```

\__spintent_spunit_luaset_canonical_str
\__spintent_spunit_luaset_lookup_status_str
\__spintent_spunit_luaset_read_str
\__spintent_spunit_luaset_is_compact_str
\__spintent_spunit_luaset_compact_exp_str
\__spintent_spunit_luaset_is_sexagesimal_str
\__spintent_spunit_luaset_status_str

```

```

532 \str_new:N \__spintent_spunit_luaset_canonical_str
533 \str_new:N \__spintent_spunit_luaset_lookup_status_str
534 \str_new:N \__spintent_spunit_luaset_read_str
535 \str_new:N \__spintent_spunit_luaset_is_compact_str
536 \str_new:N \__spintent_spunit_luaset_compact_exp_str
537 \str_new:N \__spintent_spunit_luaset_is_sexagesimal_str
538 \str_new:N \__spintent_spunit_luaset_status_str

```

(End of definition for `\__spintent_spunit_luaset_canonical_str` and others.)

```

\l__spintent_spunit_exponent_tl
\l__spintent_spunit_unit_name_tl
\l__spintent_spunit_read_slash_str
\l__spintent_spunit_exp_seq
\l__spintent_spunit_mult_seq
\l__spintent_spunit_is_mult_bool
\l__spintent_spunit_read_cdot_bool

```

These variables are used to process and format the “*unit*”. The sequences `\l__spintent_spunit_mult_seq` and `\l__spintent_spunit_exp_seq` split compound “*units*” at multiplication ‘`*`’ or exponent ‘`^`’ boundaries. The boolean variables control specific rendering options, such as whether the multiplication dot ‘`\cdot`’ is explicitly read aloud or silently skipped.

```

539 \tl_new:N \l__spintent_spunit_exponent_tl
540 \tl_new:N \l__spintent_spunit_unit_name_tl
541 \str_new:N \l__spintent_spunit_read_slash_str
542 \seq_new:N \l__spintent_spunit_exp_seq
543 \seq_new:N \l__spintent_spunit_mult_seq
544 \bool_new:N \l__spintent_spunit_is_mult_bool
545 \bool_new:N \l__spintent_spunit_read_cdot_bool

```

(End of definition for `\l__spintent_spunit_exponent_tl` and others.)

11.15.2 Definition of error messages

```

spunit-invalid-unit
spunit-no-units
inline-numeric-denominator

```

We define three error messages for the `\spunit` command. The first “error” triggers if the “*unit expression*” provided in ‘`#1`’ is unknown or malformed. The second “error” occurs if no “*units*” are found in the `{\argument}`, advising the user to use `\spnum` for pure numbers. The third “error” is raised if a numeric coefficient is illegally placed in the denominator of an inline fraction (after a slash).

```

546 \msg_new:nnnn { spintent } { spunit-invalid-unit }
547 {
548   The~expression~'#1'~contains~an~unknown~or~malformed~unit~structure.
549 }
550 {
551   Check~spelling~or~verify~the~unit~is~defined~in~the~dictionary.
552 }
553 \msg_new:nnnn { spintent } { spunit-no-units }
554 {
555   The~\token_to_str:N \spunit \c_space_tl command~requires~at~least~one~unit~('#1').
556 }
557 {
558   Use~\token_to_str:N \spnum \c_space_tl if~you~only~need~to~format~a~pure~number.
559 }
560 \msg_new:nnnn { spintent } { inline-numeric-denominator }
561 {
562   Illegal~numeric~coefficient~in~inline~denominator~'#1'.
563 }
564 {
565   The~inline~mode~of~\token_to_str:N \spunit \c_space_tl
566   does~not~allow~numbers~after~the~slash~(/).
567 }

```

(End of definition for `spunit-invalid-unit`, `spunit-no-units`, and `inline-numeric-denominator`.)

11.15.3 Definition of keys for `\spunit`

```

read-cdot
read-slash
cifras-sep
read-period-sep
notation-sym

```

Now we add the keys `print-cdot`, `read-cdot`, `read-slash`, `cifras-sep`, `read-period-sep` and `notation-sym`.

```

568 \keys_define:nn { spintent/spunit }
569 {
570   print-cdot      .bool_set:N = \l__spintent_spunit_print_cdot_bool,
571   print-cdot      .initial:n  = false,
572   print-cdot      .value_required:n = true,
573   read-cdot       .bool_set:N = \l__spintent_spunit_read_cdot_bool,
574   read-cdot       .initial:n  = false,
575   read-cdot       .value_required:n = true,
576   read-slash      .choices:nn =
577     { por , partido-por }
578     {
579       \int_case:nn { \l_keys_choice_int }
580       {
581         {1} { \str_set:Nn \l__spintent_spunit_read_slash_str { por } }
582         {2} { \str_set:Nn \l__spintent_spunit_read_slash_str { partido-por } }
583       }
584     },
585   read-slash      .initial:n  = por,
586   read-slash      .value_required:n = true,
587   read-slash / unknown .code:n =
588     \msg_error:nnee { spintent } { unknown-choice }
589     { read-slash } { por, ~partido-por } { \exp_not:n {#1} },
590   cifras-sep      .meta:nn = { spintent/spnum } { cifras-sep = {#1} },

```

```

591   cifras-sep           .value_required:n = true,
592   read-period-sep     .meta:nn = { spintent/spnum } { read-period-sep = {#1} },
593   read-period-sep     .value_required:n = true,
594   notation-sym        .meta:nn = { spintent/spnum } { notation-sym = {#1} },
595   notation-sym        .value_required:n = true,
596   unknown             .code:n =
597   {
598     \str_set:Nn \l__spintent_command_name_str { spunit }
599     \__spintent_unknown_keys_cmd:n {#1}
600   },
601 }

```

(End of definition for read-cdot and others.)

11.15.4 Internal functions for \spunit

__spintent_spunit_format_canonical:

The function `\__spintent_spunit_format_canonical:` handles the visual formatting of the “unit symbol”. It specifically checks for the *liter symbol* ‘ℓ’ to print it as `\ell`, and formats all other valid “units” in standard upright text using `\mathrm`.

```

602 \cs_new_protected:Nn \__spintent_spunit_format_canonical:
603 {
604   \str_case:VnF \l__spintent_spunit_luaset_canonical_str
605   {
606     { ℓ } { \ell }
607   }
608   { \mathrm { \l__spintent_spunit_luaset_canonical_str } }
609 }

```

(End of definition for __spintent_spunit_format_canonical:.)

__spintent_spunit_process_base_exp:n

The function `\__spintent_spunit_process_base_exp:n` evaluates a “single unit” along with its exponent, ‘#1’. It first looks up the unit in the dictionary; if not found, flags it as invalid.

Otherwise, unless this is the first unit in a product (`\l__spintent_spunit_is_mult_bool`), it handles the spacing and multiplication dot: `\cdot` when `print-cdot` is `true` (read as `por` when `read-cdot` is also `true`, silent otherwise), or a thin space `\,` when `print-cdot` is `false` (still read as `por` under `read-cdot`).

```

610 \cs_new_protected:Nn \__spintent_spunit_process_base_exp:n
611 {
612   \tl_clear:N \l__spintent_spunit_exponent_tl
613   \__spintent_luafun_spunit_lookup_alias:n { #1 }
614   \str_if_eq:VnTF \l__spintent_spunit_luaset_lookup_status_str { notfound }
615   {
616     \str_set:Nn \l__spintent_luaset_status_str { invalid }
617   }
618   {
619     \bool_if:NF \l__spintent_spunit_is_mult_bool
620     {
621       \bool_if:NTF \l__spintent_spunit_print_cdot_bool
622       {
623         \bool_if:NTF \l__spintent_spunit_read_cdot_bool
624         { \MathMLintent { por } { \cdot } }
625         { \MathMLintent { :silent } { \cdot } }
626       }
627       {
628         \bool_if:NTF \l__spintent_spunit_read_cdot_bool
629         { \MathMLintent { por } { \, } }
630         { \, }
631       }
632     }
633   }

```

Then renders the base unit and its exponent: either a pre-defined compact exponent from Lua (`\l__spintent_spunit_luaset_is_compact_str`) or by manually splitting ‘#1’ at the ‘^’ symbol otherwise.

```

633   \str_if_eq:VnTF \l__spintent_spunit_luaset_is_compact_str { true }
634   {
635     {
636       \MathMLintent { \l__spintent_spunit_luaset_read_str }
637       {
638         \__spintent_spunit_format_canonical:
639       }
640     }
641     ^ { \l__spintent_spunit_luaset_compact_exp_str }
642   }
643   {

```

```

644         \seq_set_split:Nnn \l__spintent_spunit_exp_seq { ^ } { #1 }
645         \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_unit_name_tl
646         \seq_if_empty:NF \l__spintent_spunit_exp_seq
647         {
648             \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_exponent_tl
649         }
650         \MathMLintent { \l__spintent_spunit_luaset_read_str }
651         {
652             \__spintent_spunit_format_canonical:
653         }
654         \tl_if_empty:NF \l__spintent_spunit_exponent_tl
655         {
656             ^ { \l__spintent_spunit_exponent_tl }
657         }
658     }
659     \bool_set_false:N \l__spintent_spunit_is_mult_bool
660 }
661 }

```

(End of definition for `\__spintent_spunit_process_base_exp:n`.)

`\__spintent_spunit_process_mult:n`
`\__spintent_spunit_process_mult:V`

The function `\__spintent_spunit_process_mult:n` handles compound “units” joined by an explicit multiplication star ‘*’. It splits the $\{\langle argument \rangle\}$ at each “star symbol” and applies `\__spintent_spunit_process_base_exp:n` to evaluate and render each resulting unit individually.

```

662 \cs_new_protected:Nn \__spintent_spunit_process_mult:n
663 {
664     \bool_set_true:N \l__spintent_spunit_is_mult_bool
665     \seq_set_split:Nnn \l__spintent_spunit_mult_seq { * } { #1 }
666     \seq_map_function:NN \l__spintent_spunit_mult_seq \__spintent_spunit_process_base_exp:n
667 }
668 \cs_generate_variant:Nn \__spintent_spunit_process_mult:n { V }

```

(End of definition for `\__spintent_spunit_process_mult:n`.)

`\__spintent_print_spunit_hierarchy:`

The function `\__spintent_print_spunit_hierarchy:` handles the layout of inline “unit” fractions. It first processes the “units” before the slash (the numerator). If a denominator exists, it prints the slash / ‘/’ with its corresponding speech intent, and then processes the remaining “units”.

```

669 \cs_new_protected:Nn \__spintent_print_spunit_hierarchy:
670 {
671     \tl_if_empty:NF \l__spintent_luaset_arg_above_tl
672     {
673         \__spintent_spunit_process_mult:V \l__spintent_luaset_arg_above_tl
674         \tl_if_empty:NF \l__spintent_luaset_arg_below_tl
675         {
676             \MathMLintent { \l__spintent_spunit_read_slash_str } { / }
677             \__spintent_spunit_process_mult:V \l__spintent_luaset_arg_below_tl
678         }
679     }
680 }

```

(End of definition for `\__spintent_print_spunit_hierarchy:.`)

`\__spintent_spunit_safe_exec:n`

The function `\__spintent_spunit_safe_exec:n` performs the main validation and layout for “units”. It first checks if the $\{\langle argument \rangle\}$ has units, is a valid expression, and does not contain illegal numbers in the denominator, throwing the appropriate “errors” if any check fails.

If the unit is valid, it checks if it is a sexagesimal angle (degrees, minutes, or seconds). For sexagesimal “units”, it prints the number and the symbol “without any space in between”, applying a slight negative space ‘\!’ for arcseconds. For standard units, it prints the number followed by a thin space ‘\,’ and then processes the unit sequence.

```

681 \cs_new_protected:Nn \__spintent_spunit_safe_exec:n
682 {
683     \str_if_eq:VnTF \l__spintent_luaset_has_units_str { false }
684     { \msg_error:nnn { spintent } { spunit-no-units } { #1 } }
685     {
686         \str_if_eq:VnTF \l__spintent_luaset_status_str { valid }
687         {
688             \str_if_eq:VnTF
689             \l__spintent_luaset_denom_has_number_str { true }
690             {
691                 \msg_error:nnn { spintent }
692                 { inline-numeric-denominator } { #1 }

```

```

693     }
694   {
695     \__spintent_luafun_spunit_lookup_alias:V \__spintent_luaset_arg_above_tl
696     \str_if_eq:VnTF \__spintent_spunit_luaset_is_sexagesimal_str { true }
697     {
698       \tl_if_empty:NF \__spintent_luaset_part_int_tl
699       {
700         \__spintent_spnum_print_number_core:
701       }
702       \MathMLintent { \__spintent_spunit_luaset_read_str }
703       {
704         \mathord
705         {
706           \str_case:Vn \__spintent_spunit_luaset_canonical_str
707             {
708               { " } { \prime \! \prime }
709               { ' } { \prime }
710               { ° } { ° }
711             }
712           }
713         }
714       }
715     {
716       \tl_if_empty:NF \__spintent_luaset_part_int_tl
717       {
718         \__spintent_spnum_print_number_core: \,
719       }
720       \__spintent_print_spunit_hierarchy:
721     }
722   }
723 }
724 { \msg_error:nnn { spintent } { spunit-invalid-unit } { #1 } }
725 }
726 }

```

(End of definition for `\__spintent_spunit_safe_exec:n`.)

11.15.5 Implementation of the `\spunit` command

`\spunit` The public user command `\spunit` accepts an optional [`<key = val>`] argument in ‘#1’ and a mandatory `{<unit expression>}` in ‘#2’. It first checks if it is executed inside *math mode*, raising an “error” if it is not. If correct, it opens a local group to apply the options and passes the `{<argument>}` to Lua module for parsing via `\__spintent_luafun_clean_split_and_set:n` function.

Finally, it delegates the execution to the function `\__spintent_spunit_safe_exec:n` to perform the safety checks and render the parsed data, before closing the group.

```

727 \NewDocumentCommand \spunit { O{} m }
728 {
729   \mode_if_math:TF
730   {
731     \group_begin:
732     \keys_set:nn { spintent/spunit } { #1 }
733     \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
734     \__spintent_spunit_safe_exec:n { #2 }
735     \group_end:
736   }
737   { \msg_error:nnn { spintent } { need-math-mode } { \spunit } }
738 }

```

(End of definition for `\spunit`. This function is documented on page 4.)

11.16 The commands `\spnewunit` and `\spunitalias`

The user commands `\spnewunit` and `\spunitalias` allow users to add new custom “units” or “aliases” to the package’s dictionary. Similar to `\spunit`, these commands delegate the processing and validation to the `spintent.lua` via `\__spintent_luafun_define_custom_unit:nn` and `\__spintent_luafun_define_spunit`.

During the definition, the Lua module checks if the new “unit” or “alias” is valid and ensures it does not conflict with existing “units”. It returns the result of this operation to T_EX using the `\__spintent_spunit_luaset_status_s` variable. If a collision is detected or the `{<argument>}` is invalid, the package throws the corresponding “error” message.

11.16.1 Definition of error messages

spnewunit-duplicate
spunitalias-duplicate
spunitalias-invalid-expr

We define three error messages for the creation of “units” and “aliases”. The first two “errors” trigger if the user attempts to define a “new unit” or alias passed in ‘#1’ that already exists in the dictionary. The third “error” is raised by `\spunitalias` if the target expression provided in ‘#2’ is invalid (for example, if it contains unknown “units” or incorrect syntax).

```

739 \msg_new:nnnn { spintent } { spnewunit-duplicate }
740 {
741   The~unit~symbol~or~alias~'#1'~is~already~defined!
742 }
743 {
744   You~cannot~redefine~with~\token_to_str:N \spnewunit.
745 }
746 \msg_new:nnnn { spintent } { spunitalias-duplicate }
747 {
748   The~alias~'#1'~is~already~defined~or~reserved!
749 }
750 {
751   You~cannot~redefine~an~existing~unit~or~another~alias.
752 }
753 \msg_new:nnnn { spintent } { spunitalias-invalid-expr }
754 {
755   The~expression~'#2'~for~alias~'#1'~is~invalid.
756 }
757 {
758   It~must~contain~only~defined~units,~exponents~and~one~'/'~.
759 }

```

(End of definition for `spnewunit-duplicate`, `spunitalias-duplicate`, and `spunitalias-invalid-expr`.)

`\spnewunit`

The public command `\spnewunit` defines a new custom unit. It passes the new unit symbol ‘#1’ and its speech text ‘#2’ to Lua via `\__spintent_luafun_define_custom_unit:nn`. If the module reports that the unit already exists, it throws the corresponding duplicate error.

```

760 \NewDocumentCommand \spnewunit { m m }
761 {
762   \mode_if_math:TF
763   {
764     \msg_error:nnn { spintent } { need-text-mode } { \spnewunit }
765   }
766   {
767     \__spintent_luafun_define_custom_unit:nn { #1 } { #2 }
768     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
769     {
770       \msg_error:nnn { spintent } { spnewunit-duplicate } { #1 }
771     }
772   }
773 }

```

(End of definition for `\spnewunit`. This function is documented on page 6.)

`\spunitalias`

The public command `\spunitalias` creates a shortcut for a complex unit expression. It passes the new alias name ‘#1’ and the target expression ‘#2’ to Lua via `\__spintent_luafun_define_spunit_alias:nn`. It throws an error if the alias already exists, or if the provided target expression contains invalid syntax or unknown units.

```

774 \NewDocumentCommand \spunitalias { m m }
775 {
776   \mode_if_math:TF
777   {
778     \msg_error:nnn { spintent } { need-text-mode } { \spunitalias }
779   }
780   {
781     \__spintent_luafun_define_spunit_alias:nn { #1 } { #2 }
782     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
783     {
784       \msg_error:nnn { spintent } { spunitalias-duplicate } { #1 }
785     }
786     \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { invalid-expr }
787     {
788       \msg_error:nnnn { spintent } { spunitalias-invalid-expr } { #1 } { #2 }
789     }
790   }
791 }

```

(End of definition for `\spunitalias`. This function is documented on page 6.)

11.17 The command `\spmoney`

The user command `\spmoney` is the *third* of “the big four” provided by the `spintent` package, designed to format monetary values and currencies. It delegates `{\argument}` parsing to the Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n`. The module then retrieves the correct “currency symbol”, pluralization rules, and layout structures from the database.

The command accepts *optional arguments* to override the default “currency symbol” or enable “subunit” rendering. Finally, it constructs the MathML intent for MathCAT.

These string variables store the currency data returned by the Lua module `spintent.lua`. Upon a successful database lookup, the module populates these variables with the “currency symbol”, its layout position (pre or post), the appropriate singular and plural names for both the main unit and subunits, and the execution status flags. T_EX then uses this information to render the visual layout and construct the correct MathML speech intents.

```
\__spintent_spmoney_luaset_symbol_str
\__spintent_spmoney_luaset_position_str
\__spintent_spmoney_luaset_sing_str
\__spintent_spmoney_luaset_plur_str
\__spintent_spmoney_luaset_conde_str
\__spintent_spmoney_luaset_subsing_str
\__spintent_spmoney_luaset_subplur_str
\__spintent_spmoney_luaset_print_iso_str
\__spintent_spmoney_luaset_currency_arg_str
\__spintent_spmoney_luaset_status_str
```

```
792 \str_new:N \__spintent_spmoney_luaset_symbol_str
793 \str_new:N \__spintent_spmoney_luaset_position_str
794 \str_new:N \__spintent_spmoney_luaset_sing_str
795 \str_new:N \__spintent_spmoney_luaset_plur_str
796 \str_new:N \__spintent_spmoney_luaset_conde_str
797 \str_new:N \__spintent_spmoney_luaset_subsing_str
798 \str_new:N \__spintent_spmoney_luaset_subplur_str
799 \str_new:N \__spintent_spmoney_luaset_print_iso_str
800 \str_new:N \__spintent_spmoney_luaset_currency_arg_str
801 \str_new:N \__spintent_spmoney_luaset_status_str
```

(End of definition for `\__spintent_spmoney_luaset_symbol_str` and others.)

The boolean variable tracks whether subunit rendering is enabled. The string variables assemble the specific speech text (intents) for NVDA/MathCAT, including the sign, the main currency, and the subunits. Finally, the remaining variables temporarily store the numeric value and currency code extracted from the user’s input.

```
\__spintent_spmoney_sub_units_bool
\__spintent_spmoney_intent_money_str
\__spintent_spmoney_intent_sign_str
\__spintent_spmoney_intent_sub_unit_str
\__spintent_spmoney_res_main_voice_str
\__spintent_spmoney_subnum_int
\__spintent_spmoney_extracted_curr_str
\__spintent_spmoney_extracted_num_tl
```

```
802 \bool_new:N \__spintent_spmoney_sub_units_bool
803 \str_new:N \__spintent_spmoney_intent_money_str
804 \str_new:N \__spintent_spmoney_intent_sign_str
805 \str_new:N \__spintent_spmoney_intent_sub_unit_str
806 \str_new:N \__spintent_spmoney_res_main_voice_str
807 \int_new:N \__spintent_spmoney_subnum_int
808 \str_new:N \__spintent_spmoney_extracted_curr_str
809 \tl_new:N \__spintent_spmoney_extracted_num_tl
```

(End of definition for `\__spintent_spmoney_sub_units_bool` and others.)

11.17.1 Definition of error messages

```
currency-no-subunits
invalid-currency
```

We define two messages to handle invalid currency configurations. The first “error” warns the user if subunit rendering is requested for a currency that does not support fractional divisions (such as JPY or CLP), safely ignoring the flag. The second “error” is triggered if the provided currency identifier is not registered in the Lua database.

```
810 \msg_new:nnnn { spintent } { currency-no-subunits }
811 {
812   The~currency~'#1'~does~not~support~subunits.~
813   The~option~'sub-units=true'~is~ignored.
814 }
815 {
816   Monies~like~the~Japanese~Yen~(JPY)~or~Chilean~Peso~(CLP)~
817   do~not~have~fractional~subunits~registered~in~Lua~module.~
818 }
819 \msg_new:nnnn { spintent } { invalid-currency }
820 {
821   Invalid~currency~identifier~'#1'~in~\token_to_str:N \spmoney .
822 }
823 {
824   The~provided~symbol,~alias,~or~ISO~code~does~not~exist~in~
825   the~Lua~module.
826 }
```

(End of definition for `currency-no-subunits` and `invalid-currency`.)

11.17.2 Auxiliary functions for \spmoney

```

__spintent_spmoney_set_currency:n
__spintent_spmoney_normalize_key:n
__spintent_spmoney_normalize_key:V

```

The function `\__spintent_spmoney_set_currency:n` validates the currency identifier provided in ‘#1’ against the database. It delegates the lookup to the helper function `\__spintent_spmoney_normalize_key:n`, which interfaces with the Lua module `spintent.lua`. If the module returns a not found status, an “error” is triggered. Otherwise, it stores the valid currency identifier in `\__spintent_spmoney_luaset_currency_arg_str` for subsequent processing.

```

827 \cs_new_protected:Nn \__spintent_spmoney_set_currency:n
828 {
829   \__spintent_spmoney_normalize_key:n {#1}
830   \str_if_eq:VnTF \__spintent_spmoney_luaset_status_str { notfound }
831   {
832     \msg_error:nnn { spintent } { invalid-currency } {#1}
833   }
834   { \str_set:Nn \__spintent_spmoney_luaset_currency_arg_str {#1} }
835 }
836 \cs_new_protected:Nn \__spintent_spmoney_normalize_key:n
837 {
838   \__spintent_luafun_spmoney_normalize_key:n { #1 }
839 }
840 \cs_generate_variant:Nn \__spintent_spmoney_normalize_key:n { V }

```

(End of definition for `\__spintent_spmoney_set_currency:n` and `\__spintent_spmoney_normalize_key:n`.)

11.17.3 Definition of keys for \spmoney

```

currency
sub-units
dolar-math
cifras-sep

```

Now we add the keys `currency`, `sub-units`, `dolar-math` and `cifras-sep`.

```

841 \keys_define:nn { spintent / spmoney }
842 {
843   currency      .code:n      = { \__spintent_spmoney_set_currency:n {#1} },
844   currency      .initial:n    = { pesos },
845   currency      .value_required:n = true,
846   sub-units     .bool_set:N   = \__spintent_spmoney_sub_units_bool,
847   sub-units     .value_required:n = true,
848   dolar-math    .bool_set:N   = \__spintent_spmoney_dolar_math_bool,
849   dolar-math    .initial:n    = { true },
850   dolar-math    .value_required:n = true,
851   cifras-sep    .meta:nn      = { spintent/spnum } { cifras-sep = {#1} },
852   cifras-sep    .value_required:n = true,
853   unknown      .code:n      =
854   {
855     \str_set:Nn \__spintent_command_name_str { spmoney }
856     \__spintent_unknown_keys_cmd:n {#1}
857   },
858 }

```

(End of definition for `currency` and others.)

11.17.4 Internal functions for \spmoney

```

__spintent_spmoney_validate_money:

```

The function `\__spintent_spmoney_validate_money:` verifies if the parsed input contains an embedded currency identifier by checking `\__spintent_luaset_has_units_str`. If a currency is detected, it extracts and normalizes the key from `\__spintent_luaset_arg_above_tl`. It throws an “error” if the currency is not found in the database. If valid, it updates `\__spintent_spmoney_luaset_currency_arg_str` and retrieves the full currency metadata from the `spintent.lua` via `\__spintent_luafun_spmoney_lookup_data:V`.

```

859 \cs_new_protected:Nn \__spintent_spmoney_validate_money:
860 {
861   \str_if_eq:VnT \__spintent_luaset_has_units_str { true }
862   {
863     \__spintent_spmoney_normalize_key:V \__spintent_luaset_arg_above_tl
864     \str_if_eq:VnTF \__spintent_spmoney_luaset_status_str { notfound }
865     { \msg_error:nne { spintent } { invalid-currency } { \__spintent_luaset_arg_above_tl }
866       {
867         \str_set:NV
868           \__spintent_spmoney_luaset_currency_arg_str
869           \__spintent_luaset_arg_above_tl
870       }
871     }
872     \str_set:Ne \__spintent_spmoney_luaset_status_str { found }
873     \__spintent_luafun_spmoney_lookup_data:V \__spintent_spmoney_luaset_currency_arg_str
874   }

```

(End of definition for `\__spintent_spmoney_validate_money:`.)

`\__spintent_spmoney_print_symbol:` The function `\__spintent_spmoney_print_symbol:` renders the “*currency symbol*”. If `\l__spintent_spmoney_luaset_print_iso_str` is `true`, it outputs the ISO code using `\mathrm`. Otherwise, it prints the “*standard symbol*”. For dollar signs ‘\$’, it checks `\l__spintent_spmoney_dolar_math_bool` to determine whether to render it directly as a math character ‘\$’ or wrap it inside `\text`.

```

875 \cs_new_protected:Nn \__spintent_spmoney_print_symbol:
876 {
877   \str_if_eq:VnTF \l__spintent_spmoney_luaset_print_iso_str { true }
878   { \mathrm { \l__spintent_spmoney_luaset_currency_arg_str } }
879   {
880     \str_if_eq:VnTF \l__spintent_spmoney_luaset_symbol_str { $ }
881     {
882       \bool_if:NTF \l__spintent_spmoney_dolar_math_bool
883       { \$ } { \text { \$ } }
884     }
885     { \l__spintent_spmoney_luaset_symbol_str }
886   }
887 }

```

(End of definition for `\__spintent_spmoney_print_symbol:`.)

`\__spintent_spmoney_render_layout:` The function `\__spintent_spmoney_render_layout:` determines the layout order of the “*currency symbol*” relative to the numeric value. It evaluates `\l__spintent_spmoney_luaset_position_str`; if set to `pre`, it renders the “*currency symbol*” *before* the number node. Otherwise, it places the number first, followed by the “*currency symbol*”.

```

888 \cs_new_protected:Nn \__spintent_spmoney_render_layout:
889 {
890   \str_if_eq:VnTF \l__spintent_spmoney_luaset_position_str { pre }
891   {
892     \__spintent_spmoney_print_symbol:
893     \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
894   }
895   {
896     \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
897     \__spintent_spmoney_print_symbol:
898   }
899 }

```

(End of definition for `\__spintent_spmoney_render_layout:`.)

`\__spintent_spmoney_render_number_node:` The function `\__spintent_spmoney_render_number_node:` renders the full numeric value, encompassing both the integer and decimal components. Conversely, the function `\__spintent_spmoney_render_only_integer_node:` outputs exclusively the integer portion, omitting any decimal data.

```

900 \cs_new_protected:Nn \__spintent_spmoney_render_number_node:
901 {
902   \__spintent_spmoney_render_int:
903   \__spintent_spmoney_print_part_dec:
904 }
905 \cs_new_protected:Nn \__spintent_spmoney_render_only_integer_node:
906 {
907   \__spintent_spmoney_render_int:
908 }

```

(End of definition for `\__spintent_spmoney_render_number_node:` and `\__spintent_spmoney_render_only_integer_node:`.)

`\__spintent_spmoney_print_int:` The function `\__spintent_spmoney_print_int:` evaluates the presence of an explicit sign in `\l__spintent_luaset_sign_tl`. If a sign is detected, it assigns a prefix speech intent ‘más’ or ‘menos’ and embeds the sign and subsequent layout inside a `\MathMLintent`. If no sign is present, it routes execution based on whether subunits are enabled.

```

909 \cs_new_protected:Nn \__spintent_spmoney_print_int:
910 {
911   \tl_if_empty:NTF \l__spintent_luaset_sign_tl
912   {
913     \bool_if:NTF \l__spintent_spmoney_sub_units_bool
914     { \__spintent_spmoney_set_and_print_sub_units: }
915     { \__spintent_spmoney_intent_int: }
916   }
917   {
918     \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
919     {
920       \str_set:Nn \l__spintent_spmoney_intent_sign_str { _más:prefix ($n) }
921     }
922   }

```

```

922     {
923         \str_set:Nn \__spintent_spmoney_intent_sign_str { _menos:prefix ($n) }
924     }
925     \__spintent_luafun_inner_sep:% \__spintent_invisible_sep:
926     \MathMLintent { \__spintent_spmoney_intent_sign_str }
927     {
928         \l__spintent_luaset_sign_tl
929         \MathMLarg { n }
930         {
931             \bool_if:NTF \__spintent_spmoney_sub_units_bool
932             { \__spintent_spmoney_set_and_print_sub_units: }
933             { \__spintent_spmoney_intent_int: }
934         }
935     }
936 }
937 }

```

(End of definition for `\__spintent_spmoney_print_int:`)

`\__spintent_spmoney_intent_int:`

The function `\__spintent_spmoney_intent_int:` determines the correct grammatical inflection for the currency string (singular, plural, or the prepositional ‘de’ variant for exact millions). It evaluates the integer component, decimal boundaries, and zero-fills to select the appropriate string register, applying it as a postfix MathML intent over the layout rendered by `\__spintent_spmoney_render_layout:`.

```

938 \cs_new_protected:Nn \__spintent_spmoney_intent_int:
939 {
940     \tl_if_empty:NTF \l__spintent_luaset_dec_sep_tl
941     {
942         \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
943         {
944             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_sing_str
945         }
946         {
947             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
948         }
949         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
950         {
951             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde_s
952         }
953     }
954     {
955         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
956         \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
957         {
958             \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
959             {
960                 \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_con
961             }
962             {
963                 \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plu
964             }
965         }
966     }
967     \__spintent_luafun_inner_sep:
968     \MathMLintent { \l__spintent_spmoney_intent_money_str :postfix($n) }
969     {
970         \__spintent_spmoney_render_layout:
971     }
972 }

```

(End of definition for `\__spintent_spmoney_intent_int:`)

`\__spintent_spmoney_check_sub_units:`

`\__spintent_spmoney_set_and_print_sub_units:`

The function `\__spintent_spmoney_check_sub_units:` verifies whether the selected currency supports “*subunits*”. If the database returns empty subunit data, it triggers a warning and automatically disables the “*subunit*” rendering flag.

The function `\__spintent_spmoney_set_and_print_sub_units:` processes and formats the speech intents when “*subunits*” are active. It normalizes the decimal component (for instance, padding a single-digit decimal like ‘.5’ to ‘.50’), evaluates the correct singular or plural inflections for both the main “*currency*” and the “*subunits*”, and delegates the final MathML tree assembly to `\__spintent_spmoney_build_tree:`.

```

973 \cs_new_protected:Nn \__spintent_spmoney_check_sub_units:
974 {
975     \str_if_empty:NT \l__spintent_spmoney_luaset_subsing_str

```

```

976     {
977         \msg_warning:nne { spintent } { currency-no-subunits }
978         { \l__spintent_spmoney_luaset_currency_arg_str }
979         \bool_set_false:N \l__spintent_spmoney_sub_units_bool
980     }
981 }
982 \cs_new_protected:Nn \l__spintent_spmoney_set_and_print_sub_units:
983 {
984     \int_compare:nTF { \str_count:N \l__spintent_luaset_only_part_dec_str == 1 }
985     {
986         \int_set:Nn \l__spintent_spmoney_subnum_int
987         { \l__spintent_luaset_only_part_dec_str * 10 }
988         \str_set:Ne \l__spintent_luaset_only_part_dec_str
989         { \l__spintent_luaset_only_part_dec_str 0 }
990     }
991     {
992         \int_set:Nn \l__spintent_spmoney_subnum_int
993         { \l__spintent_luaset_only_part_dec_str }
994     }
995     \int_compare:nTF { \l__spintent_spmoney_subnum_int == 1 }
996     {
997         \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
998         { \l__spintent_spmoney_luaset_subsing_str :postfix($sub) }
999     }
1000     {
1001         \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
1002         { \l__spintent_spmoney_luaset_subplur_str :postfix($sub) }
1003     }
1004     \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
1005     {
1006         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1007         \l__spintent_spmoney_luaset_sing_str
1008     }
1009     {
1010         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1011         \l__spintent_spmoney_luaset_plur_str
1012     }
1013     \str_if_eq:VnT \l__spintent_luaset_millions_str { true }
1014     {
1015         \str_set:NV \l__spintent_spmoney_res_main_voice_str
1016         \l__spintent_spmoney_luaset_conde_str
1017     }
1018     \l__spintent_spmoney_build_tree:
1019 }

```

(End of definition for `\l__spintent_spmoney_check_sub_units:` and `\l__spintent_spmoney_set_and_print_sub_units:`)

`\l__spintent_spmoney_build_tree:`

The function `\l__spintent_spmoney_build_tree:` structures the final MathML tree assembly for fractional “currencies”. It arranges the integer component, the decimal separator, and the “subunit” component based on the layout position (pre or post) of the “currency symbol”.

`\l__spintent_spmoney_print_sep_con:`

Furthermore, `\l__spintent_spmoney_print_sep_con:` applies the specific speech intent ‘con’ to the decimal separator, while `\l__spintent_spmoney_print_dec_sub_unit:` binds the computed “subunit” intent to the decimal digits, ensuring accurate accessibility output.

`\l__spintent_spmoney_print_dec_sub_unit:`

```

1020 \cs_new_protected:Nn \l__spintent_spmoney_build_tree:
1021 {
1022     \str_if_eq:VnTF \l__spintent_spmoney_luaset_position_str { pre }
1023     {
1024         \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
1025         {
1026             \l__spintent_spmoney_print_symbol:
1027             \MathMLarg { n } { \l__spintent_spmoney_render_only_integer_node: }
1028         }
1029         \l__spintent_spmoney_print_sep_con:
1030         \l__spintent_spmoney_print_dec_sub_unit:
1031     }
1032     {
1033         \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
1034         {
1035             \l__spintent_luafun_inner_sep:
1036             \MathMLarg { n } { \l__spintent_spmoney_render_only_integer_node: }
1037         }

```

```

1038     \__spintent_spmoney_print_sep_con:
1039     \__spintent_spmoney_print_dec_sub_unit:
1040     \MathMLintent { :silent } { \__spintent_spmoney_print_symbol: }
1041   }
1042 }
1043 % ^^A \cs_new_protected:Nn \__spintent_spmoney_print_integer_continue:
1044 % ^^A {
1045 % ^^A   \MathMLintent { \__spintent_spmoney_res_main_voice_str :postfix($n) }
1046 % ^^A   {
1047 % ^^A     \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
1048 % ^^A   }
1049 % ^^A }
1050 \cs_new_protected:Nn \__spintent_spmoney_print_sep_con:
1051 {
1052   \MathMLintent { con }
1053   {
1054     \mathord { \__spintent_luaset_dec_sep_tl }
1055   }
1056 }
1057 \cs_new_protected:Nn \__spintent_spmoney_print_dec_sub_unit:
1058 {
1059   \MathMLintent { \__spintent_spmoney_intent_sub_unit_str }
1060   {
1061     \__spintent_luafun_inner_sep:
1062     \MathMLarg { sub }
1063     {
1064       \__spintent_spmoney_render_dec:
1065     }
1066   }
1067 }

```

(End of definition for `\__spintent_spmoney_build_tree:`, `\__spintent_spmoney_print_sep_con:`, and `\__spintent_spmoney_print_dec_sub_unit:`.)

`\__spintent_spmoney_parse_and_route:n`

The internal function `\__spintent_spmoney_parse_and_route:n` processes the `{(argument)}` passed in ‘`#1`’ using Lua module to extract the numeric value and any “currency” identifier. If a “currency” is found (`\__spintent_spmoney_extracted_curr_str`), it sets the corresponding variables. It then calls `\__spintent_spmoney_validate_money:` to verify the “currency”. If “subunit” rendering is enabled (`\__spintent_spmoney_sub_units_bool`), it calls `\__spintent_spmoney_check_sub_units:`. Finally, it calls `\__spintent_spmoney_print_int:` to format the output.

```

1068 \cs_new_protected:Nn \__spintent_spmoney_parse_and_route:n
1069 {
1070   \__spintent_luafun_spmoney_prepare_input:n { #1 }
1071   \__spintent_luafun_clean_split_and_set:V \__spintent_spmoney_extracted_num_tl
1072   \str_if_empty:NF \__spintent_spmoney_extracted_curr_str
1073   {
1074     \str_set:Nc \__spintent_luaset_has_units_str { true }
1075     \str_set:NV \__spintent_luaset_arg_above_tl \__spintent_spmoney_extracted_curr_str
1076   }
1077   \__spintent_spmoney_validate_money:
1078   \bool_if:NT \__spintent_spmoney_sub_units_bool
1079   { \__spintent_spmoney_check_sub_units: }
1080   \__spintent_spmoney_print_int:
1081 }

```

(End of definition for `\__spintent_spmoney_parse_and_route:n`.)

`\spmoney`

The public document command `\spmoney` formats monetary values with accessible speech intents. It accepts an optional argument ‘`#1`’ for local key-value configurations and a mandatory argument ‘`#2`’ containing the numeric value and optional embedded currency identifier. It first evaluates the typesetting context, triggering a fatal error if executed outside of math mode. Upon validation, it establishes a local $\TeX$ group to safely isolate the key assignments, delegates the raw input string to `\__spintent_spmoney_parse_and_route:n` for parsing and rendering, and subsequently closes the group to prevent scope leakage.

```

1082 \NewDocumentCommand \spmoney { O{} m }
1083 {
1084   \mode_if_math:TF
1085   {
1086     \group_begin:
1087     \keys_set:nn { spintent / spmoney } { #1 }
1088     \__spintent_spmoney_parse_and_route:n { #2 }
1089     \group_end:

```

```

1090     }
1091     { \msg_error:nnn { spintent } { need-math-mode } { \spmoney } }
1092 }

```

(End of definition for `\spmoney`. This function is documented on page 6.)

11.18 The command `\spshort`

The user command `\spshort` is the *fourth* and *final* of “the big four”. It manages the semantic injection and typographical formatting of *text mode* abbreviations, ordinals, and acronyms. Unlike *math mode* commands, which rely on MathML attributes, this command uses the PDF *tagging tools* provided by the `tagpdf`[29] package. Specifically, it uses the `E` (expanded) key to ensure that NVDA interpret the full form of the abbreviation, ordinal, or acronym in *text mode*.

These internal string variables serve as the primary data interface with the Lua module `spintent.lua`. During processing, the Lua module “stores” within these registers the execution status, the targeted typographical layout parameters, the phonetic replacement text for the `E` key intended for NVDA, the final sanitized output string, and the isolated structural components for ordinals (base and suffix).

```

1093 \str_new:N \l__spintent_spshort_luaset_status_str
1094 \str_new:N \l__spintent_spshort_luaset_layout_str
1095 \str_new:N \l__spintent_spshort_luaset_expanded_str
1096 \str_new:N \l__spintent_spshort_luaset_output_str
1097 \str_new:N \l__spintent_spshort_luaset_base_str
1098 \str_new:N \l__spintent_spshort_luaset_suffix_str

```

(End of definition for `\l__spintent_spshort_luaset_status_str` and others.)

11.18.1 Definition of error messages for `\spshort`

unknown-abbreviation

The internal “error” message `unknown-abbreviation` is triggered when the command `\spshort` encounters an invalid lookup state from the Lua module. It acts as a strict fallback exception when the provided `{⟨argument⟩}` cannot be mapped to any registered database entry in `spintent.lua` nor successfully resolved through the grammatical ordinal expansion rules.

```

1099 \msg_new:nnnn { spintent } { unknown-abbreviation }
1100 { The~abbreviation~or~ordinal~'#1'~is~invalid~or~not~recognized. }
1101 { Verify~the~spelling~or~check~the~grammar~rules~for~ordinals. }

```

(End of definition for `unknown-abbreviation`.)

11.18.2 Accessibility interface

`\__spintent_spshort_tag_span:nn`

`\__spintent_spshort_tag_span:en`

The internal function `\__spintent_spshort_tag_span:nn` processes the `{⟨argument⟩}` passed in ‘#2’ using the tools provided by `tagpdf`. By utilizing the `tag` and `E` keys, it creates a native PDF /E (`#1`) structure and expands the `{⟨argument⟩}` passed in ‘#1’ for proper reading with NVDA.

```

1102 \cs_new_protected:Nn \__spintent_spshort_tag_span:nn
1103 {
1104     \tag_mc_end_push:
1105     \tag_struct_begin:n { tag=Span, E={ #1 } }
1106     \tag_mc_begin:n { tag=Span }
1107     \tag_suspend:n { \spshort }
1108     #2
1109     \tag_resume:n { \spshort }
1110     \tag_mc_end:
1111     \tag_struct_end:
1112     \tag_mc_begin_pop:n {}
1113 }
1114 \cs_generate_variant:Nn \__spintent_spshort_tag_span:nn { en }

```

(End of definition for `\__spintent_spshort_tag_span:nn`.)

11.18.3 Definition of keys for `\spshort`

small-caps

read-arg

Now we add the keys `small-caps` and `read-arg`.

```

1115 \keys_define:nn { spintent / spshort }
1116 {
1117     small-caps .bool_set:N = \l__spintent_spshort_smallcaps_bool,
1118     small-caps .initial:n = false,
1119     small-caps .value_required:n = true,
1120     read-arg .str_set:N = \l__spintent_spshort_read_arg_str,
1121     read-arg .initial:n = {},
1122     read-arg .value_required:n = true,
1123     unknown .code:n =
1124     {
1125         \str_set:Nn \l__spintent_command_name_str { spshort }
1126         \__spintent_unknown_keys_cmd:n {#1}

```

```

1127     },
1128 }

```

(End of definition for `small-caps` and `read-arg`.)

11.18.4 Internal functions for `\spshort`

`\__spinttent_spshort_print:nn`

The internal function `\__spinttent_spshort_print:nn` acts as the primary formatter for text abbreviations. It clears the string variable `\__spinttent_spshort_read_arg_str` and evaluates the optional `<keys>` passed in `#1`. It then checks if a manual reading override was provided via those keys. If that string variable remains empty, it calls `\__spinttent_spshort_process_and_render:n` to process the `<argument>` passed in `#2`. Otherwise, if an override is detected, it directly delegates the execution to `\__spinttent_spshort_render_bypass:n`.

```

1129 \cs_new_protected:Nn \__spinttent_spshort_print:nn
1130 {
1131   \str_clear:N \__spinttent_spshort_read_arg_str
1132   \keys_set:nn { spinttent / spshort } {#1}
1133   \str_if_empty:NTF \__spinttent_spshort_read_arg_str
1134   {
1135     \__spinttent_spshort_process_and_render:n {#2}
1136   }
1137   {
1138     \__spinttent_spshort_render_bypass:n {#2}
1139   }
1140 }

```

(End of definition for `\__spinttent_spshort_print:nn`.)

`\__spinttent_spshort_process_and_render:n`

The internal function `\__spinttent_spshort_process_and_render:n` interfaces with the Lua module `spinttent.lua` function `\__spinttent_luafun_spshort_process:n` to resolve the abbreviation or ordinal `<argument>` passed in `#1`.

It subsequently evaluates the execution state stored within the string variable `\__spinttent_spshort_luaset_status` and dynamically routes execution to the corresponding rendering subroutine `success`, `fallback` or `error`. Any unrecognized status automatically defaults to the “error” handler.

```

1141 \cs_new_protected:Nn \__spinttent_spshort_process_and_render:n
1142 {
1143   \__spinttent_luafun_spshort_process:n {#1}
1144   \str_case:VnF \__spinttent_spshort_luaset_status_str
1145   {
1146     { success } { \__spinttent_spshort_render_success: }
1147     { fallback } { \__spinttent_spshort_render_fallback: }
1148     { error } { \__spinttent_spshort_render_error:n {#1} }
1149   }
1150   { \__spinttent_spshort_render_error:n {#1} }
1151 }

```

(End of definition for `\__spinttent_spshort_process_and_render:n`.)

`\__spinttent_spshort_render_success:`

The function `\__spinttent_spshort_render_success:` injects the key `E` and executes the targeted typographical layout. The `\__spinttent_spshort_render_fallback:` function handles default cases, applying optional formatting such as `SMALL CAPS` to unrecognized or partially resolved terms.

`\__spinttent_spshort_render_fallback:`

Conversely, the function `\__spinttent_spshort_render_bypass:n` directly maps the user-defined “*speech*” to the `<argument>`, bypassing the automated lookup Lua module. Finally, the function `\__spinttent_spshort_render_error:n` triggers a structural exception while yielding the unformatted `<argument>` to prevent fatal compilation.

`\__spinttent_spshort_render_bypass:n`

```

1152 \cs_new_protected:Nn \__spinttent_spshort_render_success:
1153 {
1154   \__spinttent_spshort_tag_span:en
1155   { \__spinttent_spshort_luaset_expanded_str }
1156   { \__spinttent_spshort_execute_layout: }
1157 }
1158 \cs_new_protected:Nn \__spinttent_spshort_render_fallback:
1159 {
1160   \__spinttent_spshort_tag_span:en
1161   { \__spinttent_spshort_luaset_expanded_str }
1162   {
1163     \bool_if:NTF \__spinttent_spshort_smallcaps_bool
1164     { \textsc{ \__spinttent_spshort_luaset_output_str } }
1165     { \__spinttent_spshort_luaset_output_str }
1166   }
1167 }
1168 \cs_new_protected:Nn \__spinttent_spshort_render_bypass:n

```

```

1169 {
1170   \__spintent_spshort_tag_span:en { \__spintent_spshort_read_arg_str } { #1 }
1171 }
1172 \cs_new_protected:Nn \__spintent_spshort_render_error:n
1173 {
1174   \msg_error:nnn { spintent } { unknown-abbreviation } {#1}
1175 }

```

(End of definition for __spintent_spshort_render_success: and others.)

__spintent_spshort_execute_layout:

The internal function `\__spintent_spshort_execute_layout:` executes the typographical layout. It evaluates the descriptor “*stored*” within the string variable `\__spintent_spshort_luaset_layout_str` (such as `linear_regular`, `superscript`, or `small_caps_pure`) and maps it to the corresponding $\TeX$ formatting. Furthermore, it manages complex nested configurations, dynamically applying combinations such as superscripts integrated with conditional SMALL CAPS based on the active boolean variables.

```

1176 \cs_new_protected:Nn \__spintent_spshort_execute_layout:
1177 {
1178   \str_case:VnF \__spintent_spshort_luaset_layout_str
1179   {
1180     { linear_regular }
1181     {
1182       \bool_if:NTF \__spintent_spshort_smallcaps_bool
1183       { \textsc{ \__spintent_spshort_luaset_output_str } }
1184       { \__spintent_spshort_luaset_output_str }
1185     }
1186     { linear_caps }
1187     { \__spintent_spshort_luaset_output_str }
1188     { linear_mixed }
1189     { \__spintent_spshort_luaset_output_str }
1190     { superscript }
1191     {
1192       \__spintent_spshort_luaset_base_str
1193       \bool_if:NTF \__spintent_spshort_smallcaps_bool
1194       {
1195         \textsuperscript{
1196           \textsc{
1197             \str_lowercase:V \__spintent_spshort_luaset_suffix_str
1198           }
1199         }
1200       }
1201       {
1202         \textsuperscript{ \__spintent_spshort_luaset_suffix_str }
1203       }
1204     }
1205     { small_caps_pure }
1206     {
1207       \bool_if:NTF \__spintent_spshort_smallcaps_bool
1208       {
1209         \textsc{
1210           \str_lowercase:V \__spintent_spshort_luaset_output_str
1211         }
1212       }
1213       { \__spintent_spshort_luaset_output_str }
1214     }
1215     { acronym_long }
1216     { \__spintent_spshort_luaset_output_str }
1217   }
1218   { \__spintent_spshort_luaset_output_str }
1219 }

```

(End of definition for __spintent_spshort_execute_layout:.)

\spshort

The public document command `\spshort` manages the accessible formatting of *text mode* abbreviations, acronyms, and ordinals. It first triggers an error if invoked within a *math mode*, upon validation, puts `\mode_leave_vertical:` and establishes a local group to call the function `\__spintent_spshort_print:nn` for processing and printing.

```

1220 \NewDocumentCommand \spshort { 0{} m }
1221 {
1222   \mode_if_math:TF
1223   {
1224     \msg_error:nnn { spintent } { need-text-mode } { \spshort }

```

```

1225     }
1226     {
1227         \mode_leave_vertical:
1228         \group_begin:
1229             \__spintent_spsshort_print:nn {#1} {#2}
1230         \group_end:
1231     }
1232 }

```

(End of definition for `\spsshort`. This function is documented on page 7.)

11.19 The command `\spsiglo`

The public document command `\spsiglo` provides a semantic interface for typesetting century designations (e.g., siglo XXI). It accepts both Arabic and Roman numerals as $\langle \textit{argument} \rangle$, interfacing with the Lua module `spintent.lua` to validate, parse, and normalize chronological values.

This command work in dual-mode context awareness, operating uniformly across both *text mode* and *math mode*. When executed within *math mode*, it constructs an accessible MathML intent expression tree. Conversely, when invoked in standard *text mode*, it encapsulates the rendered output within a native PDF tagging structure (such as `/Alt (#1)`) containing an `alt` key.

```

\__spintent_spsiglo_luaset_error_str
\__spintent_spsiglo_luaset_arabic_str
\__spintent_spsiglo_luaset_roman_min_str
\__spintent_spsiglo_print_era_str

```

These internal string variables handle data transfer and synchronization with the Lua module `spintent.lua`. They isolate the exception evaluation states, the Arabic numeral required for speech synthesis, the lowercase Roman numeral designated for small caps typesetting via `\textsc`, and the localized chronological era modifier.

```

1233 \str_new:N \__spintent_spsiglo_luaset_error_str
1234 \str_new:N \__spintent_spsiglo_luaset_arabic_str
1235 \str_new:N \__spintent_spsiglo_luaset_roman_min_str
1236 \str_new:N \__spintent_spsiglo_print_era_str

```

(End of definition for `\__spintent_spsiglo_luaset_error_str` and others.)

11.19.1 Definition of error messages for `\spsiglo`

invalid-century-format

This internal error message is issued when the provided century identifier fails both Arabic and Roman numeral validation. It is triggered when the Lua module `spintent.lua` cannot map the $\langle \textit{argument} \rangle$ to an integer within the valid range of 1 to 4000, preventing incorrect speech synthesis.

```

1237 \msg_new:nnnn { spintent } { invalid-century-format }
1238 {
1239     Invalid~century~format~in~'#1'.
1240 }
1241 {
1242     The~argument~'#2'~is~not~a~valid~Arabic~or~Roman~numeral.~
1243     Ensure~the~value~is~between~1~and~4000.
1244 }

```

(End of definition for `invalid-century-format`.)

11.19.2 Definition of keys for `\spsiglo`

print-era

Now we add the key `print-era`. User configuration determining if an era designation is appended, available values are `none`, `ac` (antes de cristo) and `dc` (después de cristo).

```

1245 \keys_define:nn { spintent / spsiglo }
1246 {
1247     print-era .choices:nn =
1248         { none , ac , dc }
1249     {
1250         \int_case:nn { \_keys_choice_int }
1251         {
1252             {1} { \str_set:Nn \__spintent_spsiglo_print_era_str { none } }
1253             {2} { \str_set:Nn \__spintent_spsiglo_print_era_str { ac } }
1254             {3} { \str_set:Nn \__spintent_spsiglo_print_era_str { dc } }
1255         }
1256     },
1257     print-era .initial:n = none,
1258     print-era .value_required:n = true,
1259     print-era / unknown .code:n =
1260         \msg_error:nneee { spintent } { unknown-choice }
1261         { print-era } { none, ~ac, ~dc } { \exp_not:n {#1} },
1262     unknown .code:n =
1263     {
1264         \str_set:Nn \__spintent_command_name_str { spsiglo }
1265         \__spintent_unknown_keys_cmd:n {#1}
1266     },
1267 }

```

(End of definition for `print-era`.)

11.19.3 Internal functions for `\spsiglo`

The functions `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn` generate MathML structures when century designations are evaluated within *math mode*. They construct MathML intent structures, mapping the Arabic numerals to their phonetic equivalents. Furthermore, `\__spintent_spsiglo_render_with_era:nnn` integrates the era modifiers `ac` or `dc`. It adds invisible spacing and postfix intents to ensure correct NVDA/MathCAT pronunciation.

```

1268 \cs_new_protected:Nn \__spintent_spsiglo_render_simple:nn
1269 {
1270   \MathMLintent { #1:number } { \text { \textsc {#2} } } }
1271 }
1272 \cs_generate_variant:Nn \__spintent_spsiglo_render_simple:nn { VV }
1273 \cs_new_protected:Nn \__spintent_spsiglo_render_with_era:nnn
1274 {
1275   \str_if_eq:nnTF { #1 } { ac }
1276   {
1277     \MathMLintent { antes-de-cristo:postfix($z) }
1278     {
1279       \__spintent_luafun_inner_sep:
1280       \MathMLarg { z }
1281       {
1282         \MathMLintent { #2:number }
1283         {
1284           \text { \textsc {#3}~a.~C. }
1285         }
1286       }
1287     }
1288   }
1289   {
1290     \MathMLintent { después-de-cristo:postfix($z) }
1291     {
1292       \__spintent_luafun_inner_sep:
1293       \MathMLarg { z }
1294       {
1295         \MathMLintent { #2:number }
1296         {
1297           \text { \textsc {#3}~d.~C. }
1298         }
1299       }
1300     }
1301   }
1302 }
1303 \cs_generate_variant:Nn \__spintent_spsiglo_render_with_era:nnn { VVV }

```

(End of definition for `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn`.)

The function `\__spintent_spsiglo_tag_span:nn` handles century formatting within *text mode*. It utilizes the `tagpdf` package to enclose the content inside a PDF /Alt (`#1`) structure. By passing the `{\argument}` to the `alt` key, it embeds the chronological value directly into the PDF, ensuring correct NVDA/MathCAT pronunciation without relying on MathML intents.

```

1304 \cs_new_protected:Nn \__spintent_spsiglo_tag_span:nn
1305 {
1306   \tag_mc_end_push:
1307   \tag_struct_begin:n { tag=Span, alt={ #1 } }
1308   \tag_mc_begin:n { tag=Span }
1309   \tag_suspend:n { \spsiglo }
1310   #2
1311   \tag_resume:n { \spsiglo }
1312   \tag_mc_end:
1313   \tag_struct_end:
1314   \tag_mc_begin_pop:n {}
1315 }
1316 \cs_generate_variant:Nn \__spintent_spsiglo_tag_span:nn { en }

```

(End of definition for `\__spintent_spsiglo_tag_span:nn`.)

The functions `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n` format century designations based on the active mode. Both first check the “error” status in the variable `\__spintent_spsiglo_luaseta_error_str` returned by the Lua module `spintent.lua` and issue an “error” if validation fails.

If the $\langle argument \rangle$ is valid, the function `\__spintent_spsiglo_print_math:n` operates in *math mode*, applying MathML structures depending on whether an era modifier is present. Conversely, the function `\__spintent_spsiglo_print_text:n` operates in *text mode*, utilizing the `tag` and `alt` keys to expand the value of the `print-era` option into full text (e.g., “antes de cristo”) for NVDA/MathCAT.

```

1317 \cs_new_protected:Nn \__spintent_spsiglo_print_math:n
1318 {
1319   \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1320   {
1321     \msg_error:nnnn { spintent } { invalid-century-format }
1322     { \spsiglo } { #1 }
1323   }
1324   {
1325     \str_if_eq:VnTF \l__spintent_spsiglo_print_era_str { none }
1326     {
1327       \__spintent_spsiglo_render_simple:VV
1328       \l__spintent_spsiglo_luaset_arabic_str
1329       \l__spintent_spsiglo_luaset_roman_min_str
1330     }
1331     {
1332       \__spintent_spsiglo_render_with_era:VVV
1333       \l__spintent_spsiglo_print_era_str
1334       \l__spintent_spsiglo_luaset_arabic_str
1335       \l__spintent_spsiglo_luaset_roman_min_str
1336     }
1337   }
1338 }
1339 \cs_new_protected:Nn \__spintent_spsiglo_print_text:n
1340 {
1341   \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
1342   {
1343     \msg_error:nnnn { spintent } { invalid-century-format }
1344     { \spsiglo } { #1 }
1345   }
1346   {
1347     \str_case:Vn \l__spintent_spsiglo_print_era_str
1348     {
1349       { none }
1350       {
1351         \__spintent_spsiglo_tag_span:en
1352         { \l__spintent_spsiglo_luaset_arabic_str }
1353         { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } }
1354       }
1355       { ac }
1356       {
1357         \__spintent_spsiglo_tag_span:en
1358         { \l__spintent_spsiglo_luaset_arabic_str ~ antes ~ de ~ cristo }
1359         { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ a.~C. }
1360       }
1361       { dc }
1362       {
1363         \__spintent_spsiglo_tag_span:en
1364         { \l__spintent_spsiglo_luaset_arabic_str ~ después ~ de ~ cristo }
1365         { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ d.~C. }
1366       }
1367     }
1368   }
1369 }

```

(End of definition for `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n`)

`\spsiglo` The public command `\spsiglo` is the main interface for century typesetting. It opens a local group and evaluates any optional $\langle keys \rangle$ passed in ‘#1’, then executes the function `\__spintent_luafun_spsiglo_parse:n` defined in the Lua module `spintent.lua` passed in ‘#2’ and formats the output for *math mode* or *text mode*.

```

1370 \NewDocumentCommand \spsiglo { 0{} m }
1371 {
1372   \group_begin:
1373   \keys_set:nn { spintent / spsiglo } { #1 }
1374   \__spintent_luafun_spsiglo_parse:n { #2 }
1375   \mode_if_math:TF
1376   { \__spintent_spsiglo_print_math:n { #2 } }
1377   {

```

```

1378         \mode_leave_vertical:
1379         \__spintent_spsiglo_print_text:n { #2 }
1380     }
1381     \group_end:
1382 }

```

(End of definition for \spsiglo. This function is documented on page 11.)

11.20 The command \sptime

The command `\sptime` is the main public interface for typesetting and tagging time expressions. It parses time string formats (such as HH:MM or HH:MM:SS) using the Lua function `\__spintent_luafun_sptime_parse:n`. This separates the hours, minutes, seconds, and fractions to construct a MathML *intent* tree for NVDA/MathCAT.

```

\__spintent_sptime_luaset_seconds_str
\__spintent_sptime_luaset_has_seconds_str
\__spintent_sptime_luaset_error_str
\__spintent_sptime_luaset_is_fraction_str
\__spintent_sptime_luaset_base_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted time units, error flags, and boolean indicators (such as the presence of seconds or decimal fractions) used during formatting.

```

1383 \str_new:N \__spintent_sptime_luaset_seconds_str
1384 \str_new:N \__spintent_sptime_luaset_has_seconds_str
1385 \str_new:N \__spintent_sptime_luaset_error_str
1386 \str_new:N \__spintent_sptime_luaset_is_fraction_str
1387 \str_new:N \__spintent_sptime_luaset_base_str

```

(End of definition for __spintent_sptime_luaset_seconds_str and others.)

11.20.1 Definition of error messages

```
invalid-time-format
```

The error message `invalid-time-format` defines the text displayed when an invalid time input is detected. It is triggered when an argument fails the format checks or contains values outside normal ranges, such as hours exceeding 23 or minutes and seconds exceeding 59.

```

1388 \msg_new:nnnn { spintent } { invalid-time-format }
1389 {
1390     Invalid~time~format~in~#1. \\\
1391     The~argument~'#2'~does~not~match~any~supported~time~format.
1392 }
1393 {
1394     Allowed~formats:~HH:MM,~HH:MM:SS,~or~HH:MM:SS.ss. \\\
1395     Please~verify~that~hours~are~0-23~and~minutes/seconds~are~0-59.
1396 }

```

(End of definition for invalid-time-format.)

11.20.2 Internal functions for \sptime

```
\__spintent_sptime_build_seconds:nn
```

The internal function `\__spintent_sptime_build_seconds:nn` builds the MathML structure for the seconds component of a time expression. It wraps the numeric value ‘#2’ in speech intent tags, applying a connector prefix (passed in ‘#1’) and the “segundos” postfix so it is read correctly by NVDA/MathCAT.

```

1397 \cs_new_protected:Nn \__spintent_sptime_build_seconds:nn
1398 {
1399     \MathMLintent { #1:prefix($z) }
1400     {
1401         \text { : }
1402         \MathMLarg { z }
1403         {
1404             \MathMLintent { segundos:postfix($m) }
1405             {
1406                 \__spintent_luafun_inner_sep:
1407                 \MathMLarg { m }
1408                 {
1409                     \text { #2 }
1410                 }
1411             }
1412         }
1413     }
1414 }

```

(End of definition for __spintent_sptime_build_seconds:nn.)

```
\__spintent_sptime_build_tree:
```

The internal function `\__spintent_sptime_build_tree:` builds the complete MathML structure for the time expression. It checks the string variables `\__spintent_sptime_luaset_has_seconds_str` and `\__spintent_sptime_luaset_is_fraction_str` to determine if the time includes seconds or decimal fractions. Based on these values, it inserts the base time string and either calls `\__spintent_sptime_build_seconds:`

with the appropriate prefix ('con' or 'minutos-con'), or simply appends the 'minutos' intent tag if no seconds are present.

```

1415 \cs_new_protected:Nn \__spintent_sptime_build_tree:
1416 {
1417   \str_if_eq:VnTF \l__spintent_sptime_luaset_has_seconds_str { true }
1418   {
1419     \MathMLintent { :time }
1420     {
1421       \text { \l__spintent_sptime_luaset_base_str }
1422     }
1423     \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { true }
1424     {
1425       \__spintent_sptime_build_seconds:nn { con }
1426       { \l__spintent_sptime_luaset_seconds_str }
1427     }
1428     {
1429       \__spintent_sptime_build_seconds:nn { minutos-con }
1430       { \l__spintent_sptime_luaset_seconds_str }
1431     }
1432   }
1433   {
1434     \MathMLintent { :time }
1435     {
1436       \text { \l__spintent_sptime_luaset_base_str }
1437     }
1438     \str_if_eq:VnT \l__spintent_sptime_luaset_is_fraction_str { false }
1439     {
1440       \MathMLintent { minutos }
1441       {
1442         \__spintent_invisible_sep:
1443       }
1444     }
1445   }
1446 }

```

(End of definition for __spintent_sptime_build_tree:.)

\sptime The command `\sptime` is the public interface for formatting time expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it parses the input using `\__spintent_luafun_sptime_parse:n` and checks the error status in `\l__spintent_sptime_luaset_error_str`. If an invalid format is detected, it issues the `invalid-time-format` error; otherwise, it calls `\__spintent_sptime_build_tree:` to assemble the MathML output.

```

1447 \NewDocumentCommand \sptime { m }
1448 {
1449   \mode_if_math:TF
1450   {
1451     \group_begin:
1452     \__spintent_luafun_sptime_parse:n { #1 }
1453     \str_if_eq:VnTF \l__spintent_sptime_luaset_error_str { true }
1454     {
1455       \msg_error:nnnn { spintent } { invalid-time-format }
1456       { \sptime } { #1 }
1457     }
1458     {
1459       \__spintent_sptime_build_tree:
1460     }
1461     \group_end:
1462   }
1463   { \msg_error:nnn { spintent } { need-math-mode } { \sptime } }
1464 }

```

(End of definition for \sptime. This function is documented on page 11.)

11.21 The command \spdate

The command `\spdate` is the main public command for typesetting and tagging date expressions. It parses date string formats (such as DD/MM/YYYY or YYYY-MM-DD) using the Lua function `\__spintent_luafun_spdate_parse:n`. This verifies the input and constructs a MathML `intent` tree for NVDA/MathCAT.

`\l__spintent_spdate_luaset_output_str`
`\l__spintent_spdate_luaset_error_str`

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the formatted date string and the error status used during formatting.

```

1465 \str_new:N \l__spintent_spdate_luaset_output_str

```

```
1466 \str_new:N \__spintent_spdate_luaset_error_str
```

(End of definition for `\__spintent_spdate_luaset_output_str` and `\__spintent_spdate_luaset_error_str`.)

11.21.1 Definition of error messages

invalid-date-format

The error message `invalid-date-format` defines the text displayed when an invalid date input is detected. It is triggered when an argument fails the format checks or represents an invalid calendar date, such as `31/02/2024` or using an unsupported pattern like `YYYY/MM/DD`.

```
1467 \msg_new:nnnn { spintent } { invalid-date-format }
1468 {
1469   Invalid~date~or~unsupported~format~in~#1. \\\
1470   The~argument~'#2'~does~not~match~a~supported~format.
1471 }
1472 {
1473   Allowed~formats:~DD\slash MM\slash YYYY,~DD-MM-YYYY,~or~YYYY-MM-DD. \\\
1474   Ensure~the~day~and~month~are~valid~(e.g.,~avoid~31\slash 02\slash 2024).
1475 }
```

(End of definition for `invalid-date-format`.)

11.21.2 Internal functions for `\spdate`

`\__spintent_spdate_process:n`

The internal function `\__spintent_spdate_process:n` parses the input using `\__spintent_luafun_spdate_parse:n` and checks the error status in `\__spintent_spdate_luaset_error_str`. If an invalid format is detected, it issues the `invalid-date-format` error. Otherwise, it builds the `MathML intent` structure using the formatted date stored in `\__spintent_spdate_luaset_output_str`.

```
1476 \cs_new_protected:Nn \__spintent_spdate_process:n
1477 {
1478   \__spintent_luafun_spdate_parse:n { #1 }
1479   \str_if_eq:VnTF \__spintent_spdate_luaset_error_str { true }
1480   {
1481     \msg_error:nnnn { spintent } { invalid-date-format }
1482     { \spdate } { #1 }
1483   }
1484   {
1485     \MathMLintent { :date }
1486     {
1487       \text { \__spintent_spdate_luaset_output_str }
1488     }
1489   }
1490 }
```

(End of definition for `\__spintent_spdate_process:n`.)

`\spdate`

The command `\spdate` is the public interface for formatting date expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spdate_process:n` to process the input.

```
1491 \NewDocumentCommand \spdate { m }
1492 {
1493   \mode_if_math:TF
1494   {
1495     \group_begin:
1496     \__spintent_spdate_process:n { #1 }
1497     \group_end:
1498   }
1499   { \msg_error:nnn { spintent } { need-math-mode } { \spdate } }
1500 }
```

(End of definition for `\spdate`. This function is documented on page 11.)

11.22 Semantic Operators

This section provides purely semantic operators. Unlike their formatting counterparts (which parse and format numbers), these commands do not process mandatory arguments. Instead, they solely inject a mathematical glyph wrapped in its corresponding regional `MathCAT` intent. This grants teachers absolute control when building complex algebraic expressions, mixed numbers, or fraction divisions.

11.23 The command `\spusep`

For the use of parentheses, we will provide the command `\spusep` with only two *⟨keys⟩* `size` and `silent`. Most of the time we can use `\left` and `\right` or `\Uleft` and `\Uright`, but both forms create a group, and this hinders the code for *tagged* PDF when we want to interrupt something between them.

`spusep-empty-arg`
`spusep-invalid-arg`

The first “error” message `spusep-empty-arg` is raised when `\spusep` receives an empty or blank *⟨argument⟩*, the second this is when an input is invalid.

```

1501 \msg_new:nnnn { spintent } { spusep-empty-arg }
1502 {
1503   Empty~argument~'#1'~for~\token_to_str:N \spusep.
1504 }
1505 {
1506   \token_to_str:N \spusep~requires~a~non-empty~argument.
1507 }
1508 \msg_new:nnn { spintent } { spusep-invalid-arg }
1509 {
1510   Invalid~argument~'#1'~for~\token_to_str:N \spusep.
1511 }

```

(End of definition for `spusep-empty-arg` and `spusep-invalid-arg`.)

`\l__spintent_spusep_arg_tl`
`\l__spintent_spusep_left_tl`
`\l__spintent_spusep_right_tl`

The variable `\l__spintent_spusep_arg_tl` stores the *⟨argument⟩*, the variable `\l__spintent_spusep_left_tl` and `\l__spintent_spusep_right_tl` stores the parentheses.

```

1512 \tl_new:N \l__spintent_spusep_arg_tl
1513 \tl_new:N \l__spintent_spusep_left_tl
1514 \tl_new:N \l__spintent_spusep_right_tl

```

(End of definition for `\l__spintent_spusep_arg_tl`, `\l__spintent_spusep_left_tl`, and `\l__spintent_spusep_right_tl`.)

`size`
`silent`
`unknown`

We define the *⟨keys⟩* `size` and `silent`.

```

1515 \keys_define:nn { spintent / spusep }
1516 {
1517   size .choices:nn =
1518     { none , big , Big , bigg , Bigg }
1519     {
1520       \int_case:nn { \l_keys_choice_int }
1521       {
1522         { 1 }
1523         {
1524           \tl_clear:N \l__spintent_spusep_left_tl
1525           \tl_clear:N \l__spintent_spusep_right_tl
1526         }
1527         { 2 }
1528         {
1529           \tl_set:Nn \l__spintent_spusep_left_tl { \bigl }
1530           \tl_set:Nn \l__spintent_spusep_right_tl { \bigr }
1531         }
1532         { 3 }
1533         {
1534           \tl_set:Nn \l__spintent_spusep_left_tl { \Bigl }
1535           \tl_set:Nn \l__spintent_spusep_right_tl { \Bigr }
1536         }
1537         { 4 }
1538         {
1539           \tl_set:Nn \l__spintent_spusep_left_tl { \biggl }
1540           \tl_set:Nn \l__spintent_spusep_right_tl { \biggr }
1541         }
1542         { 5 }
1543         {
1544           \tl_set:Nn \l__spintent_spusep_left_tl { \Biggl }
1545           \tl_set:Nn \l__spintent_spusep_right_tl { \Biggr }
1546         }
1547       }
1548     },
1549   size .initial:n = none,
1550   size .value_required:n = true,
1551   size / unknown .code:n =
1552     \msg_error:nneee { spintent } { unknown-choice }
1553     { size } { none,~big,~Big,~bigg,~Bigg } { \exp_not:n {#1} },
1554   silent .bool_set:N = \l__spintent_spusep_silent_bool,
1555   silent .initial:n = false,

```

```

1556     silent           .value_required:n = true,
1557     unknown          .code:n =
1558     {
1559         \str_set:Nn \__spintent_command_name_str { spusep }
1560         \__spintent_unknown_keys_cmd:n {#1}
1561     },
1562 }

```

(End of definition for `size`, `silent`, and `unknown`.)

`\__spintent_spusep_set_parens:nn`

The function `\__spintent_spusep_set_parens:nn` will place the argument passed in ‘#2’ to the right of the variables `\__spintent_spusep_left_tl` and `\__spintent_spusep_right_tl` and then check the state of the variable `\__spintent_spusep_silent_bool` handled by the key `silent` to print.

```

1563 \cs_new_protected:Nn \__spintent_spusep_set_parens:nn
1564 {
1565     \tl_put_right:cn { \__spintent_spusep_ #1 _tl } { #2 }
1566     \bool_if:NTF \__spintent_spusep_silent_bool
1567     {
1568         \MathMLintenc { :silent } { \tl_use:c { \__spintent_spusep_ #1 _tl } }
1569     }
1570     {
1571         \tl_use:c { \__spintent_spusep_ #1 _tl }
1572     }
1573 }

```

(End of definition for `\__spintent_spusep_set_parens:nn`.)

`\__spintent_spusep_convert:n`

The function `\__spintent_spusep_convert:n` captures and stores the $\langle argument \rangle$ passed in ‘#1’ in the variable `\__spintent_spusep_arg_tl`, validates the cases by converting them to the macros provided by `unicode-math`, and passes them to the function `\__spintent_spusep_set_parens:nn` for printing. If the input is not supported or is empty, we return an “error”.

```

1574 \cs_new_protected:Nn \__spintent_spusep_convert:n
1575 {
1576     \tl_set:Ne \__spintent_spusep_arg_tl { \tl_trim_spaces:n {#1} }
1577     \tl_if_empty:NT \__spintent_spusep_arg_tl
1578     {
1579         \msg_error:nnn { spintenc } { spusep-empty-arg } {#1}
1580     }
1581     \str_case:VnF \__spintent_spusep_arg_tl
1582     {
1583         { ( }
1584         {
1585             \__spintenc_spusep_set_parens:nn { left } { \lparen }
1586         }
1587         { ) }
1588         {
1589             \__spintenc_spusep_set_parens:nn { right } { \rparen }
1590         }
1591         { [ ]
1592         {
1593             \__spintenc_spusep_set_parens:nn { left } { \lbrack }
1594         }
1595         { ] }
1596         {
1597             \__spintenc_spusep_set_parens:nn { right } { \rbrack }
1598         }
1599         { \{ }
1600         {
1601             \__spintenc_spusep_set_parens:nn { left } { \lbrace }
1602         }
1603         { \} }
1604         {
1605             \__spintenc_spusep_set_parens:nn { right } { \rbrace }
1606         }
1607     }
1608     {
1609         \msg_error:nnn { spintenc } { spusep-invalid-arg } {#1}
1610     }
1611 }

```

(End of definition for `\__spintenc_spusep_convert:n`.)

`\spusep` The command `\spusep` checks that it is running in math mode, opens a local group, sets the keys passed in ‘#1’ and processes the $\langle argument \rangle$ passed in ‘#2’ by means of the function `\__spintent_spusep_convert:n`.

```

1612 \NewDocumentCommand \spusep { O{} m }
1613 {
1614   \mode_if_math:TF
1615   {
1616     \group_begin:
1617     \keys_set:nn { spintent / spusep } {#1}
1618     \__spintent_spusep_convert:n {#2}
1619     \group_end:
1620   }
1621   { \msg_error:nnn { spintent } { need-math-mode } { \spusep } }
1622 }

```

(End of definition for `\spusep`. This function is documented on page 12.)

11.23.1 The command `\spread`

The command `\spread` provides a semantic read for MathCAT of mathematical symbols for situations in which it is necessary to specify gender or plurality in Spanish.

`spread-empty-args` The error message `spread-empty-args` is raised when `\spread` receives an empty or blank $\langle argument \rangle$.

```

1623 \msg_new:nnnn { spintent } { spread-empty-args }
1624 { Empty~argument~in~\token_to_str:N \spread. }
1625 { \token_to_str:N \spread~requires~two~non-empty~arguments. }

```

(End of definition for `spread-empty-args`.)

`\l__spintent_spread_intent_tl` The variable `\l__spintent_spread_intent_tl` stores the sanitized semantic reading $\langle argument \rangle$ for symbol to be injected into the `\MathMLintent`.

```

1626 \tl_new:N \l__spintent_spread_intent_tl

```

(End of definition for `\l__spintent_spread_intent_tl`.)

`\__spintent_spread_exec:nn` The function `\__spintent_spread_exec:nn` implements the core logic for `\spread`. It verifies that neither $\langle argument \rangle$ is empty or blank. If valid, it evaluates the *reading* $\langle argument \rangle$ ‘#1’ using `\__spintent_sanitize_affix:Nn` and embeds it via `\MathMLintent` into the *math symbol* ‘#2’.

```

1627 \cs_new_protected:Nn \__spintent_spread_exec:nn
1628 {
1629   \bool_lazy_or:nnTF { \tl_if_blank_p:n { #1 } } { \tl_if_blank_p:n { #2 } }
1630   { \msg_error:nn { spintent } { spread-empty-args } }
1631   {
1632     \__spintent_sanitize_affix:Nn \l__spintent_spread_intent_tl { #1 }
1633     \MathMLintent { \l__spintent_spread_intent_tl } { #2 }
1634   }
1635 }

```

(End of definition for `\__spintent_spread_exec:nn`.)

`\spread` The command `\spread` assigns a custom semantic *reading* ‘#1’ to any mathematical *symbol* ‘#2’. It enforces math mode and passes execution to `\__spintent_spread_exec:nn`.

```

1636 \NewDocumentCommand \spread { m m }
1637 {
1638   \mode_if_math:TF
1639   {
1640     \group_begin:
1641     \__spintent_spread_exec:nn { #1 } { #2 }
1642     \group_end:
1643   }
1644   { \msg_error:nnn { spintent } { need-math-mode } { \spread } }
1645 }

```

(End of definition for `\spread`. This function is documented on page 11.)

11.23.2 The command \spdsym

Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spdsym`.

```
\l__spintent_spdsym_read_sym_tl
\l__spintent_spdsym_symbol_tl
\l__spintent_spdsym_prefix_tl
\l__spintent_spdsym_suffix_tl
```

(End of definition for `\l__spintent_spdsym_read_sym_tl` and others.)

```
prefix We define the (keys) prefix, suffix, set-sym and read-sym for \spdsym.
suffix
set-sym
read-sym
unknown
```

```
1650 \keys_define:nn { spintent / spdsym }
1651 {
1652   prefix .code:n =
1653     { \l__spintent_sanitiz_ affix:Nn \l__spintent_spdsym_prefix_tl {#1} },
1654   prefix .initial:n = ,
1655   prefix .value_required:n = true,
1656   suffix .code:n =
1657     { \l__spintent_sanitiz_ affix:Nn \l__spintent_spdsym_suffix_tl {#1} },
1658   suffix .initial:n = ,
1659   suffix .value_required:n = true,
1660   set-sym .choices:nn =
1661     { old-style , two-dots , slash }
1662   {
1663     \int_case:nn { \l_keys_choice_int }
1664     {
1665       {1} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { \div } }
1666       {2} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { : } }
1667       {3} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { / } }
1668     }
1669   },
1670   set-sym .initial:n = two-dots,
1671   set-sym .value_required:n = true,
1672   set-sym / unknown .code:n =
1673     \msg_error:nneee { spintent } { unknown-choice }
1674     { set-sym } { old-style, ~two-dots, ~slash } { \exp_not:n {#1} },
1675   read-sym .choices:nn =
1676     { dividido , dividido-por , dividido-entre }
1677   {
1678     \int_case:nn { \l_keys_choice_int }
1679     {
1680       {1} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido } }
1681       {2} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido-por } }
1682       {3} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido-entre } }
1683     }
1684   },
1685   read-sym .initial:n = dividido,
1686   read-sym .value_required:n = true,
1687   read-sym / unknown .code:n =
1688     \msg_error:nneee { spintent } { unknown-choice }
1689     { read-sym } { dividido, ~dividido-por, ~dividido-entre }
1690     { \exp_not:n {#1} },
1691   unknown .code:n =
1692   {
1693     \str_set:Nn \l__spintent_command_name_str { spdsym }
1694     \l__spintent_unknown_keys_cmd:n {#1}
1695   },
1696 }
```

(End of definition for `prefix` and others.)

`\spdsym` Defines the `\spdsym` operator. It takes a single optional argument for local setup and outputs the configured intent structure, safely attaching prefixes and suffixes if present.

```
1697 \NewDocumentCommand \spdsym { 0{ } }
1698 {
1699   \mode_if_math:TF
1700   {
1701     \group_begin:
1702     \keys_set:nn { spintent / spdsym } { #1 }
1703     \MathMLintent
1704     {
```

```

1705         \tl_if_empty:NF \l__spintent_spdsym_prefix_tl
1706         { \l__spintent_spdsym_prefix_tl - }
1707         \l__spintent_spdsym_read_sym_tl
1708         \tl_if_empty:NF \l__spintent_spdsym_suffix_tl
1709         { - \l__spintent_spdsym_suffix_tl }
1710     }
1711     { \l__spintent_spdsym_symbol_tl }
1712 \group_end:
1713 }
1714 { \msg_error:nnn { spintent } { need-math-mode } { \spdsym } }
1715 }

```

(End of definition for `\spdsym`. This function is documented on page 12.)

11.23.3 The command `\spmsym`

Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spmsym`.

```

\l__spintent_spmsym_read_sym_tl
\l__spintent_spmsym_symbol_tl
\l__spintent_spmsym_prefix_tl
\l__spintent_spmsym_suffix_tl
1716 \tl_new:N \l__spintent_spmsym_read_sym_tl
1717 \tl_new:N \l__spintent_spmsym_symbol_tl
1718 \tl_new:N \l__spintent_spmsym_prefix_tl
1719 \tl_new:N \l__spintent_spmsym_suffix_tl

```

(End of definition for `\l__spintent_spmsym_read_sym_tl` and others.)

```

prefix    We define the (keys) prefix, suffix, set-sym, read-sym and not-print for \spmsym.
suffix
set-sym   1720 \keys_define:nn { spintent / spmsym }
          {
read-sym  1721 {
          prefix          .code:n =
          { \l__spintent_sanitizize_affix:Nn \l__spintent_spmsym_prefix_tl {#1} },
not-print 1722 prefix          .initial:n = ,
          1723 prefix          .value_required:n = true,
          1724 suffix          .code:n =
          1725 { \l__spintent_sanitizize_affix:Nn \l__spintent_spmsym_suffix_tl {#1} },
          1726 suffix          .initial:n = ,
          1727 suffix          .value_required:n = true,
          1728 set-sym          .choices:nn =
          1729 { cross , dot , asterisk }
          1730 {
          1731     \int_case:nn { \l_keys_choice_int }
          1732     {
          1733         {1} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \times } }
          1734         {2} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \cdot } }
          1735         {3} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \ast } }
          1736     }
          1737 },
          1738 set-sym          .initial:n = dot,
          1739 set-sym          .value_required:n = true,
          1740 set-sym / unknown .code:n =
          1741     \msg_error:nneee { spintent } { unknown-choice }
          1742     { set-sym } { cross, ~dot, ~asterisk } { \exp_not:n {#1} },
          1743 read-sym          .choices:nn =
          1744     { por , multiplicado-por , veces }
          1745     {
          1746         \int_case:nn { \l_keys_choice_int }
          1747         {
          1748             {1} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { por } }
          1749             {2} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { multiplicado-por } }
          1750             {3} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { veces } }
          1751         }
          1752     },
          1753 read-sym          .initial:n = por,
          1754 read-sym          .value_required:n = true,
          1755 read-sym / unknown .code:n =
          1756     \msg_error:nneee { spintent } { unknown-choice }
          1757     { read-sym } { por, ~multiplicado-por, ~veces } { \exp_not:n {#1} },
          1758 not-print          .bool_set:N = \l__spintent_spmsym_not_print_bool,
          1759 not-print          .initial:n = false,
          1760 not-print          .value_required:n = true,
          1761 unknown          .code:n =
          1762     {
          1763         \str_set:Nn \l__spintent_command_name_str { spmsym }
          1764         \__spintent_unknown_keys_cmd:n {#1}
          1765     }
          1766 }

```

```

1767     },
1768 }

```

(End of definition for *prefix* and others.)

\spmsym Defines the **\spmsym** operator. Similar to **\spdsym**, it outputs the correct mathematical glyph based on the local *⟨keys⟩*, safely attaching prefixes and suffixes if present.

```

1769 \NewDocumentCommand \spmsym { 0{} }
1770 {
1771   \mode_if_math:TF
1772   {
1773     \group_begin:
1774     \keys_set:nn { spintent / spmsym } { #1 }
1775     \MathMLintent
1776     {
1777       \tl_if_empty:NF \l__spintent_spmsym_prefix_tl
1778       { \l__spintent_spmsym_prefix_tl - }
1779       \l__spintent_spmsym_read_sym_tl
1780       \tl_if_empty:NF \l__spintent_spmsym_suffix_tl
1781       { - \l__spintent_spmsym_suffix_tl }
1782     }
1783     {
1784       \bool_if:NTF \l__spintent_spmsym_not_print_bool
1785       {
1786         \invisibletimes
1787       }
1788       { \l__spintent_spmsym_symbol_tl }
1789     }
1790     \group_end:
1791   }
1792   { \msg_error:nnn { spintent } { need-math-mode } { \spmsym } }
1793 }

```

(End of definition for *\spmsym*. This function is documented on page 12.)

11.24 The command **\spdiv**

The user-level command **\spdiv** formats division representations. It takes two mandatory arguments: the *⟨dividend⟩* and the *⟨divisor⟩*. Both *⟨arguments⟩* are internally processed by the **\spnum** command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the *⟨dividend⟩* nor the *⟨divisor⟩* is *blank*.

11.24.1 Definition of keys for **\spdiv**

We define the *⟨keys⟩* **wrap-parens** and **read-parens** for **\spdiv** command. Number-related *⟨keys⟩* **cifras-sep**, **read-sign**, **read-period-sep** and **notation-sym** are forwarded to **\spnum** command using **.meta:nn**, while semantic operator *⟨keys⟩* **set-sym** and **read-sym** are forwarded to the **\spdsym** command using **.meta:nn**.

```

1794 \keys_define:nn { spintent / spdiv }
1795 {
1796   cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1797   read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1798   read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1799   notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1800   set-sym         .meta:nn = { spintent / spdsym } { set-sym         = {#1} },
1801   read-sym        .meta:nn = { spintent / spdsym } { read-sym        = {#1} },
1802   wrap-parens     .bool_set:N = \l__spintent_spdiv_wrap_parens_bool,
1803   wrap-parens     .initial:n = false,
1804   wrap-parens     .value_required:n = true,
1805   read-parens     .bool_set:N = \l__spintent_spdiv_read_parens_bool,
1806   read-parens     .initial:n = true,
1807   read-parens     .value_required:n = true,
1808   unknown         .code:n =
1809   {
1810     \str_set:Nn \l__spintent_command_name_str { spdiv }
1811     \l__spintent_unknown_keys_cmd:n {#1}
1812   },
1813 }

```

(End of definition for *cifras-sep* and others.)

11.24.2 Definition of error messages

spdiv-empty-argument

Raised when either the $\langle\textit{dividend}\rangle$ or the $\langle\textit{divisor}\rangle$ is missing or *blank*.

```

1814 \msg_new:nnnn { spintent } { spdiv-empty-argument }
1815 { The~arguments~for~\token_to_str:N \spdiv~cannot~be~empty. }
1816 {
1817   You~must~provide~both~a~dividend~and~a~divisor.\\
1818 }

```

(End of definition for spdiv-empty-argument.)

11.24.3 Internal functions for \spdiv

__spintent_spdiv_render_op:nn

The function `\__spintent_spdiv_render_op:nn` parses both $\langle\textit{arguments}\rangle$ through the `\spnum` command and embeds the semantic operator from `\spdsym`.

```

1819 \cs_new_protected:Nn \__spintent_spdiv_render_op:nn
1820 {
1821   \spnum {#1}
1822   \MathMLintent { \__spintent_spdsym_read_sym_tl }
1823   {
1824     \__spintent_spdsym_symbol_tl
1825   }
1826   \spnum {#2}
1827 }

```

(End of definition for __spintent_spdiv_render_op:nn.)

__spintent_spdiv_render_parens:nn

The function `\__spintent_spdiv_render_parens:nn` handles the structural parentheses around the division.

```

1828 \cs_new_protected:Nn \__spintent_spdiv_render_parens:nn
1829 {
1830   \bool_if:NTF \__spintent_spdiv_read_parens_bool
1831   {
1832     ( \__spintent_spdiv_render_op:nn {#1} {#2} )
1833   }
1834   {
1835     \MathMLintent { :silent } { ( }
1836     \__spintent_spdiv_render_op:nn {#1} {#2}
1837     \MathMLintent { :silent } { ) }
1838   }
1839 }

```

(End of definition for __spintent_spdiv_render_parens:nn.)

__spintent_spdiv_process:nnn

The function `\__spintent_spdiv_process:nnn` sets $\langle\textit{keys}\rangle$, validates $\langle\textit{arguments}\rangle$, and decides on parenthesis wrapping.

```

1840 \cs_new_protected:Nn \__spintent_spdiv_process:nnn
1841 {
1842   \keys_set:nn { spintent / spdiv } { #1 }
1843   \tl_if_blank:nTF { #2 }
1844   { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1845   {
1846     \tl_if_blank:nTF { #3 }
1847     { \msg_error:nnn { spintent } { spdiv-empty-argument } }
1848     {
1849       \bool_if:NTF \__spintent_spdiv_wrap_parens_bool
1850       { \__spintent_spdiv_render_parens:nn {#2} {#3} }
1851       { \__spintent_spdiv_render_op:nn {#2} {#3} }
1852     }
1853   }
1854 }

```

(End of definition for __spintent_spdiv_process:nnn.)

\spdiv

Defines the user command `\spdiv`. It strictly relies on *math mode*.

```

1855 \NewDocumentCommand \spdiv { 0{ } m m }
1856 {
1857   \mode_if_math:TF
1858   {
1859     \group_begin:
1860     \__spintent_spdiv_process:nnn { #1 } { #2 } { #3 }
1861     \group_end:
1862   }
1863   { \msg_error:nnn { spintent } { need-math-mode } { \spdiv } }
1864 }

```

(End of definition for `\spd\div`. This function is documented on page 13.)

11.25 The command `\spmul`

The user-level command `\spmul` formats multiplication representations. It takes two mandatory arguments: the `{\langle first factor \rangle}` and the `{\langle second factor \rangle}`. Both `{\langle arguments \rangle}` are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the `{\langle first factor \rangle}` nor the `{\langle second factor \rangle}` is *blank*.

11.25.1 Definition of keys for `\spmul`

We define the `\keys` wrap-parens and read-parens for `\spmul` command. Number-related `\keys` `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator `\keys` `set-sym` and `read-sym` are forwarded to the `\spmsym` command using `.meta:nn`.

```

1865 \keys_define:nn { spintent / spmul }
1866 {
1867   cifras-sep      .meta:nn = { spintent / spnum } { cifras-sep      = {#1} },
1868   read-sign       .meta:nn = { spintent / spnum } { read-sign       = {#1} },
1869   read-period-sep .meta:nn = { spintent / spnum } { read-period-sep = {#1} },
1870   notation-sym    .meta:nn = { spintent / spnum } { notation-sym    = {#1} },
1871   set-sym         .meta:nn = { spintent / spmsym } { set-sym         = {#1} },
1872   read-sym        .meta:nn = { spintent / spmsym } { read-sym        = {#1} },
1873   wrap-parens     .bool_set:N = \l__spintent_spmul_wrap_parens_bool,
1874   wrap-parens     .initial:n  = false,
1875   wrap-parens     .value_required:n = true,
1876   read-parens     .bool_set:N = \l__spintent_spmul_read_parens_bool,
1877   read-parens     .initial:n  = true,
1878   read-parens     .value_required:n = true,
1879   unknown         .code:n =
1880     {
1881       \str_set:Nn \l__spintent_command_name_str { spmul }
1882       \__spintent_unknown_keys_cmd:n {#1}
1883     },
1884 }

```

(End of definition for `cifras-sep` and others.)

11.25.2 Definition of error messages

`spmul-empty-argument` Raised when either the `{\langle first factor \rangle}` or the `{\langle second factor \rangle}` is missing or *blank*.

```

1885 \msg_new:nnnn { spintent } { spmul-empty-argument }
1886 { The~arguments~for~\token_to_str:N \spmul~cannot~be~empty. }
1887 {
1888   You~must~provide~both~first~factor~and~second~factor.\\
1889 }

```

(End of definition for `spmul-empty-argument`.)

11.25.3 Internal functions for `\spmul`

`\__spintent_spmul_render_op:nn` The function `\__spintent_spmul_render_op:nn` parses both `{\langle arguments \rangle}` through the `\spnum` command and embeds the semantic operator from `\spmsym`.

```

1890 \cs_new_protected:Nn \__spintent_spmul_render_op:nn
1891 {
1892   \spnum {#1}
1893   \MathMLintent { \l__spintent_spmsym_read_sym_tl }
1894   {
1895     \l__spintent_spmsym_symbol_tl
1896   }
1897   \spnum {#2}
1898 }

```

(End of definition for `\__spintent_spmul_render_op:nn`.)

`\__spintent_spmul_render_parens:nn` The function `\__spintent_spmul_render_parens:nn` handles the structural parentheses around the multiplication.

```

1899 \cs_new_protected:Nn \__spintent_spmul_render_parens:nn
1900 {
1901   \bool_if:NTF \l__spintent_spmul_read_parens_bool
1902   {
1903     ( \__spintent_spmul_render_op:nn {#1} {#2} )
1904   }
1905   {
1906     \MathMLintent { :silent } { ( }

```

```

1907     \__spintent_spmul_render_op:nn {#1} {#2}
1908     \MathMLintent { :silent } { ) }
1909   }
1910 }

```

(End of definition for __spintent_spmul_render_parens:nn.)

__spintent_spmul_process:nnn

The function `\__spintent_spmul_process:nnn` sets `\keys`, validates `\{arguments\}`, and decides on parenthesis wrapping.

```

1911 \cs_new_protected:Nn \__spintent_spmul_process:nnn
1912 {
1913   \keys_set:nn { spintent / spmul } { #1 }
1914   \tl_if_blank:nTF { #2 }
1915   { \msg_error:nnn { spintent } { spmul-empty-argument } }
1916   {
1917     \tl_if_blank:nTF { #3 }
1918     { \msg_error:nnn { spintent } { spmul-empty-argument } }
1919     {
1920       \bool_if:NTF \__spintent_spmul_wrap_parens_bool
1921       { \__spintent_spmul_render_parens:nn {#2} {#3} }
1922       { \__spintent_spmul_render_op:nn {#2} {#3} }
1923     }
1924   }
1925 }

```

(End of definition for __spintent_spmul_process:nnn.)

`\spmul` Defines the user command `\spmul`. It strictly relies on *math mode*.

```

1926 \NewDocumentCommand \spmul { 0{} m m }
1927 {
1928   \mode_if_math:TF
1929   {
1930     \group_begin:
1931     \__spintent_spmul_process:nnn { #1 } { #2 } { #3 }
1932     \group_end:
1933   }
1934   { \msg_error:nnn { spintent } { need-math-mode } { \spmul } }
1935 }

```

(End of definition for `\spmul`. This function is documented on page 13.)

11.26 The commands `\spgcd` and `\spmcn`

The user-level commands `\spgcd` and `\spmcn` provide a semantic interface for the Greatest Common Divisor (MCD in Spanish) and the Least Common Multiple (MCM in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid `MathCAT` intent.

11.26.1 Variables for `\spgcd` and `\spmcn`

Internal variables generated dynamically via a comma-separated list to avoid code duplication for both symmetric commands.

```

\__spintent_spmcm_spmcd_luaset_error_str
\__spintent_mc_luaset_types_str
\__spintent_mc_luaset_all_numeric_str
\__spintent_mc_args_seq
\__spintent_mc_raw_args_seq
\__spintent_spmcd_print_result_bool
\__spintent_spmcm_print_result_bool
\__spintent_spmcd_read_terse_bool
\__spintent_spmcm_read_terse_bool
\__spintent_spmcd_print_upper_bool
\__spintent_spmcm_print_upper_bool
\__spintent_spmcd_prefix_tl
\__spintent_spmcm_prefix_tl
\__spintent_spmcd_suffix_tl
\__spintent_spmcm_suffix_tl

```

```

1936 \str_new:N \__spintent_spmcm_spmcd_luaset_error_str
1937 \clist_new:N \__spintent_mc_luaset_types_clist
1938 \str_new:N \__spintent_mc_luaset_all_numeric_str
1939 \seq_new:N \__spintent_mc_args_seq
1940 \seq_new:N \__spintent_mc_raw_args_seq
1941 \seq_new:N \__spintent_mc_types_seq
1942 \int_new:N \__spintent_mc_idx_int
1943 \clist_map_inline:nn { mcd , mcm }
1944 {
1945   \tl_new:c { \__spintent_sp #1 _luaset_ #1 _value_tl }
1946   \bool_new:c { \__spintent_sp #1 _read_terse_bool }
1947   \tl_new:c { \__spintent_sp #1 _prefix_tl }
1948 }

```

(End of definition for `\__spintent_spmcm_spmcd_luaset_error_str` and others.)

11.26.2 Definition of error messages

mc-invalid-arguments

Exception triggered when the comma-separated sequence contains negative integers, floats, or alphabetical characters.

```

1949 \msg_new:nnnn { spintent } { mc-invalid-arguments }
1950 {
1951   Invalid~arguments~provided~to~#1.
1952 }
1953 {
1954   The~arguments~must~be~a~comma-separated~list. \\\
1955 }
```

(End of definition for mc-invalid-arguments.)

11.26.3 Keys for \spmc and \spmcm

result
prefix
read-cmd
upper-case
concept
silent-cmd
unknown

We define the shared *(keys)* result, prefix, read-cmd, upper-case, concept and silent-cmd.

```

1956 \clist_map_inline:nn { mcd , mcm }
1957 {
1958   \keys_define:nn { spintent / sp #1 }
1959   {
1960     result      .bool_set:c = { l__spintent_sp #1 _print_result_bool },
1961     result      .initial:n = false,
1962     result      .value_required:n = true,
1963     prefix      .code:n =
1964       { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
1965     prefix      .initial:n = ,
1966     prefix      .value_required:n = true,
1967     read-cmd    .choices:nn =
1968       { terse , clear }
1969       {
1970         \int_case:nn { \l_keys_choice_int }
1971         {
1972           {1} { \bool_set_true:c { l__spintent_sp #1 _read_terse_bool } }
1973           {2} { \bool_set_false:c { l__spintent_sp #1 _read_terse_bool } }
1974         }
1975       },
1976     read-cmd    .initial:n = clear,
1977     read-cmd    .value_required:n = true,
1978     read-cmd / unknown .code:n =
1979       \msg_error:nnee { spintent } { unknown-choice }
1980       { read-cmd } { terse , ~clear } { \exp_not:n {##1} },
1981     upper-case  .bool_set:c = { l__spintent_sp #1 _print_upper_bool },
1982     upper-case  .initial:n = false,
1983     upper-case  .value_required:n = true,
1984     concept     .bool_set:c = { l__spintent_sp #1 _concept_bool },
1985     concept     .initial:n = true,
1986     concept     .value_required:n = true,
1987     silent-cmd  .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
1988     silent-cmd  .initial:n = false,
1989     silent-cmd  .value_required:n = true,
1990     unknown     .code:n =
1991       {
1992         \str_set:Nn \l__spintent_command_name_str { sp #1 }
1993         \__spintent_unknown_keys_cmd:n {##1}
1994       },
1995   }
1996 }
```

(End of definition for result and others.)

11.26.4 Internal functions for \spmc and \spmcm

l__spintent_mc_process_args:nn

The function `\__spintent_mc_process_args:nn` parses the operation ‘#1’ and its raw argument list ‘#2’. Calls `\__spintent_luafun_mc_classify_args:n`, splits ‘#2’ into `\l__spintent_mc_raw_args_seq` and `\l__spintent_mc_luaset_types_clist` into `\l__spintent_mc_types_seq`, then walks the former by position, fetching the matching type from the latter through `\seq_item:Nn`, to build `\l__spintent_mc_args_seq`: each argument wrapped through `\spnum` when its type is `number`, verbatim otherwise. Injects MathML intents around the already-formatted `\l__spintent_mc_args_seq`.

```

1997 \cs_new_protected:Nn \__spintent_mc_process_args:nn
1998 {
1999   \__spintent_luafun_mc_classify_args:n {#2}
2000   \seq_set_from_clist:Nn \l__spintent_mc_raw_args_seq {#2}
```

```

2001 \seq_set_from_clist:NN \l__spintent_mc_types_seq \l__spintent_mc_luaset_types_clist
2002 \seq_clear:N \l__spintent_mc_args_seq
2003 \int_zero:N \l__spintent_mc_idx_int
2004 \seq_map_inline:Nn \l__spintent_mc_raw_args_seq
2005 {
2006   \int_incr:N \l__spintent_mc_idx_int
2007   \str_if_eq:eeTF { \seq_item:Nn \l__spintent_mc_types_seq { \l__spintent_mc_idx_int } } {
2008     { \seq_put_right:Nn \l__spintent_mc_args_seq { \spnum {##1} } }
2009     { \seq_put_right:Nn \l__spintent_mc_args_seq {##1} }
2010   }

```

Injects MathML intents around the already-formatted `\l__spintent_mc_args_seq`, exactly as before.

```

2011 \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
2012 {
2013   \seq_set_map:Nnn \l__spintent_mc_args_seq \l__spintent_mc_args_seq
2014   { \MathMLintent { :silent } { ##1 } }
2015   \MathMLintent { _entre } { \c__spintent_invisible_sep_tl }
2016   \MathMLintent { :silent } { ( }
2017   \seq_use:Nnnn \l__spintent_mc_args_seq
2018   { \MathMLintent { _y:silent } { , } }
2019   { \MathMLintent { :silent } { , } }
2020   { \MathMLintent { _y:silent } { , } }
2021 }
2022 {
2023   \MathMLintent { _entre } { \c__spintent_invisible_sep_tl }
2024   \MathMLintent { :silent } { ( }
2025   \seq_use:Nnnn \l__spintent_mc_args_seq
2026   { \MathMLintent { _y } { , } }
2027   { \MathMLintent { :silent } { , } }
2028   { \MathMLintent { _y } { , } }
2029 }
2030 \MathMLintent { :silent } { ) }
2031 }

```

(End of definition for `\l__spintent_mc_process_args:nn`.)

`\l__spintent_spmc_process:nnnn`

The internal function `\l__spintent_spmc_process:nnnn` is the shared handler for both `\spmc d` and `\spmc m`. Takes the short identifier in ‘#1’, its full reading string in ‘#2’, the local *(keys)* in ‘#3’, and the optional arguments in ‘#4’.

The operator name (through `\operatorname`, per upper-case) is silenced with `silent-cmd true` — `\l__spintent_mc_process_args:nn` silences the arguments too, in that case — or with `concept false` alone, leaving the arguments untouched. With `concept true`, it carries the literal `intent` instead — #1 or #2 per `read-cmd`, prefixed with `prefix` when given.

With ‘#4’ given, classifies it through `\l__spintent_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting mc-invalid-arguments on failure, and, when `result` is `true`, printing the computed value. `\l__spintent_mc_process_args:n` prints the argument list itself either way.

```

2032 \cs_new_protected:Nn \l__spintent_spmc_process:nnnn
2033 {
2034   \keys_set:nn { spintent / sp #1 } { #3 }
2035   \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
2036   {
2037     \MathMLintent { :silent }
2038     {
2039       \operatorname
2040       {
2041         \bool_if:cTF { \l__spintent_sp #1 _print_upper_bool }
2042         { \text_uppercase:n {#1} } { #1 }
2043       }
2044     }
2045   }
2046   {
2047     \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
2048     {
2049       \MathMLintent
2050       {
2051         \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
2052         {
2053           \tl_use:c { \l__spintent_sp #1 _prefix_tl } -
2054         }
2055         \bool_if:cTF { \l__spintent_sp #1 _read_terse_bool } { #1 :nofix } { #2 :nofix }

```

```

2056         }
2057     {
2058         \operatorname
2059     {
2060         \bool_if:cTF { \__spintent_sp #1 _print_upper_bool }
2061         { \text_uppercase:n {#1} } { #1 }
2062     }
2063 }
2064 }
2065 {
2066     \MathMLintent { :silent }
2067     {
2068         \operatorname
2069     {
2070         \bool_if:cTF { \__spintent_sp #1 _print_upper_bool }
2071         { \text_uppercase:n {#1} } { #1 }
2072     }
2073 }
2074 }
2075 }

```

With ‘#4’ given, classifies it through `\__spintent_luafun_mc_classify_args:n`. Only when every argument is a number does it call the Lua calculation (`calculate_#1`), reporting `mc-invalid-arguments` on failure — which should not happen, since every argument already passed classification — and, when `result` is `true`, printing the computed value. With any non-numeric argument, no calculation is attempted at all. Either way, `\__spintent_mc_process_args:n` prints the argument list itself.

```

2076     \tl_if_novalue:nF { #4 }
2077     {
2078         \__spintent_luafun_mc_classify_args:n {#4}
2079         \str_if_eq:VnT \__spintent_mc_luaset_all_numeric_str { true }
2080         {
2081             \use:c { __spintent_luafun_calculate_ #1 :n } { #4 }
2082             \str_if_eq:VnT \__spintent_spmcm_spmcd_luaset_error_str { true }
2083             {
2084                 \msg_error:nne { spintent } { mc-invalid-arguments } { \exp_not:c { sp #1 } }
2085             }
2086         }
2087         \__spintent_mc_process_args:nn {#1} {#4}
2088         \str_if_eq:VnT \__spintent_mc_luaset_all_numeric_str { true }
2089         {
2090             \bool_if:cT { \__spintent_sp #1 _print_result_bool }
2091             {
2092                 \MathMLintent { es-igual-a } { = }
2093                 \tl_use:c { \__spintent_sp #1 _luaset_ #1 _value_tl }
2094             }
2095         }
2096     }
2097 }

```

(End of definition for `\__spintent_spmc_process:nnnn`.)

`\spmc` The commands `\spmc` and `\spmc` act as the public interface wrappers for typesetting the greatest common divisor and least common multiple. They enforce math mode, open a local group, and defer execution to the main processing macro `\__spintent_spmc_process:nnnn`, passing along the appropriate reading strings and arguments.

```

2098 \NewDocumentCommand \spmc { 0{} d() }
2099 {
2100     \mode_if_math:TF
2101     {
2102         \group_begin:
2103         \__spintent_spmc_process:nnnn { mcd } { máximo-común-divisor } {#1} {#2}
2104         \group_end:
2105     }
2106     { \msg_error:nnn { spintent } { need-math-mode } { \spmc } }
2107 }
2108 \NewDocumentCommand \spmc { 0{} d() }
2109 {
2110     \mode_if_math:TF
2111     {
2112         \group_begin:
2113         \__spintent_spmc_process:nnnn { mcm } { mínimo-común-múltiplo } {#1} {#2}

```

```

2114         \group_end:
2115     }
2116     { \msg_error:nnn { spintent } { need-math-mode } { \spmcm } }
2117 }

```

(End of definition for \spmcd and \spmcm. These functions are documented on page 15.)

11.27 The commands \spnM and \spnD

The user-level commands `\spnM` and `\spnD` provide a semantic interface for the Multiples of a number (“*múltiplos*” in Spanish) and the Divisors of a number (“*divisores*” in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.27.1 Definition of variables for \spnM and \spnD

Internal variables generated dynamically for multiples and divisors.

```

\__spintent_md_out_seq
\__spintent_spmmd_luaset_is_number_str
\__spintent_spm_luaset_result_tl
\__spintent_spnD_luaset_result_tl
\__spintent_spm_luaset_is_truncated_str
\__spintent_spnD_luaset_is_truncated_str
\__spintent_spmM_prefix_tl
\__spintent_spnD_prefix_tl
\__spintent_spmM_print_result_num_tl
\__spintent_spnD_print_result_num_tl
2118 \seq_new:N \__spintent_md_out_seq
2119 \str_new:N \__spintent_spmmd_luaset_is_number_str
2120 \clist_map_inline:nn { nM , nD }
2121 {
2122     \tl_new:c { \__spintent_sp #1 _luaset_result_tl }
2123     \str_new:c { \__spintent_sp #1 _luaset_is_truncated_str }
2124     \tl_new:c { \__spintent_sp #1 _prefix_tl }
2125 }

```

(End of definition for __spintent_md_out_seq and others.)

11.27.2 Keys for \spnM and \spnD

We define the shared (*keys*) `result`, `silent-cmd`, `result-num`, `prefix` and `concept`.

```

result
silent-cmd
result-num
prefix
concept
unknown
2126 \clist_map_inline:nn { nM , nD }
2127 {
2128     \keys_define:nn { spintent / sp #1 }
2129     {
2130         result      .bool_set:c = { \__spintent_sp #1 _print_result_bool },
2131         result      .initial:n = false,
2132         result      .value_required:n = true,
2133         silent-cmd .bool_set:c = { \__spintent_sp #1 _silent_cmd_bool },
2134         silent-cmd .initial:n = false,
2135         silent-cmd .value_required:n = true,
2136         result-num .tl_set:c = { \__spintent_sp #1 _print_result_num_tl },
2137         result-num .initial:n = ,
2138         prefix      .code:n =
2139             { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
2140         prefix      .initial:n = ,
2141         prefix      .value_required:n = true,
2142         concept     .bool_set:c = { \__spintent_sp #1 _concept_bool },
2143         concept     .initial:n = true,
2144         concept     .value_required:n = true,
2145         unknown     .code:n =
2146             {
2147                 \str_set:Nn \__spintent_command_name_str { sp #1 }
2148                 \__spintent_unknown_keys_cmd:n {##1}
2149             },
2150     }
2151 }

```

(End of definition for `result` and others.)

11.27.3 Definition of error messages for \spnM and \spnD

`md-invalid-argument` Exception triggered when the argument for multiples or divisors is invalid.

```

2152 \msg_new:nnnn { spintent } { md-invalid-argument }
2153 { Invalid~argument~provided~to~#1. }
2154 {
2155     The~argument~must~be~a~single~positive~integer. \
2156 }

```

(End of definition for `md-invalid-argument`.)

11.27.4 Internal functions for \spnM and \spnD

__spintent_spnM_spnD_process:nnnn

The internal function `\__spintent_spnM_spnD_process:nnnn` acts as the primary entry point for the multiples and divisors commands. It takes the command identifier ‘#1’, its printed mathematical symbol ‘#2’, its base reading string ‘#3’, the local key-value options ‘#4’, and the optional argument ‘#5’. Its sole purpose is to evaluate and set the provided keys within the current scope before delegating the core branching and typesetting logic to `\__spintent_spn_M_and_D_exec:nnnn`.

```
2157 \cs_new_protected:Nn \__spintent_spnM_spnD_process:nnnnn
2158 {
2159   \keys_set:nn { spintent / sp #1 } { #4 }
2160   \__spintent_spnM_spnD_exec:nnnn { #1 } { #2 } { #3 } { #5 }
2161 }
```

(End of definition for __spintent_spnM_spnD_process:nnnnn.)

__spintent_spnM_spnD_intent:nn

The internal function `\__spintent_spnM_spnD_intent:nn` resolves the main reading intent string. It takes the command identifier ‘#1’ (such as “nM” or “nD”) and the base reading string ‘#2’. It automatically prepends the localized prefix if defined, normalizes accentuation for “múltiplos”, and appends ‘-de’.

```
2162 \cs_new:Nn \__spintent_spnM_spnD_intent:nn
2163 {
2164   \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
2165   { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
2166   \str_case:nnF { #2 }
2167   { { múltiplos } { múltiplos } }
2168   { #2 }
2169   -de
2170 }
```

(End of definition for __spintent_spnM_spnD_intent:nn.)

__spintent_spnM_spnD_silent:nn

The internal function `\__spintent_spn_M_and_D_silent:nn` prints the atomized notation structure in full silent mode. It takes the mathematical symbol ‘#1’ (such as “D” or “M”) and the variable or explicit argument ‘#2’. Every component is wrapped in a silent intent, and an invisible separator is injected between the parentheses to prevent assistive technologies from applying default contextual reading rules like “de”.

```
2171 \cs_new_protected:Nn \__spintent_spnM_spnD_silent:nn
2172 {
2173   \MathMLintent { :silent } { \operatorname { #1 } }
2174   \MathMLintent { :silent } { ( }
2175   \MathMLintent { :silent } { \c__spintent_invisible_sep_tl }
2176   \MathMLintent { :silent } { #2 }
2177   \MathMLintent { :silent } { ) }
2178 }
```

(End of definition for __spintent_spnM_spnD_silent:nn.)

__spintent_spnM_spnD_result:n

The internal function `\__spintent_spn_M_and_D_result:n` handles the calculation and accessible formatting of the computed mathematical set. It takes the command identifier ‘#1’. It invokes the corresponding Lua backend calculation, maps the resulting clist into a sequence, and formats the final printed output utilizing custom accessibility intents for both separators and conjunctions. It also safely manages truncation indicators.

```
2179 \cs_new_protected:Nn \__spintent_spnM_spnD_result:n
2180 {
2181   \use:c { __spintent_luafun_calcular_ #1 :nn }
2182   { \l__spintent_luaset_part_int_tl }
2183   { \tl_use:c { l__spintent_sp #1 _print_result_num_tl } }
2184   \MathMLintent { _son } { = \{ }
2185   \seq_set_from_clist:Ne
2186   \l__spintent_md_out_seq { \tl_use:c { l__spintent_sp #1 _luaset_result_tl } }
2187   \seq_use:Nnnn \l__spintent_md_out_seq
2188   { \MathMLintent { _y } { , } }
2189   { \MathMLintent { :silent } { , } }
2190   { \MathMLintent { _y } { , } }
2191   \str_if_eq:vnT
2192   { l__spintent_sp #1 _luaset_is_truncated_str } { true }
2193   {
2194     \MathMLintent { :silent } { , } ~ \dots
2195   }
2196   \MathMLintent { :silent } { \} }
2197 }
```

(End of definition for __spintent_spnM_spnD_result:n.)

_spintrent_spnM_spnD_exec:nnnn

The function _spintrent_spnM_spnD_exec:nnnn takes the command name in ‘#1’, the symbol in ‘#2’, the base reading string in ‘#3’, and the optional argument in ‘#4’. The parentheses are always silenced. With ‘#4’ absent, the placeholder n stands in for the argument. With ‘#4’ given, _spintrent_luafun_spnmd_classify decides whether it is wrapped through \\spnum — and, when it is, whether _spintrent_spnM_spnD_result:n also prints the computed set, per result — or left verbatim, with no result attempted. Either way, silent-cmd silences the whole thing through _spintrent_spnM_spnD_silent:nn; otherwise concept true carries the literal intent from _spintrent_spnM_spnD_intent:nn, false silences only the operator symbol, leaving the argument to read normally.

```

2198 \\cs_new_protected:Nn \\_spintrent_spnM_spnD_exec:nnnn
2199 {
2200   \\tl_if_no_value:nTF { #4 }
2201   {
2202     \\bool_if:cTF { \\_spintrent_sp #1 _silent_cmd_bool }
2203     { \\_spintrent_spnM_spnD_silent:nn { #2 } { n } }
2204     {
2205       \\bool_if:cTF { \\_spintrent_sp #1 _concept_bool }
2206       {
2207         \\MathMLintrent
2208         { \\_spintrent_spnM_spnD_intent:nn { #1 } { #3 } }
2209         { \\operatorname { #2 } }
2210       }
2211       { \\MathMLintrent { :silent } { \\operatorname { #2 } } }
2212       \\MathMLintrent { :silent } { ( }
2213       n
2214       \\MathMLintrent { :silent } { ) }
2215     }
2216   }
2217   {
2218     \\_spintrent_luafun_spnmd_classify:n { \\exp_not:n { #4 } }
2219     \\bool_if:cTF { \\_spintrent_sp #1 _silent_cmd_bool }
2220     {
2221       \\str_if_eq:VnTF \\_spintrent_spnmd_luaset_is_number_str { true }
2222       { \\_spintrent_spnM_spnD_silent:nn { #2 } { \\spnum { #4 } } }
2223       { \\_spintrent_spnM_spnD_silent:nn { #2 } { #4 } }
2224     }
2225     {
2226       \\bool_if:cTF { \\_spintrent_sp #1 _concept_bool }
2227       {
2228         \\MathMLintrent
2229         { \\_spintrent_spnM_spnD_intent:nn { #1 } { #3 } }
2230         { \\operatorname { #2 } }
2231       }
2232       { \\MathMLintrent { :silent } { \\operatorname { #2 } } }
2233       \\MathMLintrent { :silent } { ( }
2234       \\str_if_eq:VnTF \\_spintrent_spnmd_luaset_is_number_str { true }
2235       { \\spnum { #4 } }
2236       { #4 }
2237       \\MathMLintrent { :silent } { ) }
2238     }
2239     \\str_if_eq:VnTF \\_spintrent_spnmd_luaset_is_number_str { true }
2240     {
2241       \\bool_if:cT { \\_spintrent_sp #1 _print_result_bool }
2242       { \\_spintrent_spnM_spnD_result:n { #1 } }
2243     }
2244   }
2245 }

```

(End of definition for _spintrent_spnM_spnD_exec:nnnn.)

\\spnM The commands \\spnM and \\spnD act as the public interface wrappers for typesetting multiples and divisors, respectively. They enforce math mode, open a local group, and defer execution to the function _spintrent_spnM_spnD_process:nnnnn, passing along the optional <keys> ‘#1’ and the numeric {<argument>} ‘#2’.

```

2246 \\NewDocumentCommand \\spnM { 0{} d() }
2247 {
2248   \\mode_if_math:TF
2249   {
2250     \\group_begin:
2251     \\_spintrent_spnM_spnD_process:nnnnn { nM } { M } { multiplos } { #1 } { #2 }
2252     \\group_end:

```

```

2253     }
2254     { \msg_error:nnn { spintent } { need-math-mode } { \spnM } }
2255   }
2256   \NewDocumentCommand \spnD { 0{} d() }
2257   {
2258     \mode_if_math:TF
2259     {
2260       \group_begin:
2261         \__spintent_spnM_spnD_process:nnnnn { nD } { D } { divisores } { #1 } { #2 }
2262       \group_end:
2263     }
2264     { \msg_error:nnn { spintent } { need-math-mode } { \spnD } }
2265   }

```

(End of definition for `\spnM` and `\spnD`. These functions are documented on page 14.)

11.28 The command `\sprazon`

The user command `\sprazon` typesets a ratio $a:b$ as a fraction (`\dfrac` or `\frac`, per `use-dfrac`), read natively — no Lua parsing involved.

11.28.1 Definition of keys for `\sprazon`

We define the *(keys)* `silent-cmd` and `use-dfrac`.

```

2266 \keys_define:nn { spintent / sprazon }
2267 {
2268   silent-cmd .bool_set:N = \__spintent_sprazon_silent_cmd_bool,
2269   silent-cmd .initial:n = false,
2270   silent-cmd .value_required:n = true,
2271   use-dfrac .bool_set:N = \__spintent_sprazon_use_dfrac_bool,
2272   use-dfrac .initial:n = false,
2273   use-dfrac .value_required:n = true,
2274   unknown .code:n =
2275   {
2276     \str_set:Nn \__spintent_command_name_str { sprazon }
2277     \__spintent_unknown_keys_cmd:n { #1 }
2278   },
2279 }

```

(End of definition for `silent-cmd`, `use-dfrac`, and `unknown`.)

The function `\__spintent_sprazon_exec:nnn` receives the optional *(keys)* in ‘#1’, the numerator in ‘#2’ and the denominator in ‘#3’. Under `silent-cmd`, the whole fraction is silenced as one unit. Otherwise, numerator and denominator are each tagged through a native `\MathMLarg` (num/ denom), and the outer `\MathMLintent` reads `es-a:infix($num,$denom)`.

```

2280 \cs_new_protected:Nn \__spintent_sprazon_exec:nnn
2281 {
2282   \keys_set:nn { spintent / sprazon } { #1 }
2283   \__spintent_invisible_sep:
2284   \bool_if:NTF \__spintent_sprazon_silent_cmd_bool
2285   {
2286     \MathMLintent { _- }
2287     {
2288       \__spintent_luafun_inner_sep:
2289       \bool_if:NTF \__spintent_sprazon_use_dfrac_bool
2290       { \dfrac { #2 } { #3 } }
2291       { \frac { #2 } { #3 } }
2292     }
2293   }
2294   {
2295     \MathMLintent { es-a:infix($num,$denom) }
2296     {
2297       \__spintent_luafun_inner_sep:
2298       \bool_if:NTF \__spintent_sprazon_use_dfrac_bool
2299       {
2300         \dfrac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } }
2301       }
2302       { \frac { \MathMLarg { num } { #2 } } { \MathMLarg { denom } { #3 } } }
2303     }
2304   }
2305 }

```

(End of definition for `\__spintent_sprazon_exec:nnn`.)

`\sprazon` The public command `\sprazon` enforces *math mode* and delegates execution to `\__spintent_sprazon_exec:nnn`.

```

2306 \NewDocumentCommand \sprazon { 0{} m m }
2307 {
2308   \mode_if_math:TF
2309   {
2310     \group_begin:
2311     \__spintent_sprazon_exec:nnn { #1 } { #2 } { #3 }
2312     \group_end:
2313   }
2314   { \msg_error:nnn { spintent } { need-math-mode } { \sprazon } }
2315 }

```

(End of definition for `\sprazon`. This function is documented on page ??.)

11.29 The command `\spProp`

The user command `\spProp` typesets a proportion $a:b = c:d$ as two fractions joined by $=$, reusing `\__spintent_sprazon_exec:nnn` directly (not the public `\sprazon` command) for each ratio.

`\spProp` The public command `\spProp` enforces *math mode*, and forwards the same *keys* in ‘#1’ to both ratios: a/b (‘#2’/‘#3’) and c/d (‘#4’/‘#5’), joined by a native `\MathMLintent` reading `_como` over the $=$ sign.

```

2316 \NewDocumentCommand \spProp { 0{} m m m m }
2317 {
2318   \mode_if_math:TF
2319   {
2320     \group_begin:
2321     \__spintent_sprazon_exec:nnn { #1 } { #2 } { #3 }
2322     \bool_if:NTF \l__spintent_sprazon_silent_cmd_bool
2323     {
2324       \MathMLintent { :silent } { = }
2325     }
2326     { \MathMLintent { _como } { = } }
2327     \__spintent_sprazon_exec:nnn { #1 } { #4 } { #5 }
2328     \group_end:
2329   }
2330   { \msg_error:nnn { spintent } { need-math-mode } { \spProp } }
2331 }

```

(End of definition for `\spProp`. This function is documented on page ??.)

11.30 The command `\spang`

The command `\spang` is the main public command for typesetting and tagging sexagesimal angle expressions (degrees, minutes, and seconds). It parses the input using the Lua function `\__spintent_luafun_spang_parse:n`. This separates the degrees, minutes, and seconds to construct a `MathML intent` tree with the correct postfixes for `NVDA/MathCAT`.

```

\l__spintent_spang_luaset_grado_str
\l__spintent_spang_luaset_minuto_str
\l__spintent_spang_luaset_segundo_str
\l__spintent_spang_luaset_error_str

```

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted numerical values for degrees, minutes, and seconds, as well as the error status used during formatting.

```

2332 \str_new:N \l__spintent_spang_luaset_grado_str
2333 \str_new:N \l__spintent_spang_luaset_minuto_str
2334 \str_new:N \l__spintent_spang_luaset_segundo_str
2335 \str_new:N \l__spintent_spang_luaset_error_str

```

(End of definition for `\l__spintent_spang_luaset_grado_str` and others.)

11.30.1 Definition of error messages for `\spang`

`invalid-angle-format`

The error message `invalid-angle-format` defines the text displayed when an invalid angle input is detected. It is triggered when an argument fails the format checks, does not match the expected sexagesimal pattern, or contains invalid measurement symbols.

```

2336 \msg_new:nnnn { spintent } { invalid-angle-format }
2337 { Invalid~angle~format~in~#1. }
2338 {
2339   The~argument~'#2'~could~not~be~parsed. \
2340 }

```

(End of definition for `invalid-angle-format`.)

11.30.2 Internal functions for `\spang`

`__spintent_spang_process:n`

The internal function `__spintent_spang_process:n` parses the angle input using `__spintent_luafun_spang_parse:n` after resetting the error flag in `__spintent_spang_luaaset_error_str`. If an invalid format is detected, it issues the `invalid-angle-format` error. Otherwise, it checks the extracted string variables `__spintent_spang_luaaset_grado_str`, `__spintent_spang_luaaset_minuto_str`, and `__spintent_spang_luaaset_segundo_str`. For each non-empty variable, it constructs the corresponding MathML `intent` structure with the correct measurement symbols and postfixes for NVDA/MathCAT.

```

2341 \cs_new_protected:Nn __spintent_spang_process:n
2342 {
2343   \str_set:Nn __spintent_spang_luaaset_error_str { false }
2344   __spintent_luafun_spang_parse:n { #1 }
2345   \str_if_eq:NnTF __spintent_spang_luaaset_error_str { true }
2346   {
2347     \msg_error:nnnn { spintent } { invalid-angle-format } { \spang } { #1 }
2348   }
2349   {
2350     \str_if_empty:NF __spintent_spang_luaaset_grado_str
2351     {
2352       \spunit{ __spintent_spang_luaaset_grado_str °}
2353     }
2354     \str_if_empty:NF __spintent_spang_luaaset_minuto_str
2355     {
2356       \MathMLintent { minutos : postfix($m) }
2357       {
2358         \MathMLarg { m }
2359         {
2360           \spunit{ __spintent_spang_luaaset_minuto_str '}'
2361         }
2362       }
2363     }
2364     \str_if_empty:NF __spintent_spang_luaaset_segundo_str
2365     {
2366       \MathMLintent { segundos : postfix($s) }
2367       {
2368         \MathMLarg { s }
2369         {
2370           \spunit { __spintent_spang_luaaset_segundo_str "}"
2371         }
2372       }
2373     }
2374   }
2375 }

```

(End of definition for `__spintent_spang_process:n`.)

`\spang`

The command `\spang` is the public interface for formatting sexagesimal angle expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `__spintent_spang_process:n` to process the `{(argument)}`.

```

2376 \NewDocumentCommand \spang { m }
2377 {
2378   \mode_if_math:TF
2379   {
2380     \group_begin:
2381     __spintent_spang_process:n { #1 }
2382     \group_end:
2383   }
2384   { \msg_error:nnn { spintent } { need-math-mode } { \spang } }
2385 }

```

(End of definition for `\spang`. This function is documented on page 21.)

11.31 The command `\spnZ`

The user-level command `\spnZ` formats integers (the set $\mathbb{Z}$). It includes specific options to automatically enclose numbers (particularly negative ones) in visual parentheses and allows you to control whether these parentheses are read by MathCAT.

This command verifies that the `{(argument)}` is indeed an *integer*, checking for the complete absence of decimal separators or attached units of measurement.

11.31.1 Definition of keys for \spnZ

Now we define the $\langle \text{keys} \rangle$ `cifras-sep` and `read-sign`. These are `.meta:nn` $\langle \text{keys} \rangle$ passed directly to the `\spnum` command. We also introduce `wrap-parens`, which determines whether the number is enclosed in parentheses, and `read-parens`, which controls if these parentheses are read by MathCAT.

```

2386 \keys_define:nn { spintent / spnZ }
2387 {
2388   cifras-sep .meta:nn = { spintent / spnum } { cifras-sep = {#1} },
2389   read-sign .meta:nn = { spintent / spnum } { read-sign = {#1} },
2390   wrap-parens .bool_set:N = \l__spintent_spnz_wrap_parens_bool,
2391   wrap-parens .initial:n = false,
2392   wrap-parens .value_required:n = true,
2393   read-parens .bool_set:N = \l__spintent_spnz_read_parens_bool,
2394   read-parens .initial:n = true,
2395   read-parens .value_required:n = true,
2396   unknown .code:n =
2397     {
2398       \str_set:Nn \l__spintent_command_name_str { spnZ }
2399       \__spintent_unknown_keys_cmd:n {#1}
2400     },
2401 }

```

(End of definition for `cifras-sep` and others.)

11.31.2 Definition of error messages

`spnz-not-integer` Raised when the $\{ \langle \text{argument} \rangle \}$ contains decimal separators or attached units, which conflict with the constraints of a pure integer representation.

```

2402 \msg_new:nnnn { spintent } { spnz-not-integer }
2403 { The~value~'~\the{#1}'~is~not~a~valid~integer. }
2404 {
2405   Command~\token_to_str:N \spnZ~only~accepts~integers.\
2406 }

```

(End of definition for `spnz-not-integer`.)

11.31.3 Internal functions for \spnZ

`\__spintent_spnz_render_parens:` The function `\__spintent_spnz_render_parens:` first checks the state of the boolean variable `\l__spintent_spnz_read_parens` (controlled by the `read-parens` key). If it is `true`, it prints the number surrounded by parentheses. Otherwise, it applies the `:silent function intent` to the parentheses while printing the number.

```

2407 \cs_new_protected:Nn \__spintent_spnz_render_parens:
2408 {
2409   \bool_if:NTF \l__spintent_spnz_read_parens_bool
2410   {
2411     ( \__spintent_spnum_print_num_start: )
2412   }
2413   {
2414     \MathMLintent { :silent } { ( }
2415     \__spintent_spnum_print_num_start:
2416     \MathMLintent { :silent } { ) }
2417   }
2418 }

```

(End of definition for `\__spintent_spnz_render_parens:.`)

`\__spintent_spnz_process:nn` The function `\__spintent_spnz_process:nn` first processes the $\langle \text{keys} \rangle$ passed in ‘#1’ and then calls `\__spintent_luafun_clean_split_and_set:n` on ‘#2’. It verifies that the input is a valid integer by checking if the variable `\l__spintent_luaset_has_integer_str` is `true`. If not `true`, an “error” is raised.

If it is `true`, it checks whether `\l__spintent_luaset_part_int_tl` is *empty* and raises an “error” if so. Otherwise, it checks `\l__spintent_spnz_wrap_parens_bool` set by `wrap-parens` and calls either the function `\__spintent_spnz_render_parens:` or `\__spintent_spnum_print_num_start:`.

```

2419 \cs_new_protected:Nn \__spintent_spnz_process:nn
2420 {
2421   \keys_set:nn { spintent / spnZ } {#1}
2422   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#2} }
2423   \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
2424   {
2425     \tl_if_empty:NTF \l__spintent_luaset_part_int_tl
2426     { \msg_error:nnn { spintent } { spnum-no-digits } {#2} }
2427     {
2428       \bool_if:NTF \l__spintent_spnz_wrap_parens_bool

```

```

2429         { \__spintent_spnz_render_parens: }
2430         { \__spintent_spnum_print_num_start: }
2431     }
2432 }
2433 { \msg_error:nnn { spintent } { spnz-not-integer } {#2} }
2434 }

```

(End of definition for __spintent_spnz_process:nn.)

\spnZ Defines the user command **\spnZ**, which is executed within a *group* and strictly enforces *math mode*.

```

2435 \NewDocumentCommand \spnZ { O{} m }
2436 {
2437     \mode_if_math:TF
2438     {
2439         \group_begin:
2440         \__spintent_spnz_process:nn { #1 } { #2 }
2441         \group_end:
2442     }
2443     { \msg_error:nnn { spintent } { need-math-mode } { \spnZ } }
2444 }

```

(End of definition for \spnZ. This function is documented on page 16.)

11.32 The command \spmQ

The command **\spmQ** typesets “mixed numbers” within Spanish mathematical contexts (e.g., $3\frac{1}{2}$). It delegates the scaling of the integer part to the Lua module **spintent.lua** via **__spintent_luafun_spmQ_scale_int:Vn**, which measures both the integer glyph and the fraction box at the node level and computes a scaling factor to align their heights visually. Finally, it constructs the correct MathML intent for MathCAT.

11.32.1 Definition of variables for \spmQ

The variable **__spintent_spmQ_join_word_str** stores the value set by the key **join-word** which will be verbalized by MathCAT as a grammatical connector between the integer and fractional parts.

The variable **__spintent_spnQ_intent_read_sign_str** stores the **:intent** passed to **\MathMLintent** when the signs ‘+’ or ‘−’ are present in the integer part, used by the function **__spintent_spmQ_print_scale_int:n**.

The variables **__spintent_spnQ_wrap_parens_l_tl** and **__spintent_spnQ_wrap_parens_r_tl** store the sized parentheses set by the function **__spintent_spmQ_set_wrap_parens_size:** and used by the key **wrap-parens**.

The integer variable **__spintent_saved_luamml_flag_int** temporarily stores the current value of **__luamml_flag_int** before disabling it during the injection of the scaled integer node, and restores it afterwards. This prevents **luamml** from generating a spurious **<math>** element around the **sub_box** processed in isolation, before **\frac** has expanded.

```

2445 \str_new:N \__spintent_spmQ_join_word_str
2446 \str_new:N \__spintent_spnQ_intent_read_sign_str
2447 \tl_new:N \__spintent_spnQ_wrap_parens_l_tl
2448 \tl_new:N \__spintent_spnQ_wrap_parens_r_tl
2449 \int_new:N \__spintent_saved_luamml_flag_int

```

(End of definition for __spintent_spmQ_join_word_str and others.)

11.32.2 Definition of error messages for \spmQ

The error message **spmQ-zero-denominator** is raised when a zero denominator is passed to **\spmQ**. The second error **spmQ-not-integer** is returned when the integer part passed in **{⟨argument⟩}** is not actually an integer.

```

2450 \msg_new:nnnn { spintent } { spmQ-zero-denominator }
2451 {
2452     Zero~denominator~in~\token_to_str:N \spmQ~\msg_line_context:.
2453 }
2454 {
2455     The~denominator~of~a~fraction~cannot~be~zero.\\
2456 }
2457 \msg_new:nnnn { spintent } { spmQ-not-integer }
2458 {
2459     The~value~'#1'~is~not~a~valid~integer~\msg_line_context:.
2460 }
2461 {
2462     Command~\token_to_str:N \spmQ \c_space_tl only~accepts~integers.\\
2463 }

```

(End of definition for **spmQ-zero-denominator** and **spmQ-not-integer**.)

11.32.3 Definition of keys for \spmQ

Now we add the keys `join-word`, `use-spnZ`, `wrap-parens`, `read-parens` and `print-dfrac`.

```

join-word
use-spnZ
wrap-parens
read-parens
print-dfrac
unknown
2464 \keys_define:nn { spinttent / spmQ }
2465 {
2466   join-word .choices:nn =
2467     { entero , enteros , y }
2468     {
2469       \int_case:nn { \l_keys_choice_int }
2470       {
2471         {1} { \str_set:Nn \l__spinttent_spmQ_join_word_str { entero } }
2472         {2} { \str_set:Nn \l__spinttent_spmQ_join_word_str { enteros } }
2473         {3} { \str_set:Nn \l__spinttent_spmQ_join_word_str { y } }
2474       }
2475     },
2476   join-word .initial:n = y,
2477   join-word .value_required:n = true,
2478   join-word/unknown .code:n =
2479     \msg_error:nneee { spinttent } { unknown-choice }
2480     { join-word } { entero, ~enteros, ~y } { \exp_not:n {#1} },
2481   use-spnZ .bool_set:N = \l__spinttent_spmQ_use_spnZ_bool,
2482   use-spnZ .initial:n = true,
2483   use-spnZ .value_required:n = true,
2484   print-dfrac .bool_set:N = \l__spinttent_spmQ_print_dfrac_bool,
2485   print-dfrac .initial:n = false,
2486   print-dfrac .value_required:n = true,
2487   wrap-parens .bool_set:N = \l__spinttent_spmQ_wrap_parens_bool,
2488   wrap-parens .initial:n = false,
2489   wrap-parens .value_required:n = true,
2490   read-parens .bool_set:N = \l__spinttent_spmQ_read_parens_bool,
2491   read-parens .initial:n = true,
2492   read-parens .value_required:n = true,
2493   unknown .code:n =
2494     {
2495       \str_set:Nn \l__spinttent_command_name_str { spmQ }
2496       \__spinttent_unknown_keys_cmd:n {#1}
2497     },
2498 }

```

(End of definition for `join-word` and others.)

11.32.4 Internal functions for \spmQ

`\__spinttent_spmQ_set_wrap_parens_size:`

The function `\__spinttent_spmQ_set_wrap_parens_size:` captures the current mathematical style using the primitive `\mathstyle` from LuaTeX and sets the variables `\l__spinttent_spmQ_wrap_parens_l_tl` and `\l__spinttent_spmQ_wrap_parens_r_tl` with the appropriate size for the parentheses. Display styles (0 and 1) receive `\Bigl\lparen` and `\Bigr\rparen`; text styles (2 and 3) receive `\bigl\lparen` and `\bigr\rparen`; any other style falls back to `\lparen` and `\rparen`.

If the key `print-dfrac` is active, the size is always forced to `\Bigl\lparen`/`\Bigr\rparen` regardless of the current mathematical style, since `\dfrac` always produces a display-height fraction.

```

2499 \cs_new_protected:Nn \__spinttent_spmQ_set_wrap_parens_size:
2500 {
2501   \int_case:nnF { \mathstyle }
2502   {
2503     { 0 }
2504     {
2505       \tl_set:Nn \l__spinttent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2506       \tl_set:Nn \l__spinttent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2507     }
2508     { 1 }
2509     {
2510       \tl_set:Nn \l__spinttent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2511       \tl_set:Nn \l__spinttent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2512     }
2513     { 2 }
2514     {
2515       \tl_set:Nn \l__spinttent_spmQ_wrap_parens_l_tl { \bigl \lparen }
2516       \tl_set:Nn \l__spinttent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2517     }
2518     { 3 }
2519     {
2520       \tl_set:Nn \l__spinttent_spmQ_wrap_parens_l_tl { \bigl \lparen }

```

```

2521         \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \bigr \rparen }
2522     }
2523 }
2524 {
2525     \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \lparen }
2526     \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \rparen }
2527 }

```

If the key `print-dfrac` is active, we must reset the variables `\l__spintent_spmQ_wrap_parens_l_tl` and `\l__spintent_spmQ_wrap_parens_r_tl` to `\Bigl/\Bigr` unconditionally.

```

2528     \bool_if:NT \l__spintent_spmQ_print_dfrac_bool
2529     {
2530         \tl_set:Nn \l__spintent_spmQ_wrap_parens_l_tl { \Bigl \lparen }
2531         \tl_set:Nn \l__spintent_spmQ_wrap_parens_r_tl { \Bigr \rparen }
2532     }
2533 }

```

(End of definition for `\l__spintent_spmQ_set_wrap_parens_size:`)

```

\__spintent_spmQ_start_wrap_parens:
\__spintent_spmQ_stop_wrap_parens:

```

The functions `\__spintent_spmQ_start_wrap_parens:` and `\__spintent_spmQ_stop_wrap_parens:` open and close the parentheses around the “mixed number” when the key `wrap-parens` is active. They first call `\__spintent_spmQ_set_wrap_parens_size:` to determine the correct size for the current mathematical style, then print the appropriate parenthesis stored in `\l__spintent_spmQ_wrap_parens_l_tl` or `\l__spintent_spmQ_wrap_parens_r_tl`.

If the variable `\l__spintent_spmQ_read_parens_bool`, controlled by the key `read-parens`, is *false*, each parenthesis is wrapped inside `\MathMLintent{:silent}` so that NVDA/MathCAT does not verbalize them.

```

2534 \cs_new_protected:Nn \__spintent_spmQ_start_wrap_parens:
2535 {
2536     \bool_if:NT \l__spintent_spmQ_wrap_parens_bool
2537     {
2538         \__spintent_spmQ_set_wrap_parens_size:
2539         \bool_if:NTF \l__spintent_spmQ_read_parens_bool
2540         { \l__spintent_spmQ_wrap_parens_l_tl }
2541         { \MathMLintent { :silent } { \l__spintent_spmQ_wrap_parens_l_tl } }
2542     }
2543 }
2544 \cs_new_protected:Nn \__spintent_spmQ_stop_wrap_parens:
2545 {
2546     \bool_if:NT \l__spintent_spmQ_wrap_parens_bool
2547     {
2548         \__spintent_spmQ_set_wrap_parens_size:
2549         \bool_if:NTF \l__spintent_spmQ_read_parens_bool
2550         { \l__spintent_spmQ_wrap_parens_r_tl }
2551         { \MathMLintent { :silent } { \l__spintent_spmQ_wrap_parens_r_tl } }
2552     }
2553 }

```

(End of definition for `\__spintent_spmQ_start_wrap_parens:` and `\__spintent_spmQ_stop_wrap_parens:`)

```
\__spintent_spmQ_set_join_word:
```

The function `\__spintent_spmQ_set_join_word:` injects the grammatical connector between the integer part and the fraction by dispatching on the value stored in `\l__spintent_spmQ_join_word_str`, set by the key `join-word`. In all three cases the visual output is the Unicode invisible separator U+2063 via `\c__spintent_invisible_sep_tl`, which produces a `<mi>` node in the MathML structure; what differs is the `:intent` text attached to it via `\MathMLintent`, which NVDA/MathCAT will verbalize as «entero», «enteros» or «y» respectively.

```

2554 \cs_new_protected:Nn \__spintent_spmQ_set_join_word:
2555 {
2556     \str_case:Vn \l__spintent_spmQ_join_word_str
2557     {
2558         { entero } { \MathMLintent { _entero } { \c__spintent_invisible_sep_tl } }
2559         { enteros } { \MathMLintent { _enteros } { \c__spintent_invisible_sep_tl } }
2560         { y } { \MathMLintent { _y- } { \c__spintent_invisible_sep_tl } }
2561     }
2562 }

```

(End of definition for `\__spintent_spmQ_set_join_word:`)

```
\__spintent_spmQ_print_scale_int:
```

The function `\__spintent_spmQ_print_scale_int:` prints the integer part of the “mixed number” scaled to match the visual height of the accompanying fraction. Before calling the Lua function `\__spintent_luafun_spmQ_scale_int:` it temporarily disables `luamml` by saving the current value of `\l__luamml_flag_int` into `\l__spintent_saved_luamml_flag_int` and setting it to 0 (skip completely). This prevents `luamml` from generating a

spurious `<math>` wrapper around the `sub_box` node that holds the scaled integer, which is processed in isolation before `\frac` has expanded. Once the Lua function returns, the original flag value is restored via `\l__spintenc_saved_luaamml_flag_int`.

The Lua function receives the integer digits from `\l__spintenc_luaaset_part_int_tl` and the fraction command name (`\frac` or `\dfrac`, determined by the key `print-dfrac`) as its two arguments, and inserts the appropriately scaled `sub_box` node directly into the current horizontal list.

```

2563 \cs_new_protected:Nn \l__spintenc_spmQ_print_scale_int:
2564 {
2565   \int_set_eq:Nc \l__spintenc_saved_luaamml_flag_int { l__luaamml_flag_int }
2566   \int_set:Nn \l__luaamml_flag_int { 0 }
2567   \bool_if:NTF \l__spintenc_spmQ_print_dfrac_bool
2568     { \l__spintenc_luafun_spmQ_scale_int:Vn \l__spintenc_luaaset_part_int_tl { dfrac } }
2569     { \l__spintenc_luafun_spmQ_scale_int:Vn \l__spintenc_luaaset_part_int_tl { frac } }
2570   \int_set_eq:cN { l__luaamml_flag_int } \l__spintenc_saved_luaamml_flag_int
2571 }

```

(End of definition for `\l__spintenc_spmQ_print_scale_int:`)

`\l__spintenc_spmQ_print_scale_int:n`

The function `\l__spintenc_spmQ_print_scale_int:n` takes the integer part of the “mixed number” passed in ‘#1’. It first calls `\l__spintenc_luafun_clean_split_and_set:n` on ‘#1’ to parse and populate the shared Lua variables, then checks the status of `\l__spintenc_luaaset_has_integer_str`.

If the status is `true`, it inspects `\l__spintenc_luaaset_sign_tl`. When the sign is *empty*, it delegates directly to `\l__spintenc_spmQ_print_scale_int:` to render the scaled integer. When a sign is present, it sets `\l__spintenc_spmQ_intent_read_sign_str` to either `_más:prefix($e)` or `_menos:prefix($e)` and wraps the sign glyph together with a `MathML` argument (`$e`) containing the scaled integer inside a `\MathMLintent`, so that NVDA/MathCAT verbalizes the sign as a prefix before the integer value. An `\l__spintenc_invisible_sep:` call between the sign and the `MathMLarg` forces the opening of a new `<mrow>` in the `MathML` structure, which is required for the intent tree to be well-formed.

If the status is `false`, the `spmQ-not-integer` “error” is raised immediately.

```

2572 \cs_new_protected:Nn \l__spintenc_spmQ_print_scale_int:n
2573 {
2574   \l__spintenc_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2575   \str_if_eq:VnTF \l__spintenc_luaaset_has_integer_str { true }
2576   {
2577     \tl_if_empty:NTF \l__spintenc_luaaset_sign_tl
2578     {
2579       \l__spintenc_spmQ_print_scale_int:
2580     }
2581     {
2582       \str_if_eq:VnTF \l__spintenc_luaaset_sign_tl { + }
2583       {
2584         \str_set:Nn
2585           \l__spintenc_spmQ_intent_read_sign_str { _más:prefix($e) }
2586       }
2587       {
2588         \str_set:Nn
2589           \l__spintenc_spmQ_intent_read_sign_str { _menos:prefix($e) }
2590       }
2591       \MathMLintent { \l__spintenc_spmQ_intent_read_sign_str }
2592       {
2593         \l__spintenc_luaaset_sign_tl \c__spintenc_invisible_sep_tl %%\l__spintenc_invisible_sep:
2594         \MathMLarg { e } { \l__spintenc_spmQ_print_scale_int: }
2595       }
2596     }
2597   }
2598   { \msg_error:nnn { spintenc } { spmQ-not-integer } { #1 } }
2599 }

```

(End of definition for `\l__spintenc_spmQ_print_scale_int:n`)

`\l__spintenc_spmQ_print_spmZ_dfrac:nn`

The function `\l__spintenc_spmQ_print_spmZ_dfrac:nn` typesets the fractional part of the “mixed number”. It takes the numerator in ‘#1’ and the denominator in ‘#2’. It first calls `\l__spintenc_luafun_clean_split_and_set:` on ‘#1’ to validate that the numerator is an integer; if `\l__spintenc_luaaset_has_integer_str` is `false`, the `spmQ-not-integer` “error” is raised immediately.

If the validation passes, the function dispatches on the two boolean variables `\l__spintenc_spmQ_use_spmZ_bool` and `\l__spintenc_spmQ_print_dfrac_bool`, controlled respectively by the keys `use-spmZ` and `print-dfrac`. When `use-spmZ` is `true`, both the numerator and denominator are passed through `\spnZ` to gain `MathML` integer semantics; otherwise, the raw token lists ‘#1’ and ‘#2’ are used directly. In both

branches, the `fraction` command is selected dynamically via `\use:c`: if `print-dfrac` is `true`, `dfrac` is assembled; otherwise, `frac` is used.

```

2600 \cs_new_protected:Nn \__spintent_spmQ_print_spnZ_dfrac:nn
2601 {
2602   \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
2603   \str_if_eq:VnT \__spintent_luaaset_has_integer_str { false }
2604   {
2605     \msg_error:nnn { spintent } { spmQ-not-integer } { #1 }
2606   }
2607   \bool_if:NTF \__spintent_spmQ_use_spnZ_bool
2608   {
2609     \use:c { \bool_if:NT \__spintent_spmQ_print_dfrac_bool { d } frac }
2610     { \spnZ { #1 } } { \spnZ { #2 } }
2611   }
2612   {
2613     \use:c { \bool_if:NT \__spintent_spmQ_print_dfrac_bool { d } frac }
2614     { #1 } { #2 }
2615   }
2616 }

```

(End of definition for `\__spintent_spmQ_print_spnZ_dfrac:nn`.)

`\__spintent_spmQ_print:nnn` The function `\__spintent_spmQ_print:nnn` is the main rendering orchestrator for `\spmQ`. It takes three arguments: the integer part `#1`, the numerator `#2`, and the denominator `#3`. It opens the optional parentheses via `\__spintent_spmQ_start_wrap_parens:`, then calls `\__spintent_spmQ_print_scale_int:n` to scale and render the integer part. Next, it injects the grammatical connector between the integer and the fraction via `\__spintent_spmQ_set_join_word:`. Then it calls `\__spintent_spmQ_print_spnZ_dfrac:nn` to typeset the fraction using the keys `print-dfrac` and `use-spnZ`. Finally, it closes the optional parentheses via `\__spintent_spmQ_stop_wrap_parens:`.

```

2617 \cs_new_protected:Nn \__spintent_spmQ_print:nnn
2618 {
2619   \__spintent_spmQ_start_wrap_parens:
2620   \__spintent_spmQ_print_scale_int:n { #1 }
2621   \__spintent_spmQ_set_join_word:
2622   \__spintent_spmQ_print_spnZ_dfrac:nn { #2 } { #3 }
2623   \__spintent_spmQ_stop_wrap_parens:
2624 }

```

(End of definition for `\__spintent_spmQ_print:nnn`.)

`\__spintent_spmQ_exec:nnnn` The internal function `\__spintent_spmQ_exec:nnnn` takes four arguments: the optional `<keys>` `#1`, the integer `#2`, the numerator `#3`, and the denominator `#4`. First sets the optional `<keys>` from `#1`, then processes the denominator input in `#4` using `\__spintent_luafun_clean_split_and_set:n` to validate it is a non-zero integer.

It checks the status of `\__spintent_luaaset_has_integer_str`. If `true` and the integer part is exactly zero, it issues the `spmQ-zero-denominator` “error”. Otherwise, whether the status is `false` or the integer part is *non-zero*, it calls `\__spintent_spmQ_print:nnn` to format the “mixed number”.

```

2625 \cs_new_protected:Nn \__spintent_spmQ_exec:nnnn
2626 {
2627   \keys_set:nn { spintent / spmQ } { #1 }
2628   \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
2629   \str_if_eq:VnTF \__spintent_luaaset_has_integer_str { true }
2630   {
2631     \int_compare:nNnTF { \__spintent_luaaset_part_int_tl } = { 0 }
2632     { \msg_error:nn { spintent } { spmQ-zero-denominator } }
2633     { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2634   }
2635   { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
2636 }

```

(End of definition for `\__spintent_spmQ_exec:nnnn`.)

`\spmQ` The command `\spmQ` is the public interface for formatting “mixed numbers”. It first checks if it is being used in *math mode*, issuing an “error” if not. Inside a local group, it calls `\__spintent_spmQ_exec:nnnn` to process the optional `<keys>` and the `{<arguments>}`.

```

2637 \NewDocumentCommand \spmQ { O{} m m m }
2638 {
2639   \mode_if_math:TF
2640   {
2641     \group_begin:

```

```

2642         \__spintent_spmQ_exec:nnnn { #1 } { #2 } { #3 } { #4 }
2643     \group_end:
2644 }
2645 { \msg_error:nnn { spintent } { need-math-mode } { \spmQ } }
2646 }

```

(End of definition for `\spmQ`. This function is documented on page 16.)

11.33 The commands of the geo family

The commands described in this section provide a semantic interface for geometric objects in Spanish mathematical contexts. They all delegate parsing to `spintent.lua`, sharing the variables `\l__spintent_geo_luaset_error_str`, `\l__spintent_geo_luaset_intent_str`, `\l__spintent_geo_luaset_print_tl`, `\l__spintent_geo_luaset_sub_str` and `\l__spintent_geo_luaset_tipo_str` — constructing a valid MathCAT intent — and a single point is always annotated through `\__spintent_mathmllarg_point:nnn`. Every command also activates `\l__spintent_scope_attr_int` within its own group, so the shared queue never claims content typeset outside a geo command in the same formula.

11.33.1 Definition of shared variables

Internal variables shared across all commands of the family to avoid repeated calls into Lua.

```

\l__spintent_geo_luaset_error_str
\l__spintent_geo_luaset_intent_str
\l__spintent_geo_luaset_print_tl
\l__spintent_geo_luaset_is_name_str
\l__spintent_geo_luaset_poly_sym_str
\l__spintent_geo_luaset_angulo_case_str
\l__spintent_geo_luaset_concept_str
\l__spintent_geo_luaset_letra_word_str
\l__spintent_geo_luaset_sub_spoken_str
\l__spintent_geo_luaset_nombre_es_num_str
\l__spintent_geo_luaset_letra_str
\l__spintent_geo_luaset_sub_str
\l__spintent_geo_luaset_tipo_str

```

(End of definition for `\l__spintent_geo_luaset_error_str` and others.)

11.34 The command `\spcoord`

The user command `\spcoord` typesets an ordered pair (x, y) , with the comma (or semicolon) between them carrying its own `:infix` intent, and each side wrapped with `:prefix($vp)/:prefix($vn)` when it starts with $+/-$, so the sign is always read as *más/menos* — never *positivo/negativo*, which is not how MathCAT reads a bare sign in this position.

11.34.1 Definition of variables for `\spcoord`

Set by `\__spintent_luafun_spcoord_parse_and_set:n`. `\l__spintent_spcoord_luaset_sep_str` is `,` or `;`, whichever the argument actually used. For each side, `\l__spintent_spcoord_luaset_ #1 _sign_str` is `+`, `-`, or empty; `\l__spintent_spcoord_luaset_ #1 _type_str` is `number` or `raw`; `\l__spintent_spcoord_luaset_ #1 _value_str` is the value with the sign already removed; `\l__spintent_spcoord_luaset_ #1 _has_frac_str` is `none`, `frac`, or `dfrac`.

```

\l__spintent_spcoord_luaset_error_str
\l__spintent_spcoord_luaset_sep_str
\l__spintent_spcoord_luaset_x_sign_str
\l__spintent_spcoord_luaset_x_type_str
\l__spintent_spcoord_luaset_x_value_str
\l__spintent_spcoord_luaset_x_has_frac_str
\l__spintent_spcoord_luaset_y_sign_str
\l__spintent_spcoord_luaset_y_type_str
\l__spintent_spcoord_luaset_y_value_str
\l__spintent_spcoord_luaset_y_has_frac_str

```

(End of definition for `\l__spintent_spcoord_luaset_error_str` and others.)

```

\l__spintent_spcoord_raw_arg_tl
\l__spintent_spcoord_intent_str
\l__spintent_spcoord_wrap_parens_l_tl
\l__spintent_spcoord_wrap_parens_r_tl

```

(End of definition for `\l__spintent_spcoord_raw_arg_tl` and others.)

11.34.2 Definition of error messages for \spcoord

spcoord-invalid-arg

```

2675 \msg_new:nnn { spintrent } { spcoord-invalid-arg }
2676 {
2677   Invalid~argument~'#1'~for~\token_to_str:N \spcoord.
2678 }

```

(End of definition for spcoord-invalid-arg.)

11.34.3 Definition of keys for \spcoord

silent-cmd

We define the key `silent-cmd`.

```

2679 \keys_define:nn { spintrent / spcoord }
2680 {
2681   silent-cmd .bool_set:N = \l__spintrent_spcoord_silent_cmd_bool,
2682   silent-cmd .initial:n = false,
2683   silent-cmd .value_required:n = true,
2684   unknown .code:n =
2685     {
2686       \str_set:Nn \l__spintrent_command_name_str { spcoord }
2687       \__spintrent_unknown_keys_cmd:n {#1}
2688     },
2689 }

```

(End of definition for silent-cmd.)

11.34.4 Internal functions for \spcoord

__spintrent_spcoord_set_wrap_parens_size:

With neither side carrying a fraction, the parentheses stay unsized. Otherwise, `\mathstyle` picks `\Bigl/\Bigr` for display, `\bigl/\bigr` for text — same mechanism as `\__spintrent_spmQ_set_wrap_parens_size:`. When either side is `dfrac`, the parentheses are forced to `\Bigl/\Bigr` regardless of `\mathstyle`, since `\dfrac` always renders at display height — same criterion as `print-dfrac` in `\spnQ`.

```

2690 \cs_new_protected:Nn \__spintrent_spcoord_set_wrap_parens_size:
2691 {
2692   \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \lparen }
2693   \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \rparen }
2694   \bool_if:nT
2695     {
2696       \bool_lazy_or_p:nn
2697         { \bool_not_p:n { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_x_has_frac_str { none } } }
2698         { \bool_not_p:n { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_y_has_frac_str { none } } }
2699     }
2700     {
2701       \int_case:nnF { \mathstyle }
2702       {
2703         { 0 }
2704         {
2705           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2706           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2707         }
2708         { 1 }
2709         {
2710           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2711           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2712         }
2713         { 2 }
2714         {
2715           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \bigl \lparen }
2716           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \bigr \rparen }
2717         }
2718         { 3 }
2719         {
2720           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_l_tl { \bigl \lparen }
2721           \tl_set:Nn \l__spintrent_spcoord_wrap_parens_r_tl { \bigr \rparen }
2722         }
2723       }
2724     }
2725   \bool_if:nT
2726     {
2727       \bool_lazy_or_p:nn
2728         { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_x_has_frac_str { dfraction } }
2729         { \str_if_eq_p:Vn \l__spintrent_spcoord_luaset_y_has_frac_str { dfraction } }
2730     }

```

```

2731         {
2732             \tl_set:Nn \l__spintenc_spcoord_wrap_parens_l_tl { \Bigl \lparen }
2733             \tl_set:Nn \l__spintenc_spcoord_wrap_parens_r_tl { \Bigr \rparen }
2734         }
2735     }
2736 }

```

(End of definition for `\__spintenc_spcoord_set_wrap_parens_size:.`)

`\__spintenc_spcoord_value:n`

With `\l__spintenc_spcoord_luaset_ #1 _type_str` equal to `number`, calls `\spnum` on `\l__spintenc_spcoord_value:n`. Otherwise, typesets it as given — a `\frac/\dfrac` or a letter expression, untouched. Receives `x` or `y` in ‘`#1`’.

```

2737 \cs_new_protected:Nn \__spintenc_spcoord_value:n
2738 {
2739     \str_if_eq:eeTF { \str_use:c { \l__spintenc_spcoord_luaset_ #1 _type_str } } { number }
2740     { \exp_args:Ne \spnum { \str_use:c { \l__spintenc_spcoord_luaset_ #1 _value_str } } }
2741     { \str_use:c { \l__spintenc_spcoord_luaset_ #1 _value_str } }
2742 }

```

(End of definition for `\__spintenc_spcoord_value:n`.)

`\__spintenc_spcoord_print:n`

With `\l__spintenc_spcoord_luaset_ #1 _sign_str` equal to `+` or `-`, wraps the sign and `\__spintenc_spcoord_value:n` together with `_más:prefix($vp)/_menos:prefix($vn)`, referencing the value through read as `más/menos`, never `positivo/negativo`. With no sign, delegates directly to `\__spintenc_spcoord_value:n`. Receives `x` or `y` in ‘`#1`’.

```

2743 \cs_new_protected:Nn \__spintenc_spcoord_print:n
2744 {
2745     \str_case:enF { \str_use:c { \l__spintenc_spcoord_luaset_ #1 _sign_str } }
2746     {
2747         { + }
2748         {
2749             \MathMLintent { _más:prefix ($vp) }
2750             {
2751                 \str_use:c { \l__spintenc_spcoord_luaset_ #1 _sign_str }
2752                 \MathMLarg{ vp } { \__spintenc_spcoord_value:n { #1 } }
2753             }
2754         }
2755         { - }
2756         {
2757             \MathMLintent { _menos :prefix($vn) }
2758             {
2759                 \str_use:c { \l__spintenc_spcoord_luaset_ #1 _sign_str }
2760                 \MathMLarg{ vn } { \__spintenc_spcoord_value:n { #1 } }
2761             }
2762         }
2763     }
2764     { \__spintenc_spcoord_value:n { #1 } }
2765 }

```

(End of definition for `\__spintenc_spcoord_print:n`.)

`\__spintenc_spcoord_intent_infix:`

Sets `\l__spintenc_spcoord_intent_str` to `_coma:infix` or `_punto-y-coma:infix`, per `\l__spintenc_spcoord_intent:n`, then wraps both sides — each referenced through with it, `$x/$y` standing for `\__spintenc_spcoord_print:n`’s own output.

```

2766 \cs_new_protected:Nn \__spintenc_spcoord_intent_infix:
2767 {
2768     \str_if_eq:VnTF \l__spintenc_spcoord_luaset_sep_str { ; }
2769     { \str_set:Nn \l__spintenc_spcoord_intent_str { _punto-y-coma:infix } }
2770     { \str_set:Nn \l__spintenc_spcoord_intent_str { _coma:infix } }
2771     \MathMLintent{ \l__spintenc_spcoord_intent_str ($x, $y)}
2772     {
2773         \MathMLarg{x}{ \__spintenc_spcoord_print:n { x } }
2774         \MathMLintent { :silent } { \l__spintenc_spcoord_luaset_sep_str }
2775         \MathMLarg{y}{ \__spintenc_spcoord_print:n { y } }
2776     }
2777 }

```

(End of definition for `\__spintenc_spcoord_intent_infix:.`)

`\__spintenc_spcoord_mathml:`

With `silent-cmd` `true`, everything — parentheses and argument alike — is wrapped in a single `-`, reading nothing at all: `\l__spintenc_spcoord_raw_arg_tl` is typeset exactly as given, untouched by

`\__spintent_spcoord_print:n/\__spintent_spcoord_intent_infix:.` Otherwise, the parentheses are silenced individually and `\__spintent_spcoord_intent_infix:` provides the reading in between.

```

2778 \cs_new_protected:Nn \__spintent_spcoord_mathml:
2779 {
2780   \bool_if:NTF \l__spintent_spcoord_silent_cmd_bool
2781   {
2782     \MathMLintent { _- }
2783     {
2784       \l__spintent_spcoord_wrap_parens_l_tl
2785       \l__spintent_spcoord_raw_arg_tl
2786       \l__spintent_spcoord_wrap_parens_r_tl
2787     }
2788   }
2789   {
2790     \MathMLintent { :silent } { \l__spintent_spcoord_wrap_parens_l_tl }
2791     \__spintent_spcoord_intent_infix:
2792     \MathMLintent { :silent } { \l__spintent_spcoord_wrap_parens_r_tl }
2793   }
2794 }

```

(End of definition for `\__spintent_spcoord_mathml:`.)

`\__spintent_spcoord_exec:nn`

Sets the *(keys)* in ‘#1’ and `\l__spintent_spcoord_raw_arg_tl` to ‘#2’. Calls `\__spintent_luafun_spcoord_parse_and_set:n` either way — even with `silent-cmd true`, since `\__spintent_spcoord_set_wrap_parens_size:` still needs `\l__spintent_spcoord_luaset_ #1 _has_frac_str` — but with `silent-cmd true`, forces `\l__spintent_spcoord_luaset_error_str` back to `false` rather than reporting it: the raw argument is typeset regardless of whether it parsed. Otherwise, reports `spcoord-invalid-arg` on error. Either way, sizes the parentheses, then delegates to `\__spintent_spcoord_mathml:` unless there was an error.

```

2795 \cs_new_protected:Nn \__spintent_spcoord_exec:nn
2796 {
2797   \keys_set:nn { spintent / spcoord } {#1}
2798   \tl_set:Nn \l__spintent_spcoord_raw_arg_tl {#2}
2799   \bool_if:NTF \l__spintent_spcoord_silent_cmd_bool
2800   {
2801     \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#2} }
2802     \str_set:Nn \l__spintent_spcoord_luaset_error_str { false }
2803   }
2804   {
2805     \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#2} }
2806     \str_if_eq:VnT \l__spintent_spcoord_luaset_error_str { true }
2807     { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n {#2} } }
2808   }
2809   \__spintent_spcoord_set_wrap_parens_size:
2810   \str_if_eq:VnF \l__spintent_spcoord_luaset_error_str { true }
2811   { \__spintent_spcoord_mathml: }
2812 }

```

(End of definition for `\__spintent_spcoord_exec:nn`.)

`\spcoord`

The public command enforces *math mode*, opens a local group, and delegates to `\__spintent_spcoord_exec:nn`.

```

2813 \NewDocumentCommand \spcoord { 0 } { m }
2814 {
2815   \mode_if_math:TF
2816   {
2817     \group_begin:
2818     \attribute \l__spintent_scope_attr_int = 1
2819     \__spintent_spcoord_exec:nn {#1} {#2}
2820     \group_end:
2821   }
2822   { \msg_error:nnn { spintent } { need-math-mode } { \spcoord } }
2823 }

```

(End of definition for `\spcoord`. This function is documented on page ??.)

11.35 The function `\__spintent_mathmlarg_point:nnn`

Scratch variables for `\__spintent_mathmlarg_point:nnn`: the reference name (#1), and the calling command’s own `read-cmd/silent-cmd` state, looked up by name (#2).

```

\l__spintent_mathmlarg_point_ref_tl
\l__spintent_mathmlarg_point_mode_str
\l__spintent_mathmlarg_point_kv_tl
\l__spintent_mathmlarg_point_silent_bool
2824 \tl_new:N \l__spintent_mathmlarg_point_ref_tl
2825 \str_new:N \l__spintent_mathmlarg_point_mode_str
2826 \tl_new:N \l__spintent_mathmlarg_point_kv_tl
2827 \bool_new:N \l__spintent_mathmlarg_point_silent_bool

```

(End of definition for `\l__spintent_mathmlarg_point_ref_tl` and others.)

`\__spintent_mathmlarg_point:nnn`
`\__spintent_mathmlarg_point:nnV`

Annotates one point (**#3**) with `arg` and, when applicable, a literal `intent`, without wrapping it in its own `<mrow>`. Concept `intent` is passed empty on purpose: the caller applies it externally, via a native `\MathMLintent`.

With `\l__spintent_geo_luaset_tipo_str` equal to `sub`, for `read-cmd=school` the letter alone is tagged through `\__spintent_luafun_wrap_mrow_intent_arg:nn`, and the subscript is appended separately, silenced. For `read-cmd=mathcat`, letter and subscript are typeset together, tagged as a whole through a native `\MathMLarg`.

```

2828 \cs_new_protected:Nn \__spintent_mathmlarg_point:nnn
2829 {
2830   \tl_set:Nn \l__spintent_mathmlarg_point_ref_tl {#1}
2831   \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#3} }
2832   \bool_set_eq:Nc \l__spintent_mathmlarg_point_silent_bool { \l__spintent_ #2 _silent_cmd_bool }
2833   \str_set_eq:Nc \l__spintent_mathmlarg_point_mode_str { \l__spintent_ #2 _mode_str }
2834   \__spintent_invisible_sep:
2835   \str_case:VnF \l__spintent_geo_luaset_tipo_str
2836   {
2837     { sub }
2838     {
2839       \str_if_eq:VnTF \l__spintent_mathmlarg_point_mode_str { school }
2840       {
2841         \tl_set:Ne \l__spintent_mathmlarg_point_kv_tl
2842         { arg = "\l__spintent_mathmlarg_point_ref_tl", intent = "_ \l__spintent_geo_lua
2843         \__spintent_luafun_wrap_mrow_intent_arg:nn
2844         { } { \l__spintent_mathmlarg_point_kv_tl }
2845         {
2846           \l__spintent_geo_luaset_letra_str
2847           \c_math_subscript_token
2848           { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
2849         }
2850       }
2851     }
2852     {
2853       \__spintent_luafun_inner_sep:
2854       \MathMLarg { \l__spintent_mathmlarg_point_ref_tl }
2855       {
2856         {
2857           \l__spintent_geo_luaset_letra_str
2858           \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
2859         }
2860       }
2861     }
2862   }
2863 }

```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the letter with a literal `intent` and appends the mark unsilenced (a prime is not spoken on the base, so nothing to repeat); `read-cmd=mathcat` tags letter and mark together through a native `\MathMLarg`.

```

2864 { prima }
2865 {
2866   \str_if_eq:VnTF \l__spintent_mathmlarg_point_mode_str { school }
2867   {
2868     \tl_set:Ne \l__spintent_mathmlarg_point_kv_tl
2869     { arg = "\l__spintent_mathmlarg_point_ref_tl", intent = "_ \l__spintent_geo_lua
2870     \__spintent_luafun_wrap_mrow_intent_arg:nn
2871     { } { \l__spintent_mathmlarg_point_kv_tl }
2872     { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
2873   }
2874   {
2875     {
2876       \__spintent_luafun_inner_sep:
2877       \MathMLarg { \l__spintent_mathmlarg_point_ref_tl }
2878       {
2879         { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
2880       }
2881     }
2882   }
2883 }
2884 }

```

Otherwise (no subscript, no prime), the same distinction applies to the combined `\__spintent_geo_luaset_print_tl` as a whole — the shared queue for `school` (or when `silent-cmd` is `true`), a native `\MathMLarg` for `mathcat`, never mixed within the same formula, since the queue does not recognize a native `\MathMLarg` claim as already taken.

```

2885     {
2886       \bool_if:NTF \__spintent_mathmlarg_point_silent_bool
2887       {
2888         \tl_set:Nc \__spintent_mathmlarg_point_kv_tl
2889           { arg = "\__spintent_mathmlarg_point_ref_tl", intent = "_" }
2890         \__spintent_luafun_wrap_mrow_intent_arg:nn
2891           { } % debe ir vacío
2892         { \__spintent_mathmlarg_point_kv_tl }
2893         { \__spintent_geo_luaset_print_tl }
2894       }
2895       {
2896         \str_if_eq:NnTF \__spintent_mathmlarg_point_mode_str { school }
2897         {
2898           \tl_set:Nc \__spintent_mathmlarg_point_kv_tl
2899             { arg = "\__spintent_mathmlarg_point_ref_tl", intent = " _ \__spintent_geo_luas"
2900           \__spintent_luafun_wrap_mrow_intent_arg:nn
2901             { } % debe ir vacío
2902             { \__spintent_mathmlarg_point_kv_tl }
2903             { \__spintent_geo_luaset_print_tl }
2904           }
2905           {
2906             \MathMLarg { \__spintent_mathmlarg_point_ref_tl }
2907             { \__spintent_geo_luaset_print_tl }
2908           }
2909         }
2910       }
2911     }
2912 \cs_generate_variant:Nn \__spintent_mathmlarg_point:nnn { nnV }

```

(End of definition for `\__spintent_mathmlarg_point:nnn`.)

11.36 The command `\spPoint`

The user-level command `\spPoint` provides a semantic interface for a single geometric point. It delegates argument validation to the Lua module `spintent.lua`, constructing a valid MathCAT intent.

11.36.1 Definition of variables for `\spPoint`

```

\__spintent_spPoint_mode_str
\__spintent_spPoint_prefix_tl
\__spintent_spPoint_intent_arg_tl

```

Internal variables holding the values set by `read-cmd` and `prefix`.

```

2913 \str_new:N \__spintent_spPoint_mode_str
2914 \tl_new:N \__spintent_spPoint_prefix_tl
2915 \tl_new:N \__spintent_spPoint_intent_arg_tl

```

(End of definition for `\__spintent_spPoint_mode_str`, `\__spintent_spPoint_prefix_tl`, and `\__spintent_spPoint_intent_arg_tl`.)

11.36.2 Definition of error messages for `\spPoint`

`spPoint-invalid-point`

The “error” `spPoint-invalid-point` is raised when the argument passed in ‘`#1`’ is not a single uppercase letter with an optional subscript or prime modifier.

```

2916 \msg_new:nnnn { spintent } { spPoint-invalid-point }
2917 {
2918   Invalid~point~format~in~\token_to_str:N \spPoint:~'~'#1'.
2919 }
2920 {
2921   The~argument~must~be~a~single~uppercase~letter~with~an~optional~
2922   subscript~(\_{...})~or~prime~(').\\
2923 }

```

(End of definition for `spPoint-invalid-point`.)

11.36.3 Definition of keys for `\spPoint`

`read-cmd`
`prefix`
`silent-cmd`
`unknown`

We define the `(keys)` `read-cmd`, `prefix`, `silent-cmd` and `concept`.

```

2924 \keys_define:nn { spintent / spPoint }
2925 {
2926   read-cmd
2927     {
2928       \int_case:nn { \l_keys_choice_int }
2929       {
2930         {1} { \str_set:Nn \__spintent_spPoint_mode_str { school } }

```

```

2931         {2} { \str_set:Nn \l__spintent_spPoint_mode_str { mathcat } }
2932     }
2933 },
2934 read-cmd          .initial:n = school,
2935 read-cmd          .value_required:n = true,
2936 read-cmd / unknown .code:n =
2937     \msg_error:nneee { spintent } { unknown-choice }
2938     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
2939 prefix            .code:n =
2940     { \__spintent_sanitize_affix:Nn \l__spintent_spPoint_prefix_tl {#1} },
2941 prefix            .initial:n = ,
2942 prefix            .value_required:n = true,
2943 silent-cmd        .bool_set:N = \l__spintent_spPoint_silent_cmd_bool,
2944 silent-cmd        .initial:n = false,
2945 silent-cmd        .value_required:n = true,
2946 concept           .bool_set:N = \l__spintent_spPoint_concept_bool,
2947 concept           .initial:n = true,
2948 concept           .value_required:n = true,
2949 unknown           .code:n =
2950     {
2951         \str_set:Nn \l__spintent_command_name_str { spPoint }
2952         \__spintent_unknown_keys_cmd:n {#1}
2953     },
2954 }

```

(End of definition for read-cmd and others.)

11.36.4 Internal functions for \spPoint

__spintent_spPoint_print:n

The internal function `\__spintent_spPoint_print:n` processes the `{(argument)}` passed in ‘#1’. Calls `\__spintent_luafun_geo_point_parse_and_set:n`, reporting `spPoint-invalid-point` on error. If `silent-cmd` is true or `concept` is false, sets `\l__spintent_spPoint_intent_arg_tl` to `_{($pt)}` — forced further to `_-` when `silent-cmd` is true and `read-cmd` is not `school`, since `\__spintent_spPoint_mathml:n` then tags through a native `\MathMLarg` that needs the concept silenced too. Otherwise sets it to `punto:prefix($pt)`, prefixed with `prefix` when given. Finally calls `\__spintent_spPoint_mathml:n`.

```

2955 \cs_new_protected:Nn \__spintent_spPoint_print:n
2956 {
2957     \__spintent_luafun_geo_point_parse_and_set:n { \exp_not:n {#1} }
2958     \str_if_eq:VnT \l__spintent_geo_luaseta_error_str { true }
2959     {
2960         \msg_error:nne { spintent } { spPoint-invalid-point } { \exp_not:n {#1} }
2961     }
2962     \bool_if:nTF
2963     {
2964         \bool_lazy_or:pnn
2965         { \l__spintent_spPoint_silent_cmd_bool }
2966         { ! \l__spintent_spPoint_concept_bool }
2967     }
2968     {
2969         \tl_set:Nn \l__spintent_spPoint_intent_arg_tl { _($pt) }
2970         \bool_if:NT \l__spintent_spPoint_silent_cmd_bool
2971         {
2972             \str_if_eq:VnF \l__spintent_spPoint_mode_str { school }
2973             {
2974                 \tl_set:Nn \l__spintent_spPoint_intent_arg_tl { _- }
2975             }
2976         }
2977     }
2978     {
2979         \tl_set:Ne \l__spintent_spPoint_intent_arg_tl
2980         {
2981             \tl_if_empty:NF \l__spintent_spPoint_prefix_tl
2982             {
2983                 \tl_use:N \l__spintent_spPoint_prefix_tl -
2984             }
2985             punto:prefix($pt)
2986         }
2987     }
2988     \__spintent_spPoint_mathml:n { #1 }
2989 }

```

(End of definition for __spintent_spPoint_print:n.)

`\__spintent_spPoint_mathml:n` The function `\__spintent_spPoint_mathml:n` calls `\__spintent_invisible_sep:` first, then wraps `\__spintent_mathmlarg_point:nnn` — which parses ‘#1’, tags it `arg="pt"`, and silences a real subscript or prime on its own — inside a native `\MathMLintent` carrying `\l__spintent_spPoint_intent_arg_tl` as the concept `intent`, with `\__spintent_luafun_inner_sep:` first inside that group.

```

2990 \cs_new_protected:Nn \__spintent_spPoint_mathml:n
2991 {
2992   \__spintent_invisible_sep:
2993   \MathMLintent { \l__spintent_spPoint_intent_arg_tl }
2994   {
2995     \__spintent_luafun_inner_sep:
2996     \__spintent_mathmlarg_point:nnn { pt } { spPoint } { #1 }
2997   }
2998 }

```

(End of definition for `\__spintent_spPoint_mathml:n`.)

`\__spintent_spPoint_exec:nnn` The function `\__spintent_spPoint_exec:nnn` receives the optional `<keys>` in ‘#1’ and the point `{<argument>}` in ‘#2’, delegating to `\__spintent_spPoint_print:n`. With ‘#3’ given (the optional coordinate), silences the gap between the point and the coordinate first (otherwise read as an implicit product), then temporarily aliases `\l__spintent_spcoord_silent_cmd_bool` to `\l__spintent_spPoint_silent_cmd_bool` — reverted when the calling group closes — and calls `\__spintent_luafun_spcoord_parse_and_set:n` directly, reporting `spcoord-invalid-arg` on error unless silenced, then `\__spintent_spcoord_set_wrap_parens_size` and `\__spintent_spcoord_mathml:` — the same rendering functions `\spcoord` itself uses.

```

2999 \cs_new_protected:Nn \__spintent_spPoint_exec:nnn
3000 {
3001   \keys_set:nn { spintent / spPoint } { #1 }
3002   \__spintent_spPoint_print:n { #2 }
3003   \tl_if_novalue:nF {#3}
3004   {
3005     \MathMLintent { :silent } { \c__spintent_invisible_sep_tl } % silencia el "por"
3006     \bool_set_eq:NN \l__spintent_spcoord_silent_cmd_bool \l__spintent_spPoint_silent_cmd_bool
3007     \tl_set:Nn \l__spintent_spcoord_raw_arg_tl {#3}
3008     \bool_if:NTF \l__spintent_spPoint_coord_silent_cmd_bool
3009     {
3010       \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3011       \str_set:Nn \l__spintent_spcoord_luaset_error_str { false }
3012     }
3013     {
3014       \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3015       \str_if_eq:VnT \l__spintent_spcoord_luaset_error_str { true }
3016       { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n {#3} } }
3017     }
3018     \__spintent_spcoord_set_wrap_parens_size:
3019     \str_if_eq:VnF \l__spintent_spcoord_luaset_error_str { true }
3020     { \__spintent_spcoord_mathml: }
3021   }
3022 }

```

(End of definition for `\__spintent_spPoint_exec:nnn`.)

`\spPoint` The public command `\spPoint` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spPoint_exec:nnn`.

```

3023 \NewDocumentCommand \spPoint { 0{} m d() }
3024 {
3025   \mode_if_math:TF
3026   {
3027     \group_begin:
3028     \attribute \l__spintent_scope_attr_int = 1
3029     \__spintent_spPoint_exec:nnn { #1 } { #2 } { #3 }
3030     \group_end:
3031   }
3032   { \msg_error:nnn { spintent } { need-math-mode } { \spPoint } }
3033 }

```

(End of definition for `\spPoint`. This function is documented on page 17.)

11.37 The command `\spvec`

The user command `\spvec` supports two types of notation for the input `{<argument>}`: a pair of points (with `\overrightarrow` applied), or a one-letter name (with `\vec` applied instead). Both notations are automatically detected by the Lua module. An optional coordinate, in parentheses, may follow the `{<argument>}`.

11.37.1 Definition of variables for \spvec

The variable `\__spintest_spvec_mode_str` stores the value set by the key `read-cmd` and the variable `\__spintest_spvec_prefix_tl` stores the value set by the key `prefix`.

```

3034 \str_new:N \__spintest_spvec_mode_str
3035 \tl_new:N \__spintest_spvec_prefix_tl
3036 \tl_new:N \__spintest_spvec_intent_arg_tl
3037 \tl_new:N \__spintest_spvec_kv_tl
3038 \seq_new:N \__spintest_spvec_pt_seq
3039 \tl_new:N \__spintest_spvec_refs_tl
3040 \int_new:N \__spintest_spvec_pt_idx_int

```

(End of definition for `\__spintest_spvec_mode_str` and others.)

11.37.2 Definition of error messages for \spvec

spvec-invalid-arg

The “error” `spvec-invalid-arg` is raised when the argument fails to parse, when a one-letter name carries a prime, or when the point list is not exactly two points.

```

3041 \msg_new:nnn { spintest } { spvec-invalid-arg }
3042 {
3043   Invalid~argument~'#1'~for~\token_to_str:N \spvec.
3044 }

```

(End of definition for `spvec-invalid-arg`.)

11.37.3 Definition of keys for \spvec

silent-cmd

We define the *(keys)* `silent-cmd`, `concept`, `autofit`, `read-cmd` and `prefix`.

```

concept 3045 \keys_define:nn { spintest / spvec }
autofit 3046 {
read-cmd 3047   silent-cmd      .bool_set:N = \__spintest_spvec_silent_cmd_bool,
prefix   3048   silent-cmd      .initial:n = false,
unknown  3049   silent-cmd      .value_required:n = true,
3050   concept          .bool_set:N = \__spintest_spvec_concept_bool,
3051   concept          .initial:n = true,
3052   concept          .value_required:n = true,
3053   autofit          .bool_set:N = \__spintest_spvec_autofit_bool,
3054   autofit          .initial:n = false,
3055   autofit          .value_required:n = true,
3056   read-cmd         .choices:nn = { school , mathcat }
3057   {
3058     \int_case:nn { \_keys_choice_int }
3059     {
3060       {1} { \str_set:Nn \__spintest_spvec_mode_str { school } }
3061       {2} { \str_set:Nn \__spintest_spvec_mode_str { mathcat } }
3062     }
3063   },
3064   read-cmd         .initial:n = school,
3065   read-cmd         .value_required:n = true,
3066   read-cmd / unknown .code:n =
3067     \msg_error:nnee { spintest } { unknown-choice }
3068     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3069   prefix          .code:n =
3070     { \__spintest_sanitize_affix:Nn \__spintest_spvec_prefix_tl {##1} },
3071   prefix          .initial:n = { },
3072   prefix          .value_required:n = true,
3073   unknown         .code:n =
3074     {
3075       \str_set:Nn \__spintest_command_name_str { spvec }
3076       \__spintest_unknown_keys_cmd:n {##1}
3077     },
3078 }

```

(End of definition for `silent-cmd` and others.)

11.37.4 Internal functions for \spvec

__spintest_spvec_print:n

The internal function `\__spintest_spvec_print:n` processes the *{(argument)}* passed in ‘#1’. Calls `\__spintest_luafun_geo_parse_and_set:n`, reporting `spvec-invalid-arg` on error, on a proper name with `\__spintest_geo_luasets_tipo_str` equal to `prima` (a one-letter vector cannot carry a prime), or on a point list that is not exactly two points. Otherwise calls `\__spintest_spvec_mathml:n`.

```

3079 \cs_new_protected:Nn \__spintest_spvec_print:n
3080 {
3081   \__spintest_luafun_geo_parse_and_set:n { \exp_not:n {##1} }

```

```

3082 \str_if_eq:VnT \l__spinttent_geo_luaset_is_name_str { true }
3083 {
3084   \str_if_eq:VnT \l__spinttent_geo_luaset_tipo_str { prima }
3085   { \str_set:Nn \l__spinttent_geo_luaset_error_str { true } }
3086 }
3087 \str_if_eq:VnT \l__spinttent_geo_luaset_is_name_str { false }
3088 {
3089   \int_compare:nT { \clist_count:N \l__spinttent_geo_luaset_points_clist != 2 }
3090   { \str_set:Nn \l__spinttent_geo_luaset_error_str { true } }
3091 }
3092 \str_if_eq:VnTF \l__spinttent_geo_luaset_error_str { true }
3093 { \msg_error:nne { spinttent } { spvec-invalid-arg } { \exp_not:n {#1} } }
3094 { \__spinttent_spvec_mathml:n { #1 } }
3095 }

```

(End of definition for `\__spinttent_spvec_print:n`.)

`\__spinttent_spvec_mathml:n`

The function `\__spinttent_spvec_mathml:n` branches on `\l__spinttent_geo_luaset_is_name_str`. For a proper name (a single letter, no prime), the concept `intent` is `_ $\text{\$pt}$` when `silent-cmd` is `true` or `concept` is `false` — forced further to `_-` when `silent-cmd` is `true` and `read-cmd` is not `school`; otherwise `vector:prefix( $\text{\$pt}$ )`. With `\l__spinttent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags `\vec{letra}` through `\__spinttent_luafun_wrap_mrow_intent_arg:nn` with a literal `kv`, the subscript appended and silenced; `read-cmd=mathcat` tags `\vec{...}` through a native `\MathMLarg` instead, the subscript silenced only under `silent-cmd`. `autofit` inserts a `\vphantom{b}` inside `\vec{...}` to normalize its height. Otherwise (no subscript), the same `kv` logic wraps `\vec{print\_tl}` as a whole.

```

3096 \cs_new_protected:Nn \__spinttent_spvec_mathml:n
3097 {
3098   \str_if_eq:VnTF \l__spinttent_geo_luaset_is_name_str { true }
3099   {
3100     \bool_if:nTF
3101     {
3102       \bool_lazy_or_p:nn
3103       { \l__spinttent_spvec_silent_cmd_bool }
3104       { ! \l__spinttent_spvec_concept_bool }
3105     }
3106     {
3107       \tl_set:Nn \l__spinttent_spvec_intent_arg_tl { _( $\text{\$pt}$ ) }
3108       \bool_if:NT \l__spinttent_spvec_silent_cmd_bool
3109       {
3110         \str_if_eq:VnF \l__spinttent_spvec_mode_str { school }
3111         { \tl_set:Nn \l__spinttent_spvec_intent_arg_tl { _- } }
3112       }
3113     }
3114     {
3115       \tl_set:Nn \l__spinttent_spvec_intent_arg_tl { vector:prefix( $\text{\$pt}$ ) }
3116     }
3117   }
3118   \str_case:VnF \l__spinttent_geo_luaset_tipo_str
3119   {
3120     { sub }
3121     {
3122       \str_if_eq:VnTF \l__spinttent_spvec_mode_str { school }
3123       {
3124         \bool_if:NTF \l__spinttent_spvec_silent_cmd_bool
3125         { \tl_set:Nn \l__spinttent_spvec_kv_tl { arg = "pt", intent = "_-" } }
3126         {
3127           \tl_set:Ne \l__spinttent_spvec_kv_tl
3128           { arg = "pt", intent = "_ \l__spinttent_geo_luaset_intent_str -" }
3129         }
3130         \__spinttent_invisible_sep:
3131         \MathMLintent { \l__spinttent_spvec_intent_arg_tl }
3132         {
3133           \__spinttent_luafun_inner_sep:
3134           \__spinttent_luafun_wrap_mrow_intent_arg:nn
3135           { } { \l__spinttent_spvec_kv_tl }
3136         }
3137         \vec
3138         {
3139           \bool_if:NT \l__spinttent_spvec_autofit_bool
3140           { \vphantom{ b } }
3141           \l__spinttent_geo_luaset_letra_str
3142         }
3143       }
3144     }
3145   }
3146 }

```

```

3142         \c_math_subscript_token
3143         { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
3144     }
3145 }
3146 }
3147 {
3148     \__spintent_invisible_sep:
3149     \MathMLintent { \l__spintent_sspec_intent_arg_tl }
3150     {
3151         \__spintent_luafun_inner_sep:
3152         \vec
3153         {
3154             \bool_if:NT \l__spintent_sspec_autofit_bool
3155             { \vphantom{ b } }
3156             \MathMLarg{ pt } { \l__spintent_geo_luaset_letra_str }
3157         }
3158         \bool_if:NT \l__spintent_sspec_silent_cmd_bool
3159         { \c_math_subscript_token { \l__spintent_geo_luaset_sub_str } }
3160     }
3161     \bool_if:NF \l__spintent_sspec_silent_cmd_bool
3162     { \c_math_subscript_token { \l__spintent_geo_luaset_sub_str } }
3163 }
3164 }
3165 }
3166 {
3167     \bool_if:NTF \l__spintent_sspec_silent_cmd_bool
3168     { \tl_set:Nn \l__spintent_sspec_kv_tl { arg = "pt", intent = "_-" } }
3169     {
3170         \tl_set:Nc \l__spintent_sspec_kv_tl
3171         { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3172     }
3173     \__spintent_invisible_sep:
3174     \MathMLintent { \l__spintent_sspec_intent_arg_tl }
3175     {
3176         \__spintent_luafun_inner_sep:
3177         \__spintent_luafun_wrap_mrow_intent_arg:nn
3178         { } { \l__spintent_sspec_kv_tl }
3179     }
3180     \vec
3181     {
3182         \bool_if:NT \l__spintent_sspec_autofit_bool
3183         { \vphantom{ b } }
3184         \l__spintent_geo_luaset_print_tl
3185     }
3186 }
3187 }
3188 }
3189 }

```

For a pair of points, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_sspec_pt_seq` and walked to build `\l__spintent_sspec_refs_tl`, the `$pts1`, `$pts2` reference list. The concept `intent` is `_` (refs) or `vector:prefix` (refs), same conditions as above, applied with a native `\MathMLintent` wrapping `\overrightarrow`. The sequence is walked again, calling `\__spintent_mathmlarg_point:nnn` once per point.

```

3190 {
3191     \seq_set_from_clist:NN \l__spintent_sspec_pt_seq \l__spintent_geo_luaset_points_clist
3192     \tl_clear:N \l__spintent_sspec_refs_tl
3193     \int_zero:N \l__spintent_sspec_pt_idx_int
3194     \seq_map_inline:Nn \l__spintent_sspec_pt_seq
3195     {
3196         \int_incr:N \l__spintent_sspec_pt_idx_int
3197         \tl_if_empty:NF \l__spintent_sspec_refs_tl
3198         { \tl_put_right:Nn \l__spintent_sspec_refs_tl { , } }
3199         \tl_put_right:Nc \l__spintent_sspec_refs_tl { $ pts \int_use:N \l__spintent_sspec_pt_
3200     }
3201     \bool_if:nTF
3202     {
3203         \bool_lazy_or_p:nn
3204         { \l__spintent_sspec_silent_cmd_bool }
3205         { ! \l__spintent_sspec_concept_bool }
3206     }

```

```

3207     {
3208       \tl_set:Nc \l__spintent_spvec_intent_arg_tl { _( \l__spintent_spvec_refs_tl ) }
3209     }
3210     {
3211       \tl_set:Nc \l__spintent_spvec_intent_arg_tl
3212       {
3213         \tl_if_empty:NF \l__spintent_spvec_prefix_tl
3214         { \tl_use:N \l__spintent_spvec_prefix_tl - }
3215         vector:prefix( \l__spintent_spvec_refs_tl )
3216       }
3217     }
3218   \__spintent_invisible_sep:
3219   \MathMLintent { \l__spintent_spvec_intent_arg_tl }
3220   {
3221     \__spintent_luafun_inner_sep:
3222     \overrightarrow
3223     {
3224       \int_zero:N \l__spintent_spvec_pt_idx_int
3225       \seq_map_inline:Nn \l__spintent_spvec_pt_seq
3226       {
3227         \int_incr:N \l__spintent_spvec_pt_idx_int
3228         \__spintent_mathmlarg_point:nnn
3229         { pts \int_use:N \l__spintent_spvec_pt_idx_int }
3230         { spvec }
3231         { ##1 }
3232       }
3233     }
3234   }
3235 }
3236 }

```

(End of definition for `\__spintent_spvec_mathml:n`.)

`\__spintent_spvec_exec:nnn` The function `\__spintent_spvec_exec:nnn` receives the optional $\langle keys \rangle$ in ‘#1’ and the $\{\langle argument \rangle\}$ in ‘#2’, delegating to `\__spintent_spvec_print:n`. With ‘#3’ given (the optional coordinate), aliases `\l__spintent_spcoord_silent_cmd_bool` to `\l__spintent_spvec_silent_cmd_bool` — reverted when the calling group closes — and calls `\__spintent_luafun_spcoord_parse_and_set:n` directly, reporting `spcoord-invalid-arg` on error unless silenced, then `\__spintent_spcoord_set_wrap_parens_size:` and `\__spintent_spcoord_mathml:` — the same rendering functions `\spcoord` itself uses.

```

3237 \cs_new_protected:Nn \__spintent_spvec_exec:nnn
3238 {
3239   \keys_set:nn { spintent / spvec } { #1 }
3240   \__spintent_spvec_print:n { #2 }
3241   \tl_if_novalue:NF {#3}
3242   {
3243     \equal % notación de vector =( , )
3244     \MathMLintent { :silent } { \c__spintent_invisible_sep_tl } % previene lectura de intervalo
3245     \bool_set_eq:NN \l__spintent_spcoord_silent_cmd_bool \l__spintent_spvec_silent_cmd_bool
3246     \tl_set:Nn \l__spintent_spcoord_raw_arg_tl {#3}
3247     \bool_if:NTF \l__spintent_spvec_silent_cmd_bool
3248     {
3249       \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3250       \str_set:Nn \l__spintent_spcoord_luaset_error_str { false }
3251     }
3252     {
3253       \__spintent_luafun_spcoord_parse_and_set:n { \exp_not:n {#3} }
3254       \str_if_eq:VnT \l__spintent_spcoord_luaset_error_str { true }
3255       { \msg_error:nnn { spintent } { spcoord-invalid-arg } { \exp_not:n {#3} } }
3256     }
3257     \__spintent_spcoord_set_wrap_parens_size:
3258     \str_if_eq:VnF \l__spintent_spcoord_luaset_error_str { true }
3259     { \__spintent_spcoord_mathml: }
3260   }
3261 }

```

(End of definition for `\__spintent_spvec_exec:nnn`.)

`\spvec` The public command `\spvec` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spvec_exec:nnn`.

```

3262 \NewDocumentCommand \spvec { 0{ } m d() }
3263 {

```

```

3264 \mode_if_math:TF
3265 {
3266   \group_begin:
3267     \attribute \l__spintent_scope_attr_int = 1
3268     \__spintent_spvec_exec:nnn { #1 } { #2 } { #3 }
3269   \group_end:
3270 }
3271 { \msg_error:nnn { spintent } { need-math-mode } { \spvec } }
3272 }

```

(End of definition for \spvec. This function is documented on page ??.)

11.38 The command \sprecta

The user command `\sprecta` supports two types of notation for the input $\langle argument \rangle$: when the argument contains two points (separated by an optional comma), `\overleftarrow{\overrightarrow{}}` is applied; when the argument is a single point (without comma) or a lowercase letter, it is formatted without any visual decorator. Both notations are automatically detected by the Lua module.

11.38.1 Definition of variables for \sprecta

The variable `\l__spintent_sprecta_mode_str` store the values set by the key `read-cmd` and the variable `\l__spintent_sprayo_prefix_tl` store the value set by the key `prefix`.

```

3273 \str_new:N \l__spintent_sprecta_mode_str
3274 \tl_new:N \l__spintent_sprecta_prefix_tl
3275 \seq_new:N \l__spintent_sprecta_pt_seq
3276 \tl_new:N \l__spintent_sprecta_refs_tl
3277 \tl_new:N \l__spintent_sprecta_intent_arg_tl
3278 \tl_new:N \l__spintent_sprecta_kv_tl
3279 \int_new:N \l__spintent_sprecta_pt_idx_int

```

(End of definition for \l__spintent_sprecta_mode_str and others.)

11.38.2 Definition of error messages for \sprecta

The “error” `sprecta-invalid-points` is raised when the argument matches neither the two-point notation nor the name notation.

```

3280 \msg_new:nnnn { spintent } { sprecta-invalid-points }
3281 {
3282   Invalid~format~in~\token_to_str:N \sprecta:~'#1'~\msg_line_context:.
3283 }
3284 {
3285   See~the~documentation~of~\token_to_str:N \sprecta~for~the~accepted~argument~forms.
3286 }

```

(End of definition for sprecta-invalid-points.)

11.38.3 Definition of keys for \sprecta

We define the $\langle keys \rangle$ `read-cmd`, `prefix` and `silent-cmd`.

```

3287 \keys_define:nn { spintent / sprecta }
3288 {
3289   read-cmd .choices:nn = { school , mathcat }
3290   {
3291     \int_case:nn { \l_keys_choice_int }
3292     {
3293       {1} { \str_set:Nn \l__spintent_sprecta_mode_str { school } }
3294       {2} { \str_set:Nn \l__spintent_sprecta_mode_str { mathcat } }
3295     }
3296   },
3297   read-cmd .initial:n = school,
3298   read-cmd .value_required:n = true,
3299   read-cmd / unknown .code:n =
3300     \msg_error:nnee { spintent } { unknown-choice }
3301     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
3302   prefix .code:n =
3303     { \__spintent_sanitize_affix:Nn \l__spintent_sprecta_prefix_tl {#1} },
3304   prefix .initial:n = ,
3305   prefix .value_required:n = true,
3306   silent-cmd .bool_set:N = \l__spintent_sprecta_silent_cmd_bool,
3307   silent-cmd .initial:n = false,
3308   silent-cmd .value_required:n = true,
3309   concept .bool_set:N = \l__spintent_sprecta_concept_bool,
3310   concept .initial:n = true,
3311   concept .value_required:n = true,

```

```

3312     unknown                .code:n =
3313     {
3314         \str_set:Nn \l__spintent_command_name_str { sprecta }
3315         \__spintent_unknown_keys_cmd:n {#1}
3316     },
3317 }

```

(End of definition for read-cmd and others.)

11.38.4 Internal functions for \sprecta

__spintent_sprecta_print:n

The internal function __spintent_sprecta_print:n processes the $\langle argument \rangle$ passed in ‘#1’. Calls __spintent_luafun_geo_parse_and_set:n, reporting sprecta-invalid-points on error, then __spintent_

```

3318 \cs_new_protected:Nn \__spintent_sprecta_print:n
3319 {
3320     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3321     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
3322     {
3323         \msg_error:nne { spintent } { sprecta-invalid-points } { \exp_not:n {#1} }
3324     }
3325     \__spintent_sprecta_mathml:n { #1 }
3326 }

```

(End of definition for __spintent_sprecta_print:n.)

__spintent_sprecta_mathml:n

The function __spintent_sprecta_mathml:n branches on \l__spintent_geo_luaset_is_name_str. For a proper name (m, l, ...), the concept intent is :silent(\$pt) when silent-cmd is true or concept is false; otherwise recta:prefix(\$pt), prefixed with prefix when given.

With \l__spintent_geo_luaset_tipo_str equal to sub, read-cmd=school tags the base through __spintent_luafun_wrap_mrow_intent_arg:nn with a literal kv, the subscript appended and silenced; read-cmd=mathcat tags base and subscript together through a native \MathMLarg instead.

```

3327 \cs_new_protected:Nn \__spintent_sprecta_mathml:n
3328 {
3329     \str_if_eq:VnTF \l__spintent_geo_luaset_is_name_str { true }
3330     {
3331         \__spintent_invisible_sep:
3332         \bool_if:nTF
3333         {
3334             \bool_lazy_or_p:nn
3335             { \l__spintent_sprecta_silent_cmd_bool }
3336             { ! \l__spintent_sprecta_concept_bool }
3337         }
3338         {
3339             \tl_set:Nn \l__spintent_sprecta_intent_arg_tl { :silent($pt) }
3340         }
3341         {
3342             \tl_set:Ne \l__spintent_sprecta_intent_arg_tl
3343             {
3344                 \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
3345                 { \tl_use:N \l__spintent_sprecta_prefix_tl - }
3346                 recta:prefix($pt)
3347             }
3348         }
3349         \str_case:VnF \l__spintent_geo_luaset_tipo_str
3350         {
3351             { sub }
3352             {
3353                 \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
3354                 {
3355                     \tl_set:Ne \l__spintent_sprecta_kv_tl
3356                     { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3357                     \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3358                     {
3359                         \__spintent_luafun_inner_sep:
3360                         \__spintent_luafun_wrap_mrow_intent_arg:nn
3361                         { } { \l__spintent_sprecta_kv_tl }
3362                     {
3363                         \l__spintent_geo_luaset_letra_str
3364                         \c_math_subscript_token
3365                         { \MathMLintent { :silent } { \l__spintent_geo_luaset_sub_str } }
3366                     }
3367                 }
3368             }
3369         }
3370     }
3371 }

```

```
3368         }
```

For `read-cmd=mathcat`, the same sub case tags base and subscript together through a native `\MathMLarg` instead — never mixed with the shared queue, since the queue does not recognize a native `\MathMLarg` claim as already taken.

```
3369         {
3370             \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3371             {
3372                 {
3373                     \l__spintent_luafun_inner_sep:
3374                     \MathMLarg { pt }
3375                     {
3376                         {
3377                             \l__spintent_geo_luaset_letra_str
3378                             \c_math_subscript_token { \l__spintent_geo_luaset_sub_str }
3379                         }
3380                     }
3381                 }
3382             }
3383         }
3384     }
```

With `\l__spintent_geo_luaset_tipo_str` equal to `prima`, the same distinction applies: `read-cmd=school` tags the base with the literal kv and appends the mark, unsilenced (a prime is not spelled out on the base, so nothing to repeat); `read-cmd=mathcat` tags base and mark together through a native `\MathMLarg`.

```
3385     { prima }
3386     {
3387         \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
3388         {
3389             \tl_set:Nx \l__spintent_sprecta_kv_tl
3390             { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3391             \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3392             {
3393                 \l__spintent_luafun_inner_sep:
3394                 \l__spintent_luafun_wrap_mrow_intent_arg:nn
3395                 { } { \l__spintent_sprecta_kv_tl }
3396                 { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
3397             }
3398         }
3399         {
3400             \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3401             {
3402                 {
3403                     \l__spintent_luafun_inner_sep:
3404                     \MathMLarg { pt }
3405                     {
3406                         { \l__spintent_geo_luaset_letra_str \l__spintent_geo_luaset_sub_str }
3407                     }
3408                 }
3409             }
3410         }
3411     }
3412 }
```

Otherwise (no subscript, no prime), the same kv logic wraps the combined `\l__spintent_geo_luaset_print_tl` as a whole — the shared queue for `school`, its own `arg` alone for `mathcat`.

```
3413     {
3414         \bool_if:NTF \l__spintent_sprecta_silent_cmd_bool
3415         {
3416             \tl_set:Nn \l__spintent_sprecta_kv_tl { arg = "pt", intent = "- " }
3417         }
3418         {
3419             \str_if_eq:VnTF \l__spintent_sprecta_mode_str { school }
3420             {
3421                 \tl_set:Nx \l__spintent_sprecta_kv_tl
3422                 { arg = "pt", intent = "_ \l__spintent_geo_luaset_intent_str -" }
3423             }
3424             {
3425                 \tl_set:Nn \l__spintent_sprecta_kv_tl { arg = "pt" }
3426             }
3427         }
3428         \l__spintent_luafun_wrap_mrow_intent_arg:nn
```

```

3429         { \l__spintent_sprecta_intent_arg_tl }
3430         { \l__spintent_sprecta_kv_tl }
3431     { \l__spintent_geo_luaset_print_tl }
3432 }
3433 }

```

For a list of points, `\l__spintent_geo_luaset_points_clist` is split into `\l__spintent_sprecta_pt_seq` and walked to build `\l__spintent_sprecta_refs_tl`, the `$pt1, $pt2, ...` reference list. The concept `intent` is `_(refs)` or `recta:prefix(refs)`, same conditions as above, applied with a native `\MathMLintent` wrapping `\overleftrightharrow`. The sequence is walked again, calling `\l__spintent_mathmlarg_point` once per point.

```

3434 {
3435   \seq_set_from_clist:nn \l__spintent_sprecta_pt_seq \l__spintent_geo_luaset_points_clist
3436   \tl_clear:N \l__spintent_sprecta_refs_tl
3437   \int_zero:N \l__spintent_sprecta_pt_idx_int
3438   \seq_map_inline:Nn \l__spintent_sprecta_pt_seq
3439   {
3440     \int_incr:N \l__spintent_sprecta_pt_idx_int
3441     \tl_if_empty:NF \l__spintent_sprecta_refs_tl
3442     { \tl_put_right:Nn \l__spintent_sprecta_refs_tl { , } }
3443     \tl_put_right:Ne \l__spintent_sprecta_refs_tl { $ pt \int_use:N \l__spintent_sprecta_pt_idx_int }
3444   }
3445   \bool_if:nTF
3446   {
3447     \bool_lazy_or_p:nn
3448     { \l__spintent_sprecta_silent_cmd_bool }
3449     { ! \l__spintent_sprecta_concept_bool }
3450   }
3451   {
3452     \tl_set:Ne \l__spintent_sprecta_intent_arg_tl { _ ( \l__spintent_sprecta_refs_tl ) }
3453   }
3454   {
3455     \tl_set:Ne \l__spintent_sprecta_intent_arg_tl
3456     {
3457       \tl_if_empty:NF \l__spintent_sprecta_prefix_tl
3458       { \tl_use:N \l__spintent_sprecta_prefix_tl - }
3459       recta:prefix( \l__spintent_sprecta_refs_tl )
3460     }
3461   }
3462   \l__spintent_invisible_sep:
3463   \MathMLintent { \l__spintent_sprecta_intent_arg_tl }
3464   {
3465     \l__spintent_luafun_inner_sep:
3466     \overleftrightharrow
3467     {
3468       \int_zero:N \l__spintent_sprecta_pt_idx_int
3469       \seq_map_inline:Nn \l__spintent_sprecta_pt_seq
3470       {
3471         \int_incr:N \l__spintent_sprecta_pt_idx_int
3472         \l__spintent_mathmlarg_point:nnn
3473         { pt \int_use:N \l__spintent_sprecta_pt_idx_int }
3474         { sprecta }
3475         { ##1 }
3476       }
3477     }
3478   }
3479 }
3480 }

```

(End of definition for `\l__spintent_sprecta_mathml:n`.)

`\l__spintent_sprecta_exec:nn` The function `\l__spintent_sprecta_exec:nn` receives the optional *(keys)* passed in ‘#1’ and the *(argument)* passed in ‘#2’ delegating the rendering of the latter to `\l__spintent_sprecta_print:n`.

```

3481 \cs_new_protected:Nn \l__spintent_sprecta_exec:nn
3482 {
3483   \keys_set:nn { spintent / sprecta } { #1 }
3484   \l__spintent_sprecta_print:n { #2 }
3485 }

```

(End of definition for `\l__spintent_sprecta_exec:nn`.)

`\sprecta` The public command `\sprecta` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sprecta_exec:nn`.

```

3486 \NewDocumentCommand \sprecta { 0{} m }
3487 {
3488   \mode_if_math:TF
3489   {
3490     \group_begin:
3491     \attribute \__spintent_scope_attr_int = 1
3492     \__spintent_sprecta_exec:nn { #1 } { #2 }
3493     \group_end:
3494   }
3495   { \msg_error:nnn { spintent } { need-math-mode } { \sprecta } }
3496 }

```

(End of definition for `\sprecta`. This function is documented on page 18.)

11.39 The command `\sprayo`

The user command `\sprayo` provides a semantic interface for a ray. The key `read-sty` selects the verbalized term, *rayo* or *semirrecta*, both rendered identically with `\overrightarrow`.

11.39.1 Definition of variables for `\sprayo`

The variable `\__spintent_sprayo_mode_str` store the values set by the key `read-cmd`; the variable `\__spintent_sprayo_prefix_tl` store the value set by the key `prefix` and the variable `\__spintent_sprayo_read_sty_str` stores the value set by the key `read-sty`.

The variables defined here are assigned using `.choices:nn` or `.code:n` within `\keys_define:nn`.

```

3497 \str_new:N \__spintent_sprayo_mode_str
3498 \tl_new:N \__spintent_sprayo_prefix_tl
3499 \str_new:N \__spintent_sprayo_read_sty_str

```

(End of definition for `\__spintent_sprayo_mode_str`, `\__spintent_sprayo_prefix_tl`, and `\__spintent_sprayo_read_sty_str`.)

11.39.2 Definition of error messages for `\sprayo`

`sprayo-invalid-points` Raised when the argument is not a comma-separated pair of uppercase letters.

```

3500 \msg_new:nnnn { spintent } { sprayo-invalid-points }
3501 {
3502   Invalid~point~format~in~\token_to_str:N \sprayo:~'#1'~\msg_line_context:.
3503 }
3504 {
3505   See~the~documentation~of~\token_to_str:N \sprayo~for~the~accepted~point~formats.
3506 }

```

(End of definition for `sprayo-invalid-points`.)

11.39.3 Definition of keys for `\sprayo`

We define the `(keys)` `read-cmd`, `prefix`, `silent-cmd`, `concept` and `read-sty`.

```

read-cmd 3507 \keys_define:nn { spintent / sprayo }
prefix   3508 {
silent-cmd 3509   read-cmd .choices:nn = { school , mathcat }
read-sty   3510   {
unknown    3511     \int_case:nn { \l_keys_choice_int }
3512     {
3513       {1} { \str_set:Nn \__spintent_sprayo_mode_str { school } }
3514       {2} { \str_set:Nn \__spintent_sprayo_mode_str { mathcat } }
3515     }
3516   },
3517   read-cmd .initial:n = school,
3518   read-cmd .value_required:n = true,
3519   read-cmd / unknown .code:n =
3520   { \msg_error:nneee { spintent } { unknown-choice }
3521     { read-cmd } { school,~mathcat } { \exp_not:n {#1} } },
3522   prefix .code:n =
3523   { \__spintent_sanitize_affix:Nn \__spintent_sprayo_prefix_tl {#1} },
3524   prefix .initial:n = ,
3525   prefix .value_required:n = true,
3526   silent-cmd .bool_set:N = \__spintent_sprayo_silent_cmd_bool,
3527   silent-cmd .initial:n = false,
3528   silent-cmd .value_required:n = true,
3529   concept .bool_set:N = \__spintent_sprayo_concept_bool,
3530   concept .initial:n = true,
3531   concept .value_required:n = true,

```

```

3532     read-sty           .choices:nn = { rayo , semirrecta }
3533     {
3534         \int_case:nn { \l_keys_choice_int }
3535         {
3536             {1} { \str_set:Nn \l__spintent_sprayo_read_sty_str { rayo } }
3537             {2} { \str_set:Nn \l__spintent_sprayo_read_sty_str { semirrecta } }
3538         }
3539     },
3540     read-sty           .initial:n = rayo,
3541     read-sty           .value_required:n = true,
3542     read-sty / unknown .code:n =
3543         \msg_error:nnee { spintent } { unknown-choice }
3544         { read-sty } { rayo,~semirrecta } { \exp_not:n {#1} },
3545     unknown           .code:n =
3546     {
3547         \str_set:Nn \l__spintent_command_name_str { sprayo }
3548         \__spintent_unknown_keys_cmd:n {#1}
3549     },
3550 }

```

(End of definition for read-cmd and others.)

11.39.4 Internal functions for \sprayo

Auxiliary variables used by __spintent_sprayo_mathml: to split \l__spintent_geo_luaset_points_clist and build the \$pts1,\$pts2,... reference list.

```

\l__spintent_sprayo_pt_seq
\l__spintent_sprayo_refs_tl
\l__spintent_sprayo_intent_arg_tl
\l__spintent_sprayo_pt_idx_int
3551 \seq_new:N \l__spintent_sprayo_pt_seq
3552 \tl_new:N \l__spintent_sprayo_refs_tl
3553 \tl_new:N \l__spintent_sprayo_intent_arg_tl
3554 \int_new:N \l__spintent_sprayo_pt_idx_int

```

(End of definition for \l__spintent_sprayo_pt_seq and others.)

__spintent_sprayo_print:n The internal function __spintent_sprayo_print:n process the {<argument>} passed in ‘#1’. It calls __spintent_luafun_geo_parse_and_set:n to process the {<argument>}, returning an “error” if the input is invalid, then calls __spintent_sprayo_mathml:. Unlike \sprecta, \sprayo has no proper-name argument: it always takes a list of points, so no intent is built here.

```

3555 \cs_new_protected:Nn \__spintent_sprayo_print:n
3556 {
3557     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#1} }
3558     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
3559     {
3560         \msg_error:nne { spintent } { sprayo-invalid-points } { \exp_not:n {#1} }
3561     }
3562     \__spintent_sprayo_mathml:
3563 }

```

(End of definition for __spintent_sprayo_print:n.)

__spintent_sprayo_mathml: The function __spintent_sprayo_mathml: splits \l__spintent_geo_luaset_points_clist (set by Lua, one raw point per element) into \l__spintent_sprayo_pt_seq and walks it once to build \l__spintent_sprayo_refs_tl, the comma-separated \$pts1,\$pts2,... reference list, naming each point by position.

The concept intent is _(refs) when silent-cmd is true or concept is false; otherwise it is \l__spintent_sprayo_read_sty_str:prefix(refs) — the concept word itself (rayo or semirrecta) comes from the read-sty key, not a literal, since \sprayo verbalizes under two names — prefixed with prefix when given. It is applied with a native \MathMLintent wrapping \overrightarrow, preceded by __spintent_luafun_inner_sep: (a second sibling so the sole <mrow> does not collapse into <math>). The sequence is walked a second time, calling __spintent_mathmlarg_point:nnn once per point with its ptsn reference name and silent-cmd/read-cmd looked up under sprayo: it alone decides arg/literal intent per point, so concept plays no role at that level.

```

3564 \cs_new_protected:Nn \__spintent_sprayo_mathml:
3565 {
3566     \seq_set_from_clist:NN \l__spintent_sprayo_pt_seq \l__spintent_geo_luaset_points_clist
3567     \tl_clear:N \l__spintent_sprayo_refs_tl
3568     \int_zero:N \l__spintent_sprayo_pt_idx_int
3569     \seq_map_inline:Nn \l__spintent_sprayo_pt_seq
3570     {
3571         \int_incr:N \l__spintent_sprayo_pt_idx_int
3572         \tl_if_empty:NF \l__spintent_sprayo_refs_tl
3573         { \tl_put_right:Nn \l__spintent_sprayo_refs_tl { , } }

```

```

3574         \tl_put_right:Ne \l__spintent_sprayo_refs_tl { $ pts \int_use:N \l__spintent_sprayo_pt_id
3575     }
3576     \bool_if:nTF
3577     {
3578         \bool_lazy_or_p:nn
3579         { \l__spintent_sprayo_silent_cmd_bool }
3580         { ! \l__spintent_sprayo_concept_bool }
3581     }
3582     {
3583         \tl_set:Ne \l__spintent_sprayo_intent_arg_tl { _( \l__spintent_sprayo_refs_tl ) }
3584     }
3585     {
3586         \tl_set:Ne \l__spintent_sprayo_intent_arg_tl
3587         {
3588             \tl_if_empty:NF \l__spintent_sprayo_prefix_tl
3589             { \tl_use:N \l__spintent_sprayo_prefix_tl - }
3590             \l__spintent_sprayo_read_sty_str :prefix( \l__spintent_sprayo_refs_tl )
3591         }
3592     }
3593     \__spintent_invisible_sep:
3594     \MathMLintent { \l__spintent_sprayo_intent_arg_tl }
3595     {
3596         \__spintent_luafun_inner_sep:
3597         \overrightarrow
3598         {
3599             \int_zero:N \l__spintent_sprayo_pt_idx_int
3600             \seq_map_inline:Nn \l__spintent_sprayo_pt_seq
3601             {
3602                 \int_incr:N \l__spintent_sprayo_pt_idx_int
3603                 \__spintent_mathmlarg_point:nnn
3604                 { pts \int_use:N \l__spintent_sprayo_pt_idx_int }
3605                 { sprayo }
3606                 { ##1 }
3607             }
3608         }
3609     }
3610 }

```

(End of definition for `\__spintent_sprayo_mathml:`.)

`\__spintent_sprayo_exec:nn` The function `\__spintent_sprayo_exec:nn` receives the optional $\langle keys \rangle$ passed in ‘#1’ and the $\{ \langle argument \rangle \}$ passed in ‘#2’ delegating the rendering of the latter to `\__spintent_sprayo_print:n`.

```

3611 \cs_new_protected:Nn \__spintent_sprayo_exec:nn
3612 {
3613     \keys_set:nn { spintent / sprayo } { #1 }
3614     \__spintent_sprayo_print:n { #2 }
3615 }

```

(End of definition for `\__spintent_sprayo_exec:nn`.)

`\sprayo` The public command `\sprayo` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sprayo_exec:nn`.

```

3616 \NewDocumentCommand \sprayo { 0{} m }
3617 {
3618     \mode_if_math:TF
3619     {
3620         \group_begin:
3621         \attribute \l__spintent_scope_attr_int = 1
3622         \__spintent_sprayo_exec:nn { #1 } { #2 }
3623         \group_end:
3624     }
3625     { \msg_error:nnn { spintent } { need-math-mode } { \sprayo } }
3626 }

```

(End of definition for `\sprayo`. This function is documented on page 18.)

11.40 The commands `\spside` and `\spmside`

The user commands `\spside` and `\spmside` provide a semantic interface for a side, segment or trace determined by two points. The key `read-sty` selects the verbalized term — *lado*, *segmento*, or *trazo*, the latter used in straightedge-and-compass constructions — and the key `read-arg` overrides that term entirely with a free-form string. Additionally, the command `\spmside` provides the key `print-m`, controlling whether `\operatorname{m}` is typeset before the argument.

11.40.1 Definition of variables for \spside and \spmside

Each pair of variables is declared once for both `\spside` and `\spmside` via `\clist_map_inline:nn`. The variables `\__spinttent_spside_mode_str` and `\__spinttent_spside_mode_str` store the values set by the key `read-cmd`; the variables `\__spinttent_spside_prefix_tl` and `\__spinttent_spmside_prefix_tl` store the values set by the key `prefix`.

The variables `\__spinttent_spside_read_sty_str` and `\__spinttent_spmside_read_sty_str` stores the value set by the key `read-sty`.

The variables `\__spinttent_spside_read_arg_tl` and `\__spinttent_spmside_read_arg_tl` stores the value set by the key `read-arg`.

- The variables are defined here as they are assigned using `.choices:nn` or `.code:n` rather than a direct property within `\keys_define:nn`.

```
3627 \clist_map_inline:nn { side , mside }
3628 {
3629   \str_new:c { \__spinttent_sp #1 _mode_str      }
3630   \tl_new:c { \__spinttent_sp #1 _prefix_tl     }
3631   \str_new:c { \__spinttent_sp #1 _read_sty_str  }
3632   \tl_new:c { \__spinttent_sp #1 _read_arg_tl   }
3633 }
```

(End of definition for `\__spinttent_spside_mode_str` and others.)

11.40.2 Definition of error messages for \spside and \spmside

We define an error message for the commands `\spside` and `\spmside`. The “error” is triggered if the user includes a invalid “point” in the `{\argument}` passed in ‘#1’ to the command.

```
3634 \clist_map_inline:nn { side , mside }
3635 {
3636   \msg_new:nnnn { spinttent } { sp #1 -invalid-points }
3637   {
3638     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
3639   }
3640   {
3641     See~the~documentation~of~##1~for~the~accepted~point~formats.
3642   }
3643 }
```

(End of definition for `spside-invalid-points` and `spmside-invalid-points`.)

11.40.3 Definition of keys for \spside and \spmside

We define the shared `\keys` `read-cmd`, `prefix`, `silent-cmd`, `concept`, `read-sty`, `read-arg` and `print-m`.

```
read-cmd 3644 \clist_map_inline:nn { side , mside }
prefix   3645 {
silent-cmd 3646   \keys_define:nn { spinttent / sp #1 }
concept   3647   {
read-sty   3648     read-cmd           .choices:nn = { school , mathcat }
read-arg   3649     {
print-m    3650       \int_case:nn { \l_keys_choice_int }
unknown    3651       {
3652         {1} { \str_set:cn { \__spinttent_sp #1 _mode_str } { school } }
3653         {2} { \str_set:cn { \__spinttent_sp #1 _mode_str } { mathcat } }
3654       }
3655     },
3656     read-cmd           .initial:n = school,
3657     read-cmd           .value_required:n = true,
3658     read-cmd / unknown .code:n =
3659     \msg_error:nneee { spinttent } { unknown-choice }
3660     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3661     prefix             .code:n =
3662     { \__spinttent_sanitiz Affix:cn { \__spinttent_sp #1 _prefix_tl } {##1} },
3663     prefix             .initial:n = ,
3664     prefix             .value_required:n = true,
3665     silent-cmd         .bool_set:c = { \__spinttent_sp #1 _silent_cmd_bool },
3666     silent-cmd         .initial:n = false,
3667     silent-cmd         .value_required:n = true,
3668     concept            .bool_set:c = { \__spinttent_sp #1 _concept_bool },
3669     concept            .initial:n = true,
3670     concept            .value_required:n = true,
3671     read-sty           .choices:nn = { lado , segmento , trazo }
3672     {
3673       \int_case:nn { \l_keys_choice_int }
```

```

3674         {
3675             {1} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { lado      } }
3676             {2} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { segmento } }
3677             {3} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { trazo     } }
3678         }
3679     },
3680     read-sty      .initial:n = lado,
3681     read-sty      .value_required:n = true,
3682     read-sty / unknown .code:n =
3683         \msg_error:nneee { spintent } { unknown-choice }
3684         { read-sty } { lado,~segmento,~trazo } { \exp_not:n {##1} },
3685     read-arg      .code:n =
3686         {
3687             \__spintent_sanitizate_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1}
3688         },
3689     read-arg      .initial:n = ,
3690     read-arg      .value_required:n = true,
3691     print-m       .bool_set:c = { l__spintent_sp #1 _print_m_bool },
3692     print-m       .initial:n = false,
3693     print-m       .value_required:n = true,
3694     unknown       .code:n =
3695         {
3696             \str_set:Nn \__spintent_command_name_str { sp #1 }
3697             \__spintent_unknown_keys_cmd:n {##1}
3698         },
3699     }
3700 }

```

• The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spside`.

(End of definition for `read-cmd` and others.)

```

\__spintent_spside_intent_tl
\__spintent_spside_pt_seq
\__spintent_spside_refs_tl
\__spintent_spside_intent_arg_tl
\__spintent_spside_pt_idx_int

```

`\__spintent_spside_intent_tl` holds the concept word (`read-arg` or `read-sty`). The rest are scratch variables shared by both commands (the function itself is shared) to split `\__spintent_geo_luaset_points_clist` and build the `$pts1, $pts2, ...` reference list.

```

3701 \tl_new:N \__spintent_spside_intent_tl
3702 \seq_new:N \__spintent_spside_pt_seq
3703 \tl_new:N \__spintent_spside_refs_tl
3704 \tl_new:N \__spintent_spside_intent_arg_tl
3705 \int_new:N \__spintent_spside_pt_idx_int

```

(End of definition for `\__spintent_spside_intent_tl` and others.)

```

\__spintent_spside_process:nn

```

The internal function `\__spintent_spside_process:nn` receives the short name of the command (`side` or `mside`) in `#1` and the `{⟨argument⟩}` in `#2`. It calls `\__spintent_luafun_geo_parse_and_set:n` to process `#2`, returning an “error” if invalid, then builds `\__spintent_spside_intent_tl`. When `read-arg` is empty, it is set from `read-sty`, prefixed with ‘medida-del-’ for `\spmside`. When `read-arg` is given, it is used as is, and ‘medida-del-’ is prepended only if `concept` is `true`: with `concept=false`, `read-arg` is the verbatim escape hatch for a full user-written phrase (e.g. `read-arg={medida de la hipotenusa}`), gender and article left to the user. No `intent` is built here: it needs the reference list, which `\__spintent_spside_math` builds after splitting the points.

```

3706 \cs_new_protected:Nn \__spintent_spside_process:nn
3707 {
3708     \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
3709     \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
3710     {
3711         \msg_error:nnee { spintent } { sp #1 -invalid-points }
3712         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3713     }
3714     \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
3715     {
3716         \tl_set:Ne \__spintent_spside_intent_tl { \str_use:c { l__spintent_sp #1 _read_sty_str } }
3717         \str_if_eq:eeT {#1} { mside }
3718         {
3719             \tl_put_left:Nn \__spintent_spside_intent_tl { medida-del- }
3720         }
3721     }
3722     {
3723         \tl_set:Ne \__spintent_spside_intent_tl { \tl_use:c { l__spintent_sp #1 _read_arg_tl } }
3724         \bool_if:cT { l__spintent_sp #1 _concept_bool }
3725         {
3726             \str_if_eq:eeT {#1} { mside }

```

```

3727         {
3728             \tl_put_left:Nn \l__spintent_spside_intent_tl { medida-del- }
3729         }
3730     }
3731 }
3732 \l__spintent_spside_mathml:n { #1 }
3733 }

```

(End of definition for `\l__spintent_spside_process:nn`.)

`\l__spintent_spside_mathml:n` The function `\l__spintent_spside_mathml:n` receives the short command name in ‘#1’. It splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spside_pt_seq` and walks it once to build `\l__spintent_spside_refs_tl`, the comma-separated `$pts1,$pts2,...` reference list, naming each point by position.

```

3734 \cs_new_protected:Nn \l__spintent_spside_mathml:n
3735 {
3736     \seq_set_from_clist:NN \l__spintent_spside_pt_seq \l__spintent_geo_luaset_points_clist
3737     \tl_clear:N \l__spintent_spside_refs_tl
3738     \int_zero:N \l__spintent_spside_pt_idx_int
3739     \seq_map_inline:Nn \l__spintent_spside_pt_seq
3740     {
3741         \int_incr:N \l__spintent_spside_pt_idx_int
3742         \tl_if_empty:NF \l__spintent_spside_refs_tl
3743         { \tl_put_right:Nn \l__spintent_spside_refs_tl { , } }
3744         \tl_put_right:Ne \l__spintent_spside_refs_tl { $ pts \int_use:N \l__spintent_spside_pt_idx_int , }
3745     }

```

The concept `intent` is `_ (refs)` when `silent-cmd` is `true`; otherwise, if `read-arg` is empty, it is `_ (refs)` when `concept` is `false` or `\l__spintent_spside_intent_tl:prefix(refs)` when `true`; if `read-arg` is given, it is always `\l__spintent_spside_intent_tl:prefix(refs)` — `concept` never silences a user-given `read-arg` (see `\l__spintent_spside_process:nn`). The `:prefix(...)` form is prefixed with `prefix` when given.

```

3746     \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
3747     {
3748         \tl_set:Ne \l__spintent_spside_intent_arg_tl { _ ( \l__spintent_spside_refs_tl ) }
3749     }
3750     {
3751         \tl_if_empty:cTF { \l__spintent_sp #1 _read_arg_tl }
3752         {
3753             \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
3754             {
3755                 \tl_set:Ne \l__spintent_spside_intent_arg_tl
3756                 {
3757                     \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
3758                     { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
3759                     \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
3760                 }
3761             }
3762             {
3763                 \tl_set:Ne \l__spintent_spside_intent_arg_tl { _ ( \l__spintent_spside_refs_tl ) }
3764             }
3765         }
3766         {
3767             \tl_set:Ne \l__spintent_spside_intent_arg_tl
3768             {
3769                 \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
3770                 { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
3771                 \l__spintent_spside_intent_tl :prefix( \l__spintent_spside_refs_tl )
3772             }
3773         }
3774     }

```

Applied with a native `\MathMLintent`, preceded by `\l__spintent_luafun_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`). Inside, for `#1=side` the points are always wrapped in `\overline`; for `#1=mside`, when `print-m` is `true` a silent `\operatorname{m}` precedes the `\overline`-wrapped points; when `false`, the points are typeset bare, with no `\overline` at all — the recursive lookup in the shared `mathml_filter` callback finds each point either way, whether nested inside `\overline` or sitting directly in the `\MathMLintent` group. In every case the sequence is walked a second time, calling `\l__spintent_mathmlarg_point:nnn` once per point with its `ptsn` reference name and `silent-cmd/read-cmd` looked up under `sp#1`: it alone decides `arg/literal intent` per point, so `concept` plays no role at that level.

```

3775     \l__spintent_invisible_sep: % (luamml issues #26)

```

```

3776 \MathMLintent { \l__spintent_spside_intent_arg_tl }
3777 {
3778   \__spintent_luafun_inner_sep:
3779   \str_if_eq:eeTF {#1} { side }
3780   {
3781     \overline { \__spintent_spside_points_loop:n {#1} }
3782   }
3783   {
3784     \bool_if:cTF { l__spintent_sp #1 _print_m_bool }
3785     {
3786       \__spintent_luafun_wrap_mrow_intent_arg:nn
3787       { } { intent = ":silent" }
3788       { \operatorname { m } }
3789       \overline { \__spintent_spside_points_loop:n {#1} }
3790     }
3791     {
3792       \__spintent_spside_points_loop:n {#1}
3793     }
3794   }
3795 }
3796 }

```

(End of definition for `\__spintent_spside_mathml:n`.)

`\__spintent_spside_points_loop:n`

Walks `\l__spintent_spside_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per point. Factored out of `\__spintent_spside_mathml:n` since it is needed identically in three places there (side, mside with and without `print-m`); it takes the short command name in `#1` again since a separate macro does not inherit its caller's own `#1`.

```

3797 \cs_new_protected:Nn \__spintent_spside_points_loop:n
3798 {
3799   \int_zero:N \l__spintent_spside_pt_idx_int
3800   \seq_map_inline:Nn \l__spintent_spside_pt_seq
3801   {
3802     \int_incr:N \l__spintent_spside_pt_idx_int
3803     \__spintent_mathmlarg_point:nnn
3804     { pts \int_use:N \l__spintent_spside_pt_idx_int }
3805     { sp #1 }
3806     { ##1 }
3807   }
3808 }

```

(End of definition for `\__spintent_spside_points_loop:n`.)

`\__spintent_spside_exec:nnn`

The function `\__spintent_spside_exec:nnn` receives the *short name* of the command passed in `#1`, the optional *keys* passed in `#2` and the *argument* passed in `#3`, delegating the rendering to `\__spintent_spside_process:nn`.

```

3809 \cs_new_protected:Nn \__spintent_spside_exec:nnn
3810 {
3811   \keys_set:nn { spintent / sp #1 } { #2 }
3812   \__spintent_spside_process:nn { #1 } { #3 }
3813 }

```

(End of definition for `\__spintent_spside_exec:nnn`.)

`\spside`
`\spmside`

Both public commands `\spside` and `\spmside` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spside_exec:nnn`.

```

3814 \NewDocumentCommand \spside { O{} m }
3815 {
3816   \mode_if_math:TF
3817   {
3818     \group_begin:
3819     \attribute \l__spintent_scope_attr_int = 1
3820     \__spintent_spside_exec:nnn { side } { #1 } { #2 }
3821     \group_end:
3822   }
3823   { \msg_error:nnn { spintent } { need-math-mode } { \spside } }
3824 }
3825 \NewDocumentCommand \spmside { O{} m }
3826 {
3827   \mode_if_math:TF
3828   {

```

```

3829     \group_begin:
3830     \attribute \l__spintent_scope_attr_int = 1
3831     \l__spintent_spside_exec:nnn { mside } { #1 } { #2 }
3832     \group_end:
3833   }
3834   { \msg_error:nnn { spintent } { need-math-mode } { \spmside } }
3835 }

```

(End of definition for `\spside` and `\spmside`. These functions are documented on page 19.)

11.41 The commands `\spangle` and `\spmangle`

The user commands `\spangle` and `\spmangle`, provide a semantic interface for angles, following the same generic pattern as `\spside/\spmside`. The key `print-m` chooses between the `\angle/\rightanglesqr` pair, typeset together with an explicit `\operatorname{m}`, and the `\measuredangle/\measuredrightangle` pair, which already conveys the notion of measure on its own; the key `recto` independently chooses the right-angle variant of whichever pair is active.

11.41.1 Definition of variables for `\spangle` and `\spmangle`

```

\l__spintent_spangle_mode_str
\l__spintent_spmangle_mode_str
\l__spintent_spangle_prefix_tl
\l__spintent_spmangle_prefix_tl

```

Each pair of variables is declared once for both `\spangle` and `\spmangle` via `\clist_map_inline:nn`. The variables `\l__spintent_spangle_mode_str` and `\l__spintent_spmangle_mode_str` store the values set by the key `read-cmd`; the variables `\l__spintent_spangle_prefix_tl` and `\l__spintent_spmangle_prefix_tl` store the values set by the key `prefix`.

• The variables defined here are assigned using `.choices:nn` or `.code:n` within `\keys_define:nn`.

```

3836 \clist_map_inline:nn { angle , mangle }
3837 {
3838   \str_new:c { \l__spintent_sp #1 _mode_str }
3839   \tl_new:c { \l__spintent_sp #1 _prefix_tl }
3840 }

```

(End of definition for `\l__spintent_spangle_mode_str` and others.)

11.41.2 Definition of error messages for `\spangle` and `\spmangle`

```

spangle-invalid-arg
spmangle-invalid-arg

```

We define an “error” message for the commands `\spangle` and `\spmangle`. The “error” is triggered if the user includes a `invalid` {`\langle argument \rangle`} passed in ‘`#1`’ to the command.

```

3841 \clist_map_inline:nn { angle , mangle }
3842 {
3843   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
3844   {
3845     Invalid~format~in~##1:~'##2'~\msg_line_context:.
3846   }
3847   {
3848     See~the~documentation~of~##1~for~the~accepted~argument~forms.
3849   }
3850 }

```

(End of definition for `spangle-invalid-arg` and `spmangle-invalid-arg`.)

11.41.3 Definition of keys for `\spangle` and `\spmangle`

```

read-cmd
prefix
recto
print-m
silent-cmd
concept
unknown

```

We define the shared `\keys` `read-cmd`, `prefix`, `recto`, `print-m`, `silent-cmd` and `concept`,

```

3851 \clist_map_inline:nn { angle , mangle }
3852 {
3853   \keys_define:nn { spintent / sp #1 }
3854   {
3855     read-cmd .choices:nn = { school , mathcat }
3856     {
3857       \int_case:nn { \l_keys_choice_int }
3858       {
3859         {1} { \str_set:cn { \l__spintent_sp #1 _mode_str } { school } }
3860         {2} { \str_set:cn { \l__spintent_sp #1 _mode_str } { mathcat } }
3861       }
3862     },
3863     read-cmd .initial:n = school,
3864     read-cmd .value_required:n = true,
3865     read-cmd / unknown .code:n =
3866     \msg_error:nnee { spintent } { unknown-choice }
3867     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
3868     prefix .code:n =
3869     { \l__spintent_sanitise_affix:cn { \l__spintent_sp #1 _prefix_tl } {##1} },
3870     prefix .initial:n = ,
3871     prefix .value_required:n = true,

```

```

3872     recto                .bool_set:c = { l__spintent_sp #1 _recto_bool },
3873     recto                .initial:n  = false,
3874     recto                .value_required:n = true,
3875     print-m              .bool_set:c = { l__spintent_sp #1 _print_m_bool },
3876     print-m              .initial:n  = false,
3877     print-m              .value_required:n = true,
3878     silent-cmd           .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
3879     silent-cmd           .initial:n  = false,
3880     silent-cmd           .value_required:n = true,
3881     concept              .bool_set:c = { l__spintent_sp #1 _concept_bool },
3882     concept              .initial:n  = true,
3883     concept              .value_required:n = true,
3884     unknown              .code:n =
3885     {
3886         \str_set:Nn \l__spintent_command_name_str { sp #1 }
3887         \__spintent_unknown_keys_cmd:n {##1}
3888     },
3889 }
3890 }

```

• The keys `print-m` and `concept` are defined for both commands for simplicity, but has no effect on `\spangle`.

(End of definition for `read-cmd` and others.)

11.41.4 Internal functions for `\spangle` and `\spmangle`

`\__spintent_spangle_sym:n`

The internal function `\__spintent_spangle_sym:n` dispatches on `#1`, `print-m` and `recto` to select one of four symbols: `\angle`/`\rightanglesqr` when `print-m` is true; `\measuredangle`/`\measuredrightangle` otherwise. When `#1` is `angle`, only `recto` applies, choosing between `\angle` and `\rightangle`.

```

3891 \cs_new_protected:Nn \__spintent_spangle_sym:n
3892 {
3893     \str_if_eq:eeTF {#1} { mangle }
3894     {
3895         \bool_if:cTF { l__spintent_sp #1 _print_m_bool }
3896         {
3897             \bool_if:cTF { l__spintent_sp #1 _recto_bool }
3898             { \__spintent_right_angle_sqr: } { \angle }
3899         }
3900         {
3901             \bool_if:cTF { l__spintent_sp #1 _recto_bool }
3902             { \measuredrightangle } { \measuredangle }
3903         }
3904     }
3905     {
3906         \bool_if:cTF { l__spintent_sp #1 _recto_bool }
3907         { \__spintent_right_angle_sqr: } { \angle }
3908     }
3909 }

```

(End of definition for `\__spintent_spangle_sym:n`.)

`\l__spintent_spangle_intent_tl`

`\l__spintent_spangle_intent_tl` holds the concept word (ángulo or medida-del-ángulo, ‘-recto’-suffixed when `recto` is true). The rest are scratch variables shared by both commands to split `\l__spintent_geo_lua` and build the reference list — used only in the tres-puntos and un-punto cases (see `\__spintent_spangle_mathml:n`).

`\l__spintent_spangle_pt_seq`

3910 `\tl_new:N \l__spintent_spangle_intent_tl`

`\l__spintent_spangle_refs_tl`

3911 `\seq_new:N \l__spintent_spangle_pt_seq`

`\l__spintent_spangle_intent_arg_tl`

3912 `\tl_new:N \l__spintent_spangle_refs_tl`

`\l__spintent_spangle_pt_idx_int`

3913 `\tl_new:N \l__spintent_spangle_intent_arg_tl`

`\l__spintent_spangle_print_m_now_bool`

3914 `\int_new:N \l__spintent_spangle_pt_idx_int`

3915 `\str_new:N \l__spintent_spangle_kv_tl`

3916 `\bool_new:N \l__spintent_spangle_print_m_now_bool`

(End of definition for `\l__spintent_spangle_intent_tl` and others.)

`\__spintent_spangle_process:nn`

The internal function `\__spintent_spangle_process:nn` receives the short name of the command (`angle` or `mangle`) in ‘`#1`’ and the `{\argument}` in ‘`#2`’. Calls `\__spintent_luafun_geo_angulo_parse_and_set:n`, reporting `sp#1-invalid-arg` on error. For a proper name that is a single letter (not numeric), also reports the same error when that letter is lowercase — `angle` names take an uppercase letter, number, or Greek letter, never a lowercase Latin one; the length check keeps a Greek letter command such as `\alpha` out of this comparison. Then sets `\l__spintent_spangle_intent_tl` to ángulo (or medida-del-ángulo for `\spmangle`), suffixed with `-recto` when `recto` is true, and calls `\__spintent_spangle_mathml:n`.

```

3917 \cs_new_protected:Nn \__spintent_spangle_process:nn
3918 {

```

```

3919   \__spintent_luafun_geo_angulo_parse_and_set:n { \exp_not:n {#2} }
3920   \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
3921   {
3922     \msg_error:nnee { spintent } { sp #1 -invalid-arg }
3923     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3924   }
3925   \str_if_eq:VnT \__spintent_geo_luaset_angulo_case_str { nombre }
3926   {
3927     \str_if_eq:VnF \__spintent_geo_luaset_nombre_es_num_str { true }
3928     {
3929       \int_compare:nT { \str_count:N \__spintent_geo_luaset_letra_str = 1 }
3930       {
3931         \str_if_eq:eeF { \__spintent_geo_luaset_letra_str } { \str_uppercase:n { \__sp
3932           {
3933             \msg_error:nnee { spintent } { sp #1 -invalid-arg }
3934             { \c_backslash_str sp #1 } { \exp_not:n {#2} }
3935           }
3936         }
3937       }
3938     }
3939     \str_if_eq:eeTF {#1} { mangle }
3940     {
3941       \tl_set:Nn \__spintent_spangle_intent_tl { medida-del-ángulo }
3942     }
3943     {
3944       \tl_set:Nn \__spintent_spangle_intent_tl { ángulo }
3945     }
3946     \bool_if:cT { \__spintent_sp #1 _recto_bool }
3947     {
3948       \tl_put_right:Nn \__spintent_spangle_intent_tl { -recto }
3949     }
3950     \__spintent_spangle_mathml:n {#1}
3951   }

```

(End of definition for __spintent_spangle_process:nn.)

__spintent_spangle_mathml:n

The function __spintent_spangle_mathml:n branches on __spintent_geo_luaset_angulo_case_str. For nombre, the symbol(__spintent_spangle_sym:n) is tagged through __spintent_luafun_wrap_mrow_intent on its own, with __spintent_spangle_intent_tl:prefix (no \$-reference: nombre never refers to more than one thing) – _- when silent-cmd is true or concept is false. With silent-cmd true, the name that follows is silenced as a whole.

```

3952 \cs_new_protected:Nn \__spintent_spangle_mathml:n
3953 {
3954   \__spintent_invisible_sep: % (luamml issues #26)
3955   \str_if_eq:VnTF \__spintent_geo_luaset_angulo_case_str { nombre }
3956   {
3957     \bool_if:cTF { \__spintent_sp #1 _silent_cmd_bool }
3958     {
3959       \tl_set:Nn \__spintent_spangle_intent_arg_tl { _- }
3960     }
3961     {
3962       \bool_if:cTF { \__spintent_sp #1 _concept_bool }
3963       {
3964         \tl_set:Ne \__spintent_spangle_intent_arg_tl
3965         {
3966           \tl_if_empty:cF { \__spintent_sp #1 _prefix_tl }
3967           { \tl_use:c { \__spintent_sp #1 _prefix_tl } - }
3968           \__spintent_spangle_intent_tl :prefix
3969         }
3970       }
3971       {
3972         \tl_set:Nn \__spintent_spangle_intent_arg_tl { _- }
3973       }
3974     }
3975     \tl_set:Ne \__spintent_spangle_kv_tl
3976     { intent = "\__spintent_spangle_intent_arg_tl" }
3977     \__spintent_luafun_wrap_mrow_intent_arg:nn
3978     { } { \__spintent_spangle_kv_tl }
3979     { \__spintent_spangle_sym:n {#1} }
3980     \bool_if:cTF { \__spintent_sp #1 _silent_cmd_bool }
3981     {

```

```

3982         \__spintent_luafun_wrap_mrow_intent_arg:nn { } { intent = "_" }
3983     { \__spintent_geo_luaset_print_tl }
3984 }

```

Otherwise, a numeric name is tagged as a whole, through `\__spintent_luafun_wrap_mrow_intent_arg:nn`, with a literal `intent` built from `\__spintent_geo_luaset_intent_str`.

```

3985     {
3986         \str_if_eq:VnTF \__spintent_geo_luaset_nombre_es_num_str { true }
3987     {
3988         \tl_set:Nx \__spintent_spangle_kv_tl
3989         { intent = "_" \__spintent_geo_luaset_intent_str -" }
3990         \__spintent_luafun_wrap_mrow_intent_arg:nn
3991         { } { \__spintent_spangle_kv_tl }
3992         { \__spintent_geo_luaset_print_tl }
3993     }

```

With `\__spintent_geo_luaset_tipo_str` equal to `sub`, `read-cmd=school` tags the letter through the same function, with a literal `intent`, and the subscript is appended and silenced; `read-cmd=mathcat` leaves both letter and subscript untouched, for MathCAT to read the whole `<msub>` natively (no `arg` is set in either case: there is no `$`-reference to resolve).

```

3994     {
3995         \str_case:VnF \__spintent_geo_luaset_tipo_str
3996         {
3997             { sub }
3998             {
3999                 \str_if_eq:eeTF { \use:c { \__spintent_sp #1 _mode_str } } { school }
4000             {
4001                 \tl_set:Nx \__spintent_spangle_kv_tl
4002                 { intent = "_" \__spintent_geo_luaset_intent_str -" }
4003                 \__spintent_luafun_wrap_mrow_intent_arg:nn
4004                 { } { \__spintent_spangle_kv_tl }
4005                 {
4006                     \__spintent_geo_luaset_letra_str
4007                     \c_math_subscript_token
4008                     { \MathMLintent { :silent } { \__spintent_geo_luaset_sub_str } }
4009                 }
4010             }
4011         }
4012         \__spintent_geo_luaset_letra_str
4013         \c_math_subscript_token { \__spintent_geo_luaset_sub_str }
4014     }
4015 }

```

With `\__spintent_geo_luaset_tipo_str` equal to `prima`, the prime is always read — spoken as part of the literal for `read-cmd=school`, native for `read-cmd=mathcat` — since a prime is not a repeated word to silence.

```

4016     { prima }
4017     {
4018         \str_if_eq:eeTF { \use:c { \__spintent_sp #1 _mode_str } } { school }
4019     {
4020         \tl_set:Nx \__spintent_spangle_kv_tl
4021         { intent = "_" \__spintent_geo_luaset_intent_str -" }
4022         \__spintent_luafun_wrap_mrow_intent_arg:nn
4023         { } { \__spintent_spangle_kv_tl }
4024         { \__spintent_geo_luaset_letra_str \__spintent_geo_luaset_sub_str }
4025     }
4026     {
4027         \__spintent_geo_luaset_letra_str \__spintent_geo_luaset_sub_str
4028     }
4029 }
4030 }
4031 { \__spintent_geo_luaset_print_tl }
4032 }
4033 }
4034 }

```

For `tres-puntos/un-punto`, `\__spintent_geo_luaset_points_clist` is split into `\__spintent_spangle_pts` and walked to build `\__spintent_spangle_refs_tl`, the `$pt1, $pt2, ...` reference list. A single native `\MathMLintent` wraps the symbol and the points together, with `\__spintent_spangle_intent_tl:prefix(refs` — `_` — when `silent-cmd` is `true`, `_`(refs) when `concept` is `false` alone. For `\spmangle` with `print-m`

true, a silent `\operatorname{m}` precedes `__spintent_spangle_sym:n` inside the same `\MathMLintent`. The sequence is walked again, calling `__spintent_mathmlarg_point:nnn` once per point.

```

4935     {
4936       \seq_set_from_clist:NN \l__spintent_spangle_pt_seq \l__spintent_geo_luaset_points_clist
4937       \tl_clear:N \l__spintent_spangle_refs_tl
4938       \int_zero:N \l__spintent_spangle_pt_idx_int
4939       \seq_map_inline:Nn \l__spintent_spangle_pt_seq
4940       {
4941         \int_incr:N \l__spintent_spangle_pt_idx_int
4942         \tl_if_empty:NF \l__spintent_spangle_refs_tl
4943         { \tl_put_right:Nn \l__spintent_spangle_refs_tl { , } }
4944         \tl_put_right:Ne \l__spintent_spangle_refs_tl { $ pt \int_use:N \l__spintent_spangle_pt_idx_int }
4945       }
4946       \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
4947       {
4948         \tl_set:Nn \l__spintent_spangle_intent_arg_tl { _- }
4949       }
4950       {
4951         \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4952         {
4953           \tl_set:Ne \l__spintent_spangle_intent_arg_tl
4954           {
4955             \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4956             { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4957             \l__spintent_spangle_intent_tl :prefix( \l__spintent_spangle_refs_tl )
4958           }
4959         }
4960         {
4961           \tl_set:Ne \l__spintent_spangle_intent_arg_tl { _( \l__spintent_spangle_refs_tl ) }
4962         }
4963       }
4964       \bool_set_false:N \l__spintent_spangle_print_m_now_bool
4965       \str_if_eq:eeT {#1} { mangle }
4966       {
4967         \bool_if:cT { \l__spintent_sp #1 _print_m_bool }
4968         { \bool_set_true:N \l__spintent_spangle_print_m_now_bool }
4969       }
4970       \MathMLintent { \l__spintent_spangle_intent_arg_tl }
4971       {
4972         __spintent_luafun_inner_sep:
4973         \bool_if:NT \l__spintent_spangle_print_m_now_bool
4974         {
4975           __spintent_luafun_wrap_mrow_intent_arg:nn
4976           { } { intent = ":silent" }
4977           { \operatorname { m } }
4978         }
4979         __spintent_luafun_wrap_mrow_intent_arg:nn
4980         { } { intent = ":silent" }
4981         { __spintent_spangle_sym:n {#1} }
4982         \int_zero:N \l__spintent_spangle_pt_idx_int
4983         \seq_map_inline:Nn \l__spintent_spangle_pt_seq
4984         {
4985           \int_incr:N \l__spintent_spangle_pt_idx_int
4986           __spintent_mathmlarg_point:nnn
4987           { pt \int_use:N \l__spintent_spangle_pt_idx_int }
4988           { sp#1 }
4989           { ##1 }
4990         }
4991       }
4992     }
4993   }

```

(End of definition for `__spintent_spangle_mathml:n`)

`__spintent_spangle_exec:nnn`

The function `__spintent_spangle_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *⟨keys⟩* passed in ‘#2’ and the *⟨argument⟩* passed in ‘#3’, delegating the rendering to `__spintent_spangle_process:nn`.

```

4994 \cs_new_protected:Nn __spintent_spangle_exec:nnn
4995 {
4996   \keys_set:nn { spintent / sp #1 } { #2 }
4997   __spintent_spangle_process:nn { #1 } { #3 }

```

```
4098 }
```

(End of definition for `\__spintent_spangle_exec:nnn`.)

`\spangle` Both public commands `\spangle` and `\spmangle` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_spangle_exec:nnn`.

```
4099 \NewDocumentCommand \spangle { 0{} m }
4100 {
4101   \mode_if_math:TF
4102   {
4103     \group_begin:
4104       \attribute \__spintent_scope_attr_int = 1
4105       \__spintent_spangle_exec:nnn { angle } { #1 } { #2 }
4106     \group_end:
4107   }
4108   { \msg_error:nnn { spintent } { need-math-mode } { \spangle } }
4109 }
4110 \NewDocumentCommand \spmangle { 0{} m }
4111 {
4112   \mode_if_math:TF
4113   {
4114     \group_begin:
4115       \attribute \__spintent_scope_attr_int = 1
4116       \__spintent_spangle_exec:nnn { mangle } { #1 } { #2 }
4117     \group_end:
4118   }
4119   { \msg_error:nnn { spintent } { need-math-mode } { \spmangle } }
4120 }
```

(End of definition for `\spangle` and `\spmangle`. These functions are documented on page 20.)

11.42 The commands `\sparc` and `\spmarc`

The user commands `\sparc` and `\spmarc` provide a semantic interface for arcs. Unlike `\spmside` and `\spmangle`, `\spmarc` has no `print-m` key: `\operatorname{m}` is always present, since $m\widehat{AB}$ is the only standard notation for the measure of an arc.

11.42.1 Definition of variables for `\sparc` and `\spmarc`

`\__spintent_sparc_mode_str` Each pair of variables is declared once for both `\sparc` and `\spmarc` via `\clist_map_inline:nn`. The variables `\__spintent_sparc_mode_str` and `\__spintent_spmarc_mode_str` store the values set by the key `read-cmd`; the variables `\__spintent_sparc_prefix_tl` and `\__spintent_spmarc_prefix_tl` store the values set by the key `prefix`.

```
4121 \clist_map_inline:nn { arc , marc }
4122 {
4123   \str_new:c { \__spintent_sp #1 _mode_str }
4124   \tl_new:c { \__spintent_sp #1 _prefix_tl }
4125 }
```

(End of definition for `\__spintent_sparc_mode_str` and others.)

11.42.2 Definition of error messages for `\sparc` and `\spmarc`

`sparc-invalid-points` We define an “error” message for the commands `\sparc` and `\spmarc`. The “error” is triggered if the user includes a invalid `{⟨argument⟩}` passed in ‘#1’ to the command.

```
4126 \clist_map_inline:nn { arc , marc }
4127 {
4128   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
4129   {
4130     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
4131   }
4132   {
4133     See~the~documentation~of~##1~for~the~accepted~point~formats.
4134   }
4135 }
```

(End of definition for `sparc-invalid-points` and `spmarc-invalid-points`.)

11.42.3 Definition of keys for `\sparc` and `\spmarc`

We define the shared *(keys)* `read-cmd`, `prefix`, `silent-cmd` and `concept`.

```

read-cmd  We define the shared (keys) read-cmd, prefix, silent-cmd and concept.
prefix    4136 \clist_map_inline:nn { arc , marc }
silent-cmd 4137 {
concept    4138   \keys_define:nn { spintent / sp #1 }
unknown    4139   {
              4140     read-cmd           .choices:nn = { school , mathcat }
              4141     {
              4142       \int_case:nn { \l_keys_choice_int }
              4143       {
              4144         {1} { \str_set:cn { \__spintent_sp #1 _mode_str } { school } }
              4145         {2} { \str_set:cn { \__spintent_sp #1 _mode_str } { mathcat } }
              4146       }
              4147     },
              4148     read-cmd           .initial:n = school,
              4149     read-cmd           .value_required:n = true,
              4150     read-cmd / unknown .code:n =
              4151       \msg_error:nneee { spintent } { unknown-choice }
              4152       { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
              4153     prefix           .code:n =
              4154       { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _prefix_tl } {##1} },
              4155     prefix           .initial:n = ,
              4156     prefix           .value_required:n = true,
              4157     silent-cmd        .bool_set:c = { \__spintent_sp #1 _silent_cmd_bool },
              4158     silent-cmd        .initial:n = false,
              4159     silent-cmd        .value_required:n = true,
              4160     concept           .bool_set:c = { \__spintent_sp #1 _concept_bool },
              4161     concept           .initial:n = true,
              4162     concept           .value_required:n = true,
              4163     unknown           .code:n =
              4164       {
              4165         \str_set:Nn \l__spintent_command_name_str { sp #1 }
              4166         \__spintent_unknown_keys_cmd:n {##1}
              4167       },
              4168     }
              4169   }

```

🔗 The key `print-m` is defined for both commands for simplicity, but has no effect on `\sparc`.

(End of definition for `read-cmd` and others.)

11.42.4 Internal functions for `\sparc` and `\spmarc`

The function `\__spintent_sparc_wide_arc:n` acts as a fallback when the mathematical font used in the document does not provide the command `\widearc`. The definition of this function is a direct adaptation of a response given by Clea F. Rees (@cfr) to a query in the chat room of TeX-SX.

```

4170 \cs_new_protected_nopar:Nn \__spintent_sparc_wide_arc:n
4171 {
4172   \Umathaccent 7 \symoperators "023DC \scan_stop: {#1}
4173 }

```

(End of definition for `\__spintent_sparc_wide_arc:n`)

`\__spintent_sparc_print_arc:n` The function `\__spintent_sparc_print_arc:n` applies the command `\widearc` to the argument passed to `\sparc` or `\spmarc`. If the mathematical font set in the document does not provide the command `\widearc`, we will use the built-in function `\__spintent_sparc_wide_arc:n`.

```

4174 \cs_new_protected:Nn \__spintent_sparc_print_arc:n
4175 {
4176   \cs_if_exist:NTF \widearc
4177   { \widearc {#1} }
4178   { \__spintent_sparc_wide_arc:n {#1} }
4179 }

```

(End of definition for `\__spintent_sparc_print_arc:n`)

`\l__spintent_sparc_intent_tl` `\l__spintent_sparc_intent_tl` holds the concept word (arco or medida-del-arco). The rest are scratch variables to split `\l__spintent_geo_luaset_points_clist` and build the `$pt1`, `$pt2`, ... reference list used in the concept intent.

```

4180 \tl_new:N \l__spintent_sparc_intent_tl
4181 \seq_new:N \l__spintent_sparc_pt_seq
4182 \tl_new:N \l__spintent_sparc_refs_tl
4183 \tl_new:N \l__spintent_sparc_intent_arg_tl
4184 \int_new:N \l__spintent_sparc_pt_idx_int

```

(End of definition for `\l__spintent_sparc_intent_tl` and others.)

`\__spintent_sparc_process:nn`

The internal function `\__spintent_sparc_process:nn` receives the short name of the command (arc or marc) in ‘#1’ and the $\{(\textit{argument})\}$ in ‘#2’. It calls `\__spintent_luafun_geo_parse_and_set:n` to process ‘#2’, returning an “error” if invalid, then sets `\l__spintent_sparc_intent_tl` to arco (or medida-del-arco for `\spmarc`). No `intent` is built here: it needs the reference list, which `\__spintent_sparc_mathml:n` builds after splitting the points.

```

4185 \cs_new_protected:Nn \__spintent_sparc_process:nn
4186 {
4187   \__spintent_luafun_geo_parse_and_set:n { \exp_not:n {#2} }
4188   \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
4189   {
4190     \msg_error:nnee { spintent } { sp #1 -invalid-points }
4191     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4192   }
4193   \str_if_eq:eeTF {#1} { marc }
4194   { \tl_set:Nn \l__spintent_sparc_intent_tl { medida-del-arco } }
4195   { \tl_set:Nn \l__spintent_sparc_intent_tl { arco } }
4196   \__spintent_sparc_mathml:n {#1}
4197 }

```

(End of definition for `\__spintent_sparc_process:nn`.)

`\__spintent_sparc_mathml:n`

The function `\__spintent_sparc_mathml:n` receives the short command name in ‘#1’. It splits `\l__spintent_geo_` into `\l__spintent_sparc_pt_seq` and walks it once to build `\l__spintent_sparc_refs_tl`, the comma-separated $\$pt1, \$pt2, \dots$ reference list, naming each point by position.

```

4198 \cs_new_protected:Nn \__spintent_sparc_mathml:n
4199 {
4200   \seq_set_from_clist:NN \l__spintent_sparc_pt_seq \l__spintent_geo_luaset_points_clist
4201
4202   \tl_clear:N \l__spintent_sparc_refs_tl
4203   \int_zero:N \l__spintent_sparc_pt_idx_int
4204   \seq_map_inline:Nn \l__spintent_sparc_pt_seq
4205   {
4206     \int_incr:N \l__spintent_sparc_pt_idx_int
4207     \tl_if_empty:NF \l__spintent_sparc_refs_tl
4208     { \tl_put_right:Nn \l__spintent_sparc_refs_tl { , } }
4209     \tl_put_right:Ne \l__spintent_sparc_refs_tl { $ pt \int_use:N \l__spintent_sparc_pt_idx_int }
4210   }

```

The concept `intent` is `_` - when `silent-cmd` is `true` or `concept` is `false`; otherwise `\l__spintent_sparc_intent_` prefixed with `prefix` when given.

```

4211 \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
4212 {
4213   \tl_set:Nn \l__spintent_sparc_intent_arg_tl { _- }
4214 }
4215 {
4216   \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4217   {
4218     \tl_set:Ne \l__spintent_sparc_intent_arg_tl
4219     {
4220       \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4221       { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4222       \l__spintent_sparc_intent_tl :prefix( \l__spintent_sparc_refs_tl )
4223     }
4224   }
4225   {
4226     \tl_set:Nn \l__spintent_sparc_intent_arg_tl { _- }
4227   }
4228 }

```

It is applied with a single native `\MathMLintent` wrapping `\__spintent_sparc_print_arc:n` directly (same shape as `\__spintent_spside_mathml:n`), preceded by `\__spintent_luafun_inner_sep:` (a second sibling so the sole `<mrow>` does not collapse into `<math>`, `luamml` issue #26). A silent `\operatorname{m}` precedes it for `\spmarc` (always, no `print-m` key here). The sequence is walked a second time, calling `\__spintent_mathmlarg_point:nnn` once per point with its `ptn` reference name, looked up under `sp#1`.

```

4229 \__spintent_invisible_sep: % need by MathCAT (bug in luamml issues #26)
4230 \MathMLintent { \l__spintent_sparc_intent_arg_tl }
4231 {
4232   \__spintent_luafun_inner_sep:
4233   \str_if_eq:eeT {#1} { marc }

```

```

4234     {
4235         \__spintent_luafun_wrap_mrow_intent_arg:nn
4236         { } { intent = ":silent" }
4237         { \operatorname { m } }
4238     }
4239     \__spintent_sparc_print_arc:n
4240     {
4241         \int_zero:N \__spintent_sparc_pt_idx_int
4242         \seq_map_inline:Nn \__spintent_sparc_pt_seq
4243         {
4244             \int_incr:N \__spintent_sparc_pt_idx_int
4245             \__spintent_mathmlarg_point:nnn
4246             { pt \int_use:N \__spintent_sparc_pt_idx_int }
4247             { sp #1 }
4248             { ##1 }
4249         }
4250     }
4251 }
4252 }

```

(End of definition for __spintent_sparc_mathml:n.)

__spintent_sparc_exec:nnn

The function `\__spintent_sparc_exec:nnn` receives the *short name* of the command passed in ‘#1’, the optional *keys* passed in ‘#2’ and the *argument* passed in ‘#3’, delegating the rendering to `\__spintent_sparc_process:nn`.

```

4253 \cs_new_protected:Nn \__spintent_sparc_exec:nnn
4254 {
4255     \keys_set:nn { spintent / sp #1 } { #2 }
4256     \__spintent_sparc_process:nn { #1 } { #3 }
4257 }

```

(End of definition for __spintent_sparc_exec:nnn.)

\sparc
\spmarc

Both public commands `\sparc` and `\spmarc` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sparc_exec:nnn`.

```

4258 \NewDocumentCommand \sparc { O{} m }
4259 {
4260     \mode_if_math:TF
4261     {
4262         \group_begin:
4263         \attribute \__spintent_scope_attr_int = 1
4264         \__spintent_sparc_exec:nnn { arc } { #1 } { #2 }
4265         \group_end:
4266     }
4267     { \msg_error:nnn { spintent } { need-math-mode } { \sparc } }
4268 }
4269 \NewDocumentCommand \spmarc { O{} m }
4270 {
4271     \mode_if_math:TF
4272     {
4273         \group_begin:
4274         \attribute \__spintent_scope_attr_int = 1
4275         \__spintent_sparc_exec:nnn { marc } { #1 } { #2 }
4276         \group_end:
4277     }
4278     { \msg_error:nnn { spintent } { need-math-mode } { \spmarc } }
4279 }

```

(End of definition for \sparc and \spmarc. These functions are documented on page 21.)

11.43 The command \spPoly

The user command `\spPoly` typesets a polygon defined by its vertices. The figure type and its visual symbol are inferred from the vertex count: three points produce a triangle, four a square, five or more a generic polygon with no symbol. It never sends a boolean to Lua: `\__spintent_geo_luaseta_intent_str` (the full *intent*, with the figure name).

11.43.1 Definition of variables for \spPoly

The variable `\__spintent_spPoly_mode_str` stores the value set by the key `read-cmd`; the variable `\__spintent_spPoly_prefix_tl` stores the value set by the key `prefix`.

```

4280 \str_new:N \__spintent_spPoly_mode_str
4281 \tl_new:N \__spintent_spPoly_prefix_tl

```

(End of definition for __spintent_spPoly_mode_str and __spintent_spPoly_prefix_tl.)

11.43.2 Definition of error messages for \spPoly

spPoly-invalid-points

Raised when the argument is not a comma-separated list of at least three uppercase letters.

```

4282 \msg_new:nnnn { spintent } { spPoly-invalid-points }
4283 {
4284   Invalid~point~format~in~\token_to_str:N \spPoly:~'#1'~\msg_line_context:.
4285 }
4286 {
4287   See~the~documentation~of~\token_to_str:N \spPoly~for~the~accepted~point~formats.
4288 }

```

(End of definition for spPoly-invalid-points.)

11.43.3 Definition of keys for \spPoly

read-cmd

We define the *(keys)* read-cmd, prefix, print-sym, silent-cmd and concept.

prefix

```

4289 \keys_define:nn { spintent / spPoly }

```

print-sym

```

4290 {

```

silent-cmd

```

4291   read-cmd           .choices:nn = { school , mathcat }

```

concept

```

4292   {

```

unknown

```

4293     \int_case:nn { \l_keys_choice_int }
4294     {
4295       {1} { \str_set:Nn \l__spintent_spPoly_mode_str { school } }
4296       {2} { \str_set:Nn \l__spintent_spPoly_mode_str { mathcat } }
4297     }
4298   },
4299   read-cmd           .initial:n = school,
4300   read-cmd           .value_required:n = true,
4301   read-cmd / unknown .code:n =
4302     \msg_error:nneee { spintent } { unknown-choice }
4303     { read-cmd } { school,~mathcat } { \exp_not:n {#1} },
4304   prefix             .code:n =
4305     { \__spintent_sanitize_affix:Nn \l__spintent_spPoly_prefix_tl {#1} },
4306   prefix             .initial:n = ,
4307   prefix             .value_required:n = true,
4308   print-sym          .bool_set:N = \l__spintent_spPoly_print_sym_bool,
4309   print-sym          .initial:n = true,
4310   print-sym          .value_required:n = true,
4311   silent-cmd         .bool_set:N = \l__spintent_spPoly_silent_cmd_bool,
4312   silent-cmd         .initial:n = false,
4313   silent-cmd         .value_required:n = true,
4314   concept            .bool_set:N = \l__spintent_spPoly_concept_bool,
4315   concept            .initial:n = true,
4316   concept            .value_required:n = true,
4317   unknown            .code:n =
4318     {
4319       \str_set:Nn \l__spintent_command_name_str { spPoly }
4320       \__spintent_unknown_keys_cmd:n {#1}
4321     },
4322   }

```

(End of definition for read-cmd and others.)

11.43.4 Internal functions for \spPoly

\l__spintent_spPoly_pt_seq

Scratch variables to split \l__spintent_geo_luaset_points_clist and build the \$pt1, \$pt2, ... reference list, and to hold the final concept intent once built.

\l__spintent_spPoly_refs_tl

\l__spintent_spPoly_intent_arg_tl

\l__spintent_spPoly_pt_idx_int

```

4323 \seq_new:N \l__spintent_spPoly_pt_seq
4324 \tl_new:N \l__spintent_spPoly_refs_tl
4325 \tl_new:N \l__spintent_spPoly_intent_arg_tl
4326 \int_new:N \l__spintent_spPoly_pt_idx_int

```

(End of definition for \l__spintent_spPoly_pt_seq and others.)

__spintent_spPoly_sym:

Unchanged: reads \l__spintent_geo_luaset_poly_sym_str and typesets the corresponding symbol.

```

4327 \cs_new_protected:Nn \__spintent_spPoly_sym:
4328 {
4329   \str_case:Vn \l__spintent_geo_luaset_poly_sym_str
4330   {
4331     { triangle } { \triangle }
4332     { square } { \mdlgwhtsquare }
4333   }
4334 }

```

(End of definition for `\__spintent_spPoly_sym:`.)

`\__spintent_spPoly_build_refs:` Splits `\__spintent_geo_luaset_points_clist` into `\__spintent_spPoly_pt_seq` and walks it once to build `\__spintent_spPoly_refs_tl`, the comma-separated `$pt1, $pt2, ...` reference list, naming each vertex by position.

```

4335 \cs_new_protected:Nn \__spintent_spPoly_build_refs:
4336 {
4337   \seq_set_from_clist:NN \__spintent_spPoly_pt_seq \__spintent_geo_luaset_points_clist
4338   \tl_clear:N \__spintent_spPoly_refs_tl
4339   \int_zero:N \__spintent_spPoly_pt_idx_int
4340   \seq_map_inline:Nn \__spintent_spPoly_pt_seq
4341   {
4342     \int_incr:N \__spintent_spPoly_pt_idx_int
4343     \tl_if_empty:NF \__spintent_spPoly_refs_tl
4344     { \tl_put_right:Nn \__spintent_spPoly_refs_tl { , } }
4345     \tl_put_right:Ne \__spintent_spPoly_refs_tl { $ pt \int_use:N \__spintent_spPoly_pt_idx.
4346   }
4347 }
```

(End of definition for `\__spintent_spPoly_build_refs:`.)

`\__spintent_spPoly_build_concept:` Builds `\__spintent_spPoly_intent_arg_tl`, independently of `print-sym: _`(refs) when `silent-cmd` is `true` or `concept` is `false`; otherwise `\__spintent_geo_luaset_concept_str:prefix(refs)`, prefixed with `prefix` when given. Must be called after `\__spintent_spPoly_build_refs:`.

```

4348 \cs_new_protected:Nn \__spintent_spPoly_build_concept:
4349 {
4350   \bool_if:NTF \__spintent_spPoly_silent_cmd_bool
4351   {
4352     \tl_set:Ne \__spintent_spPoly_intent_arg_tl { _ ( \__spintent_spPoly_refs_tl ) }
4353   }
4354   {
4355     \bool_if:NTF \__spintent_spPoly_concept_bool
4356     {
4357       \tl_set:Ne \__spintent_spPoly_intent_arg_tl
4358       {
4359         \tl_if_empty:NF \__spintent_spPoly_prefix_tl
4360         { \tl_use:N \__spintent_spPoly_prefix_tl - }
4361         \__spintent_geo_luaset_concept_str :prefix( \__spintent_spPoly_refs_tl )
4362       }
4363     }
4364     {
4365       \tl_set:Ne \__spintent_spPoly_intent_arg_tl { _ ( \__spintent_spPoly_refs_tl ) }
4366     }
4367   }
4368 }
```

(End of definition for `\__spintent_spPoly_build_concept:`.)

`\__spintent_spPoly_points_loop:` Walks `\__spintent_spPoly_pt_seq` a second time, calling `\__spintent_mathmlarg_point:nnn` once per vertex with its `ptn` reference name, looked up under `spPoly:` it alone decides `arg/literal intent` per vertex — replacing the single combined `\MathMLarg{pts}{...}` that lost `arg` on any vertex with a subscript or prime (the same `annotate_attribute/sub-sup` limitation already found and worked around for `\spPoint`).

```

4369 \cs_new_protected:Nn \__spintent_spPoly_points_loop:
4370 {
4371   \int_zero:N \__spintent_spPoly_pt_idx_int
4372   \seq_map_inline:Nn \__spintent_spPoly_pt_seq
4373   {
4374     \int_incr:N \__spintent_spPoly_pt_idx_int
4375     \__spintent_mathmlarg_point:nnn
4376     { pt \int_use:N \__spintent_spPoly_pt_idx_int }
4377     { spPoly }
4378     { ##1 }
4379   }
4380 }
```

(End of definition for `\__spintent_spPoly_points_loop:`.)

`\__spintent_spPoly_process:n` Calls `\__spintent_luafun_geo_poly_parse_and_set:n`, reports `spPoly-invalid-points` on failure, then delegates to `\__spintent_spPoly_mathml:`. No `intent` is built here: it needs the reference list, which `\__spintent_spPoly_mathml:` builds after splitting the vertices.

```

4381 \cs_new_protected:Nn \__spintent_spPoly_process:n
```

```

4382 {
4383   \__spintent_luafun_geo_poly_parse_and_set:n { \exp_not:n {#1} }
4384   \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
4385   {
4386     \msg_error:nne { spintent } { spPoly-invalid-points }
4387     { \exp_not:n {#1} }
4388   }
4389   \__spintent_spPoly_mathml:
4390 }

```

(End of definition for __spintent_spPoly_process:n.)

__spintent_spPoly_mathml: Builds the reference list and the concept intent(__spintent_spPoly_build_refs:, __spintent_spPoly_build_refs:) then wraps everything in a single native \MathMLintent, preceded by __spintent_luafun_inner_sep: (a second sibling so the sole <mrow> does not collapse into <math>, lua^{mm} issue #26). print-sym and concept/silent-cmd are independent: the former only decides whether the glyph (__spintent_spPoly_sym:, silenced with __spintent_luafun_wrap_mrow_intent_arg:nn) is attempted — it is skipped anyway when \l__spintent_geo_luaset_poly_sym_str is empty (no symbol defined for that vertex count, e.g. pentagon and above) — and never affects whether the concept word itself is spoken.

```

4391 \cs_new_protected:Nn \__spintent_spPoly_mathml:
4392 {
4393   \__spintent_invisible_sep: % (luamm issues #26)
4394   \__spintent_spPoly_build_refs:
4395   \__spintent_spPoly_build_concept:
4396   \MathMLintent { \l__spintent_spPoly_intent_arg_tl }
4397   {
4398     \__spintent_luafun_inner_sep:
4399     \bool_if:NT \__spintent_spPoly_print_sym_bool
4400     {
4401       \str_if_empty:NF \__spintent_geo_luaset_poly_sym_str
4402       {
4403         \__spintent_luafun_wrap_mrow_intent_arg:nn
4404         { } { intent = ":silent" }
4405         { \__spintent_spPoly_sym: }
4406       }
4407     }
4408     \__spintent_spPoly_points_loop:
4409   }
4410 }

```

(End of definition for __spintent_spPoly_mathml:.)

__spintent_spPoly_exec:nn The function __spintent_spPoly_exec:nn receives the optional keys in #1 and the argument in #2, delegating the rendering to __spintent_spPoly_process:n.

```

4411 \cs_new_protected:Nn \__spintent_spPoly_exec:nn
4412 {
4413   \keys_set:nn { spintent / spPoly } { #1 }
4414   \__spintent_spPoly_process:n { #2 }
4415 }

```

(End of definition for __spintent_spPoly_exec:nn.)

\spPoly The public command \spPoly enforces *math mode*, opens a local group, and delegates execution to __spintent_spPoly_exec:nn.

```

4416 \NewDocumentCommand \spPoly { 0{} m }
4417 {
4418   \mode_if_math:TF
4419   {
4420     \group_begin:
4421     \attribute \l__spintent_scope_attr_int = 1
4422     \__spintent_spPoly_exec:nn { #1 } { #2 }
4423     \group_end:
4424   }
4425   { \msg_error:nnn { spintent } { need-math-mode } { \spPoly } }
4426 }

```

(End of definition for \spPoly. This function is documented on page 22.)

11.44 The commands `\spAfig` and `\spPfig`

The user commands `\spAfig` and `\spPfig` typeset the area and the perimeter of a polygonal figure, using the delimited-optional argument syntax `d()` already established by `\spmcd/\spmcm`. The key `print-sym` is never sent to Lua: the symbol type is returned unconditionally in `\l__spintent_geo_luaset_poly_sym_str`, and `expl3` decides whether to wrap it in `\MathMLintent{<:silent>}` — the same mechanism already used by `\spPoly`.

11.44.1 Definition of variables for `\spAfig` and `\spPfig`

```
\l__spintent_geo_final_print_tl
```

Scratch variables used by `\__spintent_spfig_process:nn` to assemble the visual and the `intent`. `\l__spintent_geo_final_print_tl` holds the final visual placed under `\mathcal`'s subscript. The final literal `intent` and the concept forwarded when `read-cmd` is `mathcat`.

```
4427 \tl_new:N \l__spintent_geo_final_print_tl
```

(End of definition for `\l__spintent_geo_final_print_tl`.)

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `prefix` and `read-arg`, respectively. The value set by `read-arg` overrides the figure name inferred from the vertex count, independently of `print-sym`.

```
4428 \clist_map_inline:nn { Afig , Pfig }
```

```
4429 {
```

```
4430   \str_new:c { \l__spintent_sp #1 _mode_str }
```

```
4431   \str_new:c { \l__spintent_sp #1 _print_sym_str }
```

```
4432   \tl_new:c { \l__spintent_sp #1 _prefix_tl }
```

```
4433   \tl_new:c { \l__spintent_sp #1 _read_arg_tl }
```

```
4434 }
```

(End of definition for `\l__spintent_spAfig_mode_str` and others.)

11.44.2 Definition of error messages for `\spAfig` and `\spPfig`

All eight messages are generated by the same map, using the same `##1/##2` doubling convention as `\spside/\spmside`. `sp #1-invalid-points` is currently unused by `\__spintent_spfig_process:nn`, which raises only `sp #1-invalid-arg` for any parse failure, regardless of whether a comma was present. `sp #1-sym-letra-conflict` and `sp #1-sym-cardinality-conflict` are raised when `print-sym` is forced to a value incompatible with the actual argument.

```
4435 \clist_map_inline:nn { Afig , Pfig }
```

```
4436 {
```

```
4437   \msg_new:nnnn { spintent } { sp #1 -invalid-points }
```

```
4438   {
```

```
4439     Invalid~point~format~in~##1:~'##2'~\msg_line_context:.
```

```
4440   }
```

```
4441   {
```

```
4442     See~the~documentation~of~##1~for~the~accepted~point~formats.
```

```
4443   }
```

```
4444   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
```

```
4445   {
```

```
4446     Invalid~argument~format~in~##1:~'##2'~\msg_line_context:.
```

```
4447   }
```

```
4448   {
```

```
4449     See~the~documentation~of~##1~for~the~accepted~argument~forms.
```

```
4450   }
```

```
4451   \msg_new:nnnn { spintent } { sp #1 -sym-letra-conflict }
```

```
4452   {
```

```
4453     print-sym~cannot~be~combined~with~a~lettered~argument~in~'##1'~\msg_line_context:.
```

```
4454   }
```

```
4455   {
```

```
4456     See~the~documentation~of~##1~for~details~on~print-sym.
```

```
4457   }
```

```
4458   \msg_new:nnnn { spintent } { sp #1 -sym-cardinality-conflict }
```

```
4459   {
```

```
4460     print-sym~does~not~match~the~vertex~count~in~'##1'~\msg_line_context:.
```

```
4461   }
```

```
4462   {
```

```
4463     See~the~documentation~of~##1~for~details~on~print-sym.
```

```
4464   }
```

```
4465   \msg_new:nnnn { spintent } { sp #1 -read-arg-key }
```

```
4466   {
```

```
4467     read-arg~has~no~effect~with~a~single~argument~in~'##1'~\msg_line_context:.
```

```
4468   }
```

```
4469   {
```

```
4470     See~the~documentation~of~##1~for~details~on~read-arg.
```

```

4471     }
4472 }

```

(End of definition for `spAfig-invalid-points` and others.)

11.44.3 Definition of keys for `\spAfig` and `\spPfig`

The key `print-sym` chooses the visual symbol: `auto` (default) infers it from the vertex count (three points give `\triangle`, four give `\mdlgwhtsquare`, five or more give none); `triangle/square` force one, requiring the vertex count to match; `none` omits it, even with vertices. With a bare number/letter argument, `auto` and `none` behave the same (no symbol); `triangle/square` nest the argument under the forced symbol instead, and cannot be combined with a table letter (F/P/B/L). The key `read-arg` overrides the spoken figure name for real vertices, independently of `print-sym`. The key `read-sub` controls whether the bare number/letter argument is read with the word «sub» or as a plain number.

```

read-cmd  \clist_map_inline:nn { Afig , Pfig }
prefix    {
print-sym  \keys_define:nn { spintent / sp #1 }
read-arg   {
read-sub   {
silent-cmd {
concept    {
unknown    \clist_map_inline:nn { Afig , Pfig }
            {
              \keys_define:nn { spintent / sp #1 }
              {
                read-cmd      .choices:nn = { school , mathcat }
                {
                  \int_case:nn { \l_keys_choice_int }
                  {
                    {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
                    {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
                  }
                },
                read-cmd      .initial:n = school,
                read-cmd      .value_required:n = true,
                read-cmd / unknown .code:n =
                  \msg_error:nneee { spintent } { unknown-choice }
                  { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
                prefix        .code:n =
                  { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
                prefix        .initial:n = ,
                prefix        .value_required:n = true,
                print-sym     .choices:nn = { auto , triangle , square , none }
                {
                  \int_case:nn { \l_keys_choice_int }
                  {
                    {1} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { auto } }
                    {2} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { triangle } }
                    {3} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { square } }
                    {4} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { none } }
                  }
                },
                print-sym     .initial:n = auto,
                print-sym     .value_required:n = true,
                print-sym / unknown .code:n =
                  \msg_error:nneee { spintent } { unknown-choice }
                  { print-sym } { auto,~triangle,~square,~none } { \exp_not:n {##1} },
                read-arg      .code:n =
                  { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1} },
                read-arg      .initial:n = ,
                read-arg      .value_required:n = true,
                read-sub      .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
                read-sub      .initial:n = false,
                read-sub      .value_required:n = true,
                silent-cmd    .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
                silent-cmd    .initial:n = false,
                silent-cmd    .value_required:n = true,
                concept        .bool_set:c = { l__spintent_sp #1 _concept_bool },
                concept        .initial:n = true,
                concept        .value_required:n = true,
                unknown        .code:n =
                  {
                    \str_set:Nn \l__spintent_command_name_str { sp #1 }
                    \__spintent_unknown_keys_cmd:n {##1}
                  },
              },
            }
          }
        }
      }
    }
  }
}

```

(End of definition for `read-cmd` and others.)

11.44.4 Internal functions for \spAfig and \spPfig

`\__spinttent_spfig_sym:` The function `\__spinttent_spfig_sym:` reads `\l__spinttent_geo_print_sym_str` and typesets the corresponding symbol: `\triangle` for the value `triangle`, `\mdlgwhtsquare` for the value `square`.

```

4529 \cs_new_protected:Nn \__spinttent_spfig_sym:
4530 {
4531   \str_case:Vn \l__spinttent_geo_print_sym_str
4532   {
4533     { triangle } { \triangle }
4534     { square } { \mdlgwhtsquare }
4535   }
4536 }

```

(End of definition for `\__spinttent_spfig_sym:`.)

`\l__spinttent_spfig_op_tl` `\l__spinttent_spfig_op_tl` holds `área` or `perímetro`. `\l__spinttent_spfig_intent_tl` holds the concept word for the vertices case (`read-arg` or the auto-detected figure name). The rest are scratch variables to split `\l__spinttent_geo_luaset_points_clist` and build the `$pt1, $pt2, ...` reference list.

```

4537 \tl_new:N \l__spinttent_spfig_op_tl
4538 \tl_new:N \l__spinttent_spfig_intent_tl
4539 \tl_new:N \l__spinttent_spfig_desc_tl
4540 \tl_new:N \l__spinttent_spfig_symbol_tl
4541 \seq_new:N \l__spinttent_spfig_pt_seq
4542 \tl_new:N \l__spinttent_spfig_refs_tl
4543 \tl_new:N \l__spinttent_spfig_intent_arg_tl
4544 \int_new:N \l__spinttent_spfig_pt_idx_int
4545 \str_new:N \l__spinttent_geo_print_sym_str

```

(End of definition for `\l__spinttent_spfig_op_tl` and others.)

`\__spinttent_spfig_process:nn` Receives the short command name (Afig or Pfig) in ‘#1’ and the `{\argument}` in ‘#2’. Calls `\__spinttent_luafun_geo` or `\__spinttent_luafun_geo_pfig_parse_and_set:n`, reports `sp#1-invalid-arg` on failure, and `sp#1-read-arg` when `read-arg` is given for a bare number/letter argument (no vertices), where it has no effect. Sets `\l__spinttent_spfig_op_tl` to `área/perímetro`, then delegates to `\__spinttent_spfig_mathml:n`.

```

4546 \cs_new_protected:Nn \__spinttent_spfig_process:nn
4547 {
4548   \str_if_eq:eeTF {#1} { Afig }
4549   { \__spinttent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
4550   { \__spinttent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
4551   \str_if_eq:VnT \l__spinttent_geo_luaset_error_str { true }
4552   {
4553     \msg_error:nnee { spinttent } { sp #1 -invalid-arg }
4554     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
4555   }
4556   \clist_if_empty:NT \l__spinttent_geo_luaset_points_clist
4557   {
4558     \tl_if_empty:cF { \l__spinttent_sp #1 _read_arg_tl }
4559     {
4560       \msg_error:nne { spinttent } { sp #1 -read-arg-key }
4561       { \c_backslash_str sp #1 }
4562     }
4563   }
4564   \str_if_eq:eeTF {#1} { Afig }
4565   { \tl_set:Nn \l__spinttent_spfig_op_tl { área } }
4566   { \tl_set:Nn \l__spinttent_spfig_op_tl { perímetro } }
4567   \str_set:Ne \l__spinttent_geo_print_sym_str { \str_use:c { \l__spinttent_sp #1 _print_sym_str } }
4568   \__spinttent_spfig_mathml:n {#1}
4569 }

```

(End of definition for `\__spinttent_spfig_process:nn`.)

`\__spinttent_spfig_build_simple:n` Handles the argument-less, table-letter and bare number/letter cases (no vertices). Builds `\l__spinttent_geo_final_print` (the visual placed under `\mathcal`’s subscript) and `\l__spinttent_spfig_desc_tl` (the descriptive part, without `área/perímetro`). Receives the short command name in ‘#1’.

With an empty argument, both are cleared for `print-sym` auto/ `none`; a forced `print-sym` instead places the symbol alone, with its own name as the description.

```

4570 \cs_new_protected:Nn \__spinttent_spfig_build_simple:n
4571 {
4572   \tl_if_empty:nTF { \l__spinttent_geo_luaset_print_tl }
4573   {
4574     \str_case:VnF \l__spinttent_geo_print_sym_str

```

```

4575     {
4576     { auto }
4577     {
4578         \tl_clear:N \l__spintent_geo_final_print_tl
4579         \tl_clear:N \l__spintent_spfig_desc_tl
4580     }
4581     { none }
4582     {
4583         \tl_clear:N \l__spintent_geo_final_print_tl
4584         \tl_clear:N \l__spintent_spfig_desc_tl
4585     }
4586     }
4587     {
4588     \tl_set:Nn \l__spintent_geo_final_print_tl { \__spintent_spfig_sym: }
4589     \tl_set:Ne \l__spintent_spfig_desc_tl
4590     {
4591         \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triangle }
4592         { triángulo } { cuadrado }
4593     }
4594     }
4595 }

```

With a bare number/letter (empty `\l__spintent_geo_luaset_letra_word_str`), the value itself becomes the visual for `print-sym` `auto/none`, or its own subscript under a forced symbol otherwise; the description is the spoken value, prefixed with «sub-» when `read-sub` is `true` (further prefixed with the symbol's own name for a forced `print-sym`).

```

4596     {
4597     \str_if_empty:NTF \l__spintent_geo_luaset_letra_word_str
4598     {
4599         \str_case:VnF \l__spintent_geo_print_sym_str
4600         {
4601             { auto } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_p
4602             { none } { \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_p
4603         }
4604         {
4605             \tl_set:Nn \l__spintent_geo_final_print_tl
4606             {
4607                 \__spintent_spfig_sym:
4608                 \c_math_subscript_token { \l__spintent_geo_luaset_print_tl }
4609             }
4610         }
4611     \str_case:VnF \l__spintent_geo_print_sym_str
4612     {
4613         { auto }
4614         {
4615             \tl_set:Ne \l__spintent_spfig_desc_tl
4616             {
4617                 \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4618                 \l__spintent_geo_luaset_sub_spoken_str
4619             }
4620         }
4621         { none }
4622         {
4623             \tl_set:Ne \l__spintent_spfig_desc_tl
4624             {
4625                 \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4626                 \l__spintent_geo_luaset_sub_spoken_str
4627             }
4628         }
4629     }
4630     {
4631     \tl_set:Ne \l__spintent_spfig_desc_tl
4632     {
4633         \str_if_eq:VnTF \l__spintent_geo_print_sym_str { triangle }
4634         { triángulo } { cuadrado }
4635     -
4636     \bool_if:cT { \l__spintent_sp #1 _read_sub_bool } { sub - }
4637     \l__spintent_geo_luaset_sub_spoken_str
4638     }
4639     }
4640 }

```

With a table letter (F/P/B/L/T, non-empty `\l__spintent_geo_luaset_letra_word_str`), a forced `print-sym` raises `sp#1-sym-letra-conflict` — a table letter is never nested under a symbol. The visual is the value as-is; the description is its own word, connected as `\l_@@_geo_luaset_letra_conector_str` requires, with the same spoken subscript as above when present.

```

4641     {
4642         \str_case:\nF \l__spintent_geo_print_sym_str
4643         {
4644             { auto } { }
4645             { none } { }
4646         }
4647         {
4648             \msg_error:nne { spintent } { sp #1 -sym-letra-conflict }
4649             { \c_backslash_str sp #1 }
4650         }
4651     \tl_set:Nn \l__spintent_geo_final_print_tl { \l__spintent_geo_luaset_print_tl }
4652     \tl_set:Ne \l__spintent_spfig_desc_tl
4653     {
4654         \str_if_empty:NF \l__spintent_geo_luaset_letra_conector_str
4655         { \l__spintent_geo_luaset_letra_conector_str - }
4656         \l__spintent_geo_luaset_letra_word_str
4657         \str_if_empty:NF \l__spintent_geo_luaset_sub_spoken_str
4658         {
4659             -
4660             \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub - }
4661             \l__spintent_geo_luaset_sub_spoken_str
4662         }
4663     }
4664 }
4665 }
```

`\l__spintent_spfig_intent_tl` is always the `_word-` shape: `\l_@@_spfig_op_tl` alone when `\l__spintent_spfig_intent_arg_tl` is empty, both joined by a dash otherwise. `silent-cmd` overrides `\l__spintent_spfig_intent_arg_tl` with `_`; with `silent-cmd=false` and `concept false`, it becomes `\l__spintent_spfig_desc_tl-` (the descriptive part alone, without *área/perímetro*); with `concept true` it is `\l__spintent_spfig_intent_tl`.

```

4666     \tl_set:Ne \l__spintent_spfig_intent_tl
4667     {
4668         \tl_if_empty:NTF \l__spintent_spfig_desc_tl
4669         { _ \l__spintent_spfig_op_tl - }
4670         { _ \l__spintent_spfig_op_tl - \l__spintent_spfig_desc_tl - }
4671     }
4672     \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
4673     { \tl_set:Nn \l__spintent_spfig_intent_arg_tl { _- } }
4674     {
4675         \bool_if:cTF { l__spintent_sp #1 _concept_bool }
4676         { \tl_set:Ne \l__spintent_spfig_intent_arg_tl { \l__spintent_spfig_intent_tl } }
4677         { \tl_set:Ne \l__spintent_spfig_intent_arg_tl { _ \l__spintent_spfig_desc_tl - } }
4678     }
4679 }
```

(End of definition for `\l__spintent_spfig_build_simple:n`.)

`\l__spintent_spfig_build_refs:` Splits `\l__spintent_geo_luaset_points_clist` into `\l__spintent_spfig_pt_seq` and walks it once to build `\l__spintent_spfig_refs_tl`, the comma-separated `$pt1,$pt2,...` reference list, naming each vertex by position.

```

4680 \cs_new_protected:Nn \l__spintent_spfig_build_refs:
4681 {
4682     \seq_set_from_clist:NN \l__spintent_spfig_pt_seq \l__spintent_geo_luaset_points_clist
4683     \tl_clear:N \l__spintent_spfig_refs_tl
4684     \int_zero:N \l__spintent_spfig_pt_idx_int
4685     \seq_map_inline:Nn \l__spintent_spfig_pt_seq
4686     {
4687         \int_incr:N \l__spintent_spfig_pt_idx_int
4688         \tl_if_empty:NF \l__spintent_spfig_refs_tl
4689         { \tl_put_right:Nn \l__spintent_spfig_refs_tl { , } }
4690         \tl_put_right:Ne \l__spintent_spfig_refs_tl { $ pt \int_use:N \l__spintent_spfig_pt_idx_int }
4691     }
4692 }
```

(End of definition for `\l__spintent_spfig_build_refs:`.)

__spintent_spfig_build_concept_vertices:n

Handles the vertices case. Receives the short command name in ‘#1’. Must be called after `\__spintent_spfig_build_concept_vertices:n` `read-arg`, when given, overrides the auto-detected figure name in `\l__spintent_spfig_intent_tl`: with `concept true` it is still prefixed with `área-del-/perímetro-del-`, with `concept false` it is used verbatim. With `read-arg` empty, `\l__spintent_spfig_intent_tl` is simply `\l__spintent_geo_luaset_concept_str`.

```

4693 \cs_new_protected:Nn \__spintent_spfig_build_concept_vertices:n
4694 {
4695   \tl_if_empty:cTF { \l__spintent_sp #1 _read_arg_tl }
4696   { \tl_set:Ne \l__spintent_spfig_intent_tl { \l__spintent_geo_luaset_concept_str } }
4697   {
4698     \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4699     {
4700       \tl_set:Ne \l__spintent_spfig_intent_tl
4701       {
4702         \l__spintent_spfig_op_tl -del-
4703         \tl_use:c { \l__spintent_sp #1 _read_arg_tl }
4704       }
4705     }
4706     {
4707       \tl_set:Ne \l__spintent_spfig_intent_tl { \tl_use:c { \l__spintent_sp #1 _read_arg_tl } }
4708     }
4709   }

```

Determines `\l__spintent_spfig_symbol_tl` (empty when none applies): for `print-sym=auto`, the symbol matching `\l__spintent_geo_luaset_poly_sym_str` (`\triangle`/`\mdlgwhtsquare`), or none when that is empty; `print-sym=none` always clears it; a forced `print-sym` requires it to match the actual vertex count, reporting `sp#1-sym-cardinality-conflict` otherwise.

```

4710   \str_case:VnF \l__spintent_geo_print_sym_str
4711   {
4712     { auto }
4713     {
4714       \str_case:Vn \l__spintent_geo_luaset_poly_sym_str
4715       {
4716         { triangle } { \tl_set:Nn \l__spintent_spfig_symbol_tl { \triangle } }
4717         { square } { \tl_set:Nn \l__spintent_spfig_symbol_tl { \mdlgwhtsquare } }
4718       }
4719       \str_if_empty:NT \l__spintent_geo_luaset_poly_sym_str
4720       { \tl_clear:N \l__spintent_spfig_symbol_tl }
4721     }
4722     { none }
4723     { \tl_clear:N \l__spintent_spfig_symbol_tl }
4724   }
4725   {
4726     \str_if_eq:VVTF \l__spintent_geo_print_sym_str \l__spintent_geo_luaset_poly_sym_str
4727     { \tl_set:Nn \l__spintent_spfig_symbol_tl { \l__spintent_spfig_sym: } }
4728     {
4729       \msg_error:nne { spintent } { sp #1 -sym-cardinality-conflict }
4730       { \c_backslash_str sp #1 }
4731     }
4732   }

```

`silent-cmd` sets `\l__spintent_spfig_intent_arg_tl` to `_(refs)`. Otherwise, with `read-arg` empty, `concept` chooses between `\l__spintent_spfig_intent_tl:prefix(refs)` (prefixed with `prefix` when given) and `_(refs)` alone; with `read-arg` given, it is always `\l__spintent_spfig_intent_tl:prefix(refs)` — `read-arg` is never silenced by `concept` alone, only by `silent-cmd` (same convention as `\__spintent_spside_pro`

```

4733   \bool_if:cTF { \l__spintent_sp #1 _silent_cmd_bool }
4734   {
4735     \tl_set:Ne \l__spintent_spfig_intent_arg_tl { _ ( \l__spintent_spfig_refs_tl ) }
4736   }
4737   {
4738     \tl_if_empty:cTF { \l__spintent_sp #1 _read_arg_tl }
4739     {
4740       \bool_if:cTF { \l__spintent_sp #1 _concept_bool }
4741       {
4742         \tl_set:Ne \l__spintent_spfig_intent_arg_tl
4743         {
4744           \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4745           { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4746           \l__spintent_spfig_intent_tl :prefix( \l__spintent_spfig_refs_tl )
4747         }
4748       }
4749     }

```

```

4750         \tl_set:Ne \l__spintent_spfing_intent_arg_tl { _ ( \l__spintent_spfing_refs_tl ) }
4751     }
4752 }
4753 {
4754     \tl_set:Ne \l__spintent_spfing_intent_arg_tl
4755     {
4756         \tl_if_empty:cF { \l__spintent_sp #1 _prefix_tl }
4757         { \tl_use:c { \l__spintent_sp #1 _prefix_tl } - }
4758         \l__spintent_spfing_intent_tl :prefix( \l__spintent_spfing_refs_tl )
4759     }
4760 }
4761 }
4762 }

```

(End of definition for `\l__spintent_spfing_build_concept_vertices:n`)

`\l__spintent_spfing_points_loop:n` Walks `\l__spintent_spfing_pt_seq` a second time, calling `\l__spintent_mathmlarg_point:nnn` once per vertex with its `ptn` reference name, looked up under `sp#1`. Receives the short command name in ‘`#1`’.

```

4763 \cs_new_protected:Nn \l__spintent_spfing_points_loop:n
4764 {
4765     \int_zero:N \l__spintent_spfing_pt_idx_int
4766     \seq_map_inline:Nn \l__spintent_spfing_pt_seq
4767     {
4768         \int_incr:N \l__spintent_spfing_pt_idx_int
4769         \l__spintent_mathmlarg_point:nnn
4770         { pt \int_use:N \l__spintent_spfing_pt_idx_int }
4771         { sp #1 }
4772         { ##1 }
4773     }
4774 }

```

(End of definition for `\l__spintent_spfing_points_loop:n`)

`\l__spintent_spfing_mathml:n` Receives the short command name in ‘`#1`’. Branches on whether `\l__spintent_geo_luasets_points_clist` is empty.

Without vertices, `\l__spintent_spfing_build_simple:n` builds the content; a `\MathMLintent` wraps it only when `concept` is `true` — with `concept=false` the content is typeset directly, with no intent at all. With vertices, `\l__spintent_spfing_build_refs:` then `\l__spintent_spfing_build_concept_vertices:n` build the reference list and the concept intent, applied with a single native `\MathMLintent`. Inside, `\l__spintent_mathcal:n{⟨A|P⟩}` is silenced with `\l__spintent_luafun_wrap_mrow_intent_arg:nn` (it would otherwise be claimed by the queue meant for the symbol/vertices, the same ordering issue already found and fixed for `\spangle`’s own symbol); `\l__spintent_spfing_symbol_tl` is silenced the same way when non-empty; the sequence is then walked by `\l__spintent_spfing_points_loop:n`. All of this sits nested inside a subscript (`\c_math_subscript_token`), reached by the shared `mathml_filter` callback’s recursive lookup through a noad’s sub field, not just its nucleus.

`\l__spintent_invisible_sep:` precedes both branches (a second sibling so the sole `<mrow>` does not collapse into `<math>`).

```

4775 \cs_new_protected:Nn \l__spintent_spfing_mathml:n
4776 {
4777     \l__spintent_invisible_sep: % (luamml issues #26)
4778     \clist_if_empty:NTF \l__spintent_geo_luasets_points_clist
4779     {
4780         \l__spintent_spfing_build_simple:n {#1}
4781         \MathMLintent { \l__spintent_spfing_intent_arg_tl }
4782         {
4783             \l__spintent_luafun_inner_sep:
4784             \str_if_eq:eeTF {#1} { Afig } { \l__spintent_mathcal:n { A } } { \l__spintent_mathcal:n
4785             \tl_if_empty:NF \l__spintent_geo_final_print_tl
4786             {
4787                 \c_math_subscript_token { \l__spintent_geo_final_print_tl }
4788             }
4789         }
4790     }
4791     {
4792         \l__spintent_spfing_build_refs:
4793         \l__spintent_spfing_build_concept_vertices:n {#1}
4794         \MathMLintent { \l__spintent_spfing_intent_arg_tl }
4795         {
4796             \l__spintent_luafun_inner_sep:
4797             \l__spintent_luafun_wrap_mrow_intent_arg:nn

```

```

4798         { } { intent = ":silent" }
4799     {
4800         \str_if_eq:eeTF {#1} { Afig } { \__spintent_mathcal:n { A } } { \__spintent_mathcal
4801     }
4802     \c_math_subscript_token
4803     {
4804         \tl_if_empty:NF \__spintent_spfig_symbol_tl
4805         {
4806             \__spintent_luafun_wrap_mrow_intent_arg:nn
4807             { } { intent = ":silent" }
4808             { \tl_use:N \__spintent_spfig_symbol_tl }
4809         }
4810         \__spintent_spfig_points_loop:n {#1}
4811     }
4812 }
4813 }
4814 }

```

(End of definition for `\__spintent_spfig_mathml:n`.)

`\__spintent_spfig_exec:nnn` The function `\__spintent_spfig_exec:nnn` receives the short name of the command in `#1`, the optional keys in `#2`, and the argument in `#3`, delegating the rendering to `\__spintent_spfig_process:nn`.

```

4815 \cs_new_protected:Nn \__spintent_spfig_exec:nnn
4816 {
4817     \keys_set:nn { spintent / sp #1 } { #2 }
4818     \__spintent_spfig_process:nn { #1 } { #3 }
4819 }

```

(End of definition for `\__spintent_spfig_exec:nnn`.)

`\spAfig` Both public commands `\spAfig` and `\spPfig` enforce math mode, open a local group, and delegate execution to `\__spintent_spfig_exec:nnn`.

`\spPfig`

```

4820 \NewDocumentCommand \spAfig { 0{} d() }
4821 {
4822     \mode_if_math:TF
4823     {
4824         \group_begin:
4825         \attribute \__spintent_scope_attr_int = 1
4826         \tl_if_novalue:nTF {#2}
4827         { \__spintent_spfig_exec:nnn { Afig } { #1 } { } }
4828         { \__spintent_spfig_exec:nnn { Afig } { #1 } { #2 } }
4829     \group_end:
4830     }
4831     { \msg_error:nnn { spintent } { need-math-mode } { \spAfig } }
4832 }
4833 \NewDocumentCommand \spPfig { 0{} d() }
4834 {
4835     \mode_if_math:TF
4836     {
4837         \group_begin:
4838         \attribute \__spintent_scope_attr_int = 1
4839         \tl_if_novalue:nTF {#2}
4840         { \__spintent_spfig_exec:nnn { Pfig } { #1 } { } }
4841         { \__spintent_spfig_exec:nnn { Pfig } { #1 } { #2 } }
4842     \group_end:
4843     }
4844     { \msg_error:nnn { spintent } { need-math-mode } { \spPfig } }
4845 }

```

(End of definition for `\spAfig` and `\spPfig`. These functions are documented on page 23.)

11.45 The commands `\sprad` and `\spdiam`

The user-level commands `\sprad` and `\spdiam` provide a semantic interface for the radius and diameter of a circle. They delegate argument validation to the Lua module `spintent.lua`, thereby constructing a valid intent for MathCAT.

11.45.1 Definition of variables for \sprad and \spdiam

Each pair of variables is declared once for both \sprad and \spdiam via \clist_map_inline:nn.

```
\l__spintent_sprad_mode_str
\l__spintent_spdiam_mode_str
\l__spintent_sprad_upper_case_bool
\l__spintent_spdiam_upper_case_bool
\l__spintent_sprad_sub_index_str
\l__spintent_spdiam_sub_index_str
4846 \clist_map_inline:nn { rad , diam }
4847 {
4848     \str_new:c { l__spintent_sp #1 _mode_str }
4849     \str_new:c { l__spintent_sp #1 _sub_index_str }
4850     \tl_new:c { l__spintent_sp #1 _read_arg_tl }
4851 }
```

(End of definition for \l__spintent_sprad_mode_str and others.)

Scratch variables used by the function __spintent_sp_rad_diam_print:n.

```
4852 \tl_new:N \l__spintent_sp_rad_diam_letra_tl
4853 \tl_new:N \l__spintent_sp_rad_diam_word_tl
```

(End of definition for \l__spintent_sp_rad_diam_letra_tl and \l__spintent_sp_rad_diam_word_tl.)

11.45.2 Definition of keys for \sprad and \spdiam

We define the shared *keys* read-cmd, upper-case, sub-index, read-sub, read-arg, concept, silent-cmd and print-sym.

```
read-cmd
upper-case
sub-index
read-sub
read-arg
concept
silent-cmd
print-sym
unknown
4854 \clist_map_inline:nn { rad , diam }
4855 {
4856     \keys_define:nn { spintent / sp #1 }
4857     {
4858         read-cmd .choices:nn = { school , mathcat }
4859         {
4860             \int_case:nn { \l_keys_choice_int }
4861             {
4862                 {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
4863                 {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
4864             }
4865         },
4866         read-cmd .initial:n = school,
4867         read-cmd .value_required:n = true,
4868         read-cmd / unknown .code:n =
4869             \msg_error:nneee { spintent } { unknown-choice }
4870             { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
4871         upper-case .bool_set:c = { l__spintent_sp #1 _upper_case_bool },
4872         upper-case .initial:n = false,
4873         upper-case .value_required:n = true,
4874         sub-index .code:n =
4875             { \str_set:cn { l__spintent_sp #1 _sub_index_str } {##1} },
4876         sub-index .initial:n = ,
4877         sub-index .value_required:n = true,
4878         read-sub .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
4879         read-sub .initial:n = false,
4880         read-sub .value_required:n = true,
4881         read-arg .code:n =
4882             { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _read_arg_tl } {##1} },
4883         read-arg .initial:n = ,
4884         read-arg .value_required:n = true,
4885         concept .bool_set:c = { l__spintent_sp #1 _concept_bool },
4886         concept .initial:n = true,
4887         concept .value_required:n = true,
4888         silent-cmd .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
4889         silent-cmd .initial:n = false,
4890         silent-cmd .value_required:n = true,
4891         print-sym .bool_set:c = { l__spintent_sp #1 _print_sym_bool },
4892         print-sym .initial:n = false,
4893         print-sym .value_required:n = true,
4894         unknown .code:n =
4895             {
4896                 \str_set:Nn \l__spintent_command_name_str { sp #1 }
4897                 \__spintent_unknown_keys_cmd:n {##1}
4898             },
4899     }
4900 }
```

🔴 The keys `print-sym` are defined for both commands for simplicity, but has no effect on \sprad.

(End of definition for read-cmd and others.)

11.45.3 Internal functions for \sprad and \spdiam

_spinttent_sp_rad_diam_print:n

The function `\_spinttent_sp_rad_diam_print:n` sets `\l__spinttent_sp_rad_diam_letra_tl` to the bare letter — `r/R` for `\sprad`, `d/D` for `\spdiam` — per upper-case, and, for `\spdiam` with `print-sym true`, silently typesets `\diameter` first. Receives the short command name in ‘#1’ (rad/diam).

```

4901 \cs_new_protected:Nn \_spinttent_sp_rad_diam_print:n
4902 {
4903   \bool_if:cTF { l__spinttent_sp #1_upper_case_bool }
4904   {
4905     \str_if_eq:eeTF {#1} { rad }
4906     { \tl_set:Nn \l__spinttent_sp_rad_diam_letra_tl { R } }
4907     { \tl_set:Nn \l__spinttent_sp_rad_diam_letra_tl { D } }
4908   }
4909   {
4910     \str_if_eq:eeTF {#1} { rad }
4911     { \tl_set:Nn \l__spinttent_sp_rad_diam_letra_tl { r } }
4912     { \tl_set:Nn \l__spinttent_sp_rad_diam_letra_tl { d } }
4913   }
4914   \_spinttent_invisible_sep:
4915   \str_if_eq:eeT {#1} { diam }
4916   {
4917     \bool_if:NT \l__spinttent_spdiam_print_sym_bool
4918     { \MathMLintent { :silent } { \diameter } }
4919   }

```

With `silent-cmd true`, the intent is `_` — the letter alone through the shared queue when there is no sub-index, or letter and subscript together in one native `\MathMLintent` (subscript silenced) when there is.

```

4920   \bool_if:cTF { l__spinttent_sp #1_silent_cmd_bool }
4921   {
4922     \str_if_empty:cTF { l__spinttent_sp #1_sub_index_str }
4923     {
4924       \_spinttent_luafun_wrap_mrow_intent_arg:nn
4925       { } { intent = "_" }
4926       { \l__spinttent_sp_rad_diam_letra_tl }
4927     }
4928     {
4929       \MathMLintent { _- }
4930       {
4931         \_spinttent_luafun_inner_sep:
4932         \l__spinttent_sp_rad_diam_letra_tl
4933         \c_math_subscript_token
4934         {
4935           \MathMLintent { :silent }
4936           { \str_use:c { l__spinttent_sp #1_sub_index_str } }
4937         }
4938       }
4939     }
4940   }

```

Otherwise, with `concept true` (the default), it verbalizes radio/diámetro (`read-arg` replaces this word when given), with `sub-index`, if present, appended right after (`read-sub` choosing sub- $\langle value \rangle$ or $\langle value \rangle$ alone) — the letter itself is never read here. With no `sub-index`, the shared queue tags the letter alone; with one, letter and subscript are wrapped together in one native `\MathMLintent` for `read-cmd=school` (subscript silenced, already spoken), or the shared queue tags the letter alone with the subscript left bare for `read-cmd=mathcat`.

```

4941   {
4942     \bool_if:cTF { l__spinttent_sp #1_concept_bool }
4943     {
4944       \tl_if_empty:cTF { l__spinttent_sp #1_read_arg_tl }
4945       {
4946         \str_if_eq:eeTF {#1} { rad }
4947         { \tl_set:Nn \l__spinttent_sp_rad_diam_word_tl { radio } }
4948         { \tl_set:Nn \l__spinttent_sp_rad_diam_word_tl { diámetro } }
4949       }
4950       { \tl_set:Ne \l__spinttent_sp_rad_diam_word_tl { \tl_use:c { l__spinttent_sp #1_read_arg_tl } } }
4951       \str_if_empty:cTF { l__spinttent_sp #1_sub_index_str }
4952       {
4953         \_spinttent_luafun_wrap_mrow_intent_arg:nn
4954         { } { intent = "_" \l__spinttent_sp_rad_diam_word_tl - :prefix }
4955         { \l__spinttent_sp_rad_diam_letra_tl }
4956       }
4957     }

```

```

4958 \str_if_eq:eeTF
4959 { \str_use:c { \__spintent_sp #1 _mode_str } } { school }
4960 {
4961   \MathMLintent
4962   {
4963     _ \__spintent_sp_rad_diam_word_tl -
4964     \bool_if:cT { \__spintent_sp #1 _read_sub_bool } { sub- }
4965     \str_use:c { \__spintent_sp #1 _sub_index_str }
4966     :prefix
4967   }
4968   {
4969     \__spintent_luafun_inner_sep:
4970     \__spintent_sp_rad_diam_letra_tl
4971     \c_math_subscript_token
4972     {
4973       \MathMLintent { :silent }
4974       { \str_use:c { \__spintent_sp #1 _sub_index_str } }
4975     }
4976   }
4977 }
4978 {
4979   \__spintent_luafun_wrap_mrow_intent_arg:nn
4980   { } { intent = "_ \__spintent_sp_rad_diam_word_tl - :prefix" }
4981   { \__spintent_sp_rad_diam_letra_tl }
4982   \c_math_subscript_token
4983   { \str_use:c { \__spintent_sp #1 _sub_index_str } }
4984 }
4985 }
4986 }

```

With `concept false`, the letter itself is read instead — but only when `read-cmd` is `school`, built the same way from the letter (with `sub-index`, letter and subscript wrapped together, subscript silenced). With `read-cmd mathcat`, the intent is left empty: MathCAT applies its own rules to the letter and its subscript, unmarked by `us`.

```

4987 {
4988   \str_if_empty:cTF { \__spintent_sp #1 _sub_index_str }
4989   {
4990     \str_if_eq:eeTF
4991     { \str_use:c { \__spintent_sp #1 _mode_str } } { school }
4992     {
4993       \__spintent_luafun_wrap_mrow_intent_arg:nn
4994       { } { intent = "_ \__spintent_sp_rad_diam_letra_tl - :prefix" }
4995       { \__spintent_sp_rad_diam_letra_tl }
4996     }
4997     { \__spintent_sp_rad_diam_letra_tl }
4998   }
4999   {
5000     \str_if_eq:eeTF
5001     { \str_use:c { \__spintent_sp #1 _mode_str } } { school }
5002     {
5003       \MathMLintent
5004       {
5005         _ \__spintent_sp_rad_diam_letra_tl -
5006         \bool_if:cT { \__spintent_sp #1 _read_sub_bool } { sub- }
5007         \str_use:c { \__spintent_sp #1 _sub_index_str }
5008         :prefix
5009       }
5010       {
5011         \__spintent_luafun_inner_sep:
5012         \__spintent_sp_rad_diam_letra_tl
5013         \c_math_subscript_token
5014         {
5015           \MathMLintent { :silent }
5016           { \str_use:c { \__spintent_sp #1 _sub_index_str } }
5017         }
5018       }
5019     }
5020   }
5021   \__spintent_sp_rad_diam_letra_tl
5022   \c_math_subscript_token
5023   { \str_use:c { \__spintent_sp #1 _sub_index_str } }

```

```

5024         }
5025     }
5026 }
5027 }
5028 }

```

(End of definition for `\__spintent_sp_rad_diam_print:n`.)

`\__spintent_sp_rad_diam_exec:nn`

The function `\__spintent_sp_rad_diam_exec:nn` receives the *short name* of the command passed in ‘#1’ and the optional *(keys)* passed in ‘#2’ then delegating the rendering to `\__spintent_sp_rad_diam_print:n`.

```

5029 \cs_new_protected:Nn \__spintent_sp_rad_diam_exec:nn
5030 {
5031     \keys_set:nn { spintent / sp #1 } {#2}
5032     \__spintent_sp_rad_diam_print:n {#1}
5033 }

```

(End of definition for `\__spintent_sp_rad_diam_exec:nn`.)

`\sprad`
`\spdiam`

Both public commands `\sprad` and `\spdiam` enforces *math mode*, opens a local group, and delegates execution to `\__spintent_sp_rad_diam_exec:nn`.

```

5034 \NewDocumentCommand \sprad { 0{} }
5035 {
5036     \mode_if_math:TF
5037     {
5038         \group_begin:
5039         \__spintent_sp_rad_diam_exec:nn { rad } {#1}
5040         \group_end:
5041     }
5042     { \msg_error:nnn { spintent } { need-math-mode } { \sprad } }
5043 }
5044 \NewDocumentCommand \spdiam { 0{} }
5045 {
5046     \mode_if_math:TF
5047     {
5048         \group_begin:
5049         \__spintent_sp_rad_diam_exec:nn { diam } {#1}
5050         \group_end:
5051     }
5052     { \msg_error:nnn { spintent } { need-math-mode } { \spdiam } }
5053 }

```

(End of definition for `\sprad` and `\spdiam`. These functions are documented on page ??.)

11.46 The command `\spcirc`

The user command `\spcirc` provides a semantic interface for a circle or a circumference, using the delimited-optional argument syntax `d()` already established by `\spmcld/\spmcm` and `\spAfig/\spPfig`. The argument holds a required center point and an optional radius, separated by a comma. The radius is never required to declare its own kind via a key: its shape alone decides whether it is a pure number, a measure (number + unit), a bare letter, or a two-letter segment — reusing, respectively, `\spnum`, `\spunit`, and `\spside/\spmside` directly as sub-commands, rather than reimplementing any of their parsing.

11.46.1 Definition of variables for `\spcirc`

`\__spintent_geo_luaset_circ_radio_type_str`
`\__spintent_geo_luaset_circ_radio_raw_str`

Shared outputs of `\__spintent_luafun_geo_circ_parse_and_set:n`. `\__spintent_geo_luaset_circ_radio_type_str` holds one of *number*, *measure*, *label*, *segment*, or an empty string when no radius was given. `\__spintent_geo_luaset_circ_radio_raw_str` holds the corresponding raw text, ready to be forwarded to the sub-command that renders it.

```

5054 \str_new:N \__spintent_geo_luaset_circ_radio_type_str
5055 \str_new:N \__spintent_geo_luaset_circ_radio_raw_str

```

(End of definition for `\__spintent_geo_luaset_circ_radio_type_str` and `\__spintent_geo_luaset_circ_radio_raw_str`.)

`\__spintent_spcirc_mode_str`
`\__spintent_spcirc_prefix_tl`
`\__spintent_spcirc_print_sym_str`
`\__spintent_spcirc_read_sty_str`
`\__spintent_spcirc_radio_sty_str`
`\__spintent_spcirc_sub_index_str`
`\__spintent_spcirc_articulo_str`

The variable `\__spintent_spcirc_mode_str` stores the value set by the key `read-cmd`; the variable `\__spintent_spcirc_prefix_tl` stores the value set by the key `prefix`. The variable `\__spintent_spcirc_print_sym_str` stores the value set by the key `print-sym` (`usual` or `odot`). The variable `\__spintent_spcirc_read_sty_str` stores the value set by the key `read-sty` (`circun` or `circle`). The variable `\__spintent_spcirc_radio_sty_str` stores the value set by the key `radio-sty`, only meaningful when the radius is a segment. The variable `\__spintent_spcirc_sub_index_str` stores the value set by the key `sub-index`. The variable `\__spintent_spcirc_articulo_str` is set internally by `\__spintent_spcirc_word_and_article:` to `la` or `el`, matching the grammatical gender of the chosen word.

```

5056 \str_new:N \__spintent_spcirc_mode_str
5057 \tl_new:N \__spintent_spcirc_prefix_tl

```

```

5058 \str_new:N \__spintent_spcirc_print_sym_str
5059 \str_new:N \__spintent_spcirc_read_sty_str
5060 \str_new:N \__spintent_spcirc_radio_sty_str
5061 \str_new:N \__spintent_spcirc_sub_index_str
5062 \str_new:N \__spintent_spcirc_articulo_str
5063 \tl_new:N \__spintent_spcirc_read_arg_tl

```

(End of definition for `\__spintent_spcirc_mode_str` and others.)

`\__spintent_spcirc_word_tl`

Scratch variables used by `\__spintent_spcirc_process:n` to assemble the chosen word (circunferencia/círculo or the `read-arg` override) and the final literal `intent` string.

```

5064 \tl_new:N \__spintent_spcirc_word_tl

```

(End of definition for `\__spintent_spcirc_word_tl`.)

11.46.2 Definition of error messages for `\spcirc`

`spcirc-invalid-arg`

Raised when the argument does not match a single center point, optionally followed by a comma and a valid radius shape.

```

5065 \msg_new:nnnn { spintent } { spcirc-invalid-arg }
5066 {
5067   Invalid~argument~format~in~\token_to_str:N \spcirc:~'!#1'~\msg_line_context:.
5068 }
5069 {
5070   See~the~documentation~of~\token_to_str:N \spcirc~for~the~accepted~argument~forms.
5071 }

```

(End of definition for `spcirc-invalid-arg`.)

11.46.3 Definition of keys for `\spcirc`

read-cmd
prefix
print-sym
read-sty
diameter
radio-sty
sub-index
read-sub
read-arg
silent-cmd
concept
spside
spmside
unknown

The key `print-sym` chooses the visual symbol: usual for a plain C, `odot` (default) for `\odot`. The key `read-sty` chooses the verbalized word and its article: `circun` (default) for «la circunferencia», `circle` for «el círculo». The key `radio-sty` chooses, only when the radius is a two-letter segment, whether it is rendered as the *object* (side, via `\spside`) or its *measure* (mside, default, via `\spmside`). The key `sub-index` sets a subscript on the symbol; the key `read-sub` controls whether it is read with the word «sub» or as a plain number, matching the same convention as `\spAfig`.

```

5072 \keys_define:nn { spintent / spcirc }
5073 {
5074   read-cmd      .choices:nn = { school , mathcat }
5075   {
5076     \int_case:nn { \l_keys_choice_int }
5077     {
5078       {1} { \str_set:Nn \__spintent_spcirc_mode_str { school } }
5079       {2} { \str_set:Nn \__spintent_spcirc_mode_str { mathcat } }
5080     }
5081   },
5082   read-cmd      .initial:n = school,
5083   read-cmd      .value_required:n = true,
5084   prefix        .code:n =
5085     { \__spintent_sanitize_affix:Nn \__spintent_spcirc_prefix_tl {#1} },
5086   prefix        .initial:n = ,
5087   prefix        .value_required:n = true,
5088   print-sym     .choices:nn = { usual , odot }
5089   {
5090     \int_case:nn { \l_keys_choice_int }
5091     {
5092       {1} { \str_set:Nn \__spintent_spcirc_print_sym_str { usual } }
5093       {2} { \str_set:Nn \__spintent_spcirc_print_sym_str { odot } }
5094     }
5095   },
5096   print-sym     .initial:n = odot,
5097   print-sym     .value_required:n = true,
5098   read-sty      .choices:nn = { circun , circle }
5099   {
5100     \int_case:nn { \l_keys_choice_int }
5101     {
5102       {1} { \str_set:Nn \__spintent_spcirc_read_sty_str { circun } }
5103       {2} { \str_set:Nn \__spintent_spcirc_read_sty_str { circle } }
5104     }
5105   },
5106   read-sty      .initial:n = circun,
5107   read-sty      .value_required:n = true,

```

```

5108     diameter      .bool_set:N = \l__spintenc_spcirc_diameter_bool,
5109     diameter      .initial:n = false,
5110     diameter      .value_required:n = true,
5111     radio-sty      .choices:nn = { side , mside }
5112     {
5113         \int_case:nn { \l_keys_choice_int }
5114         {
5115             {1} { \str_set:Nn \l__spintenc_spcirc_radio_sty_str { side } }
5116             {2} { \str_set:Nn \l__spintenc_spcirc_radio_sty_str { mside } }
5117         }
5118     },
5119     radio-sty      .initial:n = mside,
5120     radio-sty      .value_required:n = true,
5121     sub-index      .code:n =
5122     { \str_set:Nn \l__spintenc_spcirc_sub_index_str {#1} },
5123     sub-index      .initial:n = ,
5124     sub-index      .value_required:n = true,
5125     read-sub       .bool_set:N = \l__spintenc_spcirc_read_sub_bool,
5126     read-sub       .initial:n = false,
5127     read-sub       .value_required:n = true,
5128     read-arg       .code:n =
5129     { \__spintenc_sanitizc_affix:Nn \l__spintenc_spcirc_read_arg_tl {#1} },
5130     read-arg       .initial:n = ,
5131     read-arg       .value_required:n = true,
5132     silent-cmd     .bool_set:N = \l__spintenc_spcirc_silent_cmd_bool,
5133     silent-cmd     .initial:n = false,
5134     silent-cmd     .value_required:n = true,
5135     concept        .bool_set:N = \l__spintenc_spcirc_concept_bool,
5136     concept        .initial:n = true,
5137     concept        .value_required:n = true,
5138     spside         .meta:nn = { spintenc / spside } { #1 },
5139     spside         .value_required:n = true,
5140     spmside        .meta:nn = { spintenc / spmside } { #1 },
5141     spmside        .value_required:n = true,
5142     unknown        .code:n =
5143     {
5144         \str_set:Nn \l__spintenc_command_name_str { spcirc }
5145         \__spintenc_unknown_keys_cmd:n {#1}
5146     },
5147 }

```

(End of definition for read-cmd and others.)

11.46.4 Internal functions for \spcirc

`\l__spintenc_spcirc_has_arg_bool` `\l__spintenc_spcirc_has_arg_bool` records whether the `d()` argument was given at all (`\spcirc` with no (...) at all has neither a center nor a radius). `\l__spintenc_spcirc_intent_tl` holds the word (circunferencia or círculo, with an optional `sub-index` suffix) paired with the symbol. `\l__spintenc_spcirc_intent_arg_tl` holds de-centro-en, paired with the (center) group; the center is read through `\__spintenc_mathmlarg_point:nnn`, referenced as `$pt`, not concatenated into either word — it can be subscripted or primed (e.g. $O_{\{1\}}$, as in the Euler circle) without breaking anything. The `_arg_tl` variables hold each word's final wrapped form; a third, `\l__spintenc_spcirc_radio_arg_tl`, is spoken directly on the , delimiter after the center, replacing its own reading rather than pairing with a group.

```

5148 \bool_new:N \l__spintenc_spcirc_has_arg_bool
5149 \tl_new:N \l__spintenc_spcirc_intent_tl
5150 \tl_new:N \l__spintenc_spcirc_intent_arg_tl
5151 \tl_new:N \l__spintenc_spcirc_connector_tl
5152 \tl_new:N \l__spintenc_spcirc_connector_arg_tl
5153 \tl_new:N \l__spintenc_spcirc_radio_arg_tl

```

(End of definition for \l__spintenc_spcirc_has_arg_bool and others.)

`\__spintenc_spcirc_sym:` Typesets `\bigodot` or the plain letter C, according to `print-sym`.

```

5154 \cs_new_protected:Nn \__spintenc_spcirc_sym:
5155 {
5156     \str_if_eq:VnTF \l__spintenc_spcirc_print_sym_str { odot }
5157     {
5158         \group_begin:
5159         \textstyle\bigodot
5160         \group_end:
5161     }
5162     { C }
5163 }

```

(End of definition for `\__spintent_spcirc_sym:`)

`\__spintent_spcirc_word_and_article:` Sets `\l__spintent_spcirc_word_tl` to `read-arg` or `circunferencia/círculo` from `read-sty`, and `\l__spintent_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word.

```
5164 \cs_new_protected:Nn \__spintent_spcirc_word_and_article:
5165 {
5166   \tl_if_empty:NTF \l__spintent_spcirc_read_arg_tl
5167   {
5168     \str_if_eq:VnTF \l__spintent_spcirc_read_sty_str { circun }
5169     {
5170       \tl_set:Nn \l__spintent_spcirc_word_tl { circunferencia }
5171       \str_set:Nn \l__spintent_spcirc_articulo_str { la }
5172     }
5173     {
5174       \tl_set:Nn \l__spintent_spcirc_word_tl { círculo }
5175       \str_set:Nn \l__spintent_spcirc_articulo_str { el }
5176     }
5177   }
5178   {
5179     \tl_set:Nn \l__spintent_spcirc_word_tl { \l__spintent_spcirc_read_arg_tl }
5180     \str_if_eq:VnTF \l__spintent_spcirc_read_sty_str { circun }
5181     { \str_set:Nn \l__spintent_spcirc_articulo_str { la } }
5182     { \str_set:Nn \l__spintent_spcirc_articulo_str { el } }
5183   }
5184 }
```

(End of definition for `\__spintent_spcirc_word_and_article:`)

`\__spintent_spcirc_radio_print:` Dispatches on `\l__spintent_geo_luaset_circ_radio_type_str`, reusing `\spnum/\spunit/\spside/\spmside` directly rather than reimplementing any parsing.

```
5185 \cs_new_protected:Nn \__spintent_spcirc_radio_print:
5186 {
5187   \str_case:Vn \l__spintent_geo_luaset_circ_radio_type_str
5188   {
5189     { number } { \exp_args:Ne \spnum { \l__spintent_geo_luaset_circ_radio_raw_str } }
5190     { measure } { \exp_args:Ne \spunit { \l__spintent_geo_luaset_circ_radio_raw_str } }
5191     { label } { \l__spintent_geo_luaset_circ_radio_raw_str }
5192     { segment }
5193     {
5194       \str_if_eq:VnTF \l__spintent_spcirc_radio_sty_str { side }
5195       { \exp_args:Ne \spside { \l__spintent_geo_luaset_circ_radio_raw_str } }
5196       { \exp_args:Ne \spmside { \l__spintent_geo_luaset_circ_radio_raw_str } }
5197     }
5198   }
5199 }
```

(End of definition for `\__spintent_spcirc_radio_print:`)

`\__spintent_spcirc_build_concept:` Builds two words: `\l__spintent_spcirc_intent_tl` (via `\__spintent_spcirc_word_and_article:`, plus an optional `sub-index` suffix), paired with the symbol, and `\l__spintent_spcirc_connector_tl` (`de-centro-en`), paired with the `(center)` group — built even when `\l__spintent_spcirc_has_arg_bool` is `false`, though `\__spintent_spcirc_mathml:` never uses it in that case. Neither carries the center's own text: it is read separately, by reference, through `\__spintent_mathmlarg_point:nnn` — so a subscripted or primed center (e.g. O_{1}), as in the Euler circle) is never an issue.

```
5200 \cs_new_protected:Nn \__spintent_spcirc_build_concept:
5201 {
5202   \__spintent_spcirc_word_and_article:
5203   \tl_set:Nn \l__spintent_spcirc_intent_tl { \l__spintent_spcirc_word_tl }
5204   \str_if_empty:NF \l__spintent_spcirc_sub_index_str
5205   {
5206     \tl_put_right:Nn \l__spintent_spcirc_intent_tl
5207     {
5208       -
5209       \bool_if:NT \l__spintent_spcirc_read_sub_bool { sub - }
5210       \l__spintent_spcirc_sub_index_str
5211     }
5212   }
5213   \tl_set:Nn \l__spintent_spcirc_connector_tl { de-centro-en }
```

`silent-cmd true` overrides `\__spintent_spcirc_intent_arg_tl`/`\__spintent_spcirc_radio_arg_tl` with `_-` and `\__spintent_spcirc_connector_arg_tl` with `_($pt)` (the center is always read), regardless of `read-arg`.

```

5214 \bool_if:NTF \__spintent_spcirc_silent_cmd_bool
5215 {
5216   \tl_set:Nn \__spintent_spcirc_intent_arg_tl { _- }
5217   \tl_set:Ne \__spintent_spcirc_connector_arg_tl { _($ pt) }
5218   \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _- }
5219 }

```

With `concept true`, `\__spintent_spcirc_intent_arg_tl` is `\__spintent_spcirc_intent_tl:prefix` (bare `_-` the group it wraps has nothing to reference, only a silenced symbol), prefixed with `prefix` when given; `\__spintent_spcirc_connector_arg_tl` is `\__spintent_spcirc_connector_tl:prefix($pt)`; and `\__spintent_spcirc_radio_arg_tl` — spoken directly on the `,` delimiter, replacing its own reading rather than pairing with any group — is the literal `_y-radio-`, or `_y-diámetro-` when `diameter` is `true`, read as its own phrase right before `\__spintent_spcirc_radio_print:`'s independent reading follows as the next sibling.

```

5220 {
5221   \bool_if:NTF \__spintent_spcirc_concept_bool
5222   {
5223     \tl_set:Ne \__spintent_spcirc_intent_arg_tl
5224     {
5225       \tl_if_empty:NF \__spintent_spcirc_prefix_tl
5226       { \tl_use:N \__spintent_spcirc_prefix_tl - }
5227       \__spintent_spcirc_intent_tl :prefix
5228     }
5229     \tl_set:Ne \__spintent_spcirc_connector_arg_tl
5230     { \__spintent_spcirc_connector_tl :prefix( $ pt ) }
5231     \bool_if:NTF \__spintent_spcirc_diameter_bool
5232     { \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
5233     { \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _y-radio- } }
5234   }

```

With `concept false` and `read-arg` empty, the same `_-/_($pt)` silencing applies as `silent-cmd`; with `read-arg` given, none of the three is silenced instead — the connector and `y-radio/y-diámetro` words are fixed structural pieces, not customized by `read-arg`, but once a real word is being spoken there is nothing left to silence around it (same convention as `\__spintent_spside_process:nn`).

```

5235 {
5236   \tl_if_empty:NTF \__spintent_spcirc_read_arg_tl
5237   {
5238     \tl_set:Nn \__spintent_spcirc_intent_arg_tl { _- }
5239     \tl_set:Ne \__spintent_spcirc_connector_arg_tl { _($ pt) }
5240     \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _- }
5241   }
5242   {
5243     \tl_set:Ne \__spintent_spcirc_intent_arg_tl { \__spintent_spcirc_intent_tl }
5244     \tl_set:Ne \__spintent_spcirc_connector_arg_tl
5245     { \__spintent_spcirc_connector_tl :prefix( $ pt ) }
5246     \bool_if:NTF \__spintent_spcirc_diameter_bool
5247     { \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _y-diámetro- } }
5248     { \tl_set:Nn \__spintent_spcirc_radio_arg_tl { _y-radio- } }
5249   }
5250 }
5251 }
5252 }

```

(End of definition for `\__spintent_spcirc_build_concept:`)

`\__spintent_spcirc_process:n` Receives the `d()` argument in `#1`, an explicit empty group when omitted (`\spcirc` itself converts `-NoValue-` to `{ }` before calling `\__spintent_spcirc_exec:nn`) — `\__spintent_spcirc_has_arg_bool` set to `false` in that case, and `\__spintent_luafun_geo_circ_parse_and_set:n` never called at all — there is nothing to parse. Otherwise calls it, reports `spcirc-invalid-arg` on failure, then delegates to `\__spintent_spcirc_mathml:`, which calls `\__spintent_spcirc_build_concept:` itself.

```

5253 \cs_new_protected:Nn \__spintent_spcirc_process:n
5254 {
5255   \tl_if_empty:NTF {#1}
5256   {
5257     \bool_set_false:N \__spintent_spcirc_has_arg_bool
5258   }
5259   {
5260     \bool_set_true:N \__spintent_spcirc_has_arg_bool

```

```

5261     \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#1} }
5262     \str_if_eq:VnT \l__spintent_geo_luaset_error_str { true }
5263     {
5264         \msg_error:nne { spintent } { spcirc-invalid-arg } { \exp_not:n {#1} }
5265     }
5266 }
5267 \__spintent_spcirc_mathml:
5268 }

```

(End of definition for `\__spintent_spcirc_process:n`.)

`\__spintent_spcirc_mathml:` Calls `\__spintent_spcirc_build_concept:`, then wraps the symbol (`\l__spintent_spcirc_intent_arg_tl`) in its own native `\MathMLintent` — this alone is the whole result when `\l__spintent_spcirc_has_arg_bool` is `false` (`\spcirc` with no `(...)` at all: no center, no radius).

Otherwise, the `(center)` group (`\l__spintent_spcirc_connector_arg_tl`) follows as a separate sibling, never nested inside the symbol's own `\MathMLintent` — or MathCAT falls back to its own structural reading of the underlying `<mover>/<msub>` instead, the same problem already found for `\spangle`'s symbol and `\sparc`'s accent. When a radius is present, the `,` delimiter after the center carries `\l__spintent_spcirc_radio_arg_tl` directly (replacing its own reading, not paired with any group), and `\__spintent_spcirc_radio_print:` follows as an ordinary sibling right after — wrapping it in another `\MathMLintent` would lose the reading of whatever it produces (itself a native `\MathMLintent`, from `\spnum/\spunit`, nested inside another one does not compose).

The symbol and, when `sub-index` is given, its subscript number are each silenced with `\__spintent_luafun_wrap_mrow_intent_arg:nn` *separately* — one queue entry per leaf. The two are reached differently by the shared `mathml_filter` call-back's recursive lookup (the symbol via `nucleus`, the subscript via `sub`), so a single entry covering both leaves nothing registered for the second one, which the recursion then assigns whatever comes next in the queue instead — shifting every claim after it by one. The `(` delimiter is silenced the same way, but *without* the usual `{...}` around the character: `luamml` only keeps a delimiter's own math class (giving `<mo>` instead of `<mi>`, needed for MathCAT to read the group at all) when it is not itself sitting inside a brace group.

The final `)` — following `\__spintent_spcirc_radio_print:`, whose own output the shared queue does not reliably skip over — is typeset as `\rparen` inside its own native `\MathMLintent{:silent}` instead of a bare `)`: `\rparen` keeps its own math class through the wrapping, unlike a bare delimiter character, and this sits outside the shared queue entirely, so nothing in between can claim an entry meant for it.

The center itself is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt`.

```

5269 \cs_new_protected:Nn \__spintent_spcirc_mathml:
5270 {
5271     \__spintent_invisible_sep: % (luamml issues #26)
5272     \__spintent_spcirc_build_concept:
5273     \MathMLintent { \l__spintent_spcirc_intent_arg_tl }
5274     {
5275         \__spintent_luafun_inner_sep:
5276         \__spintent_luafun_wrap_mrow_intent_arg:nn
5277         { } { intent = ":silent" }
5278         { \__spintent_spcirc_sym: }
5279         \str_if_empty:NF \l__spintent_spcirc_sub_index_str
5280         {
5281             \c_math_subscript_token
5282             {
5283                 \__spintent_luafun_wrap_mrow_intent_arg:nn
5284                 { } { intent = ":silent" }
5285                 { \l__spintent_spcirc_sub_index_str }
5286             }
5287         }
5288     }
5289     \bool_if:NT \l__spintent_spcirc_has_arg_bool
5290     {
5291         \str_if_empty:NTF \l__spintent_geo_luaset_circ_radio_type_str
5292         {
5293             \MathMLintent { \l__spintent_spcirc_connector_arg_tl }
5294             {
5295                 \__spintent_luafun_inner_sep:
5296                 \__spintent_luafun_wrap_mrow_intent_arg:nn { } { intent = ":silent" } (
5297                 \__spintent_mathmlarg_point:nnV { pt } { spcirc } \l__spintent_geo_luaset_points:
5298                 \__spintent_luafun_wrap_mrow_intent_arg:nn { } { intent = ":silent" } )
5299             }
5300         }
5301         {
5302             \MathMLintent { \l__spintent_spcirc_connector_arg_tl }
5303             {

```

```

5304         \__spintent_luafun_inner_sep:
5305         \__spintent_luafun_wrap_mrow_intent_arg:nn { } { intent = ":silent" } (
5306         \__spintent_mathmlarg_point:nnV { pt } { spcirc } \__spintent_geo_luaset_points_
5307     }
5308     \MathMLintent { \__spintent_spcirc_radio_arg_tl } { , }
5309     \__spintent_spcirc_radio_print:
5310     \MathMLintent { :silent } { \rparen }
5311 }
5312 }
5313 }

```

(End of definition for __spintent_spcirc_mathml:.)

__spintent_spcirc_exec:nn The function __spintent_spcirc_exec:nn receives the optional keys in #1 and the argument in #2, delegating the rendering to __spintent_spcirc_print:n.

```

5314 \cs_new_protected:Nn \__spintent_spcirc_exec:nn
5315 {
5316     \keys_set:nn { spintent / spcirc } { #1 }
5317     \__spintent_spcirc_process:n { #2 }
5318 }

```

(End of definition for __spintent_spcirc_exec:nn.)

\spcirc The public command \spcirc enforces math mode, opens a local group, and delegates execution to __spintent_spcirc_exec:nn. It follows the same d()-argument pattern as \spmcld/\spmcm and \spAfig/\spPfig; an omitted radius forwards an explicit empty group rather than the -NoValue- marker.

```

5319 \NewDocumentCommand \spcirc { 0{ } d() }
5320 {
5321     \mode_if_math:TF
5322     {
5323         \group_begin:
5324         \attribute \__spintent_scope_attr_int = 1
5325         \tl_if_novalue:nTF {#2}
5326         { \__spintent_spcirc_exec:nn { #1 } { } }
5327         { \__spintent_spcirc_exec:nn { #1 } { #2 } }
5328         \group_end:
5329     }
5330     { \msg_error:nnn { spintent } { need-math-mode } { \spcirc } }
5331 }

```

(End of definition for \spcirc. This function is documented on page ??.)

11.47 The commands \spAcirc and \spPcirc

The user commands \spAcirc and \spPcirc typeset the area and the perimeter of a circle or a circumference. The argument follows \spcirc's own point-and-radius grammar for a real center (with an optional radius); when no comma is present and no valid center is found, it falls back to a bare number/letter shortcut, reusing the same Lua entry point as \spAfig/\spPfig. A table letter matched by that fallback is rejected, since it has no meaning for a circle.

11.47.1 Definition of variables for \spAcirc and \spPcirc

```

\__spintent_spAcirc_mode_str
\__spintent_spPcirc_mode_str
\__spintent_spAcirc_print_sym_str
\__spintent_spPcirc_print_sym_str
\__spintent_spAcirc_read_sty_str
\__spintent_spPcirc_read_sty_str
\__spintent_spAcirc_radio_sty_str
\__spintent_spPcirc_radio_sty_str
\__spintent_spAcirc_sub_index_str
\__spintent_spPcirc_sub_index_str
\__spintent_spAcirc_prefix_tl
\__spintent_spPcirc_prefix_tl
\__spintent_spAcirc_read_arg_tl
\__spintent_spPcirc_read_arg_tl
\__spintent_geo_circfig_mode_str

```

These variables store the value set by each corresponding key: `read-cmd`, `print-sym`, `read-sty`, `radio-sty`, `sub-index`, `prefix` and `read-arg`, respectively.

```

5332 \clist_map_inline:nn { Acirc , Pcirc }
5333 {
5334     \str_new:c { \__spintent_sp #1 _mode_str }
5335     \str_new:c { \__spintent_sp #1 _print_sym_str }
5336     \str_new:c { \__spintent_sp #1 _read_sty_str }
5337     \str_new:c { \__spintent_sp #1 _radio_sty_str }
5338     \str_new:c { \__spintent_sp #1 _sub_index_str }
5339     \tl_new:c { \__spintent_sp #1 _prefix_tl }
5340     \tl_new:c { \__spintent_sp #1 _read_arg_tl }
5341 }

```

(End of definition for __spintent_spAcirc_mode_str and others.)

The variable __spintent_geo_circfig_mode_str stores whether the argument was resolved as a real circle (circle) or as the bare number/letter shortcut (bare).

```

5342 \str_new:N \__spintent_geo_circfig_mode_str

```

(End of definition for __spintent_geo_circfig_mode_str.)

11.47.2 Definition of error messages for \spAcirc and \spPcirc

spAcirc-invalid-arg
spPcirc-invalid-arg

The error message shown when the argument does not match any of the accepted forms: empty, a center point (optionally followed by a comma and a radius), or a bare number/letter.

```

5343 \clist_map_inline:nn { Acirc , Pcirc }
5344 {
5345   \msg_new:nnnn { spintent } { sp #1 -invalid-arg }
5346   {
5347     Invalid~argument~in~##1:~'##2'~\msg_line_context:.
5348   }
5349   {
5350     See~the~documentation~of~##1~for~the~accepted~argument.
5351   }

```

(End of definition for spAcirc-invalid-arg and spPcirc-invalid-arg.)

11.47.3 Definition of keys for \spAcirc and \spPcirc

read-cmd
prefix
print-sym
read-sty
concept
sub-index
read-sub
read-arg
silent-cmd
unknown

We define the shared (*keys*) read-cmd, prefix, print-sym, read-sty, concept, sub-index, read-sub, read-arg and silent-cmd.

```

5352 \keys_define:nn { spintent / sp #1 }
5353 {
5354   read-cmd .choices:nn = { school , mathcat }
5355   {
5356     \int_case:nn { \l_keys_choice_int }
5357     {
5358       {1} { \str_set:cn { l__spintent_sp #1 _mode_str } { school } }
5359       {2} { \str_set:cn { l__spintent_sp #1 _mode_str } { mathcat } }
5360     }
5361   },
5362   read-cmd .initial:n = school,
5363   read-cmd .value_required:n = true,
5364   read-cmd / unknown .code:n =
5365     \msg_error:nneee { spintent } { unknown-choice }
5366     { read-cmd } { school,~mathcat } { \exp_not:n {##1} },
5367   prefix .code:n =
5368     { \__spintent_sanitizet_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
5369   prefix .initial:n = ,
5370   prefix .value_required:n = true,
5371   print-sym .choices:nn = { usual , odot }
5372   {
5373     \int_case:nn { \l_keys_choice_int }
5374     {
5375       {1} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { usual } }
5376       {2} { \str_set:cn { l__spintent_sp #1 _print_sym_str } { odot } }
5377     }
5378   },
5379   print-sym .initial:n = odot,
5380   print-sym .value_required:n = true,
5381   print-sym / unknown .code:n =
5382     \msg_error:nneee { spintent } { unknown-choice }
5383     { print-sym } { usual,~odot } { \exp_not:n {##1} },
5384   read-sty .choices:nn = { circun , circle }
5385   {
5386     \int_case:nn { \l_keys_choice_int }
5387     {
5388       {1} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { circun } }
5389       {2} { \str_set:cn { l__spintent_sp #1 _read_sty_str } { circle } }
5390     }
5391   },
5392   read-sty .initial:n = circun,
5393   read-sty .value_required:n = true,
5394   concept .bool_set:c = { l__spintent_sp #1 _concept_bool },
5395   concept .initial:n = true,
5396   concept .value_required:n = true,
5397   sub-index .code:n =
5398     { \str_set:cn { l__spintent_sp #1 _sub_index_str } {##1} },
5399   sub-index .initial:n = ,
5400   sub-index .value_required:n = true,
5401   read-sub .bool_set:c = { l__spintent_sp #1 _read_sub_bool },
5402   read-sub .initial:n = false,
5403   read-sub .value_required:n = true,

```

```

5404         read-arg             .code:n =
5405         { \__spintent_sanitize_affix:cn { \__spintent_sp #1 _read_arg_tl } {##1} },
5406         read-arg             .initial:n = ,
5407         read-arg             .value_required:n = true,
5408         silent-cmd           .bool_set:c = { \__spintent_sp #1 _silent_cmd_bool },
5409         silent-cmd           .initial:n = false,
5410         silent-cmd           .value_required:n = true,
5411         unknown              .code:n =
5412         {
5413             \str_set:Nn \__spintent_command_name_str { sp #1 }
5414             \__spintent_unknown_keys_cmd:n {##1}
5415         },
5416     }
5417 }

```

(End of definition for `read-cmd` and others.)

`\__spintent_spcircfig_short_bool` records whether the argument resolved as the short number/letter form (via `\__spintent_luafun_geo_afig_parse_and_set:n/\__spintent_luafun_geo_pfig_parse_and_set:n`) the same entry point `\spAfig/\spPfig` use) rather than a real circle. `\__spintent_spcircfig_op_tl` holds *área* or *perímetro*. `\__spintent_spcircfig_intent_tl` holds the full spoken phrase: `\__spintent_spcircfig_del_word_tl` (via `\__spintent_spcircfig_del_word:n`: the word, contracted with `del-/de-la-`, or `read-arg`), an optional `sub-index` suffix, and, when a center is given, `-con-centro-en` — the center itself is never concatenated into it: it is read separately, by reference, through `\__spintent_mathmlarg_point:nnn`, so a subscripted or primed center (e.g. $O_{\{1\}}$, as in the Euler circle) is never an issue.

```

5418 \bool_new:N \__spintent_spcircfig_short_bool
5419 \tl_new:N \__spintent_spcircfig_op_tl
5420 \tl_new:N \__spintent_spcircfig_del_word_tl
5421 \tl_new:N \__spintent_spcircfig_intent_tl
5422 \tl_new:N \__spintent_spcircfig_intent_arg_tl

```

(End of definition for `\__spintent_spcircfig_short_bool` and others.)

`\__spintent_spcircfig_sym:n` Typesets `\bigodot` or the plain letter C, according to `print-sym`. Receives the short command name in ‘`#1`’.

```

5423 \cs_new_protected:Nn \__spintent_spcircfig_sym:n
5424 {
5425     \str_if_eq:vnTF { \__spintent_sp #1 _print_sym_str } { odot }
5426     { \bigodot }
5427     { C }
5428 }

```

(End of definition for `\__spintent_spcircfig_sym:n`.)

`\__spintent_spcircfig_word_and_article:n` Sets `\__spintent_spcirc_word_tl` to `read-arg` or `circunferencia/círculo` from `read-sty`, and `\__spintent_spcirc_articulo_str` to the matching article, which always follows `read-sty` even when `read-arg` overrides the word. Receives the short command name in ‘`#1`’.

```

5429 \cs_new_protected:Nn \__spintent_spcircfig_word_and_article:n
5430 {
5431     \str_if_eq:vnTF { \__spintent_sp #1 _read_sty_str } { circun }
5432     { \str_set:Nn \__spintent_spcirc_articulo_str { la } }
5433     { \str_set:Nn \__spintent_spcirc_articulo_str { el } }
5434     \tl_if_empty:cTF { \__spintent_sp #1 _read_arg_tl }
5435     {
5436         \str_if_eq:vnTF { \__spintent_sp #1 _read_sty_str } { circun }
5437         { \tl_set:Nn \__spintent_spcirc_word_tl { circunferencia } }
5438         { \tl_set:Nn \__spintent_spcirc_word_tl { círculo } }
5439     }
5440     {
5441         \tl_set:Ne \__spintent_spcirc_word_tl
5442         {
5443             \tl_use:c { \__spintent_sp #1 _read_arg_tl }
5444         }
5445     }
5446 }

```

(End of definition for `\__spintent_spcircfig_word_and_article:n`.)

`\__spintent_spcircfig_del_word:n` Sets `\__spintent_spcircfig_del_word_tl` to `\__spintent_spcircfig_word_and_article:n`’s word, prefixed with `del-` or `de-la-` — contracted, not `de-el-`, since a screen reader does not contract that on its own. Receives the short command name in ‘`#1`’.

```

5447 \cs_new_protected:Nn \__spintent_spcircfig_del_word:n

```

```

5448 {
5449   \__spintent_spcircfig_word_and_article:n {#1}
5450   \str_if_eq:VnTF \__spintent_spcirc_articulo_str { el }
5451   {
5452     \tl_set:Ne \__spintent_spcircfig_del_word_tl { del- \__spintent_spcirc_word_tl }
5453   }
5454   {
5455     \tl_set:Ne \__spintent_spcircfig_del_word_tl { de-la- \__spintent_spcirc_word_tl }
5456   }
5457 }

```

(End of definition for __spintent_spcircfig_del_word:n.)

__spintent_spcircfig_build_short:n

Handles the short number/letter case: builds __spintent_spcircfig_intent_arg_tl as __spintent_spcircfig_intent_arg_tl (e.g. *ber word*), wrapped in bare :prefix (no reference — there is nothing to point to), or _- when `silent-cmd` is `true` or `concept` is `false`. Receives the short command name in ‘#1’.

```

5458 \cs_new_protected:Nn \__spintent_spcircfig_build_short:n
5459 {
5460   \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
5461   {
5462     \tl_set:Nn \__spintent_spcircfig_intent_arg_tl { _- }
5463   }
5464   {
5465     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5466     {
5467       \__spintent_spcircfig_del_word:n {#1}
5468       \tl_set:Ne \__spintent_spcircfig_intent_arg_tl
5469       {
5470         \__spintent_spcircfig_op_tl - \__spintent_spcircfig_del_word_tl -
5471         \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5472         \__spintent_geo_luaset_sub_spoken_str :prefix
5473       }
5474     }
5475     {
5476       \tl_set:Nn \__spintent_spcircfig_intent_arg_tl { _- }
5477     }
5478   }
5479 }

```

(End of definition for __spintent_spcircfig_build_short:n.)

__spintent_spcircfig_finish_intent:nn

Appends an optional `sub-index` suffix and, when #2 is non-empty, `-con-centro-en` to __spintent_spcircfig_intent_arg_tl — shared by both branches of __spintent_spcircfig_build_concept:nn, after either one has set the word itself. Receives the short command name in ‘#1’ and the argument in ‘#2’.

```

5480 \cs_new_protected:Nn \__spintent_spcircfig_finish_intent:nn
5481 {
5482   \str_if_empty:cF { l__spintent_sp #1 _sub_index_str }
5483   {
5484     \tl_put_right:Ne \__spintent_spcircfig_intent_tl
5485     {
5486       -
5487       \bool_if:cT { l__spintent_sp #1 _read_sub_bool } { sub- }
5488       \str_use:c { l__spintent_sp #1 _sub_index_str }
5489     }
5490   }
5491   \tl_if_empty:nF {#2}
5492   {
5493     \tl_put_right:Nn \__spintent_spcircfig_intent_tl { -con-centro-en }
5494   }
5495 }

```

(End of definition for __spintent_spcircfig_finish_intent:nn.)

__spintent_spcircfig_build_concept:nn

Builds __spintent_spcircfig_intent_arg_tl. _- when `silent-cmd` is `true`, or when `false` and `concept` is `false` with `read-arg` empty (`( $\$$ pt)` instead when #2 is non-empty — the center is still read either way). Otherwise __spintent_spcircfig_intent_tl:prefix — with (`$\$$ pt`) appended when #2 is non-empty — prefixed with `prefix` when given, where __spintent_spcircfig_intent_tl is: with `concept` `true`, __spintent_spcircfig_op_tl- __spintent_spcircfig_del_word_tl (via __spintent_spcircfig_del_word:n: the word, contracted with `del-/de-la-`, or `read-arg` the same way); with `false` and `read-arg` given, `read-arg` verbatim instead, with __spintent_spcircfig_op_tl dropped entirely — *área/ perímetro* is itself the concept `concept` toggles, not the `-con-centro-en` that

follows, which always stays (it identifies which figure, not the concept). No automatic article either — the word (and any article it needs) is the user’s to write, same convention as `\__spintent_spcirc_process:n`. Either way, an optional `sub-index` suffix and, when `#2` is non-empty, `-con-centro-en` follow. Receives the short command name in ‘`#1`’ and the argument in ‘`#2`’.

```

5496 \cs_new_protected:Nn \__spintent_spcircfig_build_concept:nn
5497 {
5498   \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
5499   {
5500     \tl_if_empty:nTF {#2}
5501     { \tl_set:Nn \__spintent_spcircfig_intent_arg_tl { _- } }
5502     { \tl_set:Ne \__spintent_spcircfig_intent_arg_tl { _ ( $ pt ) } }
5503   }
5504   {
5505     \bool_if:cTF { l__spintent_sp #1 _concept_bool }
5506     {
5507       \__spintent_spcircfig_del_word:n {#1}
5508       \tl_set:Ne \__spintent_spcircfig_intent_tl
5509       {
5510         \__spintent_spcircfig_op_tl - \__spintent_spcircfig_del_word_tl
5511       }
5512       \__spintent_spcircfig_finish_intent:nn {#1} {#2}
5513       \tl_set:Ne \__spintent_spcircfig_intent_arg_tl
5514       {
5515         \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
5516         {
5517           \tl_use:c { l__spintent_sp #1 _prefix_tl } -
5518         }
5519         \__spintent_spcircfig_intent_tl :prefix
5520         \tl_if_empty:nF {#2} { ( $ pt ) }
5521       }
5522     }
5523     {
5524       \tl_if_empty:cTF { l__spintent_sp #1 _read_arg_tl }
5525       {
5526         \tl_if_empty:nTF {#2}
5527         { \tl_set:Nn \__spintent_spcircfig_intent_arg_tl { _- } }
5528         { \tl_set:Ne \__spintent_spcircfig_intent_arg_tl { _ ( $ pt ) } }
5529       }
5530       {
5531         \tl_set:Ne \__spintent_spcircfig_intent_tl
5532         { \tl_use:c { l__spintent_sp #1 _read_arg_tl } }
5533         \__spintent_spcircfig_finish_intent:nn {#1} {#2}
5534         \tl_set:Ne \__spintent_spcircfig_intent_arg_tl
5535         {
5536           \__spintent_spcircfig_intent_tl :prefix
5537           \tl_if_empty:nF {#2} { ( $ pt ) }
5538         }
5539       }
5540     }
5541   }
5542 }

```

(End of definition for `\__spintent_spcircfig_build_concept:nn`.)

`\__spintent_spcircfig_process:nn`

Receives the short command name in ‘`#1`’ and the argument in ‘`#2`’. Sets `\__spintent_spcircfig_op_tl` to `área/perímetro`. With `#2` empty, `\__spintent_spcircfig_short_bool` is set `false` and nothing else is parsed. With a comma in `#2`, reports `sp#1-invalid-arg` immediately — `\@@_luafun_geo_circ_parse_and_set:` (shared with `\spcirc`) still accepts a second, comma-separated part as a generic `label`-type radius, matching almost any text, so it would never itself report an error for a comma-containing argument here, where a comma is never meaningful at all. Without a comma, calls `\__spintent_luafun_geo_circ_parse_and_set:n`; on error, retries through `\__spintent_luafun_geo_afig_parse_and_set:n/\__spintent_luafun_geo_pfig_parse_and_set:n` (the same entry point `\spAfig/\spPfig` use) for the short number/letter form, rejecting a table-letter match (not meaningful for a circle) back to an error. Reports `sp#1-invalid-arg` on failure either way, then delegates to `\__spintent_spcircfig_build_short:n/\__spintent_spcircfig_build_concept:nn` and `\__spintent_spcircfig_mathml:nn`.

- Unlike `\spcirc`, there is no radius/diameter here at all: a formula’s left-hand side (`A_0`) names the figure, it does not carry the values needed to compute it — those belong on the right-hand side, elsewhere in the document.

```

5543 \cs_new_protected:Nn \__spintent_spcircfig_process:nn
5544 {
5545   \str_if_eq:eeTF {#1} { Acirc }

```

```

5546 { \tl_set:Nn \__spintent_spcircfig_op_tl { área } }
5547 { \tl_set:Nn \__spintent_spcircfig_op_tl { perímetro } }
5548 \tl_if_empty:nTF {#2}
5549 {
5550   \bool_set_false:N \__spintent_spcircfig_short_bool
5551 }
5552 {
5553   \bool_set_false:N \__spintent_spcircfig_short_bool
5554   \tl_if_in:nnTF {#2} { , }
5555   {
5556     \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5557     { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5558   }
5559   {
5560     \__spintent_luafun_geo_circ_parse_and_set:n { \exp_not:n {#2} }
5561     \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
5562     {
5563       \str_if_eq:eeTF {#1} { Acirc }
5564       { \__spintent_luafun_geo_afig_parse_and_set:n { \exp_not:n {#2} } }
5565       { \__spintent_luafun_geo_pfig_parse_and_set:n { \exp_not:n {#2} } }
5566       \str_if_eq:VnF \__spintent_geo_luaset_error_str { true }
5567       {
5568         \str_if_empty:NTF \__spintent_geo_luaset_letra_word_str
5569         { \bool_set_true:N \__spintent_spcircfig_short_bool }
5570         { \str_set:Nn \__spintent_geo_luaset_error_str { true } }
5571       }
5572     }
5573     \bool_if:NF \__spintent_spcircfig_short_bool
5574     {
5575       \str_if_eq:VnT \__spintent_geo_luaset_error_str { true }
5576       {
5577         \msg_error:nnee { spintent } { sp #1 -invalid-arg }
5578         { \c_backslash_str sp #1 } { \exp_not:n {#2} }
5579       }
5580     }
5581   }
5582 }
5583 \bool_if:NTF \__spintent_spcircfig_short_bool
5584 { \__spintent_spcircfig_build_short:n {#1} }
5585 { \__spintent_spcircfig_build_concept:nn {#1} {#2} }
5586 \__spintent_spcircfig_mathml:nn { #1 } { #2 }
5587 }

```

(End of definition for `\__spintent_spcircfig_process:nn`.)

`\__spintent_spcircfig_mathml:nn`

Renders `\__spintent_spcircfig_intent_arg_tl`, already built by `\__spintent_spcircfig_process:nn`. Three cases: the short case prints the symbol with the short value as its own subscript; an empty `#2` prints the symbol alone; a real center prints the symbol (plus optional `sub-index` subscript) and the center between parentheses. In every case a single native `\MathMLintent` wraps `\__spintent_mathcal:n{⟨A|P⟩}` and the whole subscript together — splitting the symbol and the center into separate `\MathMLintent` siblings inside the same subscript was tried and reads as `<msub>`'s own native structural narration instead. The center is read through `\__spintent_mathmlarg_point:nnn`, referenced as `$pt` (same convention as `\spcirc`). Receives the short command name in `'#1'` and the argument in `'#2'`.

```

5588 \cs_new_protected:Nn \__spintent_spcircfig_mathml:nn
5589 {
5590   \__spintent_invisible_sep: % (luamml issues #26)
5591   \MathMLintent { \__spintent_spcircfig_intent_arg_tl }
5592   {
5593     \__spintent_luafun_inner_sep:
5594     \__spintent_luafun_wrap_mrow_intent_arg:nn
5595     { } { intent = "silent" }
5596   }
5597   \str_if_eq:eeTF {#1} { Acirc } { \__spintent_mathcal:n { A } } { \__spintent_mathcal:n {
5598   }
5599   \bool_if:NTF \__spintent_spcircfig_short_bool
5600   {
5601     \c_math_subscript_token
5602     {
5603       \__spintent_spcircfig_sym:n {#1}
5604       \c_math_subscript_token { \__spintent_geo_luaset_print_tl }
5605     }

```

```

5606     }
5607     {
5608         \c_math_subscript_token
5609         {
5610             \MathMLint { :silent } { \__spintenc_spcircfig_sym:n {#1} }
5611             \str_if_empty:cF { \__spintenc_sp #1 _sub_index_str }
5612             {
5613                 \c_math_subscript_token
5614                 {
5615                     \MathMLint { :silent }
5616                     { \str_use:c { \__spintenc_sp #1 _sub_index_str } }
5617                 }
5618             }
5619             \tl_if_empty:nF {#2}
5620             {
5621                 \MathMLint { :silent } { \lparen }
5622                 \__spintenc_mathmlarg_point:nnV { pt } { sp #1 } \__spintenc_geo_luaseta_point:n
5623                 \MathMLint { :silent } { \rparen }
5624             }
5625         }
5626     }
5627 }
5628 }

```

(End of definition for `\__spintenc_spcircfig_mathml:nn`.)

`\__spintenc_spcircfig_exec:nnn`

The function `\__spintenc_spcircfig_exec:nnn` receives the short name of the command in #1, the optional keys in #2, and the argument in #3, delegating the rendering to `\__spintenc_spcircfig_process:nn`.

```

5629 \cs_new_protected:Nn \__spintenc_spcircfig_exec:nnn
5630 {
5631     \keys_set:nn { spintenc / sp #1 } { #2 }
5632     \__spintenc_spcircfig_process:nn { #1 } { #3 }
5633 }

```

(End of definition for `\__spintenc_spcircfig_exec:nnn`.)

`\spAcirc`
`\spPcirc`

Both public commands `\spAcirc` and `\spPcirc` enforce math mode, open a local group, and delegate execution to `\__spintenc_spcircfig_exec:nnn`.

```

5634 \NewDocumentCommand \spAcirc { 0{} d() }
5635 {
5636     \mode_if_math:TF
5637     {
5638         \group_begin:
5639         \attribute \__spintenc_scope_attr_int = 1
5640         \tl_if_novalue:nTF {#2}
5641         { \__spintenc_spcircfig_exec:nnn { Acirc } { #1 } { } }
5642         { \__spintenc_spcircfig_exec:nnn { Acirc } { #1 } { #2 } }
5643         \group_end:
5644     }
5645     { \msg_error:nnn { spintenc } { need-math-mode } { \spAcirc } }
5646 }
5647 \NewDocumentCommand \spPcirc { 0{} d() }
5648 {
5649     \mode_if_math:TF
5650     {
5651         \group_begin:
5652         \attribute \__spintenc_scope_attr_int = 1
5653         \tl_if_novalue:nTF {#2}
5654         { \__spintenc_spcircfig_exec:nnn { Pcirc } { #1 } { } }
5655         { \__spintenc_spcircfig_exec:nnn { Pcirc } { #1 } { #2 } }
5656         \group_end:
5657     }
5658     { \msg_error:nnn { spintenc } { need-math-mode } { \spPcirc } }
5659 }

```

(End of definition for `\spAcirc` and `\spPcirc`. These functions are documented on page ??.)

11.48 Finish package

Finish package implementation.

```

5660 \file_input_stop:
5661 \endpackage

```

12 Índice de la Implementación

The italic numbers denote the pages where the corresponding entry is described, the numbers underlined and all others indicate the line on which they are implemented in the package code.

A	
\angle	110
autofit	<u>3045</u>
C	
cifras-sep	<u>283</u> , <u>568</u> , <u>841</u> , <u>1794</u> , <u>1865</u> , <u>2386</u>
Document class:	
l3doc	29
cmd-key-unknown	<u>202</u>
cmd-key-value-unknown	<u>202</u>
Commands provide by spintent:	
\spmoney	47
\spnum	36
\spsiglo	56, 58
\spunit	41
concept <u>1956</u> , <u>2126</u> , <u>3045</u> , <u>3644</u> , <u>3851</u> , <u>4136</u> , <u>4289</u> , <u>4473</u> , <u>4854</u> , <u>5072</u> , <u>5352</u>	
currency	<u>841</u>
currency-no-subunits	<u>810</u>
D	
diameter	<u>5072</u>
dolar-math	<u>841</u>
I	
inline-numeric-denominator	<u>546</u>
invalid-angle-format	<u>2336</u>
invalid-century-format	<u>1237</u>
invalid-currency	<u>810</u>
invalid-date-format	<u>1467</u>
invalid-time-format	<u>1388</u>
J	
join-word	<u>2464</u>
K	
Keys for \spAcirc provide by spintent:	
concept	139
prefix	138, 139
print-sym	138–140
radio-sty	138
read-arg	138, 139
read-cmd	138, 139
read-sty	138, 139
read-sub	139
silent-cmd	139
sub-index	138, 139
Keys for \spAfig provide by spintent:	
prefix	121
print-sym	121, 122
read-arg	121–123
read-cmd	121
read-sub	122
Keys for \spangle provide by spintent:	
concept	109
prefix	109
print-m	109
read-cmd	109
recto	109
silent-cmd	109
Keys for \sparc provide by spintent:	
concept	115
prefix	114, 115
read-cmd	114, 115
silent-cmd	115
Keys for \spcirc provide by spintent:	
prefix	132
print-sym	132–134
radio-sty	132, 133
read-arg	133
read-cmd	132
read-sty	132, 133
read-sub	133
sub-index	132, 133
Keys for \spcoord provide by spintent:	
silent-cmd	87
Keys for \spdiam provide by spintent:	
concept	129
read-arg	129
read-cmd	129
read-sub	129
silent-cmd	129
sub-index	129
upper-case	129
Keys for \spdiv provide by spintent:	
cifras-sep	67
notation-sym	67
read-parens	67
read-period-sep	67
read-sign	67
read-sym	67
set-sym	67
wrap-parens	67
Keys for \spdsym provide by spintent:	
prefix	65
read-sym	65
set-sym	65
suffix	65
Keys for \spmangle provide by spintent:	
concept	109
prefix	109
print-m	109
read-cmd	109
recto	109
silent-cmd	109
Keys for \spmarc provide by spintent:	
concept	115
prefix	114, 115
read-cmd	114, 115
silent-cmd	115
Keys for \spmcd provide by spintent:	
concept	71
prefix	71
read-cmd	71
result	71
silent-cmd	71
upper-case	71
Keys for \spmcn provide by spintent:	
concept	71
prefix	71

read-cmd	71	radio-sty	138
result	71	read-arg	138, 139
silent-cmd	71	read-cmd	138, 139
upper-case	71	read-sty	138, 139
Keys for \spmoney provide by spintent:		read-sub	139
cifras-sep	48	silent-cmd	139
currency	48	sub-index	138, 139
dolar-math	48	Keys for \spPfig provide by spintent:	
sub-units	48	prefix	121
Keys for \spmQ provide by spintent:		print-sym	121, 122
join-word	81–83	read-arg	121–123
print-dfrac	82–85	read-cmd	121
read-parens	82, 83	read-sub	122
use-spnZ	82, 84, 85	Keys for \spPoint provide by spintent:	
wrap-parens	81–83	concept	91
Keys for \spmside provide by spintent:		prefix	91
concept	105	read-cmd	91
prefix	105	silent-cmd	91
print-m	104, 105	Keys for \spPoly provide by spintent:	
print-sym	129	concept	118
read-arg	105	prefix	117, 118
read-cmd	105, 107	print-sym	118
read-sty	105	read-cmd	117, 118
silent-cmd	105, 107	silent-cmd	118
Keys for \spmsym provide by spintent:		Keys for \sprad provide by spintent:	
not-print	66	concept	129
prefix	66	read-arg	129
read-sym	66	read-cmd	129
set-sym	66	read-sub	129
suffix	66	silent-cmd	129
Keys for \spmul provide by spintent:		sub-index	129
cifras-sep	69	upper-case	129
notation-sym	69	Keys for \sprayo provide by spintent:	
read-parens	69	concept	102
read-period-sep	69	prefix	102
read-sign	69	read-cmd	102, 103
read-sym	69	read-sty	102
set-sym	69	silent-cmd	102, 103
wrap-parens	69	Keys for \sprazon provide by spintent:	
Keys for \spnD provide by spintent:		silent-cmd	77
concept	74	use-dfrac	77
prefix	74	Keys for \sprecta provide by spintent:	
result-num	74	prefix	98
result	74	read-cmd	98
silent-cmd	74	silent-cmd	98
Keys for \spnM provide by spintent:		Keys for \spshort provide by spintent:	
concept	74	read-arg	53
prefix	74	small-caps	53
result-num	74	Keys for \spside provide by spintent:	
result	74	concept	105
silent-cmd	74	prefix	105
Keys for \spnum provide by spintent:		read-arg	104, 105
cifras-sep	35, 37	read-cmd	105, 107
notation-sym	36, 37	read-sty	104, 105
read-period-sep	36–38	silent-cmd	105, 107
read-sign	36, 37	Keys for \spsiglo provide by spintent:	
Keys for \spnZ provide by spintent:		print-era	56, 58
cifras-sep	80	Keys for \spunit provide by spintent:	
read-parens	80	cifras-sep	42
read-sign	80	notation-sym	42
wrap-parens	80	print-cdot	42
Keys for \spPcirc provide by spintent:		read-cdot	42
concept	139	read-period-sep	42
prefix	138, 139	read-slash	42
print-sym	138–140		

Keys for \spusep provide by **spintent**:

silent	62
size	62

Keys for \spvec provide by **spintent**:

autofit	94
concept	94
prefix	94
read-cmd	94
silent-cmd	94

M

mathcal-fallback	79
\MathMLintent	112
mc-invalid-arguments	1949
md-invalid-argument	2152

Lua module provide by **spintent**:

spintent.lua	29–31, 35, 36, 41, 45, 47, 48, 53, 54, 56–60, 70, 74, 78, 81, 86, 91, 128
--------------	---

N

not-print	1720
notation-sym	283, 568, 1794, 1865

P

Packages :

expl3	35, 36
lua-unicode-math	29
luamml	31, 81, 83
tagpdf	53, 57
unicode-math	29, 63
prefix	1650, 1720, 1956, 2126, 2924, 3045, 3287, 3507, 3644, 3851, 4136, 4289, 4473, 5072, 5352
print-dfrac	2464
print-era	1245
print-m	3644, 3851
print-sym	4289, 4473, 4854, 5072, 5352

R

radio-sty	5072
read-arg	1115, 3644, 4473, 4854, 5072, 5352
read-cdot	568
read-cmd	1956, 2924, 3045, 3287, 3507, 3644, 3851, 4136, 4289, 4473, 4854, 5072, 5352
read-parens	1794, 1865, 2386, 2464
read-period-sep	283, 568, 1794, 1865
read-sign	283, 1794, 1865, 2386
read-slash	568
read-sty	3507, 3644, 5072, 5352
read-sub	4473, 4854, 5072, 5352
read-sym	1650, 1720, 1794, 1865
recto	3851
result	1956, 2126
result-num	2126
right-angle-sqr-fallback	79
\rightangle	110

S

seq commands:

\seq_set_from_clist:Nn	62
set-sym	1650, 1720, 1794, 1865
\setspintent	3, 33, 190
silent	1515
silent-cmd	1956, 2126, 2266, 2679, 2924, 3045, 3287, 3507, 3644, 3851, 4136, 4289, 4473, 4854, 5072, 5352

size	1515
small-caps	1115
\spAcirc	138, 5634
spAcirc-invalid-arg	5343
\spAfig	23, 121, 4820
spAfig-invalid-arg	4435
spAfig-invalid-points	4435
spAfig-read-arg-key	4435
spAfig-sym-cardinality-conflict	4435
spAfig-sym-letra-conflict	4435
\spang	21, 78, 2376
\spangle	20, 109, 4099
spangle-invalid-arg	3841
\sparc	21, 114, 4258
sparc-invalid-points	4126
\spcirc	132, 5319
spcirc-invalid-arg	5065
\spcoord	86, 2813
spcoord-invalid-arg	2675
\spdate	11, 60, 1491
\spdiam	128, 5034
\spdiv	13, 67, 1855
spdiv-empty-argument	1814
\spdsym	12, 65, 1697
spintent internal commands:	

_spintent_collapse_hyphens:N	127
\l_spintent_command_name_str	202
\l_spintent_geo_circfig_mode_str	5342
\l_spintent_geo_final_print_tl	4427
\l_spintent_geo_luaaset_angulo_case_str	2647
\l_spintent_geo_luaaset_circ_radio_raw_str	5054
\l_spintent_geo_luaaset_circ_radio_type_str	5054
\l_spintent_geo_luaaset_concept_str	2647
\l_spintent_geo_luaaset_error_str	2647
\l_spintent_geo_luaaset_intent_str	2647
\l_spintent_geo_luaaset_letra_str	2647
\l_spintent_geo_luaaset_letra_word_str	2647
\l_spintent_geo_luaaset_nombre_es_num_str	2647
\l_spintent_geo_luaaset_points_clist\l_spintent_geo_luaaset_is_name_str	2647
\l_spintent_geo_luaaset_poly_sym_str	2647
\l_spintent_geo_luaaset_print_tl	2647
\l_spintent_geo_luaaset_sub_spoken_str	2647
\l_spintent_geo_luaaset_sub_str	2647
\l_spintent_geo_luaaset_tipo_str	2647
\l_spintent_geo_print_sym_str	4537
_spintent_inner_sep:	119
_spintent_invisible_sep:	115
\c_spintent_invisible_sep_tl	83, 123
_spintent_luafun_clean_split_and_set:n	69, 240
_spintent_luafun_spang_parse:n	79
_spintent_luafun_spdate_parse:n	61
_spintent_luafun_spmoney_lookup_data:n	69
_spintent_luafun_spmQ_scale_int:nn	69
_spintent_luafun_sptime_parse:n	60
_spintent_luafun_spunit_lookup_alias:n	69

__spintent_luafun_wrap_mrow_intent_- arg:nn	115	__spintent_spAcirc_mode_str	5332
\l__spintent_luaset_arg_above_tl	259	\l__spintent_spAcirc_prefix_tl	5332
\l__spintent_luaset_arg_below_tl	259	\l__spintent_spAcirc_print_sym_str ...	5332
\l__spintent_luaset_cal_fam_ok_str	67	\l__spintent_spAcirc_radio_sty_str ...	5332
\l__spintent_luaset_dec_is_all_zeros_- str	252	\l__spintent_spAcirc_read_arg_tl	5332
\l__spintent_luaset_dec_sep_tl	240	\l__spintent_spAcirc_read_sty_str	5332
\l__spintent_luaset_decimal_period_str	252	\l__spintent_spAcirc_sub_index_str ...	5332
\l__spintent_luaset_denom_has_number_- str	259	\l__spintent_sPAfig_mode_str	4428
\l__spintent_luaset_exponent_tl	252	\l__spintent_sPAfig_prefix_tl	4428
\l__spintent_luaset_format_part_dec_str	247	\l__spintent_sPAfig_print_sym_str	4428
\l__spintent_luaset_format_part_int_str	247	\l__spintent_sPAfig_read_arg_tl	4428
\l__spintent_luaset_has_exponent_str ..	252	\l__spintent_spang_luaset_error_str ...	2332
\l__spintent_luaset_has_integer_str ..	84, 85, 240	\l__spintent_spang_luaset_grado_str ...	2332
\l__spintent_luaset_has_natural_str ...	240	\l__spintent_spang_luaset_minuto_str ..	2332
\l__spintent_luaset_has_units_str	259	\l__spintent_spang_luaset_segundo_str .	2332
\l__spintent_luaset_millions_str	252	__spintent_spang_process:n	2341
\l__spintent_luaset_not_decimal_not_- period_str	252	__spintent_spangle_exec:nnn	4094
\l__spintent_luaset_not_decimal_period_- str	252	\l__spintent_spangle_intent_arg_tl ...	3910
\l__spintent_luaset_only_part_dec_str .	247	\l__spintent_spangle_intent_tl	3910
\l__spintent_luaset_only_part_int_str .	247	__spintent_spangle_mathml:n	3952
\l__spintent_luaset_only_part_period_- str	247	\l__spintent_spangle_mode_str	3836
\l__spintent_luaset_part_dec_tl	240	\l__spintent_spangle_prefix_tl	3836
\l__spintent_luaset_part_int_tl	84, 240	\l__spintent_spangle_print_m_now_bool .	3910
\l__spintent_luaset_part_period_tl ...	240	__spintent_spangle_process:nn	3917
\l__spintent_luaset_sign_tl	84, 240	\l__spintent_spangle_pt_idx_int	3910
\l__spintent_luaset_status_str	259	\l__spintent_spangle_pt_seq	3910
\l__spintent_luaset_sym_fam_ok_str	67	\l__spintent_spangle_refs_tl	3910
__spintent_mathcal:n	93	__spintent_spangle_sym:n	3891
__spintent_mathmlarg_point:nnn	2828	__spintent_sparc_exec:nnn	4253
\l__spintent_mathmlarg_point_kv_tl ...	2824	\l__spintent_sparc_intent_arg_tl	4180
\l__spintent_mathmlarg_point_mode_str .	2824	\l__spintent_sparc_intent_tl	4180
\l__spintent_mathmlarg_point_ref_tl ...	2824	__spintent_sparc_mathml:n	4198
\l__spintent_mathmlarg_point_silent_- bool	2824	\l__spintent_sparc_mode_str	4121
\l__spintent_mc_args_seq	1936	\l__spintent_sparc_prefix_tl	4121
\l__spintent_mc_luaset_all_numeric_str	1936	__spintent_sparc_print_arc:n	4174
\l__spintent_mc_luaset_types_str	1936	__spintent_sparc_process:nn	4185
__spintent_mc_process_args:nn	1997	\l__spintent_sparc_pt_idx_int	4180
\l__spintent_mc_raw_args_seq	1936	\l__spintent_sparc_pt_seq	4180
\l__spintent_mc_types_seq _{UUU} \l__spintent_- spmcd_print_result_bool	1936	\l__spintent_sparc_refs_tl	4180
\l__spintent_md_out_seq	2118	__spintent_sparc_wide_arc:n	4170
__spintent_print_spunit_hierarchy: ...	669	\l__spintent_spcirc_articulo_str	5056
__spintent_right_angle_sqr:	106	__spintent_spcirc_build_concept:	5200
__spintent_sanitize_affix:Nn	135	\l__spintent_spcirc_connector_arg_tl ..	5148
\l__spintent_saved_luamml_flag_int ..	83, 84, 2445	\l__spintent_spcirc_connector_tl	5148
\l__spintent_setspintent_input_arg_cmd_- tl	153	__spintent_spcirc_exec:nn	5314
__spintent_setspintent_parse_args:nn .	173	\l__spintent_spcirc_has_arg_bool	5148
__spintent_setspintent_set_keys:nn ...	162	\l__spintent_spcirc_intent_arg_tl	5148
\l__spintent_smparc_prefix_tl	4121	\l__spintent_spcirc_intent_tl	5148
__spintent_sp_rad_diam_exec:nn	5029	__spintent_spcirc_mathml:	5269
\l__spintent_sp_rad_diam_letra_tl	4852	\l__spintent_spcirc_mode_str	5056
__spintent_sp_rad_diam_print:n	4901	\l__spintent_spcirc_prefix_tl	5056
\l__spintent_sp_rad_diam_word_tl	4852	\l__spintent_spcirc_print_sym_str	5056
		__spintent_spcirc_process:n	5253
		\l__spintent_spcirc_radio_arg_tl	5148
		__spintent_spcirc_radio_print:	5185
		\l__spintent_spcirc_radio_sty_str	5056
		\l__spintent_spcirc_read_sty_str	5056
		\l__spintent_spcirc_sub_index_str	5056
		__spintent_spcirc_sym:	5154
		__spintent_spcirc_word_and_article: ..	5164
		\l__spintent_spcirc_word_tl	5064

<code>__spintent_spcircfig_build_concept:nn</code>	5496	<code>__spintent_spfig_process:nn</code>	4546
<code>__spintent_spcircfig_build_short:n</code>	5458	<code>\l__spintent_spfig_pt_idx_int</code>	4537
<code>__spintent_spcircfig_del_word:n</code>	5447	<code>\l__spintent_spfig_pt_seq</code>	4537
<code>__spintent_spcircfig_exec:nnn</code>	5629	<code>\l__spintent_spfig_refs_tl</code>	4537
<code>__spintent_spcircfig_finish_intent:nn</code>	5480	<code>__spintent_spfig_sym:</code>	4529
<code>\l__spintent_spcircfig_intent_arg_tl</code>	5418	<code>\l__spintent_spfig_symbol_tl</code>	4537
<code>\l__spintent_spcircfig_intent_tl</code>	5418	<code>\l__spintent_spmangle_mode_str</code>	3836
<code>__spintent_spcircfig_mathml:nn</code>	5588	<code>\l__spintent_spmangle_prefix_tl</code>	3836
<code>\l__spintent_spcircfig_op_tl</code>	5418	<code>\l__spintent_spmarc_mode_str</code>	4121
<code>__spintent_spcircfig_process:nn</code>	5543	<code>__spintent_spmc_process:nnnn</code>	2032
<code>\l__spintent_spcircfig_short_bool</code>	5418	<code>\l__spintent_spmcd_prefix_tl</code>	1936
<code>__spintent_spcircfig_sym:n</code>	5423	<code>\l__spintent_spmcd_print_upper_bool</code>	1936
<code>__spintent_spcircfig_word_and_-</code>		<code>\l__spintent_spmcd_read_terse_bool</code>	1936
<code> article:n</code>	5429	<code>\l__spintent_spmcd_suffix_tl</code>	1936
<code>__spintent_spcoord_exec:nn</code>	2795	<code>\l__spintent_spmcm_prefix_tl</code>	1936
<code>__spintent_spcoord_intent_infix:</code>	2766	<code>\l__spintent_spmcm_print_result_bool</code>	1936
<code>\l__spintent_spcoord_intent_str</code>	2671	<code>\l__spintent_spmcm_print_upper_bool</code>	1936
<code>\l__spintent_spcoord_luaset_error_str</code>	2661	<code>\l__spintent_spmcm_read_terse_bool</code>	1936
<code>\l__spintent_spcoord_luaset_sep_str</code>	2661	<code>\l__spintent_spmcm_spmcd_luaset_error_-</code>	
<code>\l__spintent_spcoord_luaset_x_has_frac_-</code>		<code> str</code>	1936
<code> str</code>	2661	<code>\l__spintent_spmcm_suffix_tl</code>	1936
<code>\l__spintent_spcoord_luaset_x_sign_str</code>	2661	<code>__spintent_spmoney_build_tree:</code>	1020
<code>\l__spintent_spcoord_luaset_x_type_str</code>	2661	<code>__spintent_spmoney_check_sub_units:</code>	973
<code>\l__spintent_spcoord_luaset_x_value_-</code>		<code>\l__spintent_spmoney_extracted_curr_str</code>	802
<code> str</code>	2661	<code>\l__spintent_spmoney_extracted_num_tl</code>	802
<code>\l__spintent_spcoord_luaset_y_has_frac_-</code>		<code>__spintent_spmoney_intent_int:</code>	938
<code> str</code>	2661	<code>\l__spintent_spmoney_intent_money_str</code>	802
<code>\l__spintent_spcoord_luaset_y_sign_str</code>	2661	<code>\l__spintent_spmoney_intent_sign_str</code>	802
<code>\l__spintent_spcoord_luaset_y_type_str</code>	2661	<code>\l__spintent_spmoney_intent_sub_unit_-</code>	
<code>\l__spintent_spcoord_luaset_y_value_-</code>		<code> str</code>	802
<code> str</code>	2661	<code>\l__spintent_spmoney_luaset_conde_str</code>	792
<code>__spintent_spcoord_mathml:</code>	2778	<code>\l__spintent_spmoney_luaset_currency_arg_-</code>	
<code>__spintent_spcoord_print:n</code>	2743	<code> str</code>	792
<code>\l__spintent_spcoord_raw_arg_tl</code>	2671	<code>\l__spintent_spmoney_luaset_plur_str</code>	792
<code>__spintent_spcoord_set_wrap_parens_-</code>		<code>\l__spintent_spmoney_luaset_position_-</code>	
<code> size:</code>	2690	<code> str</code>	792
<code>__spintent_spcoord_value:n</code>	2737	<code>\l__spintent_spmoney_luaset_print_iso_-</code>	
<code>\l__spintent_spcoord_wrap_parens_l_tl</code>	2671	<code> str</code>	792
<code>\l__spintent_spcoord_wrap_parens_r_tl</code>	2671	<code>\l__spintent_spmoney_luaset_sing_str</code>	792
<code>\l__spintent_sptime_luaset_error_str</code>	1465	<code>\l__spintent_spmoney_luaset_status_str</code>	792
<code>\l__spintent_sptime_luaset_output_str</code>	1465	<code>\l__spintent_spmoney_luaset_subplur_str</code>	792
<code>__spintent_sptime_process:n</code>	61, 1476	<code>\l__spintent_spmoney_luaset_subsing_str</code>	792
<code>\l__spintent_spdiam_mode_str</code>	4846	<code>\l__spintent_spmoney_luaset_symbol_str</code>	792
<code>\l__spintent_spdiam_sub_index_str</code>	4846	<code>__spintent_spmoney_normalize_key:n</code>	827
<code>\l__spintent_spdiam_upper_case_bool</code>	4846	<code>__spintent_spmoney_parse_and_route:n</code>	1068
<code>__spintent_spdiv_process:nnn</code>	1840	<code>__spintent_spmoney_print_dec_sub_-</code>	
<code>__spintent_spdiv_render_op:nn</code>	1819	<code> unit:</code>	1020
<code>__spintent_spdiv_render_parens:nn</code>	1828	<code>__spintent_spmoney_print_int:</code>	909
<code>\l__spintent_spdsym_prefix_tl</code>	1646	<code>__spintent_spmoney_print_sep_con:</code>	1020
<code>\l__spintent_spdsym_read_sym_tl</code>	1646	<code>__spintent_spmoney_print_symbol:</code>	875
<code>\l__spintent_spdsym_suffix_tl</code>	1646	<code>__spintent_spmoney_render_layout:</code>	888
<code>\l__spintent_spdsym_symbol_tl</code>	1646	<code>__spintent_spmoney_render_number_node:</code>	900
<code>__spintent_spfig_build_concept_-</code>		<code>__spintent_spmoney_render_only_integer_-</code>	
<code> vertices:n</code>	4693	<code> node:</code>	900
<code>__spintent_spfig_build_refs:</code>	4680	<code>\l__spintent_spmoney_res_main_voice_str</code>	802
<code>__spintent_spfig_build_simple:n</code>	4570	<code>__spintent_spmoney_set_and_print_sub_-</code>	
<code>__spintent_spfig_exec:nnn</code>	4815	<code> units:</code>	973
<code>\l__spintent_spfig_intent_arg_tl</code>	4537	<code>__spintent_spmoney_set_currency:n</code>	827
<code>\l__spintent_spfig_intent_tl</code>	4537	<code>\l__spintent_spmoney_sub_units_bool</code>	802
<code>__spintent_spfig_mathml:n</code>	4775	<code>\l__spintent_spmoney_subnum_int</code>	802
<code>\l__spintent_spfig_op_tl</code>	4537	<code>__spintent_spmoney_validate_money:</code>	859
<code>__spintent_spfig_points_loop:n</code>	4763	<code>__spintent_spmQ_exec:nnnn</code>	2625

151 / 152

<code>\l__spintent_sprecta_refs_tl</code>	3273	<code>\l__spintent_spunit_luaset_lookup_status_-</code> <code>str</code>	532
<code>__spintent_spsshort_execute_layout:</code> . .	1176	<code>\l__spintent_spunit_luaset_read_str</code> . .	532
<code>\l__spintent_spsshort_luaset_base_str</code> . .	1093	<code>\l__spintent_spunit_luaset_status_str</code> .	532
<code>\l__spintent_spsshort_luaset_expanded_-</code> <code>str</code>	1093	<code>\l__spintent_spunit_mult_seq</code>	539
<code>\l__spintent_spsshort_luaset_layout_str</code>	1093	<code>__spintent_spunit_process_base_exp:n</code> .	610
<code>\l__spintent_spsshort_luaset_output_str</code>	1093	<code>__spintent_spunit_process_mult:n</code> . . .	662
<code>\l__spintent_spsshort_luaset_status_str</code>	1093	<code>\l__spintent_spunit_read_cdot_bool</code> . . .	539
<code>\l__spintent_spsshort_luaset_suffix_str</code>	1093	<code>\l__spintent_spunit_read_slash_str</code> . . .	539
<code>__spintent_spsshort_print:nn</code>	1129	<code>__spintent_spunit_safe_exec:n</code>	681
<code>__spintent_spsshort_process_and_-</code> <code>render:n</code>	1141	<code>\l__spintent_spunit_unit_name_tl</code>	539
<code>__spintent_spsshort_render_bypass:n</code> . .	1152	<code>\l__spintent_spusep_arg_tl</code>	62, 63, 1512
<code>__spintent_spsshort_render_error:n</code> . .	1152	<code>__spintent_spusep_convert:n</code>	1574
<code>__spintent_spsshort_render_fallback:</code> . .	1152	<code>\l__spintent_spusep_left_tl</code>	62, 63, 1512
<code>__spintent_spsshort_render_success:</code> . .	1152	<code>\l__spintent_spusep_right_tl</code>	62, 63, 1512
<code>__spintent_spsshort_render_success:</code> . .	1152	<code>__spintent_spusep_set_parens:nn</code>	1563
<code>__spintent_spsshort_tag_span:nn</code>	1102	<code>\l__spintent_spusep_silent_bool</code>	63
<code>__spintent_spside_exec:nnn</code>	3809	<code>__spintent_spvec_exec:nnn</code>	3237
<code>\l__spintent_spside_intent_arg_tl</code>	3701	<code>\l__spintent_spvec_intent_arg_tl</code>	3034
<code>\l__spintent_spside_intent_tl</code>	3701	<code>\l__spintent_spvec_kv_tl</code>	3034
<code>__spintent_spside_mathml:n</code>	3734	<code>__spintent_spvec_mathml:n</code>	3096
<code>\l__spintent_spside_mode_str</code>	3627	<code>\l__spintent_spvec_mode_str</code>	3034
<code>__spintent_spside_points_loop:n</code>	3797	<code>\l__spintent_spvec_prefix_tl</code>	3034
<code>\l__spintent_spside_prefix_tl</code>	3627	<code>__spintent_spvec_print:n</code>	3079
<code>__spintent_spside_process:nn</code>	3706	<code>\l__spintent_spvec_pt_idx_int</code>	3034
<code>\l__spintent_spside_pt_idx_int</code>	3701	<code>\l__spintent_spvec_pt_seq</code>	3034
<code>\l__spintent_spside_pt_seq</code>	3701	<code>\l__spintent_spvec_refs_tl</code>	3034
<code>\l__spintent_spside_read_arg_tl</code>	3627	<code>__spintent_unknown_keys:n</code>	229
<code>\l__spintent_spside_read_sty_str</code>	3627	<code>__spintent_unknown_keys:nn</code>	229
<code>\l__spintent_spside_refs_tl</code>	3701	<code>\spmangle</code>	21, 109, 4099
<code>\l__spintent_spsiglo_luaset_arabic_str</code>	1233	<code>spmangle-invalid-arg</code>	3841
<code>\l__spintent_spsiglo_luaset_error_str</code> .	1233	<code>\spmarc</code>	22, 114, 4258
<code>\l__spintent_spsiglo_luaset_roman_min_-</code> <code>str</code>	1233	<code>spmarc-invalid-points</code>	4126
<code>\l__spintent_spsiglo_print_era_str</code> . . .	1233	<code>\spmcd</code>	15, 70, 2098
<code>__spintent_spsiglo_print_math:n</code>	1317	<code>\spmcm</code>	15, 70, 2098
<code>__spintent_spsiglo_print_text:n</code>	1317	<code>\spmoney</code>	6, 47, 1082
<code>__spintent_spsiglo_render_simple:nn</code> . .	1268	<code>\spmQ</code>	16, 81, 2637
<code>__spintent_spsiglo_render_with_-</code> <code>era:nnn</code>	1268	<code>spmQ-not-integer</code>	2450
<code>__spintent_spsiglo_tag_span:nn</code>	1304	<code>spmQ-zero-denominator</code>	2450
<code>__spintent_sptime_build_seconds:nn</code> . .	1397	<code>\spmside</code>	19, 104, 3814
<code>__spintent_sptime_build_tree:</code>	60, 1415	<code>spmside</code>	5072
<code>\l__spintent_sptime_luaset_base_str</code> . .	1383	<code>spmside-invalid-points</code>	3634
<code>\l__spintent_sptime_luaset_error_str</code> . .	1383	<code>\spmsym</code>	12, 66, 1769
<code>\l__spintent_sptime_luaset_has_seconds_-</code> <code>str</code>	1383	<code>\spm\l</code>	13, 69, 1926
<code>\l__spintent_sptime_luaset_is_fraction_-</code> <code>str</code>	1383	<code>spm\l-empty-argument</code>	1885
<code>\l__spintent_sptime_luaset_seconds_str</code>	1383	<code>\spnD</code>	14, 74, 2246
<code>\l__spintent_spunit_exp_seq</code>	539	<code>\spnewunit</code>	6, 45, 760
<code>\l__spintent_spunit_exponent_tl</code>	539	<code>spnewunit-duplicate</code>	739
<code>__spintent_spunit_format_canonical:</code> . .	602	<code>\spnM</code>	14, 74, 2246
<code>\l__spintent_spunit_is_mult_bool</code>	539	<code>\spnum</code>	3, 36, 37, 514
<code>\l__spintent_spunit_luaset_canonical_-</code> <code>str</code>	532	<code>spnum-has-units</code>	500
<code>\l__spintent_spunit_luaset_compact_exp_-</code> <code>str</code>	532	<code>spnum-no-digits</code>	500
<code>\l__spintent_spunit_luaset_is_compact_-</code> <code>str</code>	532	<code>\spnZ</code>	16, 79, 2435
<code>\l__spintent_spunit_luaset_is_-</code> <code>sexagesimal_str</code>	532	<code>spnz-not-integer</code>	2402
		<code>\spPcirc</code>	138, 5634
		<code>spPcirc-invalid-arg</code>	5343
		<code>\spPfig</code>	24, 121, 4820
		<code>spPfig-invalid-arg</code>	4435
		<code>spPfig-invalid-points</code>	4435
		<code>spPfig-read-arg-key</code>	4435
		<code>spPfig-sym-cardinality-conflict</code>	4435

spPfig-sym-letra-conflict	4435	spunitalias-invalid-expr	739
\spPoint	17, 91, 3023	\spusep	12, 62, 1612
spPoint-invalid-point	2916	spusep-empty-arg	1501
\spPoly	22, 117, 4416	spusep-invalid-arg	1501
spPoly-invalid-points	4282	\spvec	93, 3262
\spProp	78, 2316	spvec-invalid-arg	3041
\sprad	128, 5034	str commands:	
\sprayo	18, 102, 3616	\str_if_eq:nnTF	62
sprayo-invalid-points	3500	\str_lowercase:n	62
\sprazon	77, 2306	\str_uppercase:n	62
\spread	11, 64, 1636	sub-index	4854, 5072, 5352
spread-empty-args	1623	sub-units	841
\sprecta	18, 98, 3486	suffix	1650, 1720
sprecta-invalid-points	3280		
\spshort	7, 53, 1220	U	
\spside	19, 104, 3814	unknown	1515, 1650, 1720, 1956, 2126, 2266, 2464, 2924, 3045, 3287, 3507, 3644, 3851, 4136, 4289, 4473, 4854, 5072, 5352
spside	5072	unknown-abbreviation	1099
spside-invalid-points	3634	unknown-choice	202
\spsiglo	11, 56, 1370	unknown-command-setup	153
\sptime	11, 59, 1447	upper-case	1956, 4854
\spunit	4, 41, 42, 45, 727	use-dfrac	2266
spunit-invalid-unit	546	use-spnZ	2464
spunit-no-units	546		
\spunitalias	6, 45, 774	W	
spunitalias-duplicate	739	wrap-parens	1794, 1865, 2386, 2464